Redbooks

ibm.com/redbooks

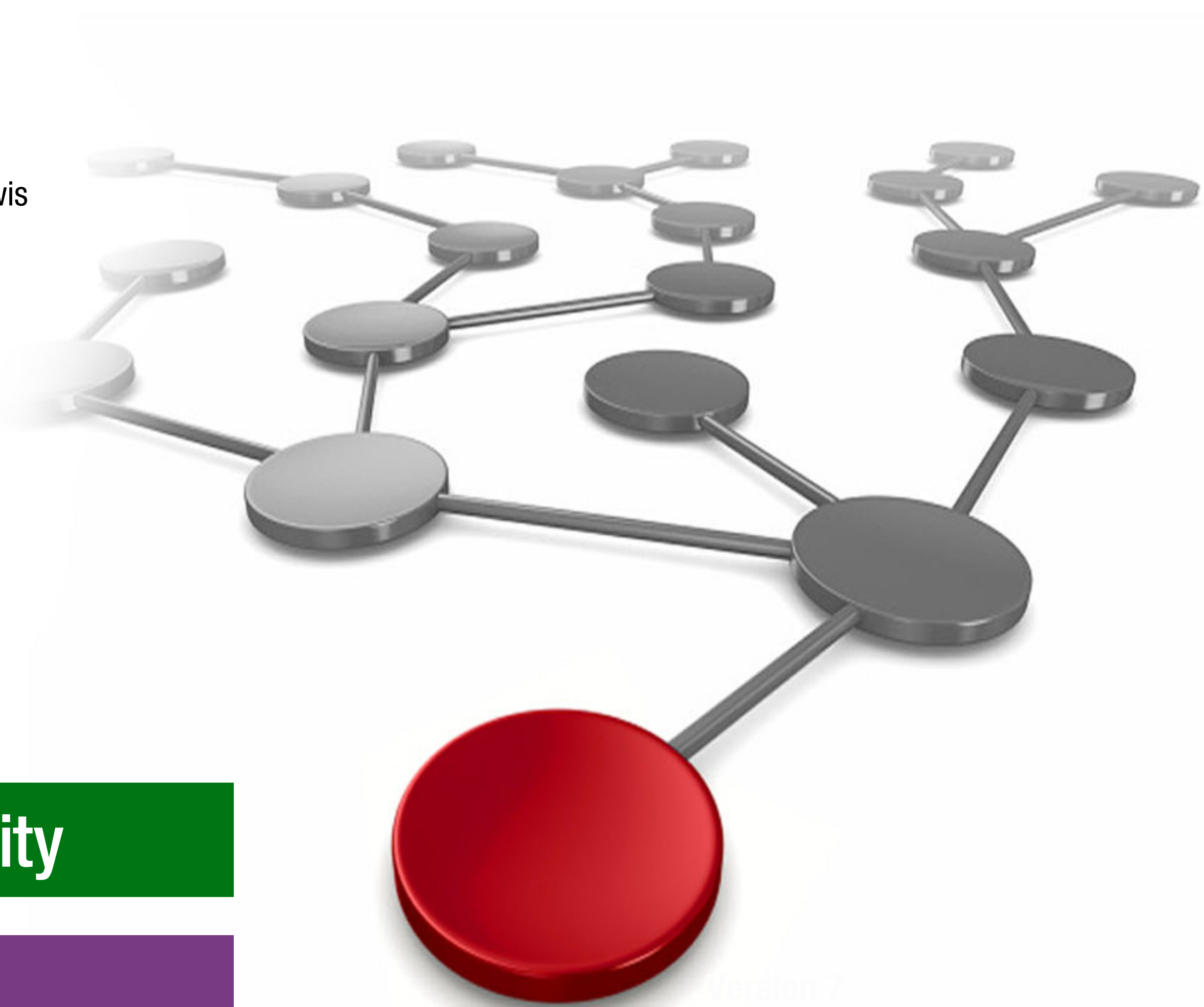
Getting Started with z/OS Data Set Encryption

Bill White

Didier Andre

Cecilia Carranza Lewis

Kasper Lindberg



Security

IBM Z

IBM

Redbooks

IBM Redbooks

Getting Started with z/OS Data Set Encryption

March 2026

SG24-8410-02

Note: Before using this information and the product it supports, read the information in “Notices” on page ix.

Third Edition (March 2026)

This edition applies to the required and optional hardware and software components needed for z/OS data set encryption.

Contents

Notices

This information was developed for products and services offered in the US. This material might be available from IBM in other languages. However, you may be required to own a copy of the product or product version in that language in order to access it.

IBM may not offer the products, services, or features discussed in this document in other countries. Consult your local IBM representative for information on the products and services currently available in your area. Any reference to an IBM product, program, or service is not intended to state or imply that only that IBM product, program, or service may be used. Any functionally equivalent product, program, or service that does not infringe any IBM intellectual property right may be used instead. However, it is the user's responsibility to evaluate and verify the operation of any non-IBM product, program, or service.

IBM may have patents or pending patent applications covering subject matter described in this document. The furnishing of this document does not grant you any license to these patents. You can send license inquiries, in writing, to:
IBM Director of Licensing, IBM Corporation, North Castle Drive, MD-NC119, Armonk, NY 10504-1785, US

INTERNATIONAL BUSINESS MACHINES CORPORATION PROVIDES THIS PUBLICATION "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Some jurisdictions do not allow disclaimer of express or implied warranties in certain transactions, therefore, this statement may not apply to you.

This information could include technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in new editions of the publication. IBM may make improvements and/or changes in the product(s) and/or the program(s) described in this publication at any time without notice.

Any references in this information to non-IBM websites are provided for convenience only and do not in any manner serve as an endorsement of those websites. The materials at those websites are not part of the materials for this IBM product and use of those websites is at your own risk.

IBM may use or distribute any of the information you provide in any way it believes appropriate without incurring any obligation to you.

The performance data and client examples cited are presented for illustrative purposes only. Actual performance results may vary depending on specific configurations and operating conditions.

Information concerning non-IBM products was obtained from the suppliers of those products, their published announcements or other publicly available sources. IBM has not tested those products and cannot confirm the accuracy of performance, compatibility or any other claims related to non-IBM products. Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products.

Statements regarding IBM's future direction or intent are subject to change or withdrawal without notice, and represent goals and objectives only.

This information contains examples of data and reports used in daily business operations. To illustrate them as completely as possible, the examples include the names of individuals, companies, brands, and products. All of these names are fictitious and any similarity to actual people or business enterprises is entirely coincidental.

COPYRIGHT LICENSE:

This information contains sample application programs in source language, which illustrate programming techniques on various operating platforms. You may copy, modify, and distribute these sample programs in any form without payment to IBM, for the purposes of developing, using, marketing or distributing application programs conforming to the application programming interface for the operating platform for which the sample programs are written. These examples have not been thoroughly tested under all conditions. IBM, therefore, cannot guarantee or imply reliability, serviceability, or function of these programs. The sample programs are provided "AS IS", without warranty of any kind. IBM shall not be liable for any damages arising out of your use of the sample programs.

Trademarks

IBM, the IBM logo, and ibm.com are trademarks or registered trademarks of International Business Machines Corporation, registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the web at "Copyright and trademark information" at http://www.ibm.com/legal/copytrade.shtml

The following terms are trademarks or registered trademarks of International Business Machines Corporation, and might also be trademarks or registered trademarks in other countries.

CICS®	IBM FlashSystem®	Tivoli®
Db2®	IBM Security®	WebSphere®
FlashCopy®	IBM Z®	z/OS®
Guardium®	RACF®	z15®
IBM®	Redbooks®	z17™
IBM Cloud®	Redbooks (logo) ®	

The registered trademark Linux® is used pursuant to a sublicense from the Linux Foundation, the exclusive licensee of Linus Torvalds, owner of the mark on a worldwide basis.

Microsoft, and the Windows logo are trademarks of Microsoft Corporation in the United States, other countries, or both.

Red Hat, OpenShift, are trademarks or registered trademarks of Red Hat, Inc. or its subsidiaries in the United States and other countries.

Other company, product, or service names may be trademarks or service marks of others.

Preface

This IBM® Redbooks® publication provides a broad explanation of data protection through encryption and IBM Z® pervasive encryption with a focus on IBM z/OS® data set encryption. It describes how the various hardware and software components interact in a z/OS data set encryption environment.

In addition, this book focuses on the planning and preparation of the environment and offers implementation, configuration, and operational examples that can be used in z/OS data set encryption environments.

This publication is intended for IT managers, IT architects, system programmers, and security administrators who plan for, deploy, and manage security on the Z platform. The reader is expected to have a basic understanding of IBM Z security concepts.

Authors

This book was produced by a team of specialists from around the world working at IBM Redbooks, Poughkeepsie Center.

Bill White is an IBM Redbooks Project Leader and Senior Infrastructure Specialist at IBM Redbooks, Poughkeepsie Center.

Didier Andre is a Principal z Security Services Delivery Leader with Technology Expert Labs at IBM US with more than 30 years of experience in IBM Z, mainframe security, and enterprise infrastructure. He works closely with global clients, product teams, and sales organizations to drive client success. A recognized IBM Z security expert, Didier has led security consulting, technical sales, and service delivery in the United States and worldwide. He is the creator of the Pervasive Encryption Readiness Assessment (PERA) a framework that has helped numerous IBM Z clients to navigate their journey toward full-scale encryption.

Cecilia Carranza Lewis is a Senior Technical Staff Member (STSM) Software Engineer within IBM Z development, based at the Silicon Valley Laboratory (SVL) in San Jose, CA. Her focus is on strategy and architecture in the area of z/OS data and storage management within z/OS Data Facilities Storage Management Subsystem (DFSMS), where she has more than 30 years of experience. Cecilia has held numerous technical leadership positions focused on supporting clients' explosive data growth on IBM Z mainframes through improvements in total cost of ownership, simplification, and data security. She is also the architect for the z/OS data set encryption solution.

Kasper Lindberg is a certified senior IT specialist working in the IBM Crypto Competence Center based in Copenhagen, Denmark. He has worked at IBM for 13 years and has experience in security, cryptography and key management, in particular in relation to the payment card industry. He has worked with IBM solutions in the field of cryptography and enterprise key management and is an expert in the IBM Enterprise Key Management Foundation solution. He holds a Master of Science in Computer Science & Engineering from the Technical University of Denmark.

Thanks to the following people for their contributions to this project:

Robert Haimowitz
IBM Redbooks®, Poughkeepsie Center

A special thanks to the authors of the first and second editions of this book for their groundbreaking contributions and insights, which laid the foundation for everything that followed.

Isabel Arnold
Andy Coulson
Jacky Doll
Brad Habbershow
Octavian Lascu
Thomas Liu
Ryan McCarry
Eysha Shirrine Powers
Philippe Richard
David Rossi
Eric Rossman
Romoaldo Santos

Now you can become a published author, too!

Here's an opportunity to spotlight your skills, grow your career, and become a published author—all at the same time! Join an IBM Redbooks residency project and help write a book in your area of expertise, while honing your experience using leading-edge technologies. Your efforts will help to increase product acceptance and customer satisfaction, as you expand your network of technical contacts and relationships. Residencies run from two to six weeks in length, and you can participate either in person or as a remote resident working from your home base.

Find out more about the residency program, browse the residency index, and apply online at:

ibm.com/redbooks/residencies.html

Comments welcome

Your comments are important to us!

We want our books to be as helpful as possible. Send us your comments about this book or other IBM Redbooks publications in one of the following ways:

- Use the online **Contact us** review Redbooks form found at:

  ibm.com/redbooks

- Send your comments in an email to:

  redbooks@us.ibm.com

- Mail your comments to:

  IBM Corporation, IBM Redbooks
  Dept. HYTD Mail Station P099
  2455 South Road
  Poughkeepsie, NY 12601-5400

Stay connected to IBM Redbooks

- Find us on Facebook:

  http://www.facebook.com/IBMRedbooks
- Follow us on Twitter:

  http://twitter.com/ibmredbooks
- Look for us on LinkedIn:

  http://www.linkedin.com/groups?home=&gid=2130806
- Explore new Redbooks publications, residencies, and workshops with the IBM Redbooks weekly newsletter:

  https://www.redbooks.ibm.com/Redbooks.nsf/subscribe?OpenForm
- Stay current on recent Redbooks publications with RSS Feeds:

  http://www.redbooks.ibm.com/rss.html

1

Protecting data in today's IT environment

Data is the foundation of every enterprise. Loss, exposure, or misuse of sensitive data can cause irreparable damage: financial loss, reputational harm, and operational disruption.

Regulatory frameworks such as GDPR, HIPAA, and PCI DSS impose strict requirements with severe penalties for noncompliance, creating intense pressure from internal and external stakeholders.

These realities are reshaping how organizations think about data security.

Encryption is one of the most effective ways to protect data, but meeting compliance requirements is the minimum threshold, not the end goal. Traditional selective encryption (protecting only certain data types) leaves gaps and increases complexity.

A best practice is pervasive encryption: encrypting all data at rest and in flight. This approach eliminates selective coverage risks, simplifies operations, and strengthens compliance posture. Pervasive encryption is not just a compliance tool. It is a strategic enabler for trust, resilience, and competitive advantage.

This chapter introduces the concept of pervasive encryption and how you can use z/OS data set encryption to protect your data. It includes the following topics:

1.1 Data considerations

How do you define data? Is your data a file, spreadsheet, row, column, or field?

Where is your data stored? Who creates it? Where is the data created? Where is the data transmitted? Who receives the data? Where is the data stored for immediate use? Where is the data backed up? Where is the data recovered from in a disaster?

When you are planning for data protection, consider the entire lifecycle of the data from creation to destruction and throughout its journey (in-flight and at-rest). Consider how the data enters and exits your environment, and also consider potential exposure after the data is out of your control.

1.1.1 Data at-rest

Data at-rest includes data that is stored in data sets and files that are written to storage devices, such as disk and tape. Data at-rest can persist even when the associated application is no longer running. When the application is restarted, the application can retrieve the data at-rest because it is stored on disk or tape.

1.1.2 Data in-use

Data in-use includes medical records that can be in memory before being written to data sets and storage devices. Data in-use is not persistent. However, it can be readable in system dumps.

1.1.3 Data in-flight

Data in-flight includes passwords and credit card numbers that are sent over a network to authenticate a user to a server, for example, to make an online purchase. Data in-flight also includes data that is sent over the storage area network from a host system to disk and tape devices. Data in-flight can be stored persistently or it might be in-use in the system temporarily to complete a transaction or operation.

1.1.4 Sensitive data

Recognizing sensitive data might not be difficult with formatted data, such as credit card numbers, social security numbers, and passwords. However, identifying all the places where that sensitive data is stored can be a challenge.

Security administrators must consider the following questions:

- Is sensitive data in a database?
- Is sensitive data in a file or data set?
- Is sensitive data in memory? Will it appear in a dump?
- Is sensitive data in the network?
- Is sensitive data on a backup tape?
- Is sensitive data shared with a third party?

To truly protect the data, you must know what data to protect and where that data is located.

1.2 Why protect data

Although data protection is often driven by industry regulations, compliance with regulations are considered only as a minimum threshold. Every organization should consider the types of scenarios that can threaten the security of sensitive data and determine the risks and effect on the organization. This section introduces the possible threats and some questions to help identify these types of threats.

1.2.1 Accidental exposure

Accidental data exposure can occur when unencrypted data is misplaced (or not stored in a tightly controlled environment), and, as a consequence, is exposed to unauthorized views.

For example, in an IBM Db2® environment, many security controls are available to control access to view or query data. However, Db2 data is stored persistently in data sets. Possible questions that would help identify potential exposure of data to accidental views include:

- Does a developer without authorization to perform SQL queries have authorization to the data sets that back the Db2 table?
- Can those data sets be viewed or copied to development environments where the data is used for internal testing?
- What view and copy operations are allowed in your organization that might enable accidental exposure of data?

1.2.2 Insider attacks

Insider attacks come from within the organization, and are usually performed by entrusted users with the intent to either steal information or damage it.

For example, in an IBM CICS® environment, millions of transactions can occur in an hour where credit card numbers and transaction details might be stored in data sets. Storage administrators are authorized to access the data sets for creating, deleting, backing up, and recovering tasks, so one of the questions that might help identify potential insider attack exposures is, can storage administrators copy sensitive data and transmit it to a nontrusted environment?

1.2.3 Data breaches

Data breaches include intentional or accidental unauthorized access of unencrypted sensitive data. Savvy security administrators ensure that known sensitive data is encrypted. However, organizations can have thousands of data sets in their environment, so the following questions might help identify possible exposures:

- How can you verify that every location where sensitive data might be is protected?
- If data is encrypted selectively, might you be giving away information to hackers about where your sensitive data is stored?

1.2.4 Regulations

Industry regulations, such as the European Union (EU) General Data Protection Regulation (GDPR), Payment Card Industry Data Security Standard (PCI-DSS), and Health Insurance Portability and Accountability Act (HIPAA) require organizations to protect sensitive data.

These regulations impose sharp penalties for the disclosure of sensitive data. Some possible questions to help with regulations awareness are:

- Which regulations apply to your organization?
- Is your sensitive data protected?

For additional information, see 1.3, “Standards and regulations overview” on page 4.

1.3 Standards and regulations overview

Worldwide, there are many standards and regulations that are related to protecting sensitive data. Most of these standards are concerned with protection of data at rest, during transactions, and while traversing network connections. Compliance starts with identifying what data is present and what data needs to be protected. Although some of the standards specify the technologies required for compliance, encryption can be applied to achieve compliance for all of them.

In addition, often there are also requirements for auditing and sufficiently granular access controls.

1.3.1 PCI Data Security Standards (PCI-DSS)

PCI-DSS applies if you accept or process payment cards. These standards cover technical and operational system components included in or connected to cardholder data and have 6 goals:

1. Build and Maintain a Secure Network
2. Protect Cardholder Data
3. Maintain a Vulnerability Management Program
4. Implement Strong Access Control Measures
5. Regularly Monitor and Test Networks
6. Maintain an Information Security Policy

For Payment Card Industry Security Standards, see PCI Security Standards Council.

1.3.2 General Data Protection Regulation (GDPR)

GDPR is a legal framework that sets guidelines for the collection and processing of personal information from individuals who live in the European Union (EU). All entities that attract European visitors, even if they don't specifically market goods or services to EU residents, must adhere to these Regulations.

For more information, see General Data Protection Regulation.

1.3.3 California Consumer Privacy Act (CCPA)

The CCPA is similar to the GDPR in purpose. It provides for new privacy rights for the State of California (US) residents.

For more information, see CCPA Regulations.

1.3.4 The Sarbanes-Oxley Act of 2002 (SOX)

The Sarbanes-Oxley Act is a United States federal law that set new or expanded requirements for all US public company boards, management and public accounting firms. It was created in response to a number of corporate scandals and requires public companies to strengthen audit committees, perform internal controls tests, make directors and officers personally liable for the accuracy of financial statements, and strengthen disclosure.

For more information, see H.R. 3763 - Sarbanes-Oxley Act of 2002.

Additionally, a legislative bill introduced in Ontario, Canada included clauses that provide SOX-equivalent provisions. This is "*The Keeping the Promise for a Strong Economy Act (Budget Measures), 2002*".

1.3.5 ISO/IEC 27001

ISO/IEC 27001:2013 specifies the requirements for establishing, implementing, maintaining, and continually improving an information security management system within the context of the organization. It also includes requirements for the assessment and treatment of information security risks tailored to the needs of the organization. The requirements set out in ISO/IEC 27001:2013 are generic and are intended to be applicable to all organizations, regardless of type, size or nature.

For more information, see ISO/IEC 27001 Information Security Management.

1.3.6 Federal Information Security Modernization Act of 2014 (FISMA 2014)

The Federal Information Security Modernization Act of 2014 (FISMA 2014) updates three Federal Government cybersecurity practices:

1. Codifying Department of Homeland Security (DHS) authority to administer the implementation of information security policies for non-national security federal Executive Branch systems, including providing technical assistance and deploying technologies to such systems.
2. Amending and clarifying the Office of Management and Budget's (OMB) oversight authority over federal agency information security practices.
3. Requiring OMB to amend or revise OMB A-130 to "eliminate inefficient and wasteful reporting".

For more information, see S.2521- Federal Information Security Modernization Act of 2014.

NIST Special Publication 800-53 (Security and Privacy Controls for Information Systems and Organizations) was developed in response to FISMA 2014 and provides a catalog of security and privacy controls for information systems and organizations.

1.3.7 Payment Card Industry (PCI) PTS HSM Security Requirements (PCI-HSM)

The PCI HSM standard covers the lifecycle of the HSM up to the point of its first delivery to the initial point of deployment facility. Subsequent stages of the HSM lifecycle continue to be of interest to PCI and are controlled by other PCI standards.

For more information, see HSM Device Evaluation: Frequently Asked Questions.

1.3.8 German Banking Industry Committee (GBIC)

The GBIC is not a standard. It is a consortium that develops common banking-industry positions on issues relating to banking law, banking policy, and banking practice. These cover, in particular, supervisory, securities and tax legislation. GBIC also drafts standardized rules for the payments sector, including card payment schemes.

For more information, see The German Banking Industry Committee.

1.3.9 Australian Payments Network (Auspaynet)

Australian Payments Network is not a standard. It is a consortium that is champions the payments system in Australia. Its role includes:

- Inspiring innovation
- Facilitating self-regulation
- Coordinating system-wide standards
- Policy development

For more information, see Australian Payments Network.

1.3.10 Common Criteria

Common Criteria certification focuses on ensuring that IT products and protection profiles are performed to high and consistent standards and are seen to contribute significantly to confidence in the security of those products and profiles. The intention of this certification is that these products can be procured or used without the need for further evaluation.

For more information, see Common Criteria.

1.3.11 FIPS PUB 140-3 (Security Requirements for Cryptographic Modules)

FIPS PUB 140-3 provides four increasing, qualitative levels of security intended to cover a wide range of potential applications and environments. The security requirements cover areas related to the secure design, implementation, and operation of a cryptographic module. These areas include:

- Cryptographic module specification
- Cryptographic module interfaces
- Roles, services, and authentication
- Software/firmware security
- Operating environment
- Physical security
- Noninvasive security
- Sensitive security parameter management
- Self-tests
- Lifecycle assurance
- Mitigation of other attacks

For more information, see Security Requirements for Cryptographic Modules.

1.3.12 HIPAA/HITECH

The Health Insurance Portability and Accountability Act of 1996 (HIPAA), Public Law 104-191, was intended to modernize the health care transaction processing, including mandating the adoption of Federal privacy protections for individually identifiable health information. It has been amended multiple times, most notably in 2009 with the HITECH (Health Information Technology for Economic and Clinical Health) Act. The HITECH Act changed the language of the HIPAA, increased penalties for compliance failures, and added incentives to encourage use of electronic healthcare records.

For more information, see H.R.3103 - Health Insurance Portability and Accountability Act of 1996.

1.3.13 Electronic IDentification, Authentication and trust Services

In 2014, the European Parliament ratified the electronic IDentification, Authentication and trust Services (eIDAS) to establish “a legal framework for electronic signatures, electronic seals, electronic time stamps, electronic documents, electronic registered delivery services and certificate services for website authentication.” This was done to facilitate secure electronic transactions within the EU.

For more information, see eIDAS Regulation.

1.4 How to protect data

One of the most effective ways to protect data is to establish a perimeter around that data by using encryption. Best practices for protecting data suggest a shift from selective encryption (protecting only specific types of data) to pervasive encryption (encrypting all data in all states).

1.4.1 Defining the perimeter

Traditionally, the network was considered the perimeter that was protected with firewalls and VPNs. Today, attackers can breach the network perimeter; therefore, the data must be protected at the *source*.

1.4.2 Methods to protect data

The following methodologies and technologies are available to protect data:

Authentication	Requires an identity (such as a user ID) and a secret (such as a password) to log in to a system.
Access control	Establishes a set of policies to determine which users can access which data and services (***authorization***).
Encryption	Applies a cryptographic key and a cryptographic algorithm to a piece of data to prevent unauthorized disclosure of the data.

1.4.3 Encryption

Encryption is a technology that is well versed in the art of hiding sensitive information in plain sight. Encryption operations require a cryptographic key and a cryptographic algorithm.

Together, a cryptographic key and algorithm can encrypt and decrypt data. Usually, the cryptographic algorithms are public, and the data is protected by using keys for encryption and decryption.

1.4.4 Forms of encryption

The following forms of encryption are available, classified by algorithm and key type:

- Symmetric

  With symmetric encryption, the same cryptographic key is used for encryption and decryption of the data (both sender and receiver of data use the same key). Symmetric encryption can be used to encrypt large amounts of data by breaking the data into blocks and encrypting each block. Common symmetric encryption algorithms include the Advanced Encryption Standard (AES) and Data Encryption Standard (DES).

- Asymmetric

  With asymmetric encryption, the receiver's public key is used for encryption and the receiver's private key is used for decryption. Because asymmetric encryption can be used to encrypt small amounts of data only, it often is used to encrypt symmetric keys. The encrypted symmetric key is then sent to a partner so that the communications (data) between them can be encrypted with the symmetric keys. Common asymmetric encryption algorithms include Rivest Shamir Adleman (RSA) and Elliptic Curve (ECC).

Standard cryptographic algorithms (such as AES, DES, RSA, and ECC) are public and validated by mathematicians, cryptographers, and cryptanalysts. Because the algorithms are well-known and established, the security of encryption depends on the security of the cryptographic keys.

1.4.5 Cryptographic keys

Symmetric encryption and symmetric keys can be used to encrypt large amounts of data, such as z/OS data sets.

Symmetric keys are created from random numbers that are generated by random number generators (RNGs) in hardware or software. The length of the random number that is required for symmetric encryption depends on the encryption algorithm, as shown in the following examples:

- DES (single-length) requires 56 random bits
- TDES requires double-length keys (112 random bits) or triple-length keys (168 random bits)
- AES requires 128, 192, or 256 random bits

The strength of a symmetric key lies in its key length.

Brute force attacks

A brute force attack is a trial-and-error method that is used to obtain information, such as a user password or personal identification number (PIN). In a brute force attack, automated software is used to generate consecutive guesses as to the value of the wanted password or PIN.

Consider a DES 56-bit key, which has 2^{56} (72,057,594,037,927,936) combinations. This combination means that a brute force attack of a 56-bit DES key requires trying up to 72 quadrillion possible keys.

In the 1970s when 56-bit DES was invented, breaking the entire DES 56-bit key was considered infeasible with the computing power that was available considering the associated cost. With today's computing power, a DES 56-bit key can be broken in less than one day, at a low cost. For more information, see the cnet.com article, Record set in cracking 56-bit crypto.

Therefore, it is recommended that symmetric encryption applications use long key lengths to reduce the possibility of brute force attacks.

Key management

Key management is a critical aspect in any encryption strategy. Cryptographic keys feature a lifecycle that includes tasks, such as key creation, key activation, key deactivation, key archival, and key deletion. Some regulations, such as PCI-DSS[1], require that key management processes are created and well-documented. For more information about key management, see 3.6, "Key management considerations" on page 49.

1.5 Pervasive encryption for IBM Z

By encrypting data at the source, pervasive encryption creates an envelope of protection around the data. Pervasive encryption implements this comprehensive security with your ongoing operations in mind. Specifically, z/OS data set encryption does not require application changes when standard access method APIs are used and can be implemented by using policy-based controls with low overhead. Centralized, policy-based data encryption controls can significantly reduce the costs that are associated with data security and regulatory compliance because data is encrypted onetime and remains encrypted, which reduces opportunities for compromise.

IBM Z pervasive encryption is enabled through tight platform integration. Several solutions and enhancements are introduced across the IBM Z platform in the areas of hardware, software, operating system, middleware, and tooling. The components of the IBM Z platform that play a role in providing pervasive encryption are shown in Figure 1-1.

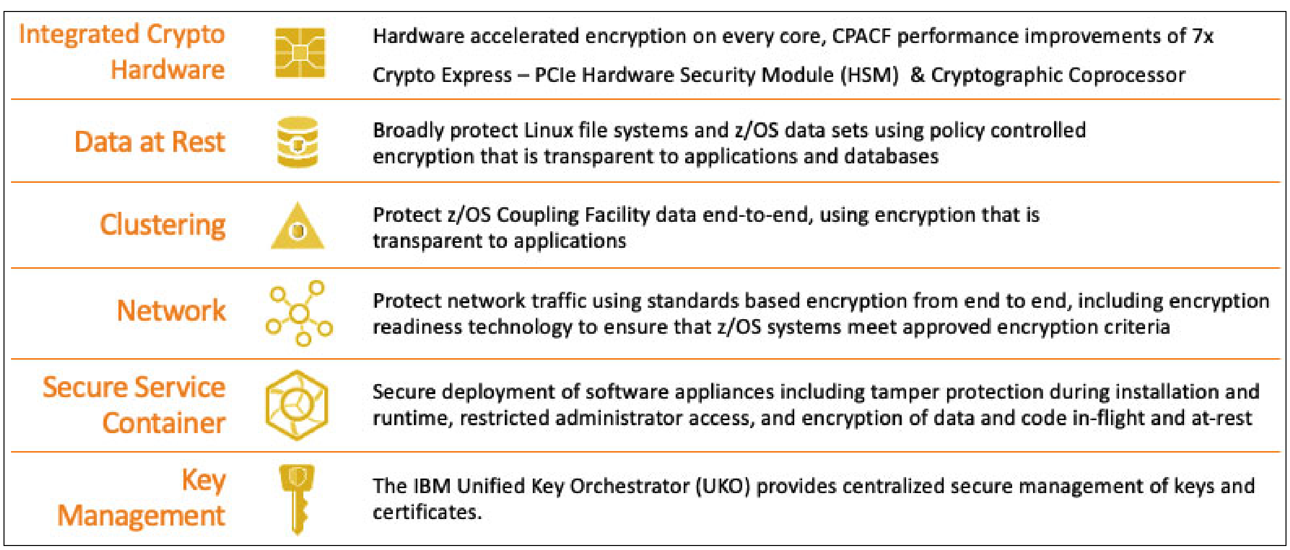


Figure 1-1 IBM Z pervasive encryption components

1 PCI DSS - Payment Card Industry - Data Security Standards - PCI Security Standards Council

1.5.1 Encrypting above and beyond compliance requirements

Complying with regulatory mandates does not necessarily mean that your data is secure. It is not uncommon to encounter regulations that were written some time ago do not incorporate current security best practices. Compliance is important, especially with the proliferation of standards, such as the Health Insurance Portability and Accountability Act (HIPAA) and the European Union (EU) General Data Protection Regulation (GDPR). For this reason, the underlying concept of pervasive encryption suggests encrypting all application and database data rather than encrypting only data required for compliance.

In addition, most organizations experience numerous audits per year. Ever evolving threats from inside and outside of an organization are causing significant security concerns, especially in the short term. Enterprises need security solutions that ensure maximum visibility into activities in their entire infrastructure, along with automated threat analysis and remediation.

The IBM Z platform provides solutions for security teams and auditors to verify up-to-date compliance statistics in near real time. Auditors can also use enhanced tooling to significantly reduce the time and effort that is required to validate compliance requirements and complete audits.

1.5.2 Encryption pyramid (data at rest)

Approaches to encryption of data at-rest can be categorized into the following levels:

- Full disk and tape encryption
- File or data set-level encryption (z/OS data set encryption is the focus of this publication)
- Database encryption
- Application encryption

Important: Disk-level and partition-level encryption typically use a single key to encrypt the entire disk or partition, with all data automatically decrypted when the system is running or an authorized user accesses it. Because of this transparent decryption, disk-level encryption is not suitable for protecting stored Primary Account Number (PAN) information on computers, laptops, servers, storage arrays, or any system that decrypts data upon user authentication.

You can choose to layer one or more of these complementary solutions, depending on your requirements.

The four encryption levels for data at-rest are mapped in Figure 1-2 on page 11 to the coverage and the security control granularity of the data.

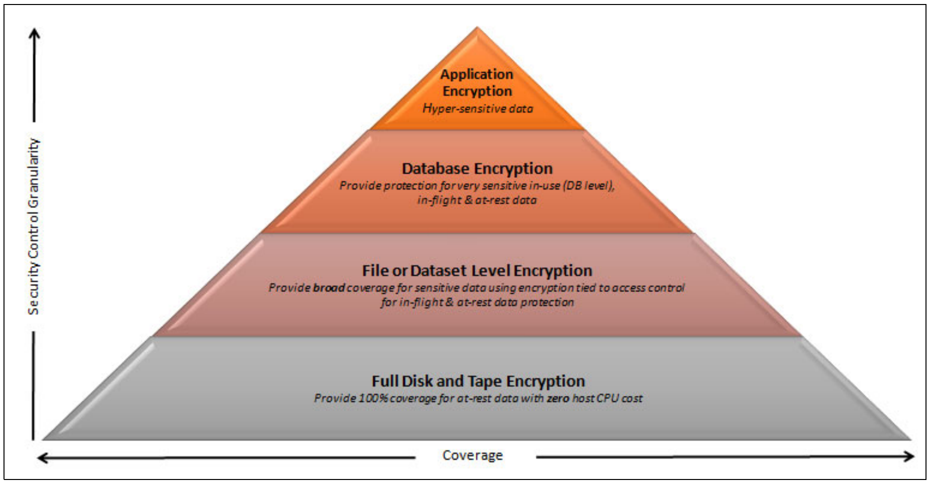


Figure 1-2 Data at rest encryption pyramid

Full disk and tape encryption

Full disk and tape encryption provides 100% coverage for *at-rest data* with limited host CPU cost. It protects against intrusion, tampering, or removal of physical infrastructure with no application overhead.

This level is "all or nothing" encryption and encrypts only data at-rest, at the storage controller subsystem level by using a single key for all encryption. Self-encrypting storage (such as IBM DS8900F, IBM FlashSystem® Storage Systems, tape, and virtual tape) solves the following security problems:

- Secure disposal of storage at the end of its lifecycle
- Tapes that are lost during shipment
- Data protection after return for repair or in case of theft

File or data set encryption

File or data set encryption provides broad coverage for sensitive data by using encryption that is tied to access control for in-flight and at-rest data protection that is enabled by policy. It is not apparent to applications and allows for *separation of duties* within your organization. This broad protection is managed by operating system components and subsystems.

Because data remains encrypted (even during operational procedures), file and data set encryption can eliminate the need to include storage administration as part of your compliance scope. The use of extra compliance controls might not be needed because the data remains encrypted when it is written.

For z/OS, data is encrypted in bulk with low overhead by using the IBM Z integrated cryptographic hardware. z/OS data set encryption supports the following data set types:

- VSAM extended format data sets (KSDS, ESDS, RRDS, VRRDS, Linear)
- Sequential extended format, basic format and large format data sets
- PDSEs (data members only)

These data set types also allow for exploitation of z/OS data set encryption by z/OS zFS, IBM Db2, IBM IMS, middleware, logs, batch, and Independent Software Vendor (ISV) solutions. Applications or middleware that use these data set types accessed with VSAM, QSAM, BSAM, or BPAM access methods can use z/OS data set encryption typically with no application changes.

However, applications or middleware that use basic or large format data sets accessed with Execute Channel Program (EXCP), a low-level I/O interface), can use z/OS data set encryption but application changes are required in this case.

To understand how to detect if an EXCP application is accessing a data set, see section “Candidates for encrypted basic and large format data sets” in z/OS DFSMS Using Data Sets. To understand how to modify EXCP applications to support encrypted data sets, refer to “Encrypting and Decrypting with the IGGENC Macro” in z/OS DFSMSdfp Advanced Services.

Database encryption

Database encryption protects data in-flight, in-use, and at-rest. It is not transparent to the application and allows for separation of duties and granular access control. Database encryption safeguards the encrypted sensitive data in logs, image copy data sets, and DASD volume backups while it uses IBM Z integrated cryptographic hardware.

Application encryption

Application encryption provides encryption and data protection that is managed by the application. It requires changes to applications to implement and maintain encryption and is highly granular in protecting data, right up to the point where it is used by the application. Applications are responsible for their own key management. This type of encryption should be used when other levels of encryption are not available or suitable.

Application-based encryption involves more maintenance overhead over time, especially when applications are modified to meet new business needs. Application programmers must know exactly which data should be encrypted. It is easier to encrypt all data seamlessly at the point that it is written, without having to rely on the programmers to determine exactly what data should be encrypted.

1.5.3 Managing the pervasive encryption environment

Managing the pervasive encryption environment is supported by several IBM Security® solutions. The essential capabilities that are needed are shown in Figure 1-3.

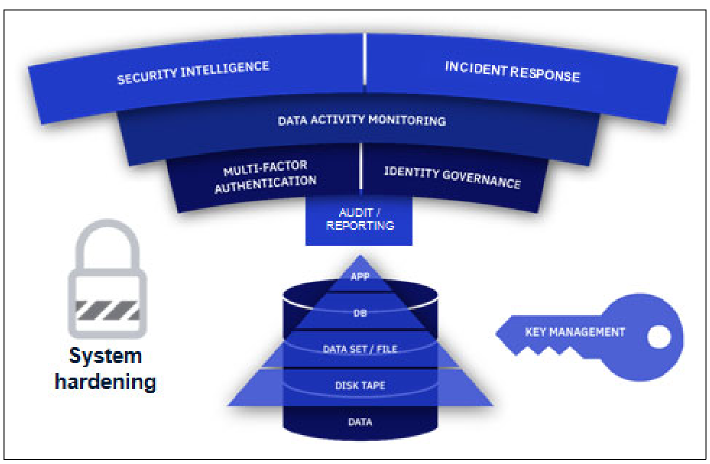


Figure 1-3 IBM Security solutions with essential capabilities

For more information about the IBM Z security solutions and their capabilities, see Chapter 2, “Identifying components and release levels” on page 25.

For more information about getting started with IBM Z pervasive encryption, see IBM Z Pervasive Encryption content solution.

1.6 Understanding z/OS data set encryption

z/OS data set encryption is one such solution introduced with IBM Z pervasive encryption. z/OS data set encryption is designed to offer high throughput and low-cost encryption. It supports encrypting data in bulk by using DFSMS access methods instead of encrypting a single field or row at a time. In addition, encrypting data at the data set level provides more data security and other capabilities than what can be provided through just enabling hardware-level encryption.

z/OS data set encryption is intended to be more accessible to the organization than many other forms of encryption. For example, usually it is not apparent to the application and requires no changes to application code. By using z/OS data set encryption, data can be encrypted at course scale without the need to perform data identification and classification first.

z/OS data set encryption helps you protect data so that you can identify new data sets or groups of data sets to be encrypted. You can also use z/OS data set encryption to identify those users who are allowed to access the data in the clear. The system encrypts the data within the host as it is written and decrypts it only for those users you have identified. When standard access method APIs (VSAM, QSAM, BSAM, or BPAM) are used to access the data, this capability typically does not require application changes.

Basic and large-format data sets can be accessed by applications by using standard access method APIs, as well as by applications using EXCP access. Applications that use EXCP to access encrypted basic and large format data sets cannot transparently access the encrypted data. In this case, they require modifications to successfully open the data set and perform encryption and decryption of the data.

> **Important:** Before converting any basic or large format data sets to encrypted basic or large format data sets:
>
> 1. Research is required to determine whether the data sets are candidates for basic and large format encryption.
> 2. Restrictions must be evaluated.
> 3. Unless the application that uses EXCP is modified to support encrypted basic and large format data sets. Otherwise, applications might fail.
>
> For information on how to identify candidates for basic and large format encryption, refer to section “Candidates for encrypted basic and large format data sets” in publication *IBM z/OS DFSMS Using Data Sets*, SC23-6855.

z/OS data set encryption is supported only for certain data set types. More data set types might follow.

The following additional design benefits are built into z/OS data set encryption:

- Offers a higher level of protection along with the high throughput of encryption by using protected keys and a crypto coprocessor. As a result, sensitive key material is not visible in clear form at any time.
- Protects data in a way that is aligned with your current security access control mechanisms, which offers a more straightforward configuration experience.
- Performs efficiently at speed by use of the integrated IBM Z crypto hardware and software stack.
- Enables encryption without requiring application or database changes.

- Supports encrypted data throughout its journey. For example, with z/OS data set encryption, any data that is replicated by using Peer-to-Peer Remote Copy (PPRC) or Extended Remote Copy (XRC), backed up, or migrated, remains encrypted.
- Provides cryptographic separation from other environments. Encryption keys can be configured so that they are owned and managed by a logical organizational environment (for example, production versus test).
- Offers System Management Facility (SMF) records, subsystem logs, and system interfaces to help simplify compliance and audit efforts.

There are a number of benefits provided by z/OS data set encryption that make it a clear choice as a method to protect data in-flight and at-rest. The following are a few of the benefits.

- Data set-level granularity

  You have control over which data sets should be encrypted and which encryption keys should be used with those data sets.

  You identify the data sets to be encrypted by specifying a key label during data set create. The key label can be supplied by a policy, such as through the IBM RACF® data set profile or SMS data class. By identifying the key label, you have control over how many unique encryption keys are to be used within your organization, and which data sets share the same encryption key. For example, you might choose to have a unique encryption key per application, where all data sets used by that specific application share that application's encryption key.

- Separation of duties

  You can limit who can view sensitive data by controlling who has access to the data in the clear.

  Having access to data in the clear requires authority to the data set's key label. This allows you to maintain a separation of duties and thus, potentially remove certain roles from compliance scope. For example, you can choose to allow only the data owner to access the data in the clear, and the storage administrator can manage only the data set as a container of the data without access to the data itself.

- Encryption in flight and at rest

  The data is encrypted in the host and remains encrypted in flight as it's transferred over the SAN to be stored in the storage subsystem, where it's encrypted at rest. If hardware encryption is in effect in the storage subsystem, the data is doubly encrypted, which is not a problem. The data also remains encrypted during backup, migration, and replication.

- Audit simplification

  Auditors can use enhanced tooling to validate compliance requirements, reducing the time and effort needed to perform an audit. System services can be used to identify encryption attributes that are displayed along with data set metadata. This allows enhanced tooling to incorporate encryption information at the data set level, such as which data sets are encrypted and which key labels are in use.

All of the benefits of z/OS data set encryption rest on establishing a robust key management strategy, which is essential to governing and safeguarding the encryption keys that protect your data. Therefore, you must ensure that the encryption keys are available whenever and wherever an encrypted data set is used. As such, it is important to understand how z/OS data set encryption works with the hardware and software components in the IBM Z cryptographic system.

1.6.1 z/OS data set encryption is quantum-safe

As organizations prepare for the quantum computing era, it is important to understand the resilience of existing cryptographic algorithms. The Advanced Encryption Standard (AES), used in z/OS data set encryption, is considered quantum-safe when implemented with 256-bit keys. Although quantum algorithms like Grover's algorithm can theoretically reduce the effective security of symmetric keys, AES-256 remains robust, offering a security level equivalent to 128 bits even in a post-quantum context. Therefore, using AES-256 provides a strong foundation for long-term data protection, including against future quantum threats.

For more information on IBM Z quantum-safe capabilities, see *Transitioning to Quantum-Safe Cryptography on IBM Z*.

1.6.2 Challenges and use cases

This section describes a few client use cases that benefited by using z/OS data set encryption. Determine whether the following use cases resonate for your organization:

- Reducing CPU cost

  Challenge: In an attempt to limit encryption, it was decided to protect only data that was deemed to be sensitive by using field level encryption. After modifying applications to apply field level encryption, the client observed significant CPU overhead after an initial benchmark test.

  Solution: Using z/OS data set encryption, the client was able to encrypt all data in the data sets without requiring application changes, and more importantly, using the IBM Z, benchmark tests showed minimal processor overhead.

- Avoiding application changes

  Challenge: Client calculated significant cost to protect data due to the requirement to change thousands of applications to add field level encryption.

  Solution: Using z/OS data set encryption, with approval by the CISO team, the client was able to save millions of dollars to protect data sets without the need for application changes for a majority of the applications. Application-level encryption was used only when needed on an exception basis.

- Achieving cryptographic isolation

  Challenge: Client was using full disk encryption to protect data at rest. However, several audit findings were raised related to lack of isolation of data between production, development, and test environments due to their sharing the same physical resources.

  Solution: Using z/OS data set encryption and taking advantage of Crypto Express domains in addition to using hardware encryption, the client was able to show separation of data across production, development, and test and resolve the audit findings.

- Reducing risk

  Challenge: Storage administrators have access to massive amounts of data and thus pose significant risk to business.

  Solution: Using z/OS data set encryption, storage administrators might be prevented from accessing data in the clear by not having authority to the key label but can still manage the data set in its entirety. Not having access to data in the clear might reduce the risk posed by some storage administrators.

1.6.3 IBM Z cryptographic system

z/OS data set encryption uses the integrated cryptographic system that is available on the IBM Z platform. In a cryptographic system, a cryptographic key and a cryptographic algorithm are required. The encryption algorithms are public, and the encryption keys are kept secret. The secure management of keys (or key material) is vital to the protection of data in a cryptographic system.

z/OS data set encryption relies on encryption keys that are in a keystore (Cryptographic Key Data Set [CKDS]) when data sets are encrypted or decrypted. Those encryption keys are managed by z/OS Integrated Cryptographic Services Facility (ICSF) or by using the IBM Unified Key Orchestrator (UKO):

- ICSF is a component of z/OS that provides APIs and utilities for creating and managing cryptographic keys, and performing crypto operations in software or hardware.
- IBM Unified Key Orchestrator (UKO) is available in three options: UKO for z/OS, UKO for Containers, and EKMF Workstation. UKO for z/OS and UKO for Containers focus on managing keys for pervasive encryption and cloud environments, and EKMF Workstation extends capabilities with advanced key and certificate management, along with PCI-level security.

Each encryption key includes a corresponding key label. The key label is used as a handle to locate the encryption key within the CKDS.

The IBM Z platform offers cryptographic engines that provide high-speed cryptographic operations. The following cryptographic engines are used with z/OS data set encryption:

- Central Processor Assist for Cryptographic Function (CPACF)

  Cryptographic function that is provided through a set of instructions that are available in hardware on every processor unit. The use of CPACF is enabled by ordering the Feature Code 3863 (CPACF DES/TDES Enablement) for IBM Z systems.

- Crypto Express in CCA mode

  Cryptographic function that is provided through high-security, tamper-responding hardware security modules[2] (HSMs).

z/OS data set encryption processing uses different types of encryption keys and features the following key terminology:

- Data-encrypting key: An encryption key that is used to encrypt and decrypt data. z/OS data set encryption supports only data-encrypting keys that are using the AES algorithm with a 256-bit key length. Both DATA and CIPHER key types are supported.

  > **Note:** Using CIPHER keys where possible is recommended due to the greater granularity in setting key usage and key management.

- DATA key: A type of data-encrypting key.
- CIPHER key: A type of data-encrypting key. CIPHER keys can be limited to more specific uses than DATA keys, preventing their use outside of data set encryption. CIPHER keys

[2] An HSM is a physical computing device that safeguards and manages digital keys for strong authentication and provides cryptographic processing.

require Crypto Express and ICSF. The key must allow encryption and decryption, any cipher mode, and export to CPACF protected key format.

> **Note:** All production, development, test, QA, and disaster recovery systems accessing z/OS data sets encrypted with AES CIPHER keys must meet the minimum system requirements. For details, see IBM Z Pervasive Encryption content solution.

- Key-encrypting key: A key that encrypts or wraps other keys.
- Master key: A special key-encrypting key (KEK) that is in a tamper-responding, Crypto Express adapter only and sits at the top level of a KEK hierarchy.
- CPACF wrapping key: A special key-encrypting key that is generated at LPAR activation and is in the Hardware System Area, which is inaccessible to applications and the operating system. It is used to create protected keys.
- Secure key: A data-encrypting key that is encrypted by a master key and never appears in clear text that is outside of a secure environment, such as a tamper-responding Hardware Security Module (HSM), or IBM Z firmware. Secure keys can be stored in an ICSF key data set or returned to the ICSF caller.
- Clear key: A data-encrypting key that is not encrypted by any other key. The key material is in clear text. Clear keys can be stored in an ICSF key data set or returned to the ICSF caller at key creation.

> **Note:** Clear keys that are stored in an ICSF key data set are not returned by using Key Record Read functions.

- Protected key: A data-encrypting key that is encrypted by a CPACF wrapping key and used within the IBM Z platform. Although protected keys are cached in ICSF, they are not persistently stored in an ICSF key data set. Protected keys can be returned to authorized ICSF callers, such as DFSMS and Db2.
- Operational key: A key that is not a master key, such as a data-encrypting key (which can be clear, secure, or protected).

Generating data encryption keys as secure keys and storing them in the CKDS is the most secure method of protecting key material on the IBM Z platform. This process requires the use of Crypto Express adapters in CCA mode.

When processing encrypted data sets, DFSMS access methods use only protected keys to ensure that the sensitive data-encrypting keys are never available in clear text within the operating system.

For more information about how the IBM Z platform manages secure keys and protected keys, see 3.6.6, “Using protected keys for high-speed encryption” on page 51.

1.7 How z/OS data set encryption works

z/OS data set encryption allows you to identify new data sets or groups of data sets to be encrypted. This process is done by using SAF controls or RACF and SMS policies.

You can specify key labels to identify encryption keys (such as secure keys) that are in the CKDS. When an encrypted data set is created, the key label is stored as an attribute of the data set in the catalog.

Authorization to view the contents of a data set is based on access to the key label that is associated with the data set. The encryption key that is associated with that key label is used by DFSMS to start CPACF to encrypt and decrypt the data.

Encrypted data sets must be SMS-managed and thus, cataloged. z/OS data set encryption supports the encryption of the following data set types:

- Sequential extended-format data sets, which are accessed through BSAM and QSAM
- Sequential basic format and large format data sets, which are accessed through BSAM, QSAM and EXCP. (Note that EXCP applications require changes to support encrypted data sets).
- VSAM extended-format data sets (KSDS, ESDS, RRDS, VRRDS, and LDS), which are accessed through base VSAM and VSAM RLS. VSAM extended-format data sets can also be accessed by using licensed Media Manager (MM) interfaces. Applications that use MM interfaces require changes to access encrypted data sets. For more information, contact your IBM representative.
- PDSEs (data members only), which are accessed through BPAM, BSAM and QSAM.

A data set is defined as encrypted when a key label is supplied on allocation of a new data set of a supported data set type. The default supported data set types for data set encryption are sequential extended format and VSAM extended format data sets.

To identify that basic and large format data sets are to be viewed as a supported data set type on the system, you must fully define the following FACILITY class resource as a discrete profile:

```
STGADMIN.SMS.ALLOW.DATASET.SEQ.ENCRYPT
```

In order to take effect, the resource needs only to be defined. The system does not require you to provide any authorization for this resource.

> **Note:** Before allowing basic and large format encryption, research must be performed to determine whether any EXCP applications will access those data sets. If so, you must determine whether those EXCP applications have been or plan to be modified in support of z/OS data set encryption. Any EXCP application that opens an encrypted basic or large format data set before it has been modified will receive an error attempting to open the data set. If you find that no EXCP applications access the data sets, consider converting the data set to sequential extended format encrypted data set. One benefit is that sequential extended format data sets support access method compression, although basic and large format data sets do not.
>
> To determine whether a data set is eligible to be an encrypted basic or large format data set, refer to section “Candidates for encrypted basic and large format data sets” in publication *z/OS DFSMS Using Data Sets*, SC23-6855. To understand how to modify EXCP applications to support encrypted basic and large format data sets, refer to section “Encrypting and Decrypting with the IGGENC Macro” in publication z/OS DFSMSdfp Advanced Services.

To identify that PDSEs are to be viewed as a supported data set type on the system, you must fully define the following FACILITY class resource as a discrete profile:

```
STGADMIN.SMS.ALLOW.PDSE.ENCRYPT
```

To take effect, the resource only needs to be defined. The system does not require you to provide any authorization for this resource.

A data set is defined as encrypted when a key label is supplied on allocation of a new data set of a supported data set type for data set encryption. You can supply a key label by using keywords in any of the following source formats on allocation of a new data set of a supported data set type for data set encryption (by using the following order of precedence):

1. Data Facility Product (DFP) segment in the RACF data set profile. Data Facility Product (DFSMSdfp) tracks all data and programs that are managed within z/OS and provides data access for z/OS applications.
2. JCL, dynamic allocation, TSO Allocate, IDCAMS DEFINE.
3. SMS data class.

An example of how a key label is used when an encrypted data set is created is shown in Figure 1-4. This example uses a key label that is specified in the DFP segment in the RACF data set profile.

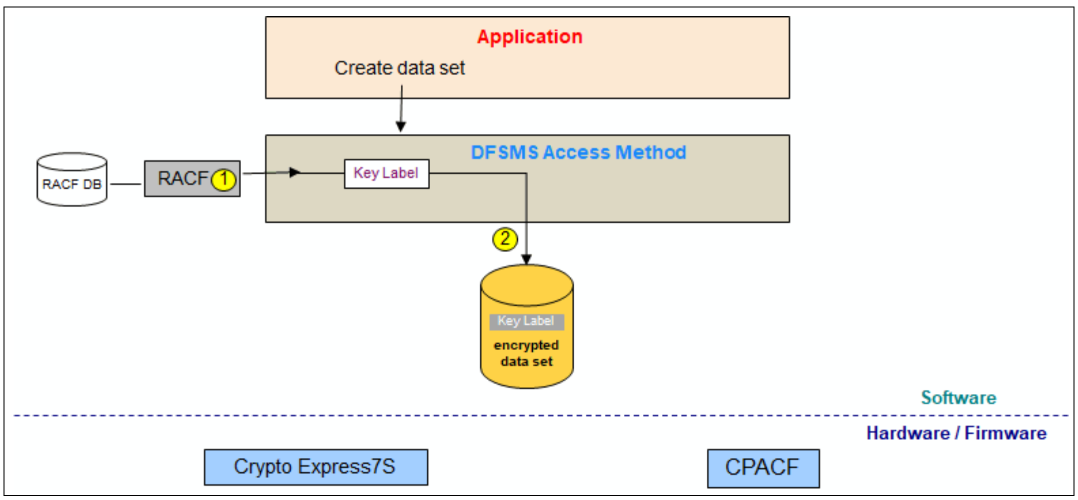


Figure 1-4 Creating an encrypted data set

Creating data sets include the following steps:

1. DFSMS calls RACF to determine whether a key label is supplied in the DFP segment in the RACF data set profile.
2. DFSMS validates the key label syntax. Then, it creates an encryption cell that is associated with the data set and stores the key label in the encryption cell.

Note: Neither Crypto Express nor CPACF are called during data set create processing.

An example of how key labels and keys are used with z/OS data set encryption is shown in Figure 1-5 on page 20. This example uses secure keys that are stored in the CKDS, and involves input processing of an encrypted data set.

Open prepares (encrypted) data to be read. When the Open process is completed, the key for decrypting the data is available for the Read/Get operation to bring data to the application storage (real memory). Open is performed once. Read/Get is repetitive (as per application request).

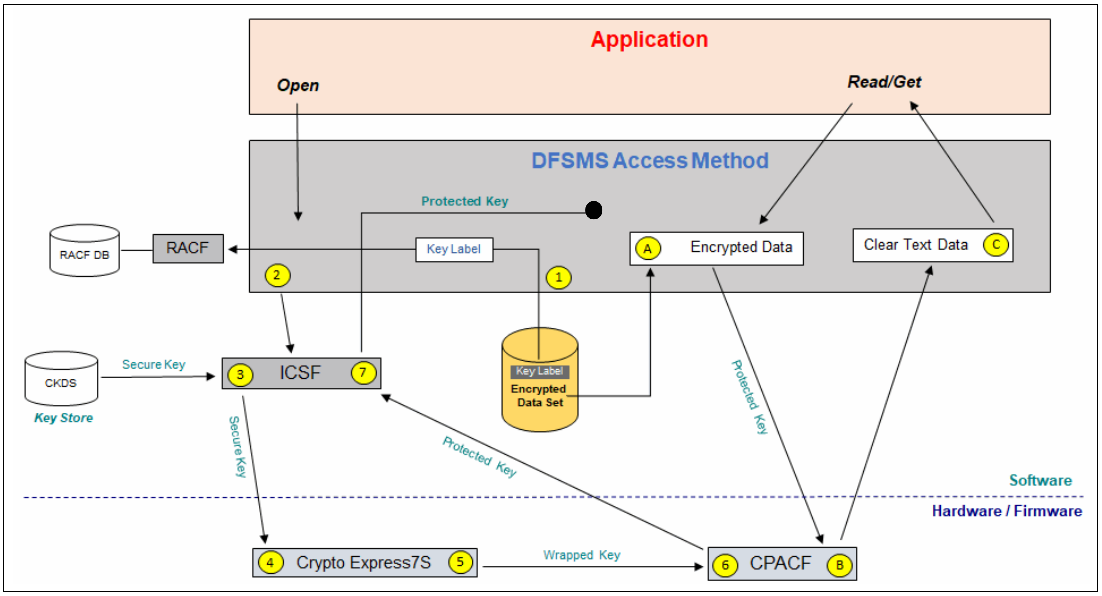


Figure 1-5 Encryption key process for decrypting an encrypted data set

Open performs the following process, as shown on the left side of Figure 1-5:

1. DFSMS receives the key label that is associated with data set from the catalog and calls RACF to verify the user's access to the key label.
2. DFSMS calls ICSF with the key label.
3. ICSF obtains the secure key from CKDS and calls the Crypto Express to unwrap the key.
4. With assistance from firmware, Crypto Express decrypts the secure key and rewraps with a transport key.
5. The wrapped key is sent to CPACF. With assistance from IBM Z firmware, CPACF unwraps the wrapped key with the transport key to expose the data-encrypting key.
6. The data-encrypting key is wrapped with the CPACF wrapping key to create the protected key.
7. The protected key is sent to ICSF, where it is cached in protected memory for future callers. ICSF sends the protected key to DFSMS to encrypt and decrypt data.

Read/Get performs the following steps, as shown on the right side of Figure 1-5:

Step A DFSMS reads encrypted data from data set and starts CPACF, which passes the protected key.

Step B CPACF decrypts data by using the protected key.

Step C Decrypted data is sent as clear text to the application through DFSMS.

1.8 Administrator's perspective of z/OS data set encryption

When data set encryption is implemented, a thorough consideration of the different roles and responsibilities regarding the handling of data is necessary to align with your organization's security strategy. Communication between the roles is also essential for a successful implementation.

One of the many benefits of z/OS data set encryption is *separation of duties* between data owners and administrators. Creating a perimeter around the data means that you can create an implementation that limits who can access the sensitive data.

At a minimum, administrators that are involved in your z/OS data set encryption implementation must have security, storage, and cryptographic roles. For more information about the necessary responsibilities of administrators, see 3.2, “Distinguishing roles and responsibilities” on page 37.

1.8.1 Security administrator

The security administrator can be responsible for identifying data sets that must be encrypted and for defining the security policies for encrypted data sets as they are being created. The security administrator might also be responsible for assigning appropriate authorization to key labels so that only data owners can access their data in clear text.

You can design your implementation in such a way that the security administrator has full control over z/OS data set encryption. This level of responsibility allows the security administrator to maintain separate duties that are required to limit who can access the sensitive data.

For example, the security administrator can perform the following tasks:

- Define or alter DATASET profiles to supply a key label for data set encryption. This process allows the security administrator to control which data sets are created as encrypted, and which key labels are to be used.
- Define a security profile through the FACILITY class so that key labels can be supplied through DATASET profiles only. This process prevents users (other than the security administrator) from deciding to encrypt data sets that are outside of the control of the security administrator.
- Assign users and groups access to CSFKEYS profiles for the key labels that permit or deny users to view sensitive data in data sets. This process allows the security administrator to limit the users who can open an encrypted data set to only those users who need access to the data in clear text, such as the data owners.

1.8.2 Storage administrator

A storage administrator might have the authority to encrypt data sets. If permitted by the security administrator, storage administrators (and other users) can encrypt data sets by supplying key labels in JCL or SMS data classes. Even if storage administrators cannot encrypt data sets, they still play a role in pervasive encryption.

The storage administrator must be aware of data set naming conventions and data set types and concern themselves with the following issues:

- Do data sets exist that should be converted to extended-format or to another data set type supported by data set encryption?
- Do data sets exist that cannot be converted to extended-format or to another data set type supported by data set encryption?
- Do specific data set types exist that are not supported?
- Do those unsupported data sets contain sensitive information?

The storage administrator works closely with the security administrator and ICSF administrator to ensure that all necessary data sets are protected.

The storage administrator might be responsible for data migration, backup, and replication. They can perform these functions for encrypted data sets without requiring authority to the key label because these functions process the data in the encrypted form. However, there might be other functions that storage administrators in your organization can perform that require them to have authority to the key label. Examples include IEBGENER or XMIT. In such cases, it is beneficial to limit the storage administrators that requires access to the key labels.

1.8.3 Cryptographic administrator

The cryptographic (or ICSF) administrator governs the cryptographic system, which represents the foundation for z/OS data set encryption. Management of the operational keys is the responsibility of key managers.

Responsibilities for cryptographic administrator include generating and managing master keys and ensuring that the cryptographic system is operational. This role can also involve monitoring the use of the Crypto Express coprocessor and CPACF.

The ICSF administrator works closely with the security administrators and key managers to ensure that keys are available for z/OS data set encryption.

1.8.4 Key manager

The key managers govern the key management process for the operational keys that are used for z/OS data set encryption. The responsibilities of the key managers include generating and managing keys and ensuring that keys are available when and where they are needed.

The key managers rely on ICSF administrators to provide a working cryptographic environment and work closely with security administrators to define key label naming conventions that ensure an easy mapping of DATASET profile resources to key labels.

1.9 Zero trust and IBM Z encryption strategy

The zero trust security model is based on the principle *Never trust, always verify*. It assumes no implicit trust for users, devices, or networks, whether inside or outside the traditional security perimeter. This model aligns well with IBM Z security capabilities, particularly z/OS data set encryption and pervasive encryption.

1.9.1 Core principles of zero trust

IBM Z supports the core tenets of zero trust, which includes the following integrated security features:

- Verify explicitly
  - Enforce strong identity verification using RACF and Multi-Factor Authentication (MFA)
  - Validate encryption key usage through ICSF and SMF auditing
- Use least privilege access
  - Implement granular RACF roles to separate key management from data access
  - Use ICSF callable services with controlled permissions

- Assume breach
  - Encrypt all data at rest and in transit by using pervasive encryption
  - Regularly rotate keys and enforce cryptoperiods to limit exposure

1.9.2 How z/OS data set encryption enables zero trust

z/OS data set encryption provides a policy-driven, transparent encryption mechanism that supports zero trust with the following features:

- Policy-based encryption: Automatically encrypts eligible data sets without relying on application-level trust.
- Separation of duties: Ensures that key administrators cannot access encrypted data, and data owners cannot manage keys.
- Continuous monitoring: SMF records and UKO audit logs provide real-time visibility into key usage and access patterns.
- Hardware root of trust: IBM Z Crypto Express features and CPACF ensure secure key storage and cryptographic operations.

1.9.3 Integration steps for zero trust on IBM Z

To implement zero trust effectively on IBM Z, consider the following steps:

1. Enable pervasive encryption for all sensitive data sets.
2. Use centralized key management (like UKO) with strict access controls.
3. Implement multi-factor authentication for administrative roles.
4. Automate audit log collection and integrate with SIEM tools.
5. Regularly test and validate encryption policies against compliance frameworks.

For more information on zero trust, see NIST Special Publication 800-207, Zero Trust Architecture.

2

Identifying components and release levels

This chapter describes the IBM Z hardware and software components that are required or optional for z/OS data set encryption. It includes the following topics:

2.1 Starting a z/OS data set encryption implementation

z/OS data set encryption uses tight integration that spans the capabilities of IBM Z hardware, firmware, and software. Although z/OS data set encryption delivers significant value, IBM designed it with extensibility in mind to further strengthen its capabilities. Updates continue to align with emerging industry needs and evolving client requirements.

Implementations of data set encryption on earlier levels of the IBM Z platform can provide a base for establishing appropriate processes and procedures before full-scale production. However, cryptographic technology with the latest IBM Z generation can drastically reduce computational overhead compared to previous generations.

Also, every new release of z/OS continues to provide enhancements for various aspects of data security beyond encryption, such as identity management, access control, auditability, monitoring, and reporting.

The enhancements that are provided by the latest levels of IBM Z hardware, firmware, and software ensure the best scalability, performance, and enhanced system and security management capabilities.

At the time of this publication, the preferred and most optimized IBM Z platform for data set encryption is the combination of the IBM z17™ (including CPACF enablement and Crypto Express8S) with z/OS 3.2 and ICSF HCR77F0.

For the minimum supported levels of the IBM Z platform components that are needed to implement z/OS data set encryption, see: z/OS data set encryption.

2.2 Required and optional hardware features

In principle, cryptographic algorithms can run without extra hardware. However, cryptographic algorithms are computationally intense, and the secure handling of keys requires special hardware protection. Therefore, z/OS data set encryption takes advantage of IBM Z cryptographic hardware features to meet the requirements for bulk cryptographic processing.

This section introduces the following hardware components to consider for use with z/OS data set encryption:

- Central Processor Assist for Cryptographic Function (CPACF)
- Crypto Express adapters
- Trusted Key Entry (TKE) workstation
- IBM Unified Key Orchestrator (UKO)

2.2.1 Central Processor Assist for Cryptographic Function

The no-cost CP Assist for Cryptographic Function enablement (FC 3863) is required on the IBM Z hardware platform to support z/OS data set encryption.

CPACF is a set of instructions that is available on every processor unit that accelerates encryption. CPACF is designed to facilitate the privacy of cryptographic key material when used for data encryption through a key wrapping implementation. It ensures that key material is not visible to applications or operating systems during encryption operations.

2.2.2 Crypto Express adapters

Crypto Express adapters are required to generate the secure keys that are stored in the CKDS. They are also required to generate protected keys from secure keys for z/OS data set encryption.

Crypto Express adapters are tamper-responding hardware security modules (HSM). An HSM is a physical computing device that safeguards and manages digital keys for strong authentication and provides cryptographic processing. A Crypto Express adapter provides high-security, high-throughput cryptographic functions, which adds a layer of protection for the storage and use of a master key.

> **Note:** All installed crypto coprocessors cards must be loaded with the same level of code. Otherwise, unpredictable results can occur.

The following Crypto Express configuration options are available:

- Accelerator
- CCA coprocessor
- EP11 coprocessor

> **Note:** For z/OS data set encryption, the Crypto Express adapters must be configured as Common Cryptographic Architecture (CCA) coprocessors.

CCA is an architecture and a set of application programming interfaces (APIs) that support cryptographic operations and key management.

To access and use the Crypto Express adapters, applications must use APIs and panel utilities that are provided by the operating system. For z/OS, the Integrated Cryptographic Service Facility (ICSF) provides these APIs and manages access to the hardware cryptographic features.

Minimum number of adapters

A minimum of two Crypto Express adapters are recommended . If one adapter must be taken offline (for example, MCL upgrade), the second adapter (loaded with the same AES master key [MK] as the first) can handle the same number of requests. In this case, the usage threshold for each Crypto Express adapter should not exceed 50%.

2.2.3 Trusted Key Entry workstation

The Trusted Key Entry (TKE) workstation is an optional hardware feature that can be used for z/OS data set encryption.

TKE securely manages multiple cryptographic coprocessors (including master keys) on various generations of IBM Z and other platforms, from a single point of control. Manually managing master keys across a complex installation can require significant systems management effort, introduce audit and secrecy complexity, and can be error prone at critical master key entry stages.

An IBM Z system supports up to 16 Crypto Express features, each of which can include 85 domains and a duplicate environment for disaster recovery. The IBM Z platform uses the concept of a cryptographic domain to virtualize the physical coprocessor of the Crypto Express adapter. A Crypto Express coprocessor can be shared by multiple logical partitions (LPARs) and different operating systems. IBM Z firmware enforces domain usage. The Crypto

Express coprocessor manages assignment of master keys to cryptographic domains. Cryptographic key material for one domain is not usable by another domain with a distinct master key. In this environment, master key management on the coprocessors is not a trivial endeavor.

The TKE consists of a secure, hardware-based workstation and a Crypto Express adapter that is fully compatible with IBM Z Crypto Express adapters. It uses smartcard-controlled key management to ensure secure, fast, and accurate deployment of new cryptographic material across production, test, and disaster recovery (DR) environments.

For more information about other TKE workstation versions that can be used with different generations of the IBM Z platform, see Trusted Key Entry support (TKE).

2.2.4 IBM Unified Key Orchestrator

IBM Unified Key Orchestrator (UKO) is a flexible and highly secure key management system for the enterprise. It provides centralized key management on IBM Z and distributed platforms for streamlined, efficient, and secure key and certificate management operations. UKO serves as the foundation on which remote cryptographic solutions and analytics for the cryptographic infrastructure can be provided.

The UKO family consists of UKO for z/OS, UKO for Containers, and the EKMF Workstation:

- UKO for z/OS

  Released as a product in April 2020, it was developed at the IBM Crypto Competence Center in Copenhagen, Denmark with the EKMF Workstation. Both UKO for z/OS and EKMF Workstation use the same UKO Agent, which communicates with ICSF through APIs.

  UKO for z/OS runs in a z/OS LPAR (image) and is accessible through a web browser. Currently, UKO for z/OS supports the generation and management of keys for IBM Z, including those used for data set encryption, and sets up keys for cloud deployments.

  > **Note:** UKO for z/OS is an orderable software product.

- UKO for Containers

  UKO for Containers provides the same functionality as UKO for z/OS but is deployed as containers in Red Hat OpenShift Container Platform on IBM LinuxONE or IBM Z (IFLs on IBM Z). It uses the Crypto Express capabilities provided by the IBM LinuxONE platform to create keys and can manage keys on z/OS by using UKO agents on z/OS.

  Statements made in this IBM Redbooks publication about UKO for z/OS generally also apply to UKO for Containers.

  > **Note:** UKO for Containers is an orderable software product.

- EKMF Workstation

  IBM implements EKMF Workstation as a self-contained hardware appliance (including a dedicated workstation, Linux operating system, and an embedded crypto card). It is feature-rich, supporting many key types, with authentication performed by using smart cards with PINs. With the security controls, EKMF Workstation can achieve a higher security rating and administer more complex key types.

Note: EKMF Workstation is a service-based offering.

The EKMF Workstation, UKO for z/OS, and UKO for Containers all provide the following key management services:

- Generate operational keys
- Facilitate backup and recovery of key material
- Provide monitoring, auditing, and planning capabilities

For more information, see Unified Key Orchestrator for IBM z/OS.

For additional information, see Chapter 10, “IBM Unified Key Orchestrator” on page 223.

2.3 Required and optional software features

This section introduces the required and optional software components to consider with z/OS data set encryption as listed in Table 2-1.

Table 2-1 Software components for z/OS data set encryption

Required	Optional
▶ IBM z/OS DFSMS ▶ IBM z/OS Integrated Cryptographic Service Facility ▶ IBM System Authorization Facility ▶ IBM Resource Access Control Facility for z/OS[a]	▶ IBM Multi-Factor Authentication for z/OS ▶ IBM Security zSecure Suite ▶ IBM zBNA[b]

a. An equivalent Enterprise Security Manager can be used, such as CA ACF2 or Top Secret for z/OS.

b. Use this tool to help estimate the potential performance effects of implementing z/OS data set encryption.

2.3.1 IBM z/OS DFSMS

The z/OS Data Facility Storage Management Subsystem (DFSMS) is required to implement z/OS data set encryption.

For z/OS data set encryption, DFSMS supports the creation of and access to encrypted data sets, in addition to backup, migration, and replication of encrypted data sets. Applications that use the access method APIs can transparently access encrypted data if the user has the appropriate authorization to access the key label.

Note: Applications that use EXCP to access encrypted basic and large format data sets cannot transparently access the encrypted data. In this case, they require modifications to successfully open the data set and perform encryption and decryption of the data.

Before converting any basic or large format data sets to encrypted basic or large format data sets, research is required to determine whether the data sets are candidates for basic and large format encryption. Restrictions must be evaluated. In addition, if the data sets are accessed by any application (including vendor software), using EXCP, they should not be converted to encryption unless the application is modified to support encrypted basic and large format data sets. Otherwise, application failures can result. For information on how to identify candidates for basic and large format encryption, refer to section "*Candidates for encrypted basic and large format data sets*" in publication z/OS DFSMS Using Data Sets. To understand how to modify EXCP applications to support encrypted basic and large format data sets, refer to section "Encrypting and Decrypting with the IGGENC Macro" in publication z/OS DFSMSdfp Advanced Services. If you find that no EXCP applications access the data sets, consider converting the data set to sequential extended format encrypted data set. One benefit is that sequential extended format data sets support access method compression, and basic and large format data sets do not.

For more information about DFSMS, see Understanding the DFSMS Environment.

For an overview of z/OS data set encryption, refer to section 'Data set encryption' in publication *IBM z/OS DFSMS Using Data Sets*, SC23-6855.

Tip: For more information about identifying fixes for z/OS data set encryption, see IBM Fix Category Values and Descriptions.

2.3.2 IBM z/OS Integrated Cryptographic Service Facility

The z/OS Integrated Cryptographic Service Facility (ICSF) is required to implement z/OS data set encryption.

ICSF is a component of z/OS that provides APIs and utilities for creating and managing cryptographic keys, and performing crypto operations in software or hardware. The symmetric keys that are used for data set encryption are stored in the Cryptographic key data set (CKDS).

For z/OS data set encryption, DFSMS calls ICSF services to retrieve a protected key that is used to encrypt/decrypt data. Therefore, ICSF must be available to access any encrypted data set. For more information, see 3.5.2, "Starting ICSF early in the IPL process" on page 45.

ICSF calls the System Authorization Facility (SAF) interface to verify user authorization to APIs and Time-Sharing Option (TSO) Interactive System Productivity Facility (ISPF) panels. SAF policies (such as RACF profiles) control access to ICSF keys and services.

For more information about the use of SAF policies to protect ICSF keys and services, see z/OS Cryptographic Services ICSF Administrator's Guide.

ICSF supports features for viewing and managing keys, and provides more audit information. For example, the CKDS KEYS utility, which is an ISPF-based browser for the CKDS.

The browser shows the state of the record in CKDS (for example, Active or Archived) and options to display the key attributes and metadata. ICSF also includes enhanced SMF formatting, with which you monitor cryptographic usage statistics and reveal compliance warning events.

2.3.3 IBM System Authorization Facility

IBM System Authorization Facility (SAF) is an interface that is defined by z/OS that enables programs to call system authorization services to check access to resources, such as data sets and z/OS commands.

SAF processes security authorization requests directly or works with RACF, or other third-party security products, such as CA Top Secret or ACF2 for z/OS.

For more information about z/OS data set encryption support with Top Secret and ACF2, see Pervasive Encryption Compatibility Matrix.

2.3.4 IBM Resource Access Control Facility for z/OS

The Resource Access Control Facility (RACF) is an External Security Manager (ESM) and a component of the Security Server for z/OS that controls access to all protected z/OS resources. RACF or an equivalent ESM is required for z/OS data set encryption.

A key feature of RACF is its hierarchical management structure. The RACF security administrator is defined at the top of the hierarchy and can issue any RACF command or change any RACF profile (except for some auditing specific operands).

With z/OS data set encryption, you can define RACF profiles to assign ICSF key labels to data sets and authorize users and groups to ICSF keys and services.

For more information about the use of RACF commands, see z/OS Security Server RACF.

2.3.5 IBM Multi-Factor Authentication for z/OS

IBM Multi-Factor Authentication (MFA) for z/OS is an optional software feature that can increase the security of z/OS data set encryption.

Certain system user IDs include powerful access privileges, which might warrant more security protection. Without some form of multi-factor authentication, the security of the system might be relying solely on a user ID and password combination to protect access to sensitive resources.

The user ID and password combination can become a weak link that is vulnerable to various threats, including social engineering and password cracking. These risks can be minimized by using a multi-factor authentication product, such as IBM Multi-Factor Authentication for z/OS.

By using MFA software, you can define more authentication factors for users of IBM z/OS systems. It works with IBM RACF to authenticate users with multiple factors and define policies for these factors and apply them to specific IDs. IBM MFA integrates directly with the security server, and not any specific authentication factor. Therefore, factors can be added without changes to the RACF/MFA infrastructure. For more information, see IBM Z Multi-Factor Authentication.

2.3.6 IBM Security zSecure Suite

IBM zSecure Suite is an optional software feature that provides alert, audit, and administration capability for z/OS operations, such as z/OS data set encryption.

This suite aids in the detection of concealed and complex risks by using a built-in technology base to perform extensive automated analysis of the operating system, security system, and major subsystems. It integrates security event information from critical IBM z/OS subsystems and applications.

IBM Security zSecure Suite was enhanced with many new features to assist with the secure management of z/OS data set encryption and network encryption. These features include audit capabilities for data at-rest and data in-flight. For more information, see IBM Security zSecure.

Monitoring and reporting

A securely managed data set encryption implementation must satisfy many challenges, including regulatory, legal, internal, and external audit. The ability to automate, alert, report, and respond quickly to ad hoc requests for changes and for information is vital. In addition, having a single place to manage the administration of changes to key labels, SAF settings, and other security settings can ensure accuracy and consistency.

In its most basic respect, RACF and other SAF products are designed to protect resources by answering the question of is this access to be permitted.

However, modern z/OS installations must review their security setup from a number of different perspectives, including the following examples:

- Automated compliance capability; for example, prove every month that only these specific user IDs can access a particular sensitive resource and prove that the protection is still in place.
- Automated Audit capability, such as, How many people can access this resource? How many accessed it this week?
- Lockdown protects the system from dangerous intended or accidental changes.
- Alerting highlights that immediately a violation is occurring or is attempted.
- Safe housekeeping; for example, ensuring least-privilege access, that no redundant entities exist and are assisting with role-based access requirements.
- Analysis of how vulnerable a system might be with the ability to perform routine or ad hoc health checks.

IBM Security zSecure Suite provides these capabilities. IBM Security zSecure also includes the following capabilities to assist with the systems management of a z/OS data set encryption implementation:

- DFP segment administration
- Audit of the ICSF
- Enforcement and configuration options
- Key label reporting that is associated with sensitive data sets
- Verification whether a data set can be used (decrypted) on the system
- Key label information

System input to IBM Security zSecure can include SMF records, the RACF (or SAF) databases, and a proprietary zSecure Freeze data set that includes system information from z/OS control blocks, among other sources.

For more information about zSecure, see IBM Security zSecure Suite.

2.3.7 IBM zBNA

The IBM Z Batch Network Analyzer (zBNA) tool is supplied as-is by IBM at no charge. It was enhanced to include analysis capability for z/OS data set encryption.

zBNA uses SMF workload data (SMF records 113 are required) to generate graphical and text-based reports. zBNA also requires SMF record type 42 subtype 6.

zBNA can help your capacity planners estimate the effect of implementing data set encryption by using your own performance data on current and projected hardware and software.

For more information, see IBM Z Batch Network Analyzer (zBNA) Tool.

2.4 Cost and performance effect

In choosing which hardware and software components to configure for your z/OS data set encryption environment, the following performance considerations are important:

- Operational encryption and decryption performance depends heavily on the capability of the CPACF processor and the data block size.
- Encrypting or decrypting larger blocks of data improves performance.
- Utilities (such as zBNA) help with pre-production analysis.

Note: Each implementation is unique.

The cost of widespread data set encryption on an IBM Z (compared to not encrypting on the same systems might be a low single-digit percentage increase.

The use of IBM Z Data Compression (zEDC)[1] before encryption can significantly lower encryption costs. By reducing the volume of data to be encrypted and decrypted, CPU usage decreases. Therefore, compress data first whenever possible.

Combining the Integrated Accelerator for zEDC (on-chip) compression with BSAM/QSAM file encryption can improve elapsed time and reduce CPU consumption compared to not using compression and encryption. Also, using the Integrated Accelerator for zEDC compression with BSAM/QSAM files can significantly reduce file size and improve CPU costs and elapsed times compared to using no compression.

Note: The zEDC compression with dataset encryption measurements were completed in a controlled environment. Your results can vary, depending on individual workload, configuration, and software levels.

[1] Sequential extended-format data sets support zEDC compression, in addition to generic and tailored compression. For VSAM extended-format data sets, only generic compression for a KSDS is supported.

3

Planning for z/OS data set encryption

This chapter covers planning considerations for implementing z/OS data set encryption from several perspectives, and includes the following topics:

3.1 Creating an implementation plan

The following approach is suggested for starting with a z/OS data set encryption implementation plan:

- Understand the scope of the data that you want to protect.

  For example, consider what data you want to protect. Must the data be protected to satisfy an encryption initiative, such as to satisfy regulatory compliance, or other security requirements?

- Consider a pilot project for an internal proof of technology.

  Develop a use case for the project. Based on the data to be protected, choose an application similar to what is used to access the protected data; for example, batch workload, CICS/VSAM, Db2, and IMS.

  Run a zBNA study to evaluate the potential CPU effect for the application and identify noneligible datasets as early as possible. For more information, see 2.3.7, “IBM zBNA” on page 33 for more information.

- Define the criteria for eligible z/OS data sets.

  Start by determining where the data is to be stored; that is, what data set types contain the data to be protected. Is it contained in sequential or VSAM data sets, Db2, IMS, zFS, or PDSE?

  Identify all z/OS data sets and groupings of data sets that might be eligible for encryption, such as sequential and VSAM extended format data sets or data sets that must be converted to extended format.

- Ensure that the IBM Z platform is ready for z/OS data set encryption.

  At a minimum, ensure that the system to be used for the proof of technology satisfies the prerequisite hardware and software levels. Also, consider any other middleware that is required based on the use case to be evaluated.

  In addition, consider the items in the Readiness checklists. Most of the items might not be needed during a proof of technology, but should be evaluated before implementation on production systems. For more information, see 5.1, “Readiness checklists for deployment” on page 126.

  > **Note:** You can get assistance from IBM Technology Expert Labs, which provides a pervasive encryption readiness assessment (PERA) along with implementation assistance services.

- Implement the proof of technology and review and assess carefully regarding expected performance and operational outcomes.

  Prepare the environment by completing the configuration and setup steps. Then, complete the required deployment steps to enable the creation and access of encrypted data sets. Run the application to ensure that it can successfully process encrypted data sets.

  After the results of the proof of technology are satisfactory, continue with developing a strategy for the broader z/OS data set encryption implementation.

- Develop operational processes that protect and maintain the implementation.

  Operational processes might include, but are not limited to, the areas of access control policies, key management, auditing, high availability, disaster recovery, and backup/restore. Consider practicing and refining these operational processes over time.

- Determine how z/OS data set encryption is to be rolled out to production systems.

  Because the implementation process requires allocating new data sets or reallocating existing data sets, gradual increments of encrypting data sets might be the preferred approach. Therefore, your implementation plan should include multiple phases that are based on the criteria that are used to identify the eligible data sets.

3.2 Distinguishing roles and responsibilities

A z/OS data set encryption implementation includes an understanding of the different roles and responsibilities that are involved from your organization. After the implementation plan is determined, assigning task ownership is required. It is likely that roles and their assigned tasks differ for every organization. The roles and potential tasks that you might map to your organization are listed in Table 3-1.

Using a RACI chart to assign the different roles and task can be helpful and can help to build a deployment workflow that is specific to your organization.

Table 3-1 Roles and tasks sample

Roles	Tasks	How	Benefit
Security Administrator	► Identify data sets that must be encrypted ► Tie encryption to user access ► Create RACF profiles, assigning access to key labels ► Permit the creation of encrypted data sets	► Updates key label in RACF data set profile ► Modifies data set profiles with key labels and access permissions to files ► Provides access permission to FACILITY class resource to allow data set encryption	► Encrypts sensitive data ► Prevents unauthorized access to data based on profiles ► Prevents unauthorized access to creating encrypted data sets
Storage Administrator (data set manager)	► Assign encryption to specific data classes ► Manage backup, migration, and replication of encrypted data ► Manage backup, migration, and replication of keys	► Sets key labels for data class by using storage management panels (ISMF) ► Updates ACS routines	► Manages SMS constructs that enable encryption ► Manages HA/DR of data and keys
Key Manager	► Manage keys (defining keys and creating key labels); works with key management ► Manage key changes, including encryption key transportation to other systems	► Generates encryption keys ► Defines key labels in CKDS that are associated with secure AES256 DATA or CIPHER keys	Manages the key lifecycle from creation to destruction
ICSF Administrator	► Manage ICSF, CKDS	► Creates and maintains CKDS	Manages the ICSF environment

Roles	Tasks	How	Benefit
Security Auditor	► Update audit reports ► Ensure audit and reporting compliance	► Lists the catalog, and so on, to display encryption status ► Uses zSecure for audit	Determines encryption status to meet compliance
System Programmer	► Ensure system hardware and software support encryption ► Work with the Security Administrator to determine whether migration action is needed to allow creation of encrypted data sets	Ensures that all systems that might need to access the data include the CKDS	Manages hardware and software level on systems to support encryption
User (data owner)	► Automatically create encrypted data sets ► Run applications, submits jobs	If the key labels are not assigned by RACF (preferred method), adds key label to JCL or `IDCAMS DEFINE CLUSTER`	Automates creating encrypted files and accessing clear text without code changes

3.3 Data set administration considerations

This section describes the following considerations for data set administration:

- Supported data set types
- Data set compression
- Data set naming conventions
- Encrypted data set availability at IPL
- Using z/OS data set encryption
- Copying, backing up, migrating, and replicating encrypted z/OS data sets

3.3.1 Supported data set types

z/OS data set encryption encrypts the following types of data sets:

- Sequential extended format data sets, which are accessed through basic sequential access method (BSAM) and queued sequential access method (QSAM)
- VSAM extended format data sets, such as a Key-sequenced data set (KSDS), Entry-sequenced data set (ESDS), Relative record data set (RRDS), variable relative record data set (VRRDS), and linear data set (LDS), which are accessed through base VSAM and VSAM record-level sharing (RLS)
- PDSE data sets (containing data members), which are accessed through basic partitioned access method (BPAM), BSAM, and QSAM.
- Basic format data sets, which are accessed through BSAM, QSAM and EXCP.
- Large format data sets, which are accessed through BSAM, QSAM and EXCP.

To identify that PDSEs are to be viewed as a supported data set type on the system, define the following FACILITY class resource as a discrete profile:

STGADMIN.SMS.ALLOW.PDSE.ENCRYPT

To identify that basic and large format data sets are to be viewed as a supported data set type on the system, define the following FACILITY class resource as a discrete profile:

```
STGADMIN.SMS.ALLOW.DATASET.SEQ.ENCRYT
```

For more information about restrictions and dependencies for z/OS data set encryption, see Data Set Encryption.

The following rules apply when encrypting data sets:

- Encrypted data sets must be SMS-managed.
- Encrypted sequential extended format data sets can be created in compressed format.
- Encrypted VSAM extended format KSDS data sets can be created in compressed format.
- System data sets, such as catalogs, sharing control data set (SHCDS), and DFSMShsm control data sets, must not be encrypted, unless otherwise specified.
- Data sets that are needed before ICSF is started *must not be encrypted*.
- Sequential (non-compressed) extended format data sets with a BLKSIZE of less than 16 bytes cannot be encrypted.
- Encrypted data sets are supported on IBM 3390 DASD device types only.
- The DFSMSdss REBLOCK keyword is ignored on RESTORE functions. DFSMSdss ADRREBLK installation exit is not called for encrypted data sets.
- DFSMSdss does not support VALIDATE processing when backing up encrypted indexed VSAM data sets. For more information, see VALIDATE.

The following additional rules apply when encrypting basic and large format data sets:

- Basic and large format data sets with an LRECL of less than 16 bytes for RECFM F(B(S)) or less than 12 bytes for RECFM V(B(S)) cannot be encrypted.
- Basic and large format data sets with user blocks less than 16 bytes cannot be encrypted.
- Basic and large format data sets with hardware keys (nonzero DCBKEYLE) cannot be encrypted.
- Basic and large format data sets opened for BDAM processing cannot be encrypted.
- Checkpoint/restart fails if an encrypted data set is open at the time of checkpoint, or if the checkpoint data set is created as an encrypted data set.
- EXCP applications that access encrypted basic and large format data sets must be modified to successfully open the data set and to perform encryption/decryption of the data.
- Applications that provide the data set block size as input to the TRKCALC macro might need to be modified because of the length of the block prefix added for encrypted basic and large format data sets. For more information regarding the block prefix, refer to section "Specifying a key label for a basic or large format data set" in publication *IBM z/OS DFSMS Using Data Sets*, SC23-6855.

> **Note:** As a best practice, for basic and large format data sets that are not accessed by any EXCP application, consider converting them to encrypted sequential extended format. Sequential extended format data sets support access method compression, but basic and large format data sets do not.

If extended format data sets are to be used with data set encryption, complete the following steps to prepare for requesting extended format on new data set allocation:

1. Set up an SMS policy to request extended format. A storage administrator can update specific data classes through Interactive Storage Management Facility (ISMF) to request extended format by using the DSNTYPE option with EXTR or EXTP.
2. A storage administrator can update automatic class selection (ACS) routines through ISMF to select data classes that are enabled for extended format.

For more information about allocating extended format data sets, including guidelines and restrictions, see Data Set Encryption.

3.3.2 Data set compression

Because encrypted data does not compress, any compression that occurs after encryption is ineffective. Therefore, consider the use of access method compression when encrypted data sets are created.

Note: Only sequential extended format data sets and VSAM extended format KSDSs can be compressed. If you must use the support of Basic and Large dataset encryption, you cannot use hardware assist compression. Assess the impact on the backup compression of your tapes.

The following considerations apply when z/OS encrypted data sets are compressed:

- When a data set is allocated as compressed format, DFSMS first compresses the data; then, it encrypts the data.
- Data sets remain encrypted during DFSMShsm and DFSMSdss migration and backup processing.
- Data sets remain encrypted during hardware-based data replication services.
- zEDC is expected to significantly reduce the CPU cost of encryption with compression ratios five times or more for most files. zEDC can be used with sequential extended format data sets.
- Less data to encrypt accrues lower encryption costs.
- Compressed data sets use large block size for I/O, which is more efficient with encryption.
- Sequential extended format data sets support generic, tailored, or IBM Z Enterprise Data Compression (zEDC).
- A VSAM extended format KSDS supports generic compression (only KSDS can be compressed format).

If compressed format data sets are not yet used, complete the following steps to prepare for compression on new data set allocation:

1. Set up an SMS policy to request compression. A storage administrator can update specific data classes through the Interactive Storage Management Facility (ISMF) to request compression by using the COMPACTION option.

2. A storage administrator can update automatic class selection (ACS) routines through ISMF to select data classes that are enabled for compression.

For more information about allocating compressed format data sets, including guidelines and restrictions, see Data Set Encryption.

3.3.3 Data set naming conventions

Your organization might use an established data set naming convention that considers the following information:

- Environment (such as PROD, DEV, QA, and TEST)
- LPAR (PROD1, PROD2, and so on)
- Application (such as BANKING or MEDICAL)
- Users (such as JOHN or MIKE)

For z/OS data set encryption, key labels can be associated with the generic profiles that are protecting the data sets. Therefore, data set readability is determined by access to the generic profile.

During planning, you might want to revisit your data set naming convention to ensure that you can separate access to the data set to only those users that must access the data.

For more information about role separation considerations, see 3.4.1, “Organizing DATASET resource profiles” on page 44.

3.3.4 Encrypted data set availability at IPL

Access to encrypted data sets requires ICSF to retrieve the data encryption keys. Therefore, ICSF must be started before those encrypted data sets are used. This requirement is especially true when you plan to encrypt SMF data sets or other data sets that are used during z/OS initialization.

When you are choosing which data sets to encrypt, you must ensure that those data sets are not accessed during IPL before ICSF is started.

For more information about when to start ICSF in the IPL process, see 3.5.2, “Starting ICSF early in the IPL process” on page 45.

3.3.5 Using z/OS data set encryption

The following applications can use z/OS data set encryption:

- All in-service releases of IBM Db2
- All in-service releases of IBM Information Management System (IMS)
- All in-service releases of IBM CICS Transaction Server for z/OS
- All in-service releases of IBM MQ
- New and existing zFS data can be encrypted and compressed

Also see Data Set Encryption.

3.3.6 Copying, backing up, migrating, and replicating encrypted z/OS data sets

The following system services that manage the data set (as opposed to the data) ensure that data remains in encrypted form:

- During DFSMSdss functions, COPY, DUMP, and RESTORE.

  The backup, migration, and restore functions are performed by DFSMSdss and DFSMShsm. DFSMSdss and DFSMShsm are priced features of z/OS

- During DFSMShsm functions, Migrate/Recall, Backup/Recover, Abackup/Arecover, Dump/Data Set Restore, and FRBACKUP/FRRECOV DSNAME.
- During IBM disk copy services functions, such as Metro Mirror, Global Copy, Global Mirror, z/OS Global Mirror (XRC/ZGM), IBM FlashCopy®, and Concurrent Copy, because the copy operation copies data as it is written (already encrypted).

The recovery system must include the same master keys and data set encryption keys. Storage administrators (or others) that perform these system services do not require access to the key label.

Replicating encrypted data sets

Replication technologies that move data in physical format maintains data in encrypted (and compressed) format. Take advantage of the reduced storage requirements with data compression.

For sequential extended format data sets, zEDC compression is recommended to significantly reduce the amount of data that is transferred and the elapsed time to complete the transfer. Data transmission between systems with noncompressed and compressed data is shown in Figure 3-1 .

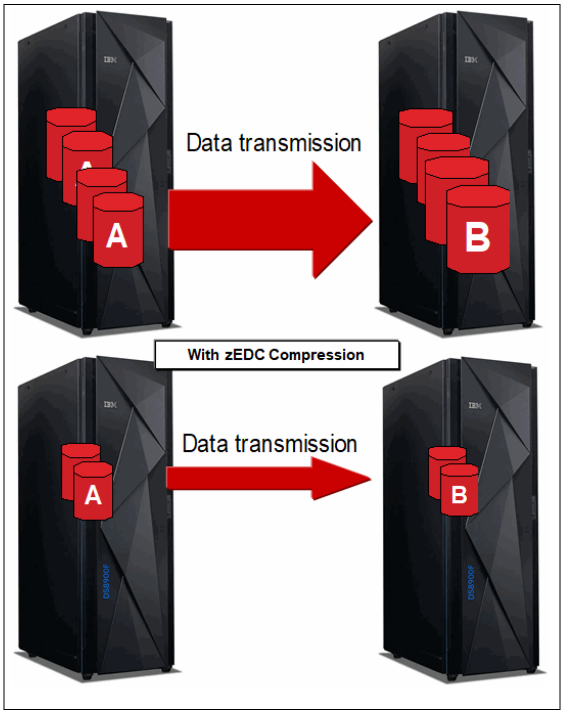


Figure 3-1 Data transmission between systems

> **Note:** The key material must be available on the target systems to access the encrypted data sets.

Transmitting encrypted data sets

System services that transmit data typically retrieve the data by using the access methods. The data in encrypted data sets is decrypted within these services before transmission. When transmitting sensitive data, use the secure versions of the following services:

- Connect: Direct[1]
- FTP
- XMIT

> **Note:** Users and system administrators who perform these functions require access to the appropriate key labels.

Copying encrypted data sets with DFSMSdss

When you copy a data set by using DFSMSdss, the allocation of the target data set is based on the *original attributes* from the source data set, regardless of the HLQ or the RACF profile that is associated with the target data set.

[1] Except when using the ADRDSSU Interface program exit, which copifes files in their current format.

Consider the following example:

- Data set: ITSO2.MY.DATASET
- HLQ ITSO2 is not an encrypted HLQ according to your RACF profile and SMS/ACS routines
- HLQ ITSO1 is an encrypted HLQ according to your RACF profile ND SMS/ACS routines

If you use an ADRDSSU JCL to copy ITSO2.MY.DATASET to a new data set ITSO1.MY.DATASET, data set ITSO1.MY.DATASET is *not* encrypted.

When you use ADRDSSU, the program does not drive any ACS routines. Your data set ITSO1.MY.DATASET includes the correct SMS and allocation attributes, but is *not* encrypted.

For more information, see ACS routines invoked for copying and importing data sets.

ACS routines started for copying and importing data sets.

The ACS routines that are started when initial allocation, importing, restoring, and recalling data sets is performed are listed in Table 3-2.

Table 3-2 Allocation, IMPORT, and COPY conditions

Type of processing	Data class ACS	Storage class ACS	Management class ACS	Storage class ACS
Initial allocation	Yes	Yes	SC	SC
IMPORT (Access Method Services)	No	Yes	SC	SC
COPY (DFMSdss)	No	Yes	SC	SC
COPY BYPASSACS (DFSMSdss)	No	No	No	SC

Consider the following points:

- Yes = ACS routine is started.
- No = ACS routine is not started. The ACS routine is not started for the data set that is copied or imported because their attributes are defined. The ACS routine might be started for other new data sets that are allocated to the job.
- SC = ACS routine is started only if storage class is assigned.

Backing up and restoring encrypted data sets

When you restore a data set by using DFSMShsm, the allocation of the restored data set is based on the attributes of the original data set, regardless of the HLQ or the RACF profile that is associated with the data set at restoration.

Consider the following points:

- If the source data set was *not* encrypted before backup, the restored data set is *not* encrypted.
- If the source data set was encrypted before backup, the restored data set is encrypted.

Consider the following scenario:

1. A nonencrypted data set, ITSO1.PROJECT1, was backed up.
2. A RACF DATASET profile, ITSO1.**, was updated with a key label.
3. ITSO1.PROJECT1 was restored.

Is ITSO1.PROJECT1 non-encrypted or encrypted? It is non-encrypted because its attributes are based on the original data set, which was non-encrypted. The key label in the DATASET profile is ignored.

3.4 Resource authorization considerations

This section includes considerations to aid in the administration of granting appropriate access to keys, data sets, and resources.

3.4.1 Organizing DATASET resource profiles

Authorization to data sets involves the DATASET class with commands, such as the following examples:

- ADDSD
- ALTDSD
- LISTDSD
- DELDSD

With z/OS data set encryption, you can reuse DATASET resource profiles or create more granular resource profiles so that you can assign different keys to different applications or business areas.

For example, if you have a resource profile PROD.APP1.*, you can assign a single key label to cover all data sets that are protected by that profile. However, if you want to ensure that certain information is available to only a subset of users, granularity can be added. You define more granular resource profiles (such as PROD.APP1.ACCOUNTS.* and PROD.APP1.BANKING.*) and associate a different key label with each resource profile to enforce separation of access to information associated with the application.

3.4.2 Separating duties of data owners and administrators

One of the key features of z/OS data set encryption is the separation of duties between data owners and administrators. With z/OS data set encryption, you can grant a storage administrator ALTER access to a data set (by using the DATASET class) to perform operations, such as create, move, and delete.

However, if the storage administrator has NONE access to the key label that is protecting the data set (by using the CSFKEYS class), the storage administrator cannot view the contents of the data set.

When assigning users or groups to CSFKEYS resources, security administrators should be careful not to copy the access control list directly from the DATASET without modification. Otherwise, all users and groups can view the sensitive data sets.

For key managers, the separation of duties means that although they need access to the keys that are used to protect data sets to do their job maintaining the data set keys, they do not have access to the data sets themselves. Similar to storage administrators, this effectively prevents key managers from accessing protected data.

When using UKO, keys in the CKDS are managed by the UKO key management system, as a proxy for the key managers. This reduces the need to grant key managers full access to all keys.

The security administrator must verify whether a user requires access to the data set or the contents within the data set. Consider the following points:

- If the user requires access to the data set to run a storage backup, they require access to only the data set profile that protects it.

- If the user needs access to read the encrypted data, they also need access to the key label and CSFKEYS entry that is protecting it.

Only a small subset of users should have access to the key label.

3.4.3 Considering multi-factor authentication

Privileged users and administrators must be identified and protected with multi-factor authentication. This type of authentication ensures that if a single authentication credential is compromised (such as a password), another form of authentication can prevent unauthorized access. For more information, see 2.3.5, “IBM Multi-Factor Authentication for z/OS” on page 31.

3.5 ICSF administration considerations

This section describes several considerations for ICSF administration and configuration.

3.5.1 Upgrading an IBM Z platform

If your IBM Z platform is being upgraded, you must consider how existing key data sets[2] (KDSs) and master keys are moved to the new system. This process is unlike starting from scratch, where you initialize a new KDS and set new master keys.

For the upgrade, the following items are required:

- ICSF CSFPRMxx configuration (for possible duplication)
- All KDSs (containing existing operational keys)
- All master keys that are associated with the KDSs (for reloading onto the Crypto Express adapters if needed)

Unknown master keys

If your system includes active KDSs and active master keys, but the master keys are not documented, you can perform a Coordinated Change Master Key operation to rotate the master key to a new, known master key. The new master key should be securely stored for future reentry. For more information, see 7.2.1, “Rotating the AES master key” on page 158.

3.5.2 Starting ICSF early in the IPL process

The ICSF address space becomes a critical component after you start using z/OS data set encryption because all access to encrypted data sets depends on calls that are made to ICSF. As a result, ICSF must always be up and running.

You can think of ICSF as having the same essential role as other system components, such as RACF or CATALOG. Would you expect CATALOG A/S or RACF to fail? This issue has serious implications in terms of continuous availability and resiliency.

If you are planning to use the z/OS data set encryption function, you must ensure that ICSF is started early in the IPL process. This requirement is especially true if you plan to encrypt SMF data sets or other data sets that are used during z/OS initialization.

[2] A key data set (KDS) can be a cryptographic key data set (CKDS), public key data set (PKDS), or token data set (TKDS).

This support is provided by entries in the IEASYSxx parmlib member. You can find more information in Starting ICSF during IPL-time.

ICSF starts as a SUB=MSTR address space, so you cannot access the ICSF job log by using the System Display and Search Facility (SDSF) or vendor products, such as (E)JES. In that case, you cannot browse the content of the ICSF job log (JCT not available). See Figure 3-1 on page 42.

Example 3-1 JCT not available

```
SDSF DA SC74     (ALL)    PAG  0  CPU/L     2/  1       JCT NOT AVAILABLE
COMMAND INPUT ===>                                            SCROLL ===> CSR
NP   JOBNAME  StepName ProcStep JobID    Owner    C         Pos DP Real Paging
S    CSF      CSF                                           NS  FE 2277   0.00
```

Not having access to the ICSF job log has operational implications in that you can no longer easily view the status of the ICSF startup, except by completing the following steps:

1. Search for ICSF startup messages in the system log (SYSLOG).
2. Check the time window where the ICSF messages were issued.
3. Determine whether a problem occurred during startup.

Typically, messages can be found in the ICSF job log, as shown in Figure 3-2 on page 47.

Example 3-2 ICSF startup messages

```
CSFM129I MASTER KEY DES ON CRYPTO EXPRESS6 COPROCESSOR 6C00,
CSFM129I MASTER KEY AES ON CRYPTO EXPRESS6 COPROCESSOR 6C00,
CSFM129I MASTER KEY RSA ON CRYPTO EXPRESS6 COPROCESSOR 6C00,
CSFM129I MASTER KEY ECC ON CRYPTO EXPRESS6 COPROCESSOR 6C00,
CSFM111I CRYPTOGRAPHIC FEATURE IS ACTIVE. CRYPTO EXPRESS6 COP
CSFM111I CRYPTOGRAPHIC FEATURE IS ACTIVE. CRYPTO EXPRESS6 ACC

CSFM127I CRYPTOGRAPHY - AES SERVICES ARE AVAILABLE.
CSFM126I CRYPTOGRAPHY - FULL CPU-BASED SERVICES ARE AVAILABLE
CSFM001I ICSF INITIALIZATION COMPLETE
```

3.5.3 Using the Common Record Format (KDSRL) cryptographic key data set

The Large common record format (KDSRL) is common to all key data sets. The LRECL of the CKDS determines its format, for example:

- 252 creates a fixed-length CKDS.
- 1024 creates a variable-length CKDS.
- 2048 creates a KDSR format CKDS.
- 32768 creates a KDSRL format CKDS (recommended)

Note: Upgrade all sysplex members to ICSF HCR77D1 or higher.

Use the newest format, the common record format (KDSRL), for all your key data sets. The KDSRL format supports more functions to manage cryptographic keys, such as tracking key reference dates, archiving keys, and setting cryptoperiods. For more information about creating a common record format CKDS, see 4.3.2, “Creating a Common Record Format (KDSRL) CKDS” on page 81.

For more information about converting your CKDS to KDSRL format, see the following resources:

- "Converting a CKDS" on page 82
- Migrating to the common record format (KDSR) key data set

3.5.4 Planning the size of your CKDS

With z/OS data set encryption, cryptographic keys are stored in a VSAM CKDS data set. The CKDS must be defined to support and manage secure keys. The CKDS contains individual entries for each key that is added to it by way of ICSF. CKDS is an essential part of z/OS data set encryption environment.

Plan the allocation, size, and format of the CKDS. If the data set is defined, verify the size allocation and formatting. Consider reallocating the CKDS to a larger size. For more information, see 8.3, "Refreshing the CKDS" on page 178.

Also, consider reformatting it to the recommended common record format (KDSRL) with an LRECL of 32768. For more information, see 4.3.2, "Creating a Common Record Format (KDSRL) CKDS" on page 81.

The KDSRL format supports the functions and fields that are used for metadata.

The formula to determine the size that is required for the CKDS is shown in Figure 3-2 .

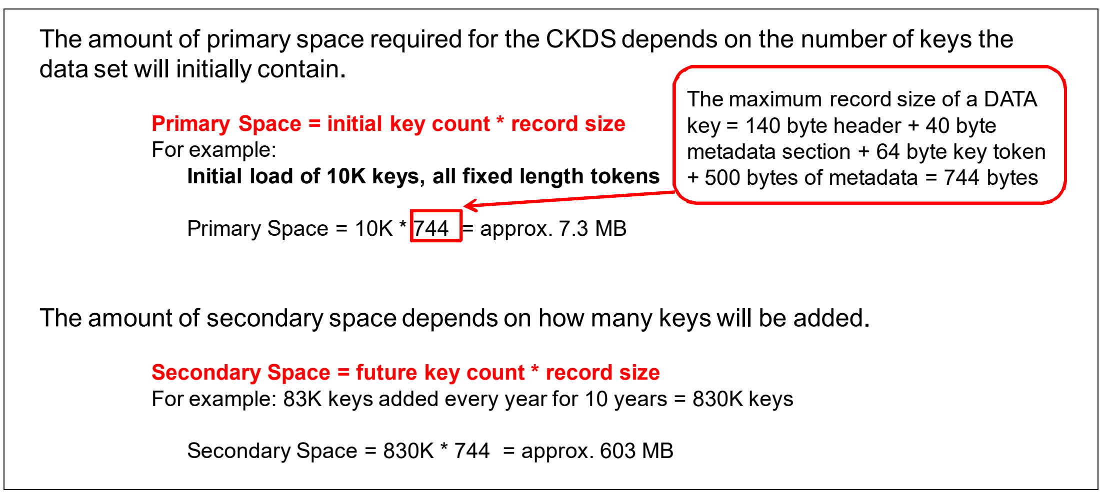


Figure 3-2 Primary and secondary space calculations

3.5.5 Calculating the virtual storage that is needed for the CKDS

Plan for the system resources that are required for managing the CKDS copy in virtual storage, particularly when the installation is deploying a large CKDS.

> **Note:** The term *very large* is a relative assessment (depending on the installation) and might be expressed, for example, in terms of tens or hundreds of thousands of symmetric keys in the CKDS or even millions of keys.

An in-storage copy of a CKDS that is not experiencing significant dynamic key creation or deletion activity uses a stable amount of virtual storage by using a stable amount of system backing resources. However, occasional but unavoidable ICSF functions (such as CKDS refresh) generate a significant spike in the amount of virtual storage that is used, which causes a greater temporary demand for system resources that are backing up that virtual storage.

Because of these circumstances, it is important to calculate and plan for the system main storage and auxiliary paging space that is required to support an active in-storage copy. For a CKDS shared across a sysplex environment, every active ICSF in the sysplex includes an equivalent resource requirement.

> **Tip:** A preferred practice is to always allocate enough main storage to avoid the use of auxiliary paging.

Each symmetric key in the CKDS is managed with one VSAM record. You must plan for the appropriate amount of combined main storage and auxiliary paging space for each VSAM record, per active ICSF.

A formula that can be used to calculate the required system virtual storage backing resource for an active in-storage CKDS is shown in Example 3-3 on page 48.

Example 3-3 Formula to calculate the required system virtual storage

```
HI-A-RBA x ( ( 100 - %Free Space ) / 100 ) x 6
```

In this formula, HI-A-RBA is the allocated relative byte address for the data component of a CKDS VSAM data set. The output from IDCAMS LISTCAT for a CKDS VSAM data set can be used to determine the HI-A-RBA value for the data component. The `%Free Space` value represents the percentage of free space in the CKDS VSAM data set. The IDCAMS EXAMINE DATATEST command output can be reviewed to determine the percentage of free space.

3.5.6 Sharing the CKDS in a sysplex

Any ICSF instance in the system-wide parallel sysplex that enables sysplex communications (**SYSPLEXCKDS** option in the CSFPRMxx member) and uses the same Key Data Set name (**CKDSN** option in the CSFPRMxx member) is considered a member of the CKDS sysplex.

All members of the CKDS sysplex must share the master key and key data set. With this configuration, the following conditions are inherent:

- Updates are automatically propagated across the sysplex (by way of signaling) to maintain cache coherency.
- Keystore management operations are coordinated across the sysplex.

Cryptographic key management for high availability with a single system image view of cryptographic key material is shown in Figure 3-3 on page 49.

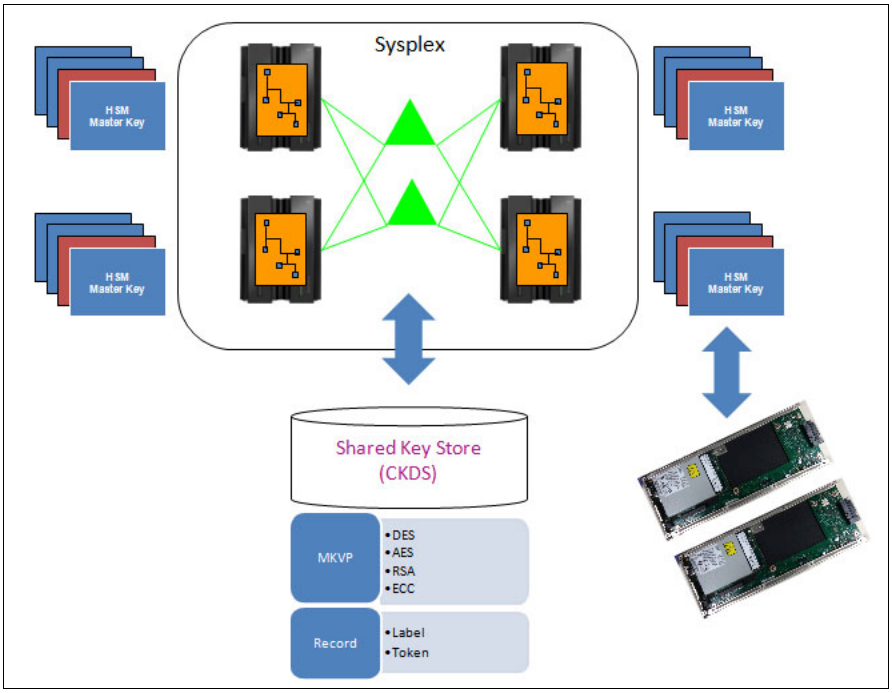


Figure 3-3 Single system view of cryptographic key material

3.6 Key management considerations

Consider creating a key management plan to be reviewed by your auditors, security administrators, and ICSF administrators.

3.6.1 Understanding key management

Cryptographic keys feature a lifecycle that includes tasks, such as key creation, key activation, key deactivation, key archival, and key deletion. Some regulations, such as PCI-DSS, require that key management processes are created and well-documented.

Consider the following questions:

- What regulations must be considered?
- What key types and lengths will be used?
- Will the keys be stored in the clear or encrypted?
- How long will keys be active?
- What happens to a key after it is deactivated?
- When should keys be archived?
- How will a key be handled if it is compromised?
- How often will keys be backed up?
- How will keys be distributed to other systems?
- Who will own the key (such as the user or the application owner)?
- What metadata should be associated with a key?
- Will keys be rotated? How often? Which keys?

The various key management areas are briefly explained in the following sections.

3.6.2 Reviewing industry regulations

Before your key management plan is created, identify and review any regulations that might be required for compliance. For regulations that are generic or non-specific, work with your auditors to clarify ambiguities and review your key management plan.

For more information, see 1.2.4, “Regulations” on page 3, and 1.5.1, “Encrypting above and beyond compliance requirements” on page 10.

3.6.3 Choosing key algorithms and lengths

In “Brute force attacks” on page 8, the nature of brute force attacks that can be attempted to compromise keys was described. The conclusion was that longer key lengths provide better protection for symmetric keys.

For z/OS data set encryption, only 256-bit AES keys are supported. However, your organization might include other crypto applications. If so, are those applications using strong keys?

As of this writing, the National Institute for Standards and Technology (NIST) approves only TDES and AES for symmetric encryption. For more information, see NIST.

To check whether your installation includes applications that are using weak keys, you can view the following records:

- SMF Type 82 Subtype 31 ICSF audit records.

  For more information, see 6.5, “Auditing crypto engine, service, and algorithm usage” on page 151.
- SMF type 119 Subtype 11 and 12 network encryption audit records. For more information, see z/OS Encryption Readiness Technology (zERT).

3.6.4 Determining key security

Data set encryption keys can be stored in the Cryptographic Key Data Set (CKDS) as clear keys or secure keys. Consider the following points:

- Clear keys require CPACF.
- Secure keys require Crypto Express adapters and CPACF.

The authors recommend the use of secure keys. A secure key is a data key that is encrypted by using a master key. Therefore, all encrypted data is unreadable without a master key. The master key should be created from two or more key parts by using a different key owner for each master key part. By using this configuration, reading sensitive key material requires access to the CKDS and access to all master key parts.

When clear keys are used and the CKDS is dumped, all keys are readable. By using secure keys, dumping the CKDS does not yield any sensitive data.

Protected keys are not stored in the CKDS. They are created from secure or clear keys and stored in memory. When ICSF is restarted, all protected keys are cleared from memory.

For more information, see 3.6.11, “Establishing a process for handling compromised operational keys” on page 56.

3.6.5 Choosing key officers

Master keys should be made up of two or more key parts. A different key owner should be used for each part. In the case of disaster recovery or when Crypto Express adapters are added, all key officers must be available and present to load their master key part.

Based on this criteria, the following decisions must be made:

- How many key officers will you have?
- Who will be those key officers?
- How often will master keys need to be changed?
- How will you ensure that the key officers will not collude and compromise the master key?

3.6.6 Using protected keys for high-speed encryption

The use of secure keys and protected keys in the z/OS data set encryption process ensures that key material is not available to unauthorized users at any time, which gives unauthorized users the ability to decrypt encrypted data sets.

The Central Processor Assist for Cryptographic Functions (CPACF) wrapping key is used to rewrap (encrypt) a secure key after it is decrypted. The CPACF wrapping key is in a protected area of the hardware system area (HSA), which is not visible to the operating system or applications.

The process of converting a secure key to a protected key involves a master key that is stored in the Crypto Express adapter and authorization to the following segments in the CSFKEYS class profiles:

- `SYMCPACFWRAP [YES | NO is the default]`

  SYMCPACFWRAP specifies whether symmetric keys can be rewrapped by CPACF.
- `SYMCPACFRET [YES | NO is the default]`
- `SYMCPACFRET specifies whether the encrypted symmetric keys that were rewrapped by CPACF can be returned to an authorized caller (such as ICSF).`

Rewrapping a secure key into a protected key with Crypto Express

The key wrapping process on IBM Z with a Crypto Express7S or later features is shown in Figure 3-4 . Notice that the data key material is not in the clear at any point in the process.

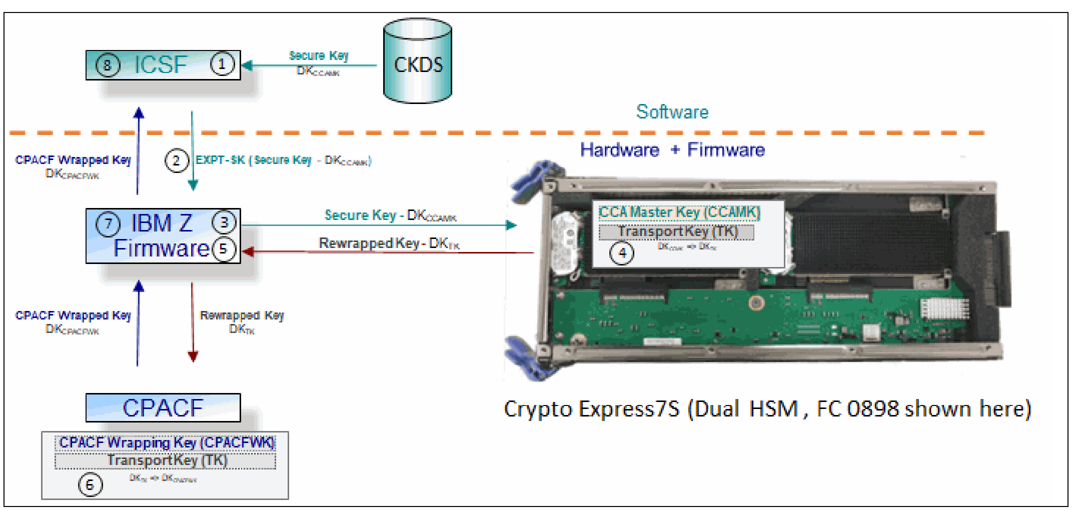


Figure 3-4 Key wrapping process with the Crypto Expess7S

The following process, which is similar for Crypto Express6S features, is used to rewrap a secure key into a protected key (as shown in Figure 3-4 on page 51):

1. ICSF retrieves the data key (DK) that is stored as a secure key (encrypted by using a master key [CCAMK]) in the CKDS.
2. ICSF starts rewrapping the data key by sending a command and the secure key to Z firmware.
3. Z firmware sends the secure key with transport key (TK) information to Crypto Express7S. Transport keys are derived for each cryptographic domain by way of a key agreement protocol between IBM Z firmware and Crypto Express firmware.
4. Crypto Express7S decrypts the secure key by using the master key and rewraps the data key by using a transport key (TK).
5. The rewrapped data key (encrypted by using the transport key) is sent back to Z firmware.
6. Z firmware starts CPACF to unwrap and rewrap the data key by using a CPACF wrapping-key to create a protected key. CPACF Wrapping Key and Transport Key are in a protected area of HSA that is not visible to the operating system or application.
7. Z firmware returns the CPACF wrapped-key (protected key) to ICSF.
8. ICSF caches the protected key in the ICSF address space and optionally returns the protected key to the authorized caller.

3.6.7 Creating a key label naming convention

Key labels can be used to protect groups of data sets or single data sets. The granularity is determined by the organization and must be aligned with policy. The data set naming convention provides a way to group data sets logically so that related groups of data sets can include a common level of protection.

> **Tip:** Having a good key label naming convention is also important when using the UKO as a key management system. UKO creates keys based on key templates that specify a key label pattern with variable fields. This enables UKO to enforce a given key label naming convention. It is also flexible enough to support any required key label that conforms to the specified key label naming convention.

With key labels, you can limit or prevent access to data through policies, as shown in the following examples:

- Storage administrators who manage data sets need access to only the data set and not to the key label; therefore, the data is protected.
- Different key labels can be used to protect different data sets, which is ideal for multiple tenant environments or data set specific policies.
- Administrators can be prevented from accessing data; utilities can process data preserving its encrypted form.

Because creating key labels is an ongoing process, the naming convention for key labels must be planned so that key labels are easier to maintain.

A key label can consist of up to 64 characters. The first character must be an alphabetic or a national character (#,$, or @). The remaining characters can be alphanumeric, a national character, or a period (.).

In your environment, you can use naming conventions that are based on the following conditions:

- LPAR that is associated with the key
- Type of data that is encrypted
- Owner that is associated with the key
- Date that the key was created
- Application that is intended to use the key[3]
- Sequence number for the key

A recommendation for the naming convention of key labels is shown in the following example:

```
DATASET.<dataset_resource_description>.ENCRKEY.<seqno>
```

By using the DATASET keyword (or others like DB2 for Db2 table spaces) in the key label, the key administrator can easily recognize that the key label is associated with a data set and be handled differently from short-term keys (for example, archiving rather than deleting at "end of life").

The corresponding sample RACF resource class to protect this data set generically is shown in the following example:

```
CSFKEYS DATASET.<dataset_resource_description>.ENCRKEY.* ICSF(SYMCPACFWRAP(YES)
SYMCPACFRET(YES))
```

The reasoning is that if future data sets are allocated, they are automatically eligible to be encrypted data sets. It is not recommended to change the ICSF segment for the generic entry because you must ensure that the naming standards are being used before new allocations of encrypted data sets are performed.

Also, plan how to transport an encrypted data set with a key label to another site as part of your key label naming standard. You must ensure that the key label meets the naming standard of that site; otherwise, you might need to rekey the data set before it is transmitted.

3.6.8 Deciding whether to archive or delete keys

After a key is generated, it progresses through multiple states during its lifecycle. The key and its lifecycle must be managed by an authorized administrator.

A simplified key lifecycle from creation (start) to deletion (destroyed) is shown in Figure 3-5 on page 54. Keys can be defined with a valid start date and end date, which is known as the *cryptoperiod*. A cryptoperiod can be used to control when a key is allowed for use in crypto operations.

[3] Useful if you want to track encryption progress at the applications level.

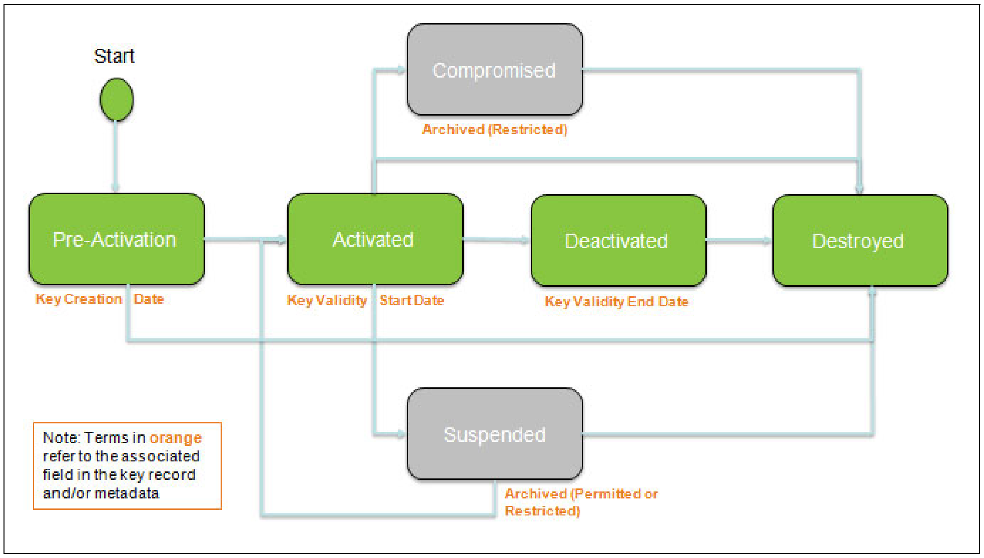


Figure 3-5 Key lifecycle (simple)

The following terms are also used in referencing the key lifecycle:

- Creating

  Creating is the starting point for generating keys and creating key labels. You must determine who has the authority to generate keys and create labels.

  The administrators and users require access to the CSFKEYS profile entry that corresponds to the name of the key label that is used. The administrators also require access to the CSFSERV profiles that protect the callable services and utility panels.

- Updating

  The process of updating or changing key labels (rekeying) is performed in instances where a key was determined to be compromised or a security policy states that data must be rekeyed regularly. In both cases, a new key must be generated and a key label must be defined. This process involves a reallocation and copy of the data set.

- Deleting

  In most installations, the use of z/OS data set encryption is not recommended to delete key labels that are used for data set encryption after they are created and tagged to a dataset. Key deletion makes the data set inaccessible and unrecoverable. The preferred method to retire a key is to use archiving.

- Archiving

  Archiving is the process of marking a key unusable rather than removing or deleting the key completely. This operation is preferred over deletion because archived keys can be recalled later, if needed.

  If a user attempts to access a key that was archived, a `S213 RC8 RSND10 access error` occurs and an SMF record is written (type `82 subtype 30`). However, if an installation wants to monitor usage but allow access to archived key labels, the XFACILIT class `CSF.KDS.KEY.ARCHIVE.USE` can be enabled.

  If this step is done, it is also recommended to enable the `KEYARCHMSG` option to provide a message to the user the first time it is accessed after it is archived.

  The use of PE01.TEST.KEY as the sample key label in the output is shown in Figure 3-6 on page 55.

```
**************************************************
Subtype=001E KDS Key Archive/Recall/Cryptoperiod
Written whenever an inactive or archived record is referenced
6 Nov 2017 17:18:37.12
   TME... 005F16A0 DTE... 0117310F SID... SC60     SSI... 00000000 STY... 001E
   Flags = 80800000
     80000000 Key Data Set was CKDS
     00800000 Archived record referenced, request failed
   Key Data Set Name... SYS1.SC60NEW.SCSFCKDS
   Key Data Set Label.. PE01.TEST.KEY
ICSF Server Identity...
   USRI.. IBMUSER
   GRPN.. SYS1
   JBN... CSF
   RST... 18:18:42.82
   RSD... 26 Oct 2017
   SUID.. 4040404040404040
End User Identity...
   USRI.. PE01
   GRPN.. SYS1
   TRM... SC60TC20
   JBN... PE01
   RST... 16:13:26.64
   RSD... 2 Nov 2017
   SUID.. 4040404040404040
```

Figure 3-6 Output of CSFSMFJ to show access to archive or inactive key labels

An option is also available to prohibit archiving for certain keys if you do not want unexpected access errors for key labels that are associated with critical data sets.

Note: With UKO, keys can be 'deactivated' which removes them from the CKDS, but UKO retains a copy of the key in the UKO repository. UKO can then reactivate and restore the key to the CKDS if necessary.

Key deactivation allows removing the data set encryption key from the CKDS but is still able to restore the key if necessary.

Any attempts to access a deactivated key behaves similarly to a deleted key and will not produce the SMF records that attempts to access an archived key would.

See also Chapter 10, “IBM Unified Key Orchestrator” on page 223.

3.6.9 Defining key rotation

The following options are available to rotate keys:

- Rotate the master key

  This option is the simplest method. The new master key must be loaded and then the key data sets can be reenciphered by using an ICSF panel utility.

- Rotate the operational key

  This option can be simple or complex, depending on which of the following approaches is used:

  – Approach 1: Establish a period (for example, one month) for which a key can be used to encrypt new data. When that period ends, all new data is encrypted with a new key and data remains encrypted with the old key.

  – Approach 2: Encrypt all new data with a new key. Decrypt and reencrypt all data with the new key as well.

  Approach 2 is similar to the process for handling a compromised key, but potentially on a much larger scale.

Based on the industry regulations for which you must comply, you might choose to use either approach, or both. You must also consider the cost and benefit for your environment when Approach 1 is used versus Approach 2.

3.6.10 Establishing cryptoperiods

A cryptoperiod defines the time in which a key is active. It is the time between the key activation start date and end date.

Some regulations require keys to have a clearly defined cryptoperiod. When the end date is reached, the key reached its end of life and can be revoked or destroyed. The data that is protected by that key is destroyed or reencrypted with a new key.

ICSF enables cryptoperiods by supporting key validity start and end dates for keys that are in the CKDS. This information can be found in the metadata view of the key. ICSF Option 5 can be used to display the metadata of a particular key.

Attempts to use keys within their start and end dates are successful. Attempts to use keys outside of their start and end dates fail with a S213 access error and `RC8 RSNDOE` error code.

Cryptoperiods are supported for z/OS data set encryption; however, any data sets must be reencrypted (or rekeyed) before the key expires. For this reason, some organizations might decide to not establish cryptoperiods for data set encryption keys.

> **Note:** For more information about how to rekey a data set, see Chapter 7, “Maintaining encrypted data sets” on page 157.

Before a cryptoperiod is established, consider the following questions:

- Should cryptoperiods be established for data set encryption keys?
- Does a regulatory requirement exist?
- What is an appropriate cryptoperiod?
- Does a regulatory requirement exist?
- What happens to a key at the end of its cryptoperiod?
  - Would the cryptoperiod ever be extended?
  - Will encrypted data be reencrypted with a new key?
- What happens to the data set at the end of the key’s cryptoperiod?
  - Should the data set be destroyed?
  - Should the data set be rekeyed?
- How will administrators identify expired or soon-to-expire keys?

To audit cryptoperiods, you can view SMF Type 82 Subtype 40 ICSF audit records. For more information, see 6.6, “Auditing key lifecycle transitions” on page 152 for more information.

To identify expiring keys, you can regularly run the ICSF_KEY_EXPIRATION z/OS Health Check. For more information, see “Key expiration check” on page 65.

> **Note:** For keys managed by UKO, UKO can be used to determine which keys will expire within a given time. See also Chapter 10, “IBM Unified Key Orchestrator” on page 223.

3.6.11 Establishing a process for handling compromised operational keys

When a data set encryption key is compromised, a plan should be in place to manage the key and the encrypted data.

Key handling

When a key is compromised, the key should not be available for use and any attempted use of the key should be audited.

The following options are available to prevent a key from being used:

- Set the key validity end date to the current date. For more information, see Chapter 7, “Maintaining encrypted data sets” on page 157.

  This process forces the key to expire the next day. However, the key can still be used on the current date. SMF type 82 subtype 30 audit records shows any attempts to use the compromised key.
- Mark the key as archived and disable the use of archived keys. For more information, see Chapter 7, “Maintaining encrypted data sets” on page 157.

  This process forces the key to be unavailable the same day. Any archived keys that are unrelated to the compromise are also unusable. SMF type 82 subtype 30 audit records show any attempts to use the compromised key.
- Delete the key.

  This process forces the key to be unavailable the same day. No audit records are created that show attempts to use the compromised key.

You might decide to use a combination of these options, depending on the risk.

Data handling

When a key is compromised, any data that was protected with the compromised key should be identified and rekeyed. For more information about how to identify encrypted data sets and rekey data sets, see Chapter 7, “Maintaining encrypted data sets” on page 157.

3.6.12 Establishing a process for handling compromised master keys

When a master key is compromised, it should be immediately changed and the key data sets must be re-enciphered twice. Crypto Express adapters contain the following master key registers for each master key type:

- Current master key (CMK)
- Old master key (OMK)
- New master key (NMK)

New master keys are loaded into the NMK register. When the master key is set or the KDS is initialized, the NMK register contents are moved to the CMK register. The CMK register contents are then moved to the OMK register, and the NMK register is cleared. In this way, keys that are encrypted by the OMK can still be used on the system.

In the case of a compromise, the OMK should be cleared or overwritten. Therefore, two master key change operations must occur to completely clear the compromised master key from the system.

For more information about loading a master key, see “Loading master keys” on page 59.

For more information about re-enciphering the key data sets, see Chapter 7, “Maintaining encrypted data sets” on page 157.

> **Note:** Remember to perform a change master key operation twice to completely remove a compromised master key from the environment.

3.6.13 Choosing key management tools

Managing cryptographic keys is vital to the overall security of your encrypted data. If the cryptographic keys are compromised, your encrypted data can also be compromised.

The following types of keys must be managed in a z/OS data set encryption environment:

- Master keys

  These keys are stored in a Crypto Express adapter and used to encrypt operational keys.

- Operational keys

  These keys are stored on the host system in a keystore (such as the CKDS) or in memory. They are used to perform various cryptography operations.

> **Terminology:** Data-encrypting keys are *operational keys*. Data-encrypting keys can be clear, secure, or protected keys.

For more information about the key types that are associated with z/OS data set encryption, see 1.6.3, "IBM Z cryptographic system" on page 16.

Available tools

The tools that are available for key management in a z/OS data set encryption environment and which keys you can manage with them are listed in Table 3-3.

Table 3-3 Tools and what they manage

Tool	Manage master keys?	Manage operational keys?
Integrated Cryptographic Service Facility (ICSF)	Yes	Yes
Trusted Key Entry (TKE)	Yes	Yes (small scale)
IBM Unified Key Orchestrator (UKO): ► UKO for z/OS ► UKO for Containers ► EKMF Workstation	No	Yes
IBM Guardium® Key Lifecycle Manager (GKLM)	No	Yes (for self-encrypting devices only)

Consider the following points:

- Master keys can be managed with ICSF or a TKE Workstation.
- Operational keys that are used for z/OS data set encryption can be managed with ICSF or UKO. For additional details, see Chapter 10, "IBM Unified Key Orchestrator" on page 223.

> **Note:** TKE features limited support for operational keys. Users typically manage no more than 50 operational keys by using TKE.

Management of master keys

The master key is stored in the Crypto Express hardware security module (HSM) and can be managed by way of utilities and panels as part of the Common Cryptographic Architecture (CCA) or Enterprise PKCS #11 (EP11) Architecture.

Choosing master key owners

Best practices for master key management involve building master keys from multiple key parts where each key part is owned by a different user. You should define which users are key owners and what process they follow for loading master keys.

Loading master keys

The following methods that can be used to load and set a master key are listed in order of strongest security first:

- TKE workstation

  This method is the most secure way to load and set a master key. It involves smart cards and smart card readers. The master key material can be generated directly onto the smart cards and cloned to backup smart cards. Each key owner generates their own key part and all owners must be present to complete the key loading process. The key material never needs to be displayed on a computer. For more information, see 2.2.3, "Trusted Key Entry workstation" on page 27.

- ICSF master key entry panels

  This method involves logging on to the system and going through the steps of generating random numbers, generating checksums, and loading the master key parts. For more information, see 4.3.5, "Loading the AES master key" on page 86. Each key owner can generate and load their key by using the ICSF panels.

  Because the key material is displayed in a panel, a process must be in place to ensure that the key material is not disclosed to unauthorized users. Also, the generated key material must be captured and saved for future reentry if disaster recovery is needed or new adapters are installed.

- ICSF PassPhrase Initialization (PPINIT) panels

  This method is the least secure way to load and set a master key. It involves entering a 16 - 64 character passphrase that generates and loads the master keys onto the Crypto Express adapters and initializes the key data sets. The use of PPINIT means that a single user controls the master key. Separate master key parts cannot be created by using PPINIT. You cannot rotate master keys with the PPINIT method. Therefore, it is not recommended for production.

A sample primary ICSF panel is shown in Figure 3-7 .

```
OPTION ===>
System Name:  SC74                                  Crypto Domain: 10
Enter the number of the desired option.

   1  COPROCESSOR MGMT -  Management of Cryptographic Coprocessors
   2  KDS MANAGEMENT   -  Master key set or change, KDS Processing
   3  OPSTAT           -  Installation options
   4  ADMINCNTL        -  Administrative Control Functions
   5  UTILITY          -  ICSF Utilities
   6  PPINIT           -  Pass Phrase Master Key/KDS Initialization
   7  TKE              -  TKE PKA Direct Key Load
   8  KGUP             -  Key Generator Utility processes
   9  UDX MGMT         -  Management of User Defined Extensions

      Licensed Materials - Property of IBM
      5650-ZOS Copyright IBM Corp. 1989, 2019.
      US Government Users Restricted Rights - Use, duplication or
      disclosure restricted by GSA ADP Schedule Contract with IBM Corp.

Press ENTER to go to the selected option.
Press END   to exit to the previous menu.
```

Figure 3-7 Sample primary ICSF panel

Managing operational keys

Operational keys are defined as all keys that are not master keys. They can be in a CKDS or in memory on the host system. Operational keys can be managed by way of a set of application programming interfaces (APIs), utility panels, or the UKO key management system. See 10.3, “Key management with UKO for z/OS” on page 231.

The operational keys that are used with data set encryption are stored in the CKDS and returned to DFSMS to perform the encryption and decryption with CPACF. Every record in the CKDS includes a corresponding key label.

z/OS data set encryption uses symmetric key encryption and the same key to encrypt and decrypt data.

Operational key owners

Different applications can need different sets of keys or different types of keys. The crypto or security administrator defines a process by which users can request operational keys for their applications. The process often includes the following information:

- Type of key that is needed
- Type of data (such as data sets) that is protected with the key
- How long the key is valid
- Number of needed keys
- Key label naming convention that might be used
- Metadata that is associated with the key
- Contact person regarding the key

Generating operational keys

An operational key can be generated by using the following methods:

- UKO for z/OS and UKO for Containers

UKO for z/OS and UKO for Containers support key templates for key definitions. Keys are generated and stored in the UKO repository before distributing the key to keystores (such as the CKDS).

- EKMF Workstation

  EKMF Workstation supports key templates for key definitions. Keys are generated and stored in the UKO repository before distributing those keys to other keystores (such as the CKDS). Although it is part of the UKO family, the EKMF Workstation is set up to meet a higher level of security (like PCI) than UKO for z/OS and UKO for Containers.

- ICSF CKDS KEYS utility

  ICSF (HCR77D1) provides a utility to generate AES DATA keys and browse keys in the CKDS. ICSF HCR77E0 extends this capability with support for generating AES Cipher keys, which is the recommended key type for data set encryption.

  The ICSF options for managing keys in the CKDS are shown in Figure 3-8 .

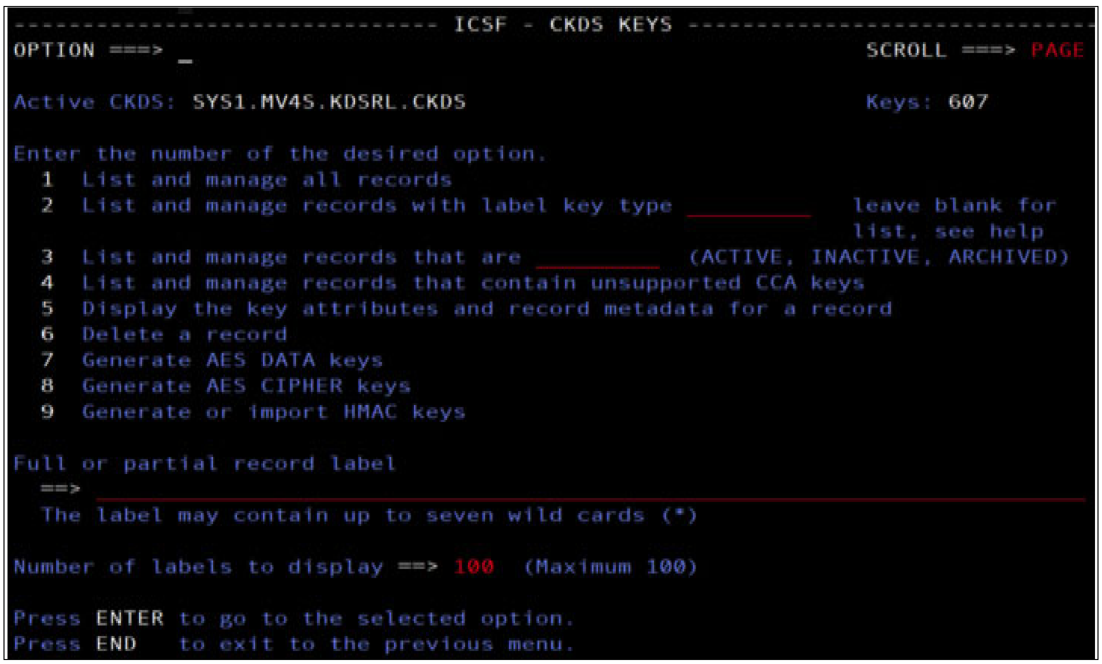


Figure 3-8 ICSF option 5.5 panel to manage keys in the CKDS

- ICSF Key Generator Utility Program (KGUP)

  This program defines and generates keys in bulk and stores them in the CKDS. The CKDS must be refreshed after the key is generated. For more information about KGUP, see **5.3.4, "Using CSFKGUP" on page 137**.

- ICSF Callable Services and APIs

  The CSNBKGN callable service generates AES DATA keys. The CSNBKGN2 callable service generates AES CIPHER keys (recommended), and the CSNBKRC2 callable service stores keys in the CKDS.

Key management activities and tools

The four tools that are supported for Key Management are ICSF, TKE, UKO, and GKLM. Only ICSF, TKE, and UKO[4] can be used to manage keys that are used for z/OS data set encryption (see Figure 3-9 on page 62).

[4] For UKO, see Chapter 10, "IBM Unified Key Orchestrator" on page 223

	Activity	ICSF	TKE	UKO	SKLM
Authorization Tasks	SAF Authorization (CSFKEYS and CSFSERV)	YES	YES	YES	SKLM for z/OS
	Key Auditing (master keys, operational keys)	YES	YES	OPERATIONAL KEYS	YES
Master Key Tasks	Master Key Entry	YES, PANELS	YES, SECURE	NO	NO
	Master Key Change	YES, PANELS	YES, SECURE	NO	NO
	Master Key Zeroize	NO, HMC / SE	YES	NO	NO
Basic KDS Tasks	Operational Key Record Creation (and naming)	YES	NO	YES, GUI-BASED	SEDs
	Operational Key Record Update	YES	NO	YES, GUI-BASED	SEDs
	Operational Key Record Deletion	YES	NO	YES, GUI-BASED	SEDs
Basic Key Tasks	Operational Key Generation	YES	SMALL SCALE	YES, GUI-BASED	SEDs
	Operational Key Import	YES	SMALL SCALE	YES, GUI-BASED. With EKMF Workstation	SEDs
	Operational Key Export	YES	NO	YES, GUI-BASED. With EKMF Workstation	SEDs
KDS Metadata Tasks	Operational Key Archival	YES	NO	NON-KDS,GUI-BASED	NO
	Operational Key Restore	YES	NO	NON-KDS,GUI-BASED	NO
	Operational Key Expiration	YES	NO	NON-KDS,GUI-BASED	NO
Maintenance Tasks	Rekeying encrypted data (operational keys)	YES	NO	NO	SEDs
Recovery Tasks	Disaster Recovery (master keys, operational keys)	YES	YES	OPERATIONAL KEYS	SEDs

SEDs = Self-encrypting devices

Figure 3-9 Key management activities

3.6.14 Determining key availability needs

Systems that are not sharing the CKDS can need access to shared encrypted data sets. In environments with different master keys, encryption keys cannot be read from one CKDS and written to another. If you use this environment, you might need to establish key importers and exporters on each system for securely sending keys between systems. For more information about this process, see Chapter 7, “Maintaining encrypted data sets” on page 157.

Tip: UKO manages keys on multiple systems and eliminates the need to manually transfer keys between systems.

3.6.15 Creating backups of keys

The CKDS is a critical component of z/OS data set encryption. Not only must you protect it from unauthorized users, you must also have backup procedures in place as part of your normal housekeeping routines to ensure that regular and valid copies or dumps of your CKDS are available.

Note: Any keys that were created between the time of the backup and the date of recovery are lost. Therefore, it is important that backups are taken regularly.

Manually back up the CKDS *before* a major operation (such as rotating data set encryption keys or manually transporting a key from one CKDS to another). After the operation is completed and the CKDS contents are verified, the backup can be deleted. If verification is unsuccessful, the CKDS can be recovered from the backup and ICSF can be refreshed to use the backup CKDS.

> **Tip:** The UKO key management system holds a copy (backup) of all keys managed by UKO in a separate key repository (Db2 database).

Backup and restoration include the following considerations:

- How often will the CKDS be backed up?
  - How often are new keys created?
  - Will keys be backed up before and after major key operations?
- How many backup versions will be kept?
- What tools will be used for backup?
- Will the CKDS be backed up at the data set or volume level or both?

The CKDS can be backed up and restored by using one of the following methods:

- Manual backup and restore
- Automated backup and restore:
  - Data set level
  - Volume level

For more information about these methods and tools, see 9.1, “Backing up and restoring data set encryption keys” on page 194.

3.6.16 Planning for disaster recovery

To plan for disaster recovery, you must determine whether your remote site meets the following requirements for data set encryption:

- Replicated copies of encrypted z/OS data sets are also encrypted and protected.
- Cryptographic coprocessor configurations are replicated across both sites, including master key and CKDS. This replication must be done initially and with every master key change. The process can be simplified by using TKE domain groups.

Cryptographic Key Management for disaster recovery replication of cryptographic key material for multi-site disaster recovery solutions is shown in Figure 3-10 .

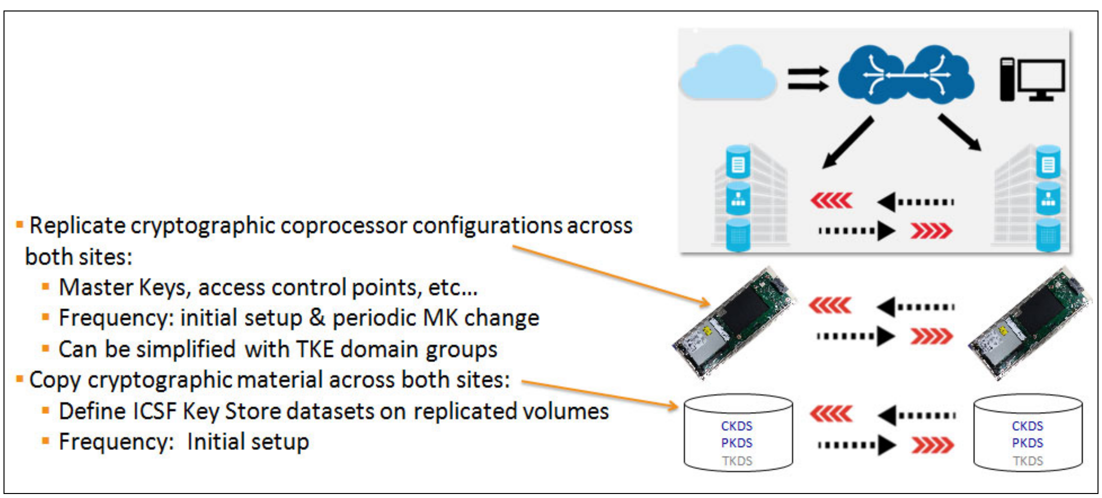


Figure 3-10 Multi-site disaster recovery solutions

3.7 General considerations

This section provides information about performing health checks and maintaining your data set encryption environment. It also includes steps for backing out of data set encryption should it be necessary.

For information about auditing and compliance see 3.7.4, "Auditing and compliance" on page 67 and 6.9, "Auditing Key management in UKO" on page 155.

3.7.1 Defining a maintenance policy

A robust corrective and preventive maintenance policy is one of the best ways to ensure your operating system and all associated products (including hardware, firmware, and middleware) are as stable and securable as possible. Resolving known defects quickly helps deliver a platform where any new issues can be resolved more quickly.

IBM flags fixes in numerous categories, including High Impact PERvasive (HIPER), Program temporary fix in Error (PE), and Pervasive. Security Vulnerability (SECINT) is of particular importance. SECINT is a classification (SOURCEID) of vulnerability PTFs that are related to the Common Vulnerability Scoring System (CVSS).

Security and Integrity Vulnerability APARs address problems that are associated with potential unauthorized access or potentially compromised system controls. Because of the highly sensitive nature of any such identified defects, the content is classified as "IBM Confidential" and access is restricted to those APARs. Access is permitted to authorized customers through the IBM z Systems Security Portal.

3.7.2 Performing z/OS health checks

The IBM Health Checker for z/OS can be accessed by way of the System Display and Search Facility (SDSF) by using the `CK` command. The health checker can help identify potential problems before they affect availability or cause outages. ICSF provides a set of health checks to inform of potential ICSF problems.

For more information about the relevant health checks, see the following publications (log in required):

- *IBM Health Checker for z/OS: User's Guide*, SC23-6843
- *z/OS Cryptographic Services ICSF Administration Guide*, SC14-7506

These publications provide more information about the following health checks:

- ICSF_COPROCESSOR_STATE_NEGCHANGE
- ICSF_DEPRECATED_SERV_WARNINGS
- ICSF_KEY_EXPIRATION
- ICSF_MASTER_KEY_CONSISTENCY
- ICSF_OPTIONS_CHECKS
- ICSF_UNSUPPORTED_CCA_KEYS
- RACF_SENSITIVE_RESOURCES
- RACF_CSFSERV_ACTIVE
- RACF_CSFKEYS_ACTIVE

You can also check the health checker components that start with ICSF.

Master key consistency check

Two health checks that are enabled are shown in Example 3-4 on page 65.

Example 3-4 Health check enabled

```
ICSF_MASTER_KEY_CONSISTENCY      IBMICSF          ACTIVE(ENABLED)    SUCCE
ICSF_OPTIONS_CHECKS              IBMICSF          ACTIVE(ENABLED)    SUCCE
```

The CHECK (IBMICSF, ICSF_MASTER_KEY_CONSISTENCY) command and its results are shown in Example 3-5 on page 65.

Example 3-5 CHECK (IBMICSF, ICSF_MASTER_KEY_CONSISTENCY)

```
CHECK(IBMICSF,ICSF_MASTER_KEY_CONSISTENCY)
SYSPLEX:    PLEX60    SYSTEM: SC60
START TIME: 11/08/2017 17:18:43.440431
CHECK DATE: 20120101  CHECK SEVERITY: MEDIUM

CSFH0014I (ICSF,ICSF_MASTER_KEY_CONSISTENCY): The master keys are
consistent across the current set of coprocessors.

END TIME: 11/08/2017 17:18:43.440540  STATUS: SUCCESSFUL
CHECK(IBMICSF,ICSF_OPTIONS_CHECKS)
SYSPLEX:    PLEX60    SYSTEM: SC60
START TIME: 11/08/2017 17:18:43.440428
CHECK DATE: 20160401  CHECK SEVERITY: MEDIUM
CHECK PARM: CHECKAUTH(NO)
CSFH0036I (ICSF,ICSF_OPTIONS_CHECKS): All ICSF options checked were set
to the specified values.

END TIME: 11/08/2017 17:18:43.442068  STATUS: SUCCESSFUL
```

Key expiration check

An SDSF Health Check exception for ICSF_KEY_EXPIRATION, which indicates the situation that a key is about to expire, is shown in Figure 3-11 .

Note: The key data sets must be in the KDSR format to have key material validity dates.

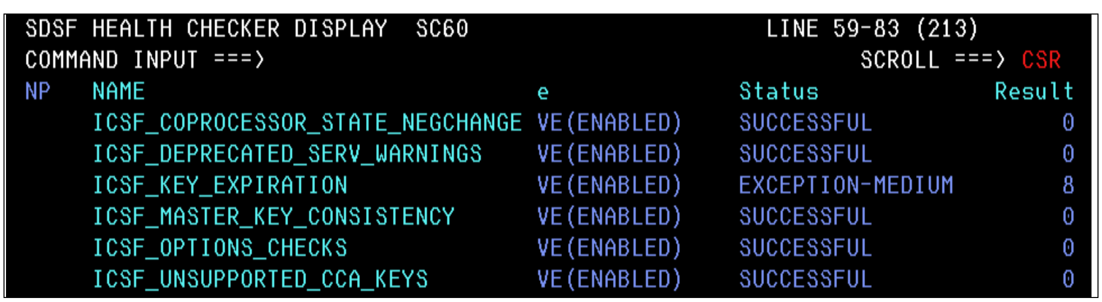
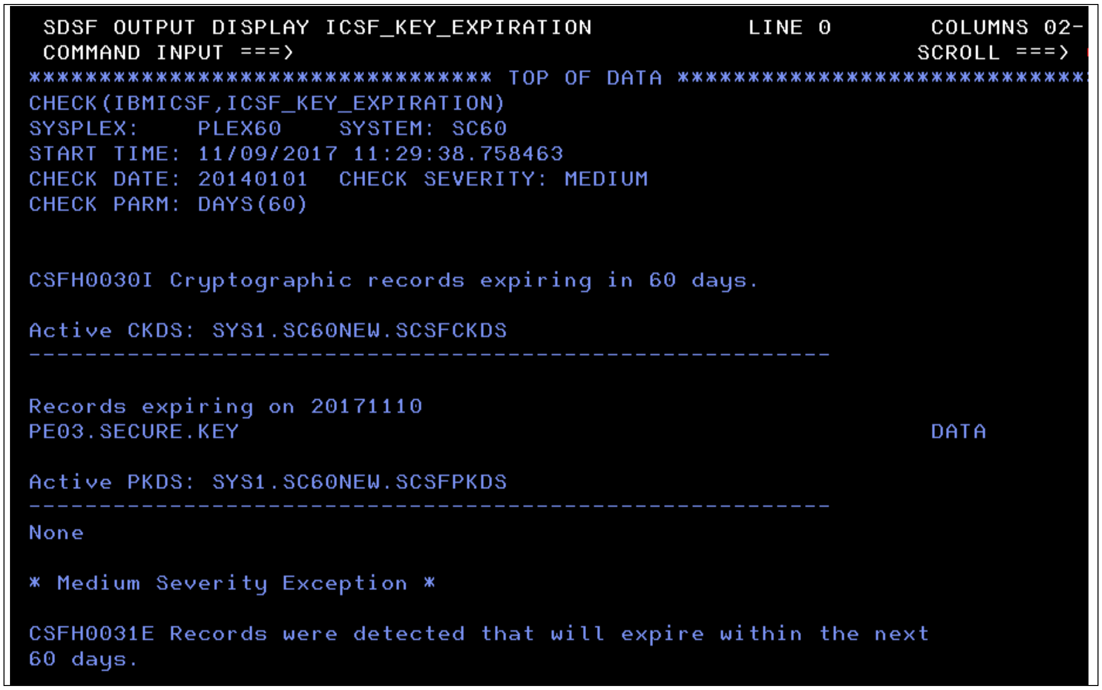


Figure 3-11 CK Status showing exception

More information about the ICSF key expiration is shown in Figure 3-12 .



```
 SDSF OUTPUT DISPLAY ICSF_KEY_EXPIRATION          LINE 0       COLUMNS 02-
 COMMAND INPUT ===>                                            SCROLL ===>
********************************* TOP OF DATA ****************************
CHECK(IBMICSF,ICSF_KEY_EXPIRATION)
SYSPLEX:    PLEX60    SYSTEM: SC60
START TIME: 11/09/2017 11:29:38.758463
CHECK DATE: 20140101  CHECK SEVERITY: MEDIUM
CHECK PARM: DAYS(60)


CSFH0030I Cryptographic records expiring in 60 days.

Active CKDS: SYS1.SC60NEW.SCSFCKDS
---------------------------------------------------------

Records expiring on 20171110
PE03.SECURE.KEY                                                 DATA

Active PKDS: SYS1.SC60NEW.SCSFPKDS
---------------------------------------------------------
None

* Medium Severity Exception *

CSFH0031E Records were detected that will expire within the next
60 days.
```

Figure 3-12   Display ICSF key expiration

Note: Consider adding system automation alerts for message CSFH0031E.

3.7.3 Backing out of z/OS data set encryption

Part of any implementation plan is the preparation for backing out, if required. It is recommended to plan for a simple or gradual implementation of z/OS data set encryption so that backout is straightforward and easy.

If the process is followed and you have the basic knowledge of your encryption criteria, the easiest way to backout is to copy the encrypted data set into a non-encrypted data set. If the implementation requires a backout for all encrypted data sets, this process must be done for each z/OS data set that was encrypted.

To ensure that no other z/OS data sets become encrypted, check that the FACILITY profile for `STGADMIN.SMS.ALLOW.DATASET.ENCRYPT` includes a `UACC(NONE)`. A backout procedure also includes removing DATAKEY statements from the RACF data set profiles.

Attention: Deleting key labels from the CKDS makes the associated encrypted data sets unusable.

The backout process includes the following steps:

- Changing FACILITY class for STGADMIN.SMS.ALLOW.DATASET.ENCRYPT to UACC(NONE)
- Removing data set encryption keys from the DFP segment in the RACF data set profiles

Copying encrypted data sets into non-encrypted data sets.

3.7.4 Auditing and compliance

A chain is only as strong as its weakest link. Encrypting data is good, but if malicious actors with access to the encrypted data can get to the encryption keys the encryption doesn't help.

To protect data, the means for protecting the data must also be secured.

There are several best practices that can be followed to reduce or eliminate the risk of compromise by malicious actors. These practices are reflected in various standards and regulations (see 1.3, “Standards and regulations overview” on page 4).

Each standard and regulation have different levels of detail and focus, but overall, their goal is to ensure the protection of information and to reduce or eliminate the possibility of a security compromise.

If a regulation applies to the encrypted data, the key management process may also be within the scope of the regulation and be required to be compliant.

Having a log is important for monitoring which operations/actions that have been done and when required to be compliant with regulations, the audit log becomes a tool which the organization can use to demonstrate compliance.

The UKO key management system provides an audit log, which shows what actions were done.

In addition, UKO key templates can be used to document compliance with a process/policy with specific requirements for keys (see 6.9, “Auditing Key management in UKO” on page 155).

Auditing the z/OS data set encryption environment and the UKO key management system is described in Chapter 6, “Auditing z/OS data set encryption” on page 149.

4

Preparing for z/OS data set encryption

In preparation of deploying data set encryption, it is important to understand which settings are mandatory or of particular use to your installation. This issue also includes deciding which utilities, tools, and program offerings can assist or complement the setup and management of your environment.

This chapter includes the following topics:

4.1 Data set configuration

This section describes the configuration process for encrypted data sets. The following data set types are supported for data set encryption:

- VSAM extended format data sets
- Sequential extended format data sets
- PDSE data sets
- Sequential basic format data sets
- Sequential large format data sets

Where possible, the best practice for VSAM and sequential data sets is for encrypted data sets to be created as extended format and to be compressed in the access methods before encryption.

> **Note:** At the time of this writing, the latest ICSF FMID available is HCR77F0. Since the release of z/OS 2.5, ICSF is not delivered as a web download. It is part of z/OS itself. For more information, see z/OS: ICSF Version and FMID Cross Reference.

4.1.1 Migrating to extended format data sets

A storage administrator must validate the current list of data sets that are eligible for encryption and the current ACS routines, which are used to determine storage and allocation requirements. For example, some data sets might need to be converted to extended format before they are eligible for encryption.

Do not automatically use SMS data classes for data set encryption because this configuration can cause all new allocations to be encrypted before the environment is fully prepared to implement.

Also, be aware of the implications of the use of SMS data classes for the allocation of encrypted data sets after the crypto environment is in place because it gives the storage administrator the authority to manage encryption of data sets, which might not be intended.

For more information about other methods of implementation that reference how SMS can be used for setting up, see Chapter 5, “Deploying z/OS data set encryption” on page 125.

4.1.2 Compressing data sets before encryption

Encrypted data does not compress; therefore, data must be compressed before it is encrypted wherever possible. When enabled for compression, DFSMS access methods compress data sets before encryption.

Data sets eligible for compression

The following data sets can be compressed:

- Sequential extended format data sets support generic, tailored, or zEDC compression.
- A VSAM extended format KSDS supports generic compression (only KSDS can be compressed format).
- Data members (Partitioned Data Set Extended) support zEDC compression.

Compressing data sets

To prepare for compression on new data set allocation, set up an SMS policy to request compression. A storage administrator can update the following information:

- Specific data classes through ISMF to request compression by using the COMPACTION option
- Automatic class selection (ACS) routines through Interactive Storage Management Facility (ISMF) to select data classes that are enabled for compression

4.2 RACF configuration

New authorization checks might need to be performed for users and applications to read and write encrypted data sets. Security administrators fully control over who is allowed to enable (and disable) the encryption of data sets. Security administrators also control who can access the following components:

- Data set
- Encryption key
- Encryption services

4.2.1 Restricting data set encryption to security administrators

By using the FACILITY class, data set encryption can be restricted to only security administrators. This profile with the universal access set to NONE makes data set encryption is unavailable to users who are not explicitly authorized to use it:

```
RDEFINE FACILITY STGADMIN.SMS.ALLOW.DATASET.ENCRYPT UACC(NONE)
```

Restricting nonsecurity administrators from encrypting data sets ensures that administrators can be sure that encryption is not enabled before the environment is fully configured. Administrators can also be sure that a key management policy is in place, and that they have oversight on which data sets are encrypted with which keys.

4.2.2 Defining DATASET, CSFSERV, CSFKEYS, and other resources

This section describes the following access controls to consider based on your policies for data set encryption:

- Protecting the cryptographic key data set (CKDS)

  The data encryption keys that are used for data set encryption are stored in a CKDS.

  Access to the VSAM data set for the CKDS is provided by the DATASET resource class. For more information about the DATASET class, see *z/OS Security Server RACF Security Administrator's Guide*.

  > **Note:** The DATASET resource that protects the CKDS must not be encrypted with data set encryption because the key is rendered inaccessible at ICSF startup. The keys in the CKDS can be encrypted by using a master key. For more information, see 1.6.3, "IBM Z cryptographic system" on page 16.

The RACF profile that is used for protecting the CKDS data set is listed in Table 4-1 on page 72.

Table 4-1 RACF profile for protecting the CKDS data set

Class	Entry in class	Description	Examples
DATASET	*<KDS_dsname>*	Allow users access to the CKDS and PKDS data sets	PERMIT '*<KDS_dsname>*' ID(*groupid*) ACCESS(READ)

Protecting encrypted data setsAccess to data sets is provided by the DATASET resource class. For more information about the DATASET class, see *z/OS Security Server RACF Security Administrator's Guide*.

For more information about data set encryption, see other resource classes that might be needed, such as CSFKEY, CSFSERV, and FACILITY.

RACF profiles that are used for protecting encrypted data sets are listed in Table 4-2.

Table 4-2 RACF profiles for protecting encrypted data sets

Class	Entry in class	Description	Examples
DATASET	*<dsname>*	Allow users access to the data set.	PERMIT '<dsname>' ID(*groupid*) ACCESS(READ)
	DFP segment to profile	Allow encryption of data set through DFSMS access methods by using a key label that is specified in DFP segment. **Note 1:** For data set encryption, a key label can also be supplied by way of other sources, such as JCL and SMS data class. **Note 2**: In z/OS V3R2, a new field ENCRYPTTYPES was introduced to further filter what datasets are eligible to encryption. See z/OS Security Server RACF Command Language Reference.	ALTDSD '<dsname>' UACC(NONE) DFP(RESOWNER(owner) DATAKEY(keylabel_name))
CSFKEYS	**	Protects access to crypto key, enable, and disable, protected key use *RECOMMENDED* not to use ICSF segment for this profile.	RDEFINE CSFKEYS name UACC(NONE)
	keylabel_name	Protects access to the corresponding key label. Must include ICSF segment with options that are required for data set encryption: ▶ SYMCPACFWRAP(YES) and ▶ SYMCPACFRET(YES).	RDEFINE CSFKEYS keylabel_name UACC(NONE) ICSF(SYMCPACFWRAP(YE S) SYMPACFRET(YES))
CSFSERV	*	General access to ICSF services *REQUIRED* UACC(NONE) is recommended	RDEFINE CSFSERV * UACC(NONE)

Class	Entry in class	Description	Examples
	CSFKRR2	CKDS Key Record Read2 callable service. *REQUIRED* for data set encryption *only* when CHECKAUTH(YES) is specified in the ICSF installation options data set. CHECKAUTH(NO) is the default.	RDEFINE CSFSERV CSFKRR2 UACC(READ)
FACILITY	STGADMIN.*	Generic entry to protect access to STGADMIN.*, assumed to be created previously.	RDEFINE STGADMIN.* UACC(NONE)
	STGADMIN.SMS.ALLOW.DATASET.ENCRYPT	Recommended to be defined with UACC(NONE) to restrict any users from attempting to allocate a data set before encryption is fully implemented. With at least read authority, allows creating encrypted data sets by using a key label that is specified through a source other than the DATASET DFP segment. **Note:** This resource does not protect against creating encrypted data sets by using a key label in the DATASET DFP segment.	RDEFINE FACILITY STGADMIN.SMS.ALLOW.DATASET.ENCRYPT UACC(NONE)
	STGADMIN.SMS.FAIL.INVALID.DSNTYPE.ENC	Control whether an allocation succeeds or fails if a key label is specified for a DASD data set that is not a supported data set type for data set encryption. Recommended to be defined with UACC(NONE) to prevent allocations from failing if a key label is specified for a data set that is not a supported data set type for data set encryption.	RDEFINE FACILITY STGADMIN.SMS.FAIL.INVALID.DSNTYPE.ENC UACC(NONE)
	STGADMIN.SMS.ALLOW.PDSE.ENCRYPT	When the resource name is fully defined, so the system can treat PDSEs as a supported data set type for data set encryption. Does not require read authority to take effect.	RDEFINE FACILITY STGADMIN.SMS.ALLOW.PDSE.ENCRYPT UACC(NONE)
	STGADMIN.SMS.ALLOW.DATASET.SEQ.ENCRYPT	When the resource name is fully defined, the system can treat sequential basic and large format data sets as supported data set types for data set encryption. Does not require read authority to take effect.	RDEFINE FACILITY STGADMIN.SMS.ALLOW.DATASET.SEQ.ENCRYPT UACC(NONE)

Class	Entry in class	Description	Examples
	STGADMIN.IGG.DIRCAT	Ability to LISTCAT a data set, which shows the encryption attribute *OPTIONAL*	
FIELD	DATASET.DFP.DATAKEY	Controls specification of DATAKEY in DFP segment. Required if FIELD entries are present for data sets.	RDEFINE FIELD DATASET.DFP.DATAKEY UACC(NONE)
FACILITY	STGADMIN.*	Generic entry to protect access to STGADMIN.* Assumed to be created previously.	RDEFINE STGADMIN.* UACC(NONE)
	STGADMIN.SMS.ALLOW.DATASET.ENCRYPT	Recommended to be defined with UACC(NONE) to restrict any users from attempting to allocate a data set before encryption is fully implemented. With at least read authority, allows creating encrypted data sets by using a key label that is specified through a source other than the DATASET DFP segment. **Note:** This resource does not protect against creating encrypted data sets by using a key label in the DATASET DFP segment.	RDEFINE FACILITY STGADMIN.SMS.ALLOW.DATASET.ENCRYPT UACC(NONE)
	STGADMIN.SMS.FAIL.INVALID.DSNTYPE.ENC	Control whether an allocation succeeds or fails if a key label is specified for a DASD data set that is not a supported data set type for data set encryption. Recommended to be defined with UACC(NONE) to prevent allocations from failing if a key label is specified for a data set that is not a supported data set type for data set encryption.	RDEFINE FACILITY STGADMIN.SMS.FAIL.INVALID.DSNTYPE.ENC UACC(NONE)
	STGADMIN.SMS.ALLOW.PDSE.ENCRYPT	When the resource name is fully defined, allows the system to treat PDSEs as a supported data set type for data set encryption. Does not require read authority to take effect.	RDEFINE FACILITY STGADMIN.SMS.ALLOW.PDSE.ENCRYPT UACC(NONE)
	STGADMIN.SMS.ALLOW.DATASET.SEQ.ENCRYPT	When the resource name is fully defined, allows the system to treat sequential basic and large format data sets as supported data set types for data set encryption. Does not require read authority to take effect.	RDEFINE FACILITY STGADMIN.SMS.ALLOW.DATASET.SEQ.ENCRYPT UACC(NONE)
	STGADMIN.IGG.DIRCAT	Ability to LISTCAT a data set, which shows the encryption attribute *OPTIONAL*	

Class	Entry in class	Description	Examples
FIELD	DATASET.DFP.DATAKEY	Controls specification of DATAKEY in DFP segment. Required if FIELD entries are present for data sets.	RDEFINE FIELD DATASET.DFP.DATAKEY UACC(NONE)

- Protecting key labels

  Access to key labels is provided by the CSFKEYS resource class. For more information about the CSFKEYS class, see *z/OS Cryptographic Services ICSF Administrator's Guide.*

  See other resources that might be used when key labels and associated cryptographic keys, such as XFACILIT and XCSFKEY, are managed.

  RACF profiles for protecting key labels are listed in Table 4-3.

Table 4-3 RACF profiles for protecting key labels

Class	Entry in class	Description	Examples
CSFKEYS	**	Access to crypto key, enable/disable, protected key use *RECOMMENDED* not to use ICSF segment for this profile	RDEFINE CSFKEYS name UACC(NONE)
	DATASET.<dataset_resource>.ENCRKEY.<seqno>	Protects access to the corresponding key label DATASET.<dataset_resource>.ENCRKEY.<seqno>	RDEFINE CSFKEYS keylabel_name UACC(NONE) ICSF(SYMCPACFWRAP(YES) SYMPACFRET(YES))
XFACILIT	CSF.CSFKEYS.AUTHORITY.LEVELS.FAIL CSF.CSFKEYS.AUTHORITY.LEVELS.WARN	Enables granular key label access; recommended to activate at least WARNING mode and then FAIL mode after ready to implement	
	CSF.XCSFKEY.ENABLE.AES	Allows symmetric key label export for AES keys by using profiles in XCSFKEY class	
	CSF.CKDS.TOKEN.NODUPLICATES		Not allow duplicates of key labels *RECOMMENDED*
	CSF.KDS.KEY.ARCHIVE.USE	Allows users to use archived keys in cryptographic operations with warning messages and SMF records indicating use. For more information, see 3.6.8, "Deciding whether to archive or delete keys" on page 53.	
	CSF.SSM.ENABLE	Ability for users to change to secure mode	
XCSFKEY	*	Used when exporting keys; transferring a secure symmetric key to encryption under an RSA key Checks for entry XFACILIT CFS.XCSFKEY.ENABLE.AES, which is used for exporting of symmetric keys	

- Protecting crypto services

Access to cryptographic services is provided by the CSFSERV resource class. For more information about the CSFSERV class, see *z/OS Cryptographic Services ICSF Administrator's Guide*.

RACF profiles for protecting crypto services are listed in Table 4-4.

Table 4-4 RACF profiles for protecting crypto services

Class	Entry in class	Description	Examples
CSFSERV	*	General access to ICSF services *REQUIRED* UACC(NONE) is recommended	RDEFINE CSFSERV * UACC(NONE)
	CSFBRCK	ICSF panel utility - CKDS KEYS - Browse CKDS (option 5.5)	
	CSFKGUP	ICSF panel utility- Key Generator Utility Program	
	CSFKGN	Key Generate callable service	
	CSFKRR2	CKDS Key Record Read2 callable service	
	CSFKRC2	CKDS Key Record Create2 callable service	
	CSFREFR	ICSF panel utility - Refresh CKDS using ISPF panels	
	CSFCRC	Coordinated Change Master Key and Coordinated Refresh KDS panel utilities, Coordinated KDS Administration callable service *RECOMMENDED* to turn on AUDIT(ALL)	RDEFINE CSFSERV CSFCRC UACC(NONE) AUDIT(ALL)
	CSFDKCS	ICSF panel utility - Load master keys using ISPF panels *RECOMMENDED* to turn on AUDIT(ALL)	See CSFCRC example
	CSFSMK	ICSF panel utility - Set master keys using ISPF panels *RECOMMENDED* to turn on AUDIT(ALL)	See CSFCRC example
	CSFCMK	ICSF panel utility - Change master keys using ISPF panels *RECOMMENDED* to turn on AUDIT(ALL)	See CSFCRC example

- Protecting operator commands

  Access to MVS operator commands is provided by the OPERCMDS resource class. For more information about the OPERCMDS class, see *z/OS Security Server RACF Security Administrator's Guide*.

  RACF profiles for protecting operator commands with a brief description of each class, a list of optional profiles, and a sample entry, are listed in Table 4-5 on page 77.

Table 4-5 RACF profiles for protecting operator commands

Class	Entry in class	Description	Examples
OPERCMDS	MVS.DISPLY.ICSF	Allow users to enter D ICSF commands	RDEFINE MVS.DISPLAY.ICSF CLASS(OPERCMDS) UACC(NONE)
	MVS.SETICSF.*	Allow users to enter SETICSF commands, for example SETICSF OPT,REFRESH	RDEFINE MVS.SETICSF CLASS(OPERCMDS) UACC(NONE)

- Ability to LISTCAT an encrypted data set to show catalog encryption attribute

  Access to LISTCAT is provided by the STGADMIN.IGG.DIRCAT resource in the FACILITY resource class. For more information about the FACILITY class, see Storage Administration (STGADMIN) Profiles in the FACILITY Class.

  An RACF profile for showing catalog encryption attribute is listed in Table 4-6.

Table 4-6 RACF profiles for showing catalog encryption attribute

Class	Entry in class	Description	Examples
FACILITY	STGADMIN.*	Generic entry assumed to be created previously	UACC(NONE)
	STGADMIN.IGG.DIRCAT	Ability to LISTCAT a data set that shows the encryption attribute	

Protecting ICSF start and stop

It is imperative to have tight controls in place to protect the use of z/OS and JES2 commands to **START**, **STOP**, or **FORCE** ICSF. You might also want to control the use of the command **SETICSF** and perform the following tasks:

- Ensure that the OPERCMDS class is active and RACLISTed and that RACLIST processing is enabled.
- Define the OPERCMDS class profile by using a security product; for example, RACF.
- Grant ICSF access to the OPERCMDS class and then refresh the OPERCMDS class.

Some RACF profiles that restrict who can issue the **START**, **STOP**, **FORCE**, or **SETICSF** operator commands are described next (see Example 4-1).

Note: ICSF does not support the CANCEL or MODIFY operator commands.

Example 4-1 Operator commands to restrict

```
MVS.START.STC.ICSF.*
MVS.START.STC.ICSF

MVS.STOP.STC.ICSF.*
MVS.STOP.STC.ICSF

MVS.FORCE.STC.ICSF.*
MVS.FORCE.STC.ICSF
```

```
MVS.SETICSF.*
```

Replace ICSF with your actual proc name; for example, CSF if it is different in your installation.

Typically, RACF generic profiles are available that restrict who can issue the **MVS START**, **STOP**, and **FORCE** commands. The specific **PERMIT** command that you must issue depends on how your RACF administrator defined the profiles in class OPERCMDS to protect these commands.

Alternatively, you can create profiles in class OPERCMDS and permit the necessary access. The sample job performs this task by defining new profiles (see Example 4-2).

Example 4-2 Sample job defining new profiles

```
RDEFINE OPERCMDS MVS.START.STC.ICSF*.** UACC(NONE) OWNER(SECADM)
PERMIT MVS.START.STC.ICSF*.** CLASS(OPERCMDS) RESET
PERMIT MVS.START.STC.ICSF*.** CLASS(OPERCMDS) +
ID(ICSFADM) ACCESS(UPDATE)
SETROPTS RACLIST(opercmds) REFRESH
```

You must repeat the same commands and profiles for the **STOP**, **FORCE**, and **SETICSF** commands.

For more information about protecting operator commands, see Planning MVS operations.

For more information about controlling the use of operator commands, see Administering the use of operator commands.

For more information related to these services, see z/OS Cryptographic Services.

System Display and Search Facility

System Display and Search Facility (SDSF) generates console commands (z/OS or JES) when the rules you established to SDSF dictate that the user is allowed to stop or purge the job. If the JESSPOOL class is active, the user's authority to the JESSPOOL profile that is protecting the job determines whether SDSF generates the command on behalf of the user. If JESSPOOL is inactive, the native SDSF controls in ISFPARMs or the SDSF statements control if the user is authorized to the job.

If the user is authorized from the SDSF perspective, SDSF generates and issues the command on the user's behalf under the users own ID, but originating from the SDSF console. By using this method, you can authorize the console commands that are generated by SDSF, as shown in the following example:

```
PERMIT JES2.STOP.ICSF CLA(OPERCMDS) ID(*) WHEN(CONSOLE(SDSF))
```

This command allows an SDSF generated stop command to run.

If the user directly issues the command on their own (you gave them console access or the `/` command in SDSF), it does not originate from CONSOLE SDSF.

4.2.3 Setting a policy to control the use of archived keys

Archiving data set encryption keys is preferred over deletion because archived keys can be recalled later, if needed.

When a key is archived, an SMF Type 82 Subtype 30 record is created for each attempted use and an optional job log message is displayed.

You can decide to allow or deny archived keys to be used for cryptographic operations with the `CSF.KDS.KEY.ARCHIVE.USE` resource in the XFACILIT class. When that resource is defined, ICSF allows archived keys to be used for crypto operations. When that resource is undefined, ICSF fails any attempts to use an archived key.

4.2.4 Configuring the RACF environment for key generation

The following methods are available to generated keys. RACF authorization is required to perform key generation by using the following methods:

- UKO
- ICSF Panels
- ICSF APIs
- ICSF KGUP
- TKE

For more information, see 5.3, “Generating a secure 256-bit data set encryption key” on page 131.

CSFSERV

For all methods, the CSFSERV class must be active and RACLISTed. Also enable it for generic use. To enable CSFSERV for generics, use:

```
SETROPTS CLASSACT(CSFSERV)
SETROPTS RACLIST(CSFSERV)
SETROPTS GENERIC(CSFSERV)
```

Initially set the CSFSERV class to disallow access to all users, as shown in the following example:

```
RDEFINE CSFSERV * UACC(NONE)
```

Granting access to CSFSERV resources depends on the key generation method. The process includes the following steps:

1. Define the profiles.
   - For ICSF KGUP:
     ```
     RDEFINE CSFSERV CSFKGUP UACC(NONE)
     RDEFINE CSFSERV CSFKGN UACC(NONE)
     ```
   - For ICSF Panel Utilities and APIs:
     ```
     RDEFINE CSFSERV CSFKGN UACC(NONE)
     RDEFINE CSFSERV CSFKRC2 UACC(NONE)
     RDEFINE CSFSERV CSFKRR2 UACC(NONE)
     ```
2. Grant the users (preferably groups) access permission as shown in the following example:
   - For ICSF KGUP:
     ```
     PERMIT CSFKGUP CLASS(CSFSERV) ID(groupid) ACCESS(READ)
     PERMIT CSFKGN CLASS(CSFSERV) ID(groupid) ACCESS(READ)
     ```
   - For ICSF Panel Utilities and APIs
     ```
     PERMIT CSFKGN CLASS(CSFSERV) ID(groupid) ACCESS(READ)
     PERMIT CSFKRC2 CLASS(CSFSERV) ID(groupid) ACCESS(READ)
     ```

```
PERMIT CSFKRR2 CLASS(CSFSERV) ID(groupid) ACCESS(READ)
```

3. Refresh the class, as shown in the following example:

```
SETROPTS RACLIST(CSFSERV) REFRESH
```

CSFKEYS

For all methods, the CSFKEYS class must be active and RACLISTed and also enabled for generic use. To enable CSFSERV for generics, use:

```
SETROPTS CLASSACT(CSFKEYS)
SETROPTS RACLIST(CSFKEYS)
SETROPTS GENERIC(CSFKEYS)
```

Activating the CSFKEYS class is the only preparation step. For more information about granting access to the CSFKEYS resources, see 5.7, “Granting access to encrypted data sets” on page 144.

4.3 ICSF configuration

z/OS data set encryption requires key labels and data-encrypting keys to be defined in an ICSF CKDS. The data-encrypting keys can be DATA keys or CIPHER keys. CIPHER keys require Crypto Express and ICSF. To create an encrypted data set, a key label must be supplied on new data set allocation. The key label must point to an AES 256-bit data-encrypting key to be used for encryption and decryption.

For each encrypted data set, its key label is stored in the data set catalog. The key label is not typically considered sensitive information; however, it identifies the data set encryption key, which is sensitive. Therefore, the use of secure keys is recommended.

Data-encrypting keys are enciphered with a master key to create a secure key that is stored in CKDS. The master key is stored securely in the HSM of an assigned Crypto Express adapter. When a secure key is later accessed, the master key is used in the transformation to a protected key, as described in 3.6.6, “Using protected keys for high-speed encryption” on page 51.

Data set encryption is flexible, which allows as much granularity as wanted when key labels are identified for data sets. The number of key labels and encryption keys that are used across z/OS data sets is unlimited.

The data set encryption management process is provided by z/OS through SAF (interfacing with RACF or an equivalent access control software security system, such as CA ACF2 for z/OS), ICSF, DFSMS, and UKO.

Key labels that are used to retrieve secure keys

Every record in the ICSF CKDS includes an associated key label. When applications or z/OS components start ICSF callable services, the application can specify a key label by way of a parameter to identify the key for the callable service to use.

After a key label is stored in the catalog for an encrypted data set, it cannot be altered. Any subsequent change to SAF or RACF data set profile or data class does not affect existing data sets. This approach ensures that data cannot be tampered with.

SAF or RACF policies define the users that are authorized to use distinct key labels and ICSF callable services. The CSFKEYS resource class controls access to cryptographic keys in the

ICSF CKDS and enables the use of protected key. The CSFSERV resource class controls access to ICSF callable services.

How a key label is used to retrieve a secure key from CKDS to be unwrapped by a Crypto Express adapter is shown in Figure 4-1.

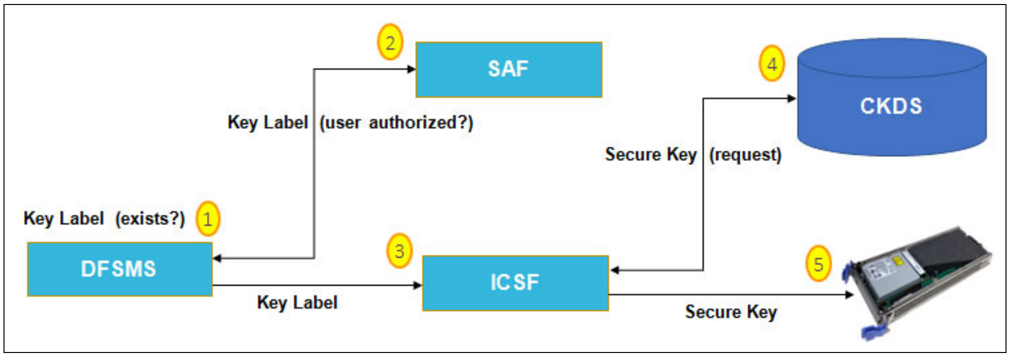


Figure 4-1 Using a key label to retrieve a secure key from CKDS

The following key label process steps are shown in Figure 4-1:

1. At data set allocation, DFSMS checks if a key label exists in the DATASET class profile, which protects the data set and saves the key label.
2. At data set open, DFSMS checks if the user is authorized to the class resource that protects the saved key label.
3. DFSMS specifies the saved key label to ICSF and requests to retrieve the secure key from the CKDS.
4. ICSF uses the key label that is specified by DFSMS to locate the secure key in the CKDS.
5. ICSF calls the Crypto Express adapter to convert the secure key to a protected key by using the master key.

For more information about creating a protected key from a secure key, see 3.6.6, “Using protected keys for high-speed encryption” on page 51.

4.3.1 Configuring Crypto Express adapters

To use Crypto Express adapters with ICSF, they must be configured online and assigned to the appropriate LPARs. Each Crypto Express adapter supports multiple cryptographic domains. Each domain can be assigned a unique master key, which prevents access to the key material from other domains.

The maximum number of domains matches the maximum number of LPARs that is available on the server. For example, the IBM z17 supports up to 85 LPARs; therefore, the Crypto Express adapters support up to 85 domains.

For z/OS data set encryption, at least two Crypto Express adapters must be configured as CCA coprocessors.

4.3.2 Creating a Common Record Format (KDSRL) CKDS

A CKDS can be created or converted to a common record format (KDSR) format. Any system that shares the KDSR format key data set must be running ICSF FMID HCR77D1 or later. For more information, see 3.5.3, “Using the Common Record Format (KDSRL) cryptographic key data set” on page 46.

The first step is to allocate a KDSRL format CKDS. The CKDS must be a key-sequenced data set and must be allocated on a permanently resident volume.

The sample job in SYS1.SAMPLIB(CSFCKDS) can be used as a template. Our JCL is shown in Example 4-3.

Example 4-3 Job to define a new cluster

```
//DEFINE  EXEC PGM=IDCAMS,REGION=4M
//SYSPRINT DD SYSOUT=*
//SYSIN DD *
  DEFINE CLUSTER (NAME(PLEX75.SHARED.COPY.SCSFCKDS)    -
                 VOLUMES(BH5CAT)          -
                 RECORDS(500 50)          -
                 RECORDSIZE(372,32768)    -
                 KEYS(72 0)               -
                 FREESPACE(10,10)         -
                 SHAREOPTIONS(2,3))       -
           DATA (NAME(PLEX75.SHARED.COPY.SCSFCKDS.DATA)       -
                 BUFFERSPACE(100000)      -
                 ERASE)                   -
                 CISZ(32768)              -
           INDEX (NAME(PLEX75.SHARED.COPY.SCSFCKDS.INDEX))
/*
```

ICSF CKDS must include a single volser that is specified in the VOLUMES parameter. Do not code more than one volume or specify VOLUME(*).

Converting a CKDS

If CKDS is available to convert to KDSR or KDSRL format, the conversion can be done by calling the CSFCRC callable service or by using the ICSF panels. During the conversion, all updates to the key data set that is converted are suspended.

At the end of the conversion, all systems in the sysplex that share the key data set use the KDSR format key data set as the active key data set. All new updates are made to the KDSR format key data set.

Complete the following steps to convert a key data set to KDSR format by using the ICSF panels:

1. In the ICSF Primary Menu panel, select **option 2 KDS MANAGEMENT** and press Enter.
2. In the ICSF Key Data Set Management panel, select the type of key data set **option 6 COORDINATED CKDS CONVERSION** and press Enter.
3. In the next panel, select the **COORDINATED xKDS CONVERSION** option and press Enter.
4. When the ICSF Coordinated KDS conversion panel appears, complete the required fields and press Enter.
5. Specify the new KDSRL format CKDS to switch to (see Example 4-4 on page 83) then press Enter to perform a dynamic, nondisruptive conversion.

Example 4-4 Coordinated KDS conversion panel

```
---------------------- ICSF - Coordinated KDS conversion ----------------------
 COMMAND ===>

 To perform a coordinated KDS conversion, enter the KDS names below
 and optionally select the rename option.

     KDS Type ===> CKDS

   Active KDS ===> 'PLEX75.SHARED.SCSFCKDS'

      New KDS ===> 'PLEX75.SHARED.NEW'SCSFCKDS'

           Rename Active to Archived and New to Active (Y/N) ===> Y

           Archived KDS ===> 'PLEX75.SHARED.OLD'SCSFCKDS'

           Create a backup of the converted KDS (Y/N) ===> N

           Backup KDS ===>

 Press ENTER to perform a coordinated KDS conversion.
 Press END to exit to the previous menu.
```

For more information, see Migrating to the common record format (KDSR) key data set.

4.3.3 CSFPRMxx and installation options

The CSFPRMxx member in PARMLIB contains the installation options that are used for ICSF initialization. A sample CSFPRMxx for z/OS data set encryption is shown in Example 4-5. The CKDS in our example is shared with other systems.

Example 4-5 CSFPRMxx options

```
CKDSN(PLEXNAME.NEW.SCSFCKDS)
SYSPLEXCKDS(YES,FAIL(YES))
CHECKAUTH(NO)
AUDITKEYLIFECKDS(TOKEN(YES),LABEL(YES))
AUDITKEYUSGCKDS(TOKEN(YES),LABEL(YES),INTERVAL(24))
KEYARCHMSG(YES)
KDSREFDAYS(1)
STATS(ENG,SRV,ALG)
STATSFILTERS(NOTKUSERID)
```

If you do not intend to share the CKDS across multiple systems, the following option is used:

```
CKDSN($SYSNAME.NEW.SCSFCKDS)
```

The SYSPLEXCKDS option can be used for a single system or multi-system sysplex.

Display the current options and defaults by using the ICSF utility panel option 3.1 (OPSTAT options) or the **D ICSF,OPT** command. For more information, see 8.2, “Viewing ICSF options” on page 176.

For a complete list and more information about default values, see *z/OS Cryptographic Services Integrated Cryptographic Service Facility System Programmer's Guide*, SC14-7507.

The following settings are available:

- CKDSN

  The name of the VSAM data set for the CKDS.

- SYSPLEXCKDS

  Enables the current ICSF instance to join the CKDS sysplex and synchronize updates with other ICSF instances with the same CKDSN.

- CHECKAUTH

  The recommended setting for CHECKAUTH is NO. Access to CSFKEYS, CSFSERV, and XCSFKEYS resources are not checked for authorized callers. This setting increases crypto throughput and improves performance.

- AUDITKEYLIFECKDS, AUDITKEYUSGCKDS

  Enables auditing for key lifecycle events and key usage events.

- KEYARCHMSG

  Specify YES for ICSF to issue a console message when an archived key is attempted to be used in a cryptographic operation. ICSF creates an SMF Type 82 Subtype 30 record, which indicates the attempted use whether KEYARCHMSG is enabled or disabled.

- KDSREFDAYS

  The common record format (KDSR) for the CKDS can be used to take advantage of the extra metadata and use the key usage and key lifecycle audit records. The key usage, key lifecycle, and crypto usage statistics must be turned on in the CSFPRMxx PARMLIB member.

  KDS Key Utilization Statistics provides the following capabilities:

  – A new key record format (KDSR) for internally saving metadata and statistics about each cryptographic key is available.

  – The Coordination KDS Administration (CSFCRC and CSFCRC6) callable service was enhanced to perform a coordinated conversion of an old format *KDS to the new KDSR format. Included in this new record format is a section tracked the *last referenced* date for each cryptographic key.

  – The reference date is the last time that a record was used in a cryptographic operation or read, such that the retrieved key might be used in a cryptographic operation. Because the read is interpreted as a show of interest, the reference date is updated.

  – A new ICSF startup option, KDSREFDAYS(n), was added that specifies (in days) how often a record is written for a reference date and time change.

  KDSREFDAYS(0) means that ICSF does not track key reference dates. The default is KDSREFDAYS(1) and the maximum value that is allowed is KDSREFDAYS(30).

- STATS, STATSFILTERS

  Crypto usage tracking can be controlled with the STATS and STATSFILTERS options. The STATS option enables usage tracking for various cryptographic statistics.

Note: SMF must be configured to record ICSF events and set the time interval for recording. Also, ICSF must be configured to track the wanted usage statistics.

For more information about configuring SMF, see 4.4.2, “Configuring SMF recording options in SMFPRMxx” on page 106.

Keywords can be combined to track multiple statistics. If the STATS option is not specified, usage tracking is not performed. The STATS and STATSFILTERS parameters use the following syntax:

```
STATS(ENG,SRV,ALG)
STATSFILTERS(NOTKUSERID)
```

The following STATS parameter values available:

- ENG enables usage tracking of cryptographic engines and supports Crypto Express coprocessors, Regional Cryptographic Servers, CP Assist for Cryptographic Functions (CPACF), and cryptographic software.
- SRV enables usage tracking of ICSF callable services and UDXes.
- ALG enables usage tracking of the cryptographic algorithms that are referenced in cryptographic operations.

> **Tip:** The ALG option can help you identify users that are using weak keys (such as DES).

Limited support is available for key generation, key derivation, and key import.

STATSFILTERS controls the identifier that is used to collect usage information. Usage is tracked by unique user and job identifiers. The user identifier consists of an address space user ID and a task level user ID.

For applications, such as CICS that can generate a unique task level user ID for each transaction, many SMF records can be generated. To prevent this problem, use the `STATSFILTER(NOTKUSERID)` option. Specifying `STATSFILTER(NOTKUSERID)` ignores the task level user ID and uses only the address space user ID when the job or user identifier is created.

`STATSFILTER(NOTKUSERID)` reduces the number of SMF records that are written.

The **`SETICSF OPT,STATS`** command can be used to enable STATS, as shown in Example 4-6.

Example 4-6 Output of the SETICSF command

```
SETICSF OPT,STATS=(ENG,SRV,ALG),SYSPLEX=NO
CSFM667I 13.27.39 SETICSF Options 802
    STATS:
      SC60     : ENG, SRV, ALG
```

4.3.4 Starting and stopping ICSF

ICSF runs as a started task on z/OS. A sample started task from SYS1.SAMPLIB(CSF) is shown in Example 4-7.

Example 4-7 Sample ICSF started task

```
//CSF PROC
//CSF EXEC PGM=CSFINIT,REGION=0M,TIME=1440,MEMLIMIT=NOLIMIT
//CSFPARM DD DSN=SYS1.PARMLIB(CSFPRM00),DISP=SHR
```

The CSFPARM DD statement points to the CSFPRMxx PARMLIB member that contains the ICSF installation options.

To start ICSF, issue the **START CSF,SUB=MSTR** command.

> **Note:** ICSF must be started before any components that intend to access encrypted data sets (for example, SMF data sets or data sets that are used during z/OS initialization). For more information, see 3.5.2, "Starting ICSF early in the IPL process" on page 45.

To stop ICSF, issue the **STOP CSF** command.

Verifying that ICSF is running

Verify that the ICSF address space is up and running and that it was started with no error. To verify that the ICSF address space is up and running, use command **DISPLAY JOB,ICSF** (or **CSF**, depending on your PROC name), as shown in Example 4-8.

Example 4-8 DISPLAY JOB,ICSF with results

```
DISPLAY JOB,CSF
CNZ4106I 14.50.30 DISPLAY ACTIVITY 604
 JOBS     M/S    TS USERS    SYSAS    INITS   ACTIVE/MAX VTAM     OAS
00003    00037    00006      00037    00011    00006/00030       00022
 CSF      CSF               NSW  S   A=002C   PER=NO   SMC=000
                                     PGN=N/A  DMN=N/A  AFF=NONE
                                     CT=002.211S  ET=00333.31
                                     WUID=STC05636 USERID=IBMUSER
                                     WKL=SYSTEM   SCL=SYSSTC   P=1
                                     RGP=N/A      SRVR=NO  QSC=NO
                                     ADDR SPACE ASTE=3E603B00
```

Review the ICSF the SYSLOG and check that you see the following messages:

```
CSFM126I CRYPTOGRAPHY - FULL CPU-BASED SERVICES ARE AVAILABLE
CSFM001I ICSF INITIALIZATION COMPLETE
```

4.3.5 Loading the AES master key

Use two or more key officers to manage master keys. Each key officer generates and owns their master key part. All key parts must be entered (in any order) to assemble the final key part. In this way, no single person has the entire master key.

The following high-level process is used to load the new master key registers:

1. Generate a random number for the AES master key part.
2. Generate a checksum for the AES master key part.
3. Load the first AES master key part.
4. Repeat Steps 1 -3 for the wanted number of middle key parts.
5. Load the final AES master key part.
6. Generate and load the remaining master keys.
7. Verify the new master key registers.

This process is shown in Figure 4-2.

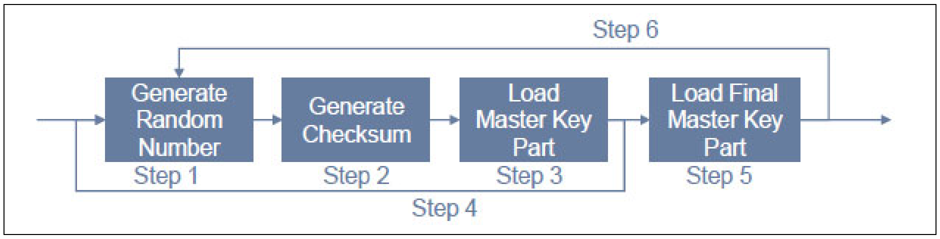


Figure 4-2 Steps to loading a new master key

Attention: Although this scenario shows a change of MASTER KEY for AES, the process is the same for all other types of master keys (DES, RSA, and ECC).

Checking the status of the master key register

Complete the following steps to check the AES master key:

1. From the ICSF Main panel, choose **Option 5 Utility** and press Enter (see Figure 4-3).

```
HCR77C1  -------------- Integrated Cryptographic Service Facility -------
OPTION ===> _
System Name:  SC74                                  Crypto Domain: 3
Enter the number of the desired option.

   1  COPROCESSOR MGMT -  Management of Cryptographic Coprocessors
   2  KDS MANAGEMENT   -  Master key set or change, KDS Processing
   3  OPSTAT           -  Installation options
   4  ADMINCNTL        -  Administrative Control Functions
   5  UTILITY          -  ICSF Utilities
   6  PPINIT           -  Pass Phrase Master Key/KDS Initialization
   7  TKE              -  TKE PKA Direct Key Load
   8  KGUP             -  Key Generator Utility processes
   9  UDX MGMT         -  Management of User Defined Extensions

      Licensed Materials - Property of IBM
      5650-ZOS Copyright IBM Corp. 1989, 2017.
      US Government Users Restricted Rights - Use, duplication or
      disclosure restricted by GSA ADP Schedule Contract with IBM Corp.

Press ENTER to go to the selected option.
Press END   to exit to the previous menu.
```

Figure 4-3 ICSF main panel

2. Check that the new master key register is empty by selecting action character to **s** (status) and press Enter (see Figure 4-4).

```
------------------------- ICSF Coprocessor Management -------- Row 1 to 2 of 2
COMMAND ===>                                                SCROLL ===> PAGE

 Select the cryptographic features to be processed and press ENTER.
 Action characters are: A, D, E, K, R, S and V. See the help panel for details.

  CRYPTO    SERIAL
  FEATURE   NUMBER    STATUS                  AES    DES    ECC    RSA    P11
  -------   --------  --------------------    ---    ---    ---    ---    ---
s _ 6C00    DV785304  Active                   A      A      A      A
. 6A01      N/A       Active
```

Figure 4-4 Check status of new master key register

3. View the register status on the Coprocessor Hardware status panel (see Figure 4-5 on page 88).

```
---------------------- ICSF - Coprocessor Hardware Status ---------------------
COMMAND ===> _                                                  SCROLL ===>
                                                               CRYPTO DOMAIN: 3

REGISTER STATUS                    COPROCESSOR 6C00
                                                                      More:     +
 Crypto Serial Number             : DV785304
 Status                           : ACTIVE
 PCI-HSM Compliance Mode          : INACTIVE
 Compliance Migration Mode        : INACTIVE
 AES Master Key
   New Master Key register        : EMPTY
     Verification pattern         :
   Old Master Key register        : VALID
     Verification pattern         : 183F88A73F6ECB8B
   Current Master Key register    : VALID
     Verification pattern         : 81A5742A3004D79B
 DES Master Key
   New Master Key register        : EMPTY
     Verification pattern         :
     Hash pattern                 :
                                  :
   Old Master Key register        : EMPTY
     Verification pattern         :
     Hash pattern                 :
                                  :
   Current Master Key register    : VALID
     Verification pattern         : 5B8EAE2289D07CF7

Press ENTER to refresh the hardware status display.
Press END   to exit to the previous menu.
```

Figure 4-5 Register status

Generating a checksum for the AES master key part

Complete the following steps to generate a checksum:

1. In the ICSF Utilities panel, select **Option 3 - Random** and press Enter (see Figure 4-6).

```
Enter the number of the desired option.

  1  ENCODE        -  Encode data
  2  DECODE        -  Decode data
  3  RANDOM        -  Generate a random number
  4  CHECKSUM      -  Generate a checksum and verification and
                      hash pattern
  5  CKDS KEYS     -  Manage keys in the CKDS
  6  PKDS KEYS     -  Manage keys in the PKDS
  7  PKCS11 TOKEN  -  Management of PKCS11 tokens




Press ENTER to go to the selected option.
Press END   to exit to the previous menu.
```

Figure 4-6 ICSF Utilities panel Option 3

2. View the Random Number Generator (RNG) panel (see Figure 4-7), which includes Random Number 1 - 4 with all 0s.

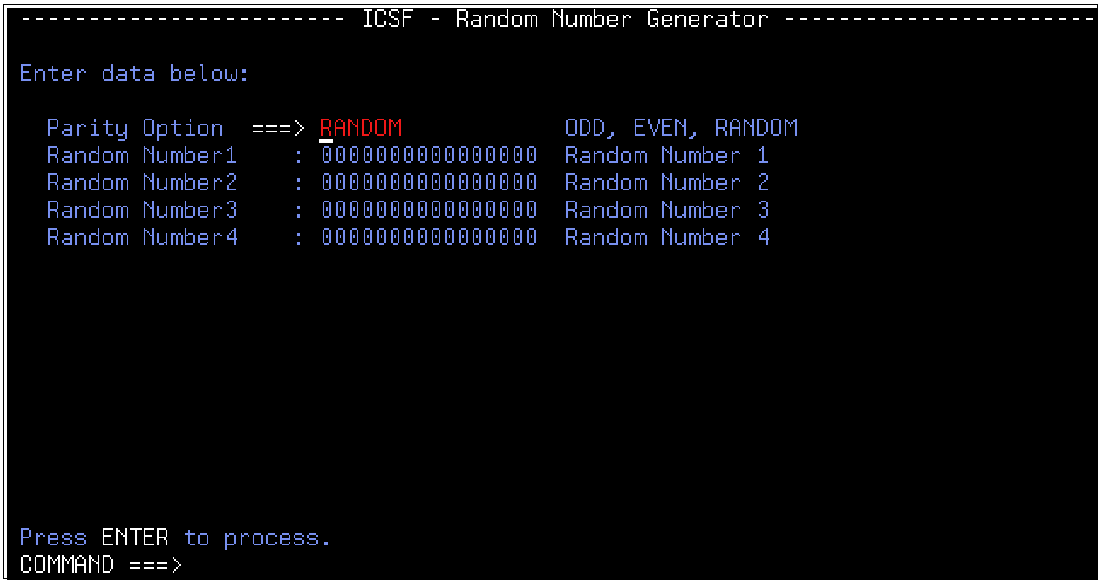


Figure 4-7 Random Number Generator panel entries all zeros

3. Press F1 for Help (see Figure 4-8).

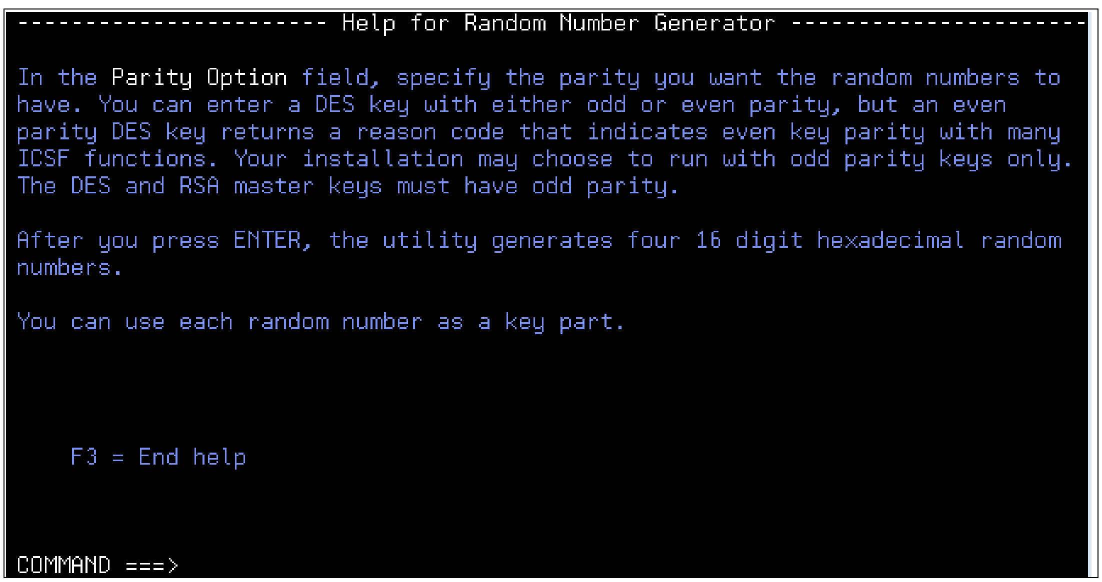


Figure 4-8 Help panel

4. Press F3 to return to the Random Number Generator panel. Enter `RANDOM` and then press Enter. The new random numbers that are generated are shown in Figure 4-9.

Figure 4-9 New random numbers generated

5. Press F3 to return to the Random Number Generator panel and choose **Option 4 - Checksum**. The Checksum Panel with the random numbers pre-populated is shown in Figure 4-10.

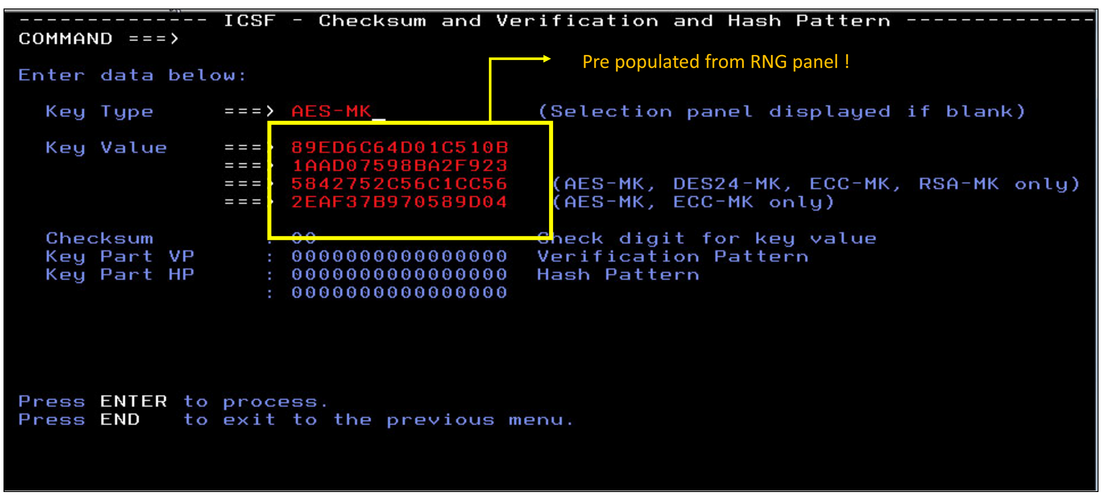


Figure 4-10 Checksum panel

6. Press F1 for Help. Choose **Option 1** for Key Type (see Figure 4-11).

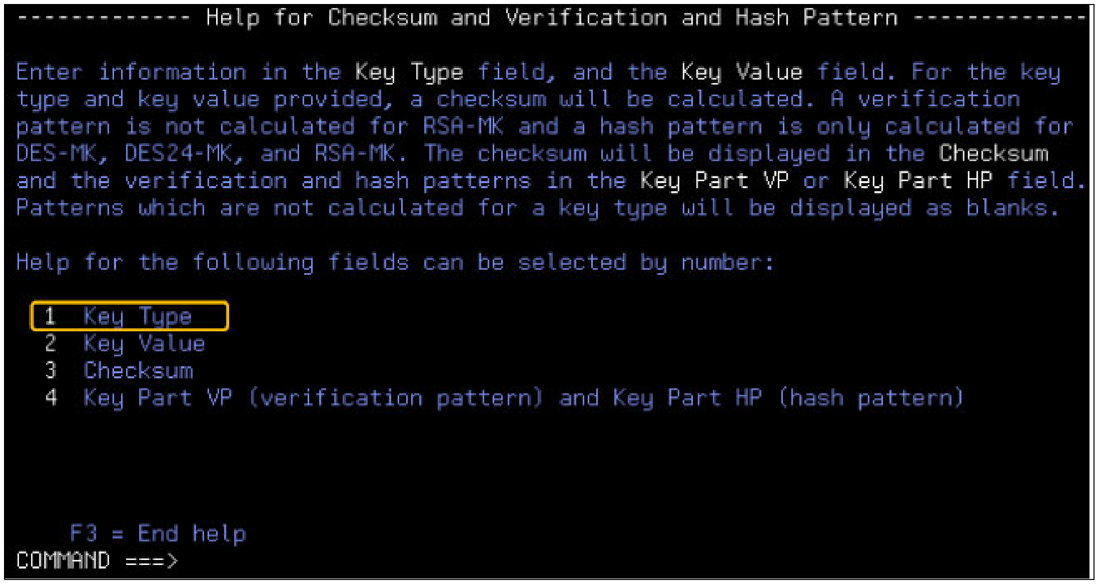

```
------------- Help for Checksum and Verification and Hash Pattern ------------

Enter information in the Key Type field, and the Key Value field. For the key
type and key value provided, a checksum will be calculated. A verification
pattern is not calculated for RSA-MK and a hash pattern is only calculated for
DES-MK, DES24-MK, and RSA-MK. The checksum will be displayed in the Checksum
and the verification and hash patterns in the Key Part VP or Key Part HP field.
Patterns which are not calculated for a key type will be displayed as blanks.

Help for the following fields can be selected by number:

  1  Key Type
  2  Key Value
  3  Checksum
  4  Key Part VP (verification pattern) and Key Part HP (hash pattern)



    F3 = End help
COMMAND ===>
```

Figure 4-11   Selecting key type from Help panel

7. View the possible key type values and lengths (see Figure 4-12).

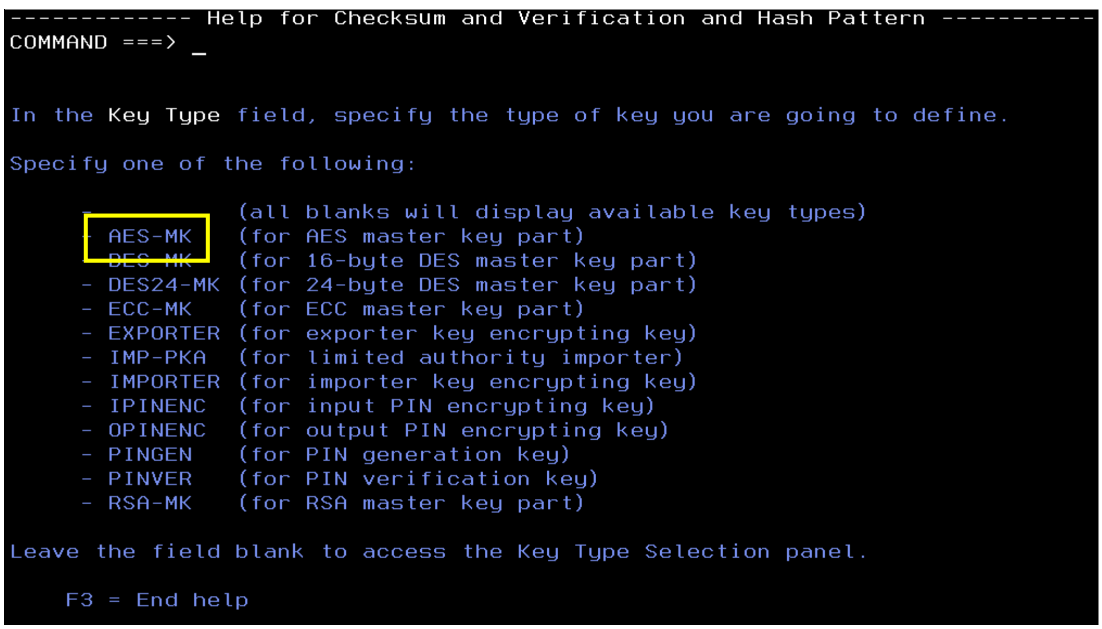

```
------------- Help for Checksum and Verification and Hash Pattern -----------
COMMAND ===> _


In the Key Type field, specify the type of key you are going to define.

Specify one of the following:

      -          (all blanks will display available key types)
      - AES-MK   (for AES master key part)
      - DES-MK   (for 16-byte DES master key part)
      - DES24-MK (for 24-byte DES master key part)
      - ECC-MK   (for ECC master key part)
      - EXPORTER (for exporter key encrypting key)
      - IMP-PKA  (for limited authority importer)
      - IMPORTER (for importer key encrypting key)
      - IPINENC  (for input PIN encrypting key)
      - OPINENC  (for output PIN encrypting key)
      - PINGEN   (for PIN generation key)
      - PINVER   (for PIN verification key)
      - RSA-MK   (for RSA master key part)

Leave the field blank to access the Key Type Selection panel.

    F3 = End help
```

Figure 4-12   Key types

8. Press F3 to return to the Checksum panel. Enter `AES-MK` for the key type and press Enter (see Figure 4-13).

> **Note:** Use AES-MK for data set encryption.

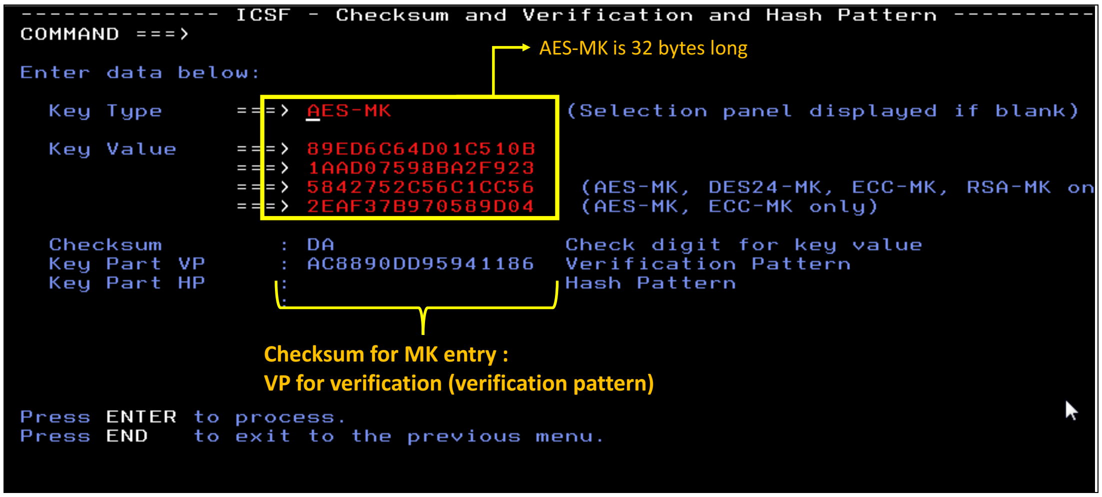


Figure 4-13 Keys with type AES-MK

> **Important:** It is strongly recommended to save a copy of the key value and checksum. If the master key is lost, you cannot be re-create it, and you cannot decrypt your data.

9. Press F3 to return to the Utility Panel. Press F3 to return to the ICSF Main panel.

Loading the first AES Master Key part

Complete the following steps to load the first AES Master Key part:

1. Select **Option 1 - Coprocessor Management** from the ICSF Main panel. The Coprocessor Management panel is shown in Figure 4-14.

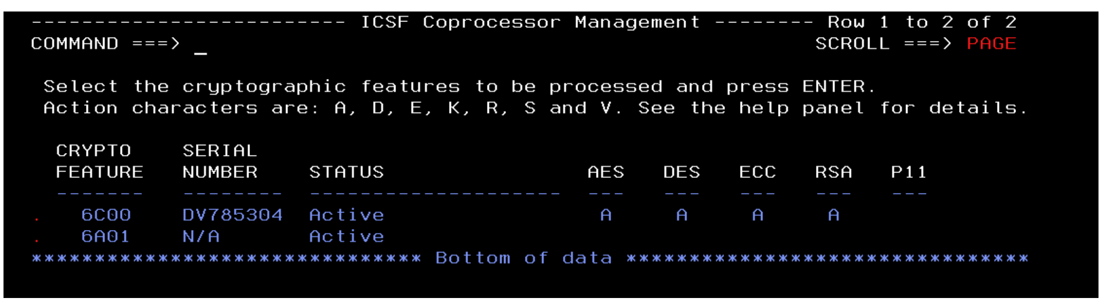


Figure 4-14 Coprocessor Management Panel

2. Press F1 for Help and scroll down (by pressing Enter) to see the Master Key State definitions (see Figure 4-15).

```
Cryptographic Coprocessor Master Key State:
    A:  Master key Verification Pattern matches the Key Store (CKDS, PKDS, or
        TKDS) and the master key is available for use
    C:  Master key Verification Pattern matches the Key Store, but the master
        key is not available for use
    E:  Master key Verification Pattern mismatch for Key Store or, for P11, no
        TKDS was specified in the options data set
    I:  The Master key Verification Pattern in the Key Store is not set,
        so the contents of the Master key are Ignored
    U:  Master key is not initialized
    -:  Not supported
     :  Not applicable
```

Figure 4-15 Cryptographic Coprocessor master key states

3. Press F3 to return to the Coprocessor Management panel. Enter `s` next to the crypto feature to view its status (see Figure 4-16).

```
 ------------------------- ICSF Coprocessor Management -------- Row 1 to 2 of 2
 COMMAND ===>                                                 SCROLL ===> PAGE

  Select the cryptographic features to be processed and press ENTER.
  Action characters are: A, D, E, K, R, S and V. See the help panel for details.

   CRYPTO    SERIAL
   FEATURE   NUMBER    STATUS                AES    DES    ECC    RSA    P11
   -------   --------  --------------------  ---    ---    ---    ---    ---
 .   6C00    DV785304  Active                 A      A      A      A
 .   6A01    N/A       Active
 ******************************* Bottom of data ********************************
```

Figure 4-16 Return to ICSF Coprocessor Management panel

The Coprocessor Hardware Status panel is shown in Figure 4-17. Notice that the new master key register is empty.

```
---------------------- ICSF - Coprocessor Hardware Status ---------------------
COMMAND ===> _                                                 SCROLL ===>
                                                            CRYPTO DOMAIN: 3

REGISTER STATUS                  COPROCESSOR 6C00
                                                                More:     +
 Crypto Serial Number           : DV785304
 Status                         : ACTIVE
 PCI-HSM Compliance Mode        : INACTIVE
 Compliance Migration Mode      : INACTIVE
 AES Master Key
   New Master Key register      : EMPTY
     Verification pattern       :
   Old Master Key register      : VALID
     Verification pattern       : 81A5742A3004D79B
   Current Master Key register  : VALID
     Verification pattern       : 1A7DFDEAFFEEDAC4
 DES Master Key
   New Master Key register      : EMPTY
     Verification pattern       :
     Hash pattern               :
                                :
   Old Master Key register      : EMPTY
     Verification pattern       :
     Hash pattern               :
                                :
   Current Master Key register  : VALID
     Verification pattern       : 5B8EAE2289D07CF7
```

Figure 4-17 Coprocessor Hardware Status panel with empty New Master Key register

4. Press F3 to return to the Coprocessor Management panel. In the Coprocessor Management panel, enter `e` next to the crypto feature to enter key parts (see Figure 4-18) and press Enter.

> **Note:** To simultaneously load multiple Crypto Express adapters (that are assigned to the current LPAR or domain) with the same master key, enter `e` next to all of the necessary cryptographic adapters and press Enter.

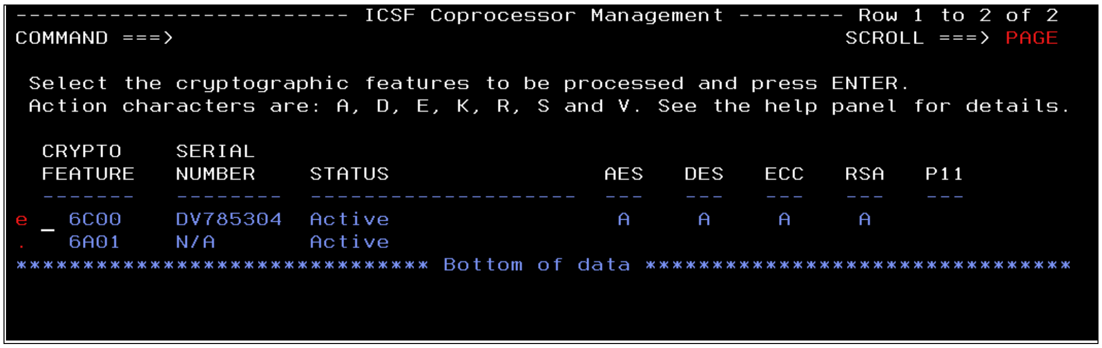


Figure 4-18 Coprocessor Management panel with e by a crypto feature

5. You see the Master Key Entry panel. For the Key Type, enter `AES-MK`; for Part, enter `first`; and for Checksum enter, `DA` (see Figure 4-19). Then, press Enter.

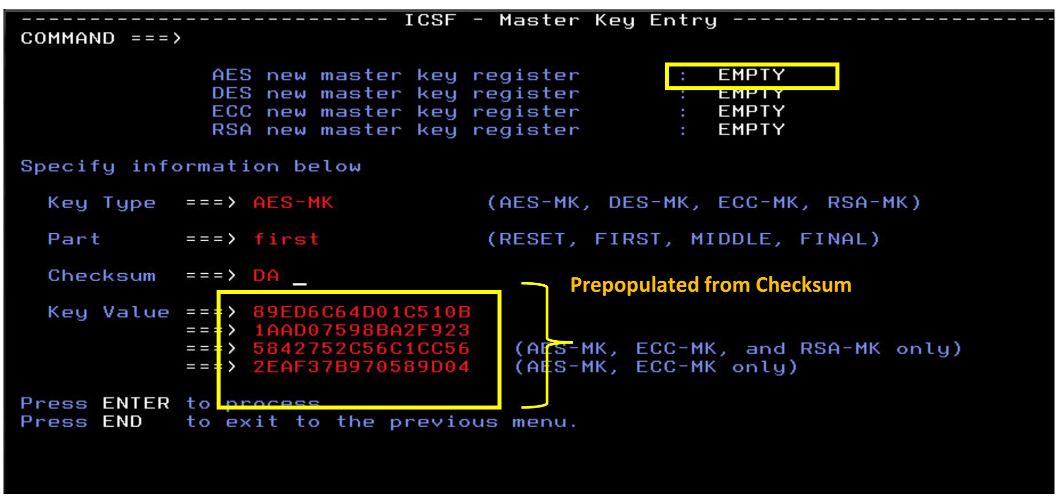


Figure 4-19 Master Key Entry panel

6. Check the new master key register status; in our example, it is PART FULL (see Figure 4-20).

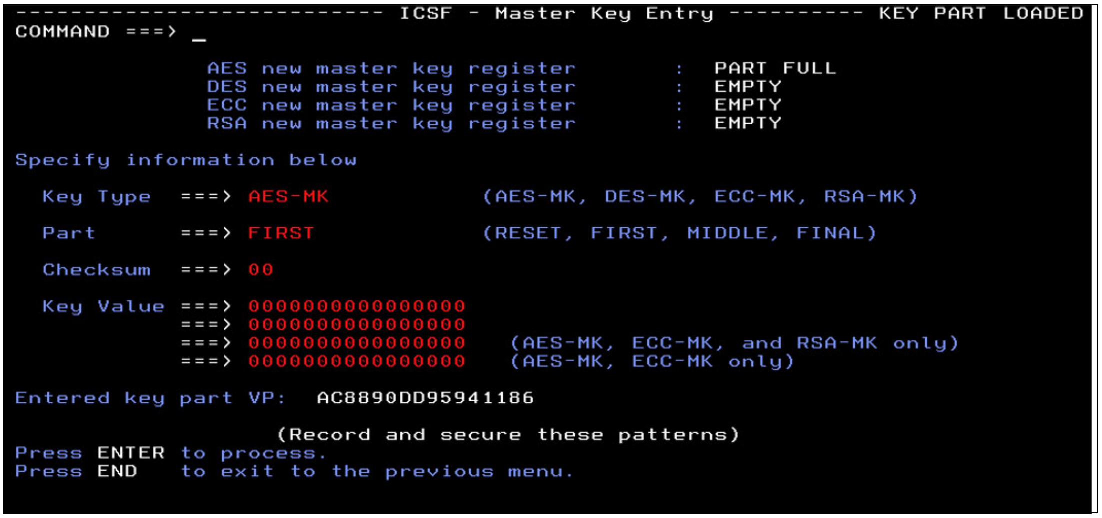


Figure 4-20 Master Key Entry - Part Full

Generating the wanted number of middle parts

Repeat the following steps for the wanted number of key parts:

- “Checking the status of the master key register” on page 87
- “Generating a checksum for the AES master key part” on page 88
- “Loading the first AES Master Key part” on page 92

Repeat the steps for each intermediate AES master key part. Each key custodian generates and loads (and securely saves) the following individual key parts:

- Generate a random key (and save the results) by using ICSF Option 5.3.
- Generate a checksum, VP (and save the results) by using ICSF Option 5.4.
- Load the key part into the new master key register by using ICSF Option 1.e (with part value entered as MIDDLE).

After all intermediate (middle) key parts are generated and saved, the final AES master key part must be loaded. This process is described next.

Loading the final AES master key part

Complete the following steps to load the final AES master key part:

1. Choose **Option 1 - Coprocessor Management** panel from the ICSF Main panel. In the Coprocessor Management panel, enter e next to the crypto feature to enter the final key part (see Figure 4-21). Press Enter.

```
------------------------- ICSF Coprocessor Management -------- Row 1 to 2 of 2
COMMAND ===>                                                 SCROLL ===> PAGE

 Select the cryptographic features to be processed and press ENTER.
 Action characters are: A, D, E, K, R, S and V. See the help panel for details.

  CRYPTO     SERIAL
  FEATURE    NUMBER     STATUS                  AES    DES    ECC    RSA    P11
  -------    --------   --------------------    ---    ---    ---    ---    ---
e _ 6C00     DV785304   Active                   A      A      A      A
.   6A01     N/A        Active
******************************* Bottom of data ********************************
```

Figure 4-21 Coprocessor Management panel (final part)

2. Use the Random Number Generator to generate numbers for the final part. In the Parity Option, enter RANDOM and press Enter (see Figure 4-22).

```
------------------------- ICSF - Random Number Generator ----------
COMMAND ===>

Enter data below:

  Parity Option    ===> RANDOM             ODD, EVEN, RANDOM
  Random Number1      : C6A03CD17C45F64B   Random Number 1
  Random Number2      : 231551662103 65E6  Random Number 2
  Random Number3      : FA6B02825F5F9D45   Random Number 3
  Random Number4      : B7128BDB2CAE4E57   Random Number 4









Press ENTER to process.
Press END   to exit to the previous menu.
```

Figure 4-22 Random Number Generator for final part

3. In the Checksum and Verification and Hash Pattern panel, enter `AES-MK` for the Key Type (see Figure 4-23). Press Enter.

```
-------------- ICSF - Checksum and Verification and Hash Pattern -------------
COMMAND ===>

Enter data below:

   Key Type      ===> AES-MK_            (Selection panel displayed if blank)

   Key Value     ===> C6A03CD17C45F64B
                 ===> 23155166210365E6
                 ===> FA6B02825F5F9D45    (AES-MK, DES24-MK, ECC-MK, RSA-MK only)
                 ===> B7128BDB2CAE4E57    (AES-MK, ECC-MK only)

   Checksum          : 00                Check digit for key value
   Key Part VP       : 0000000000000000  Verification Pattern
   Key Part HP       : 0000000000000000  Hash Pattern
                     : 0000000000000000



Press ENTER to process.
Press END   to exit to the previous menu.
```

Figure 4-23 Checksum and Verification and Hash Pattern panel before Checksum

The results of final key part generation are shown in Figure 4-24.

```
-------------- ICSF - Checksum and Verification and Hash Pattern -------------
COMMAND ===>

Enter data below:

   Key Type      ===> AES-MK             (Selection panel displayed if blank)

   Key Value     ===> C6A03CD17C45F64B
                 ===> 23155166210365E6
                 ===> FA6B02825F5F9D45    (AES-MK, DES24-MK, ECC-MK, RSA-MK only)
                 ===> B7128BDB2CAE4E57    (AES-MK, ECC-MK only)

   Checksum          : C2                Check digit for key value
   Key Part VP       : 0EC44AEBA8600619  Verification Pattern
   Key Part HP       :                   Hash Pattern
                     :



Press ENTER to process.
Press END   to exit to the previous menu.
```

Figure 4-24 Checksum and Verification and Hash Pattern panel with final key part

4. In the Master Key Entry panel, enter `AES-MK` for the Key Type. Enter `FINAL` for the Part type, and enter `C2` for Checksum (see Figure 4-25 on page 98). Press Enter.

> **Note:** If you enter an incorrect checksum value, ICSF warns you with a message "Incorrect checksum value" in the upper right corner of the panel. Ensure that you enter the correct value that was reported as the generated checksum value.

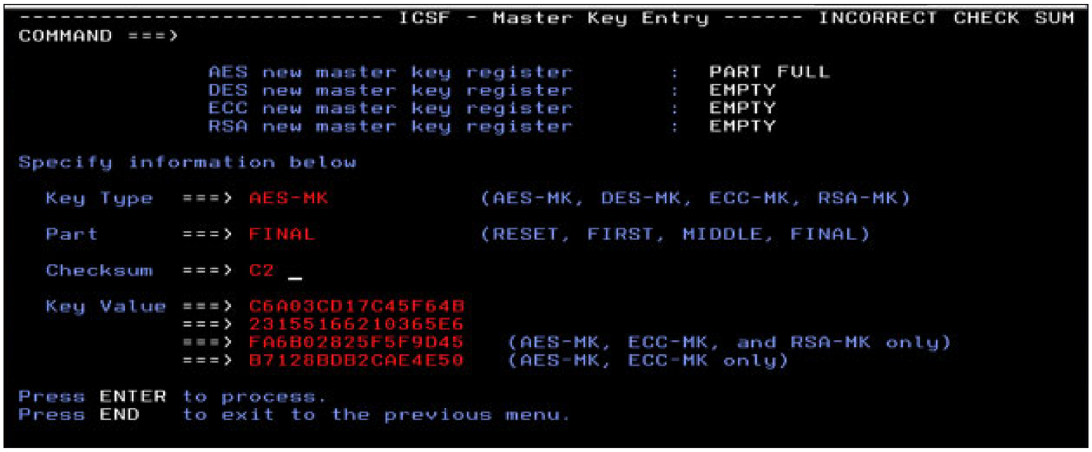


Figure 4-25 Master Key Entry panel set final part

5. Check the Master Key Entry panel for the new master key register. Confirm that the status is FULL and the final key part VP matches the checksum value (see Figure 4-26).

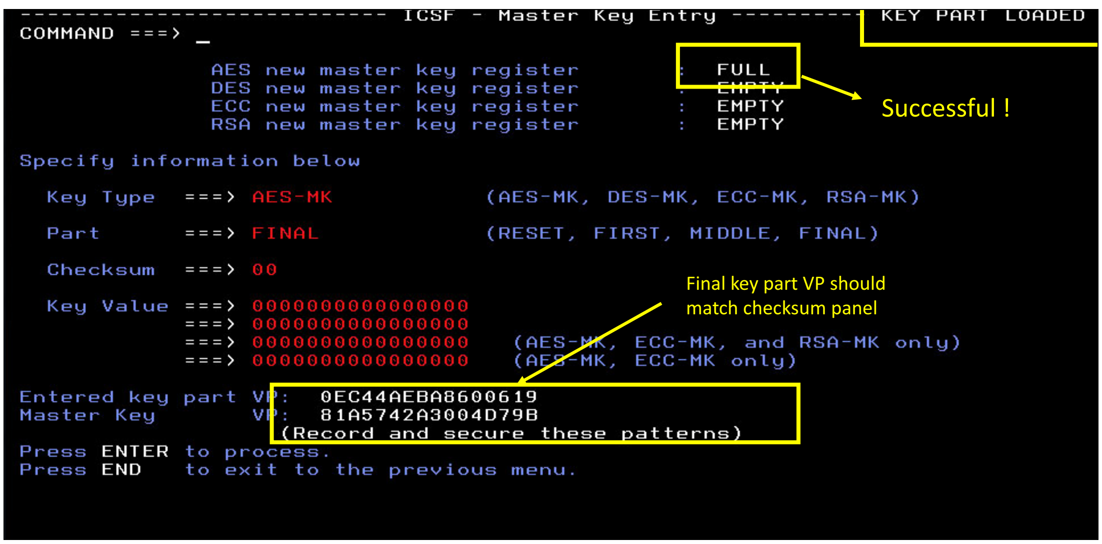


Figure 4-26 Master Key full

6. Press F3 to return to the Coprocessor Management panel. Enter `s` next to the crypto feature to view the status (see Figure 4-27). Press Enter.

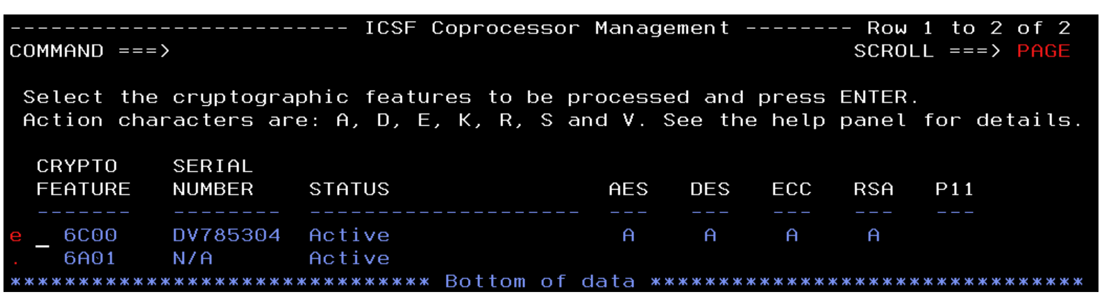


Figure 4-27 Coprocessor Management check full key

7. Review the Coprocessor Hardware Status panel (see Figure 4-28 on page 99), which shows the new master key register as FULL.

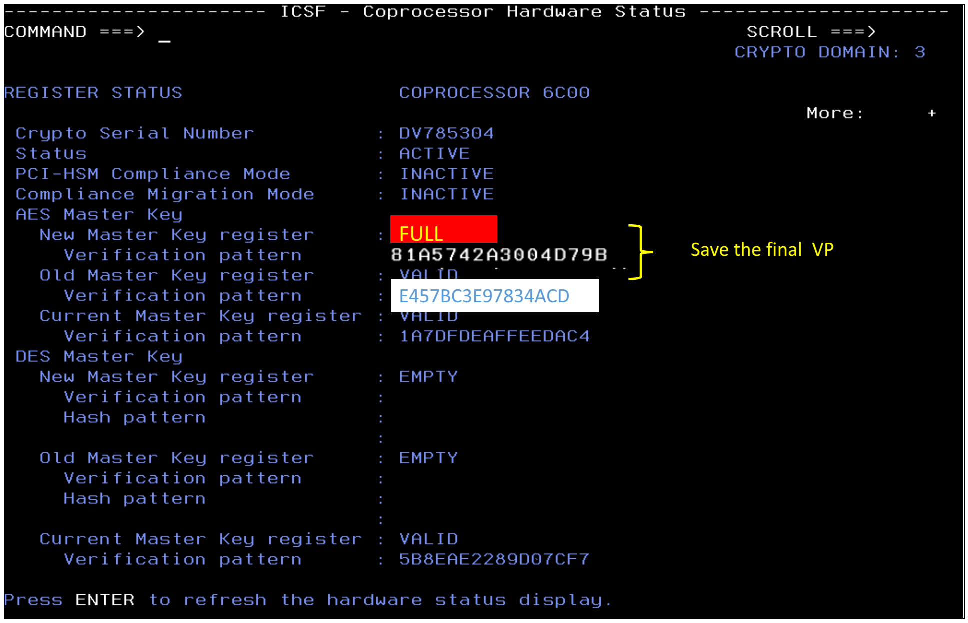


Figure 4-28 Coprocessor Hardware Status with full key

If other members in your sysplex share the CKDS and use a different domain number, repeat the process for each member of the sysplex (except for steps 1 and 2).

In this scenario, these steps were not repeated for the second member (SC75) of the sysplex, which is sharing the CKDS. When we attempted to perform the coordinated master key change, we received the error messages that are shown in Figure 4-29.

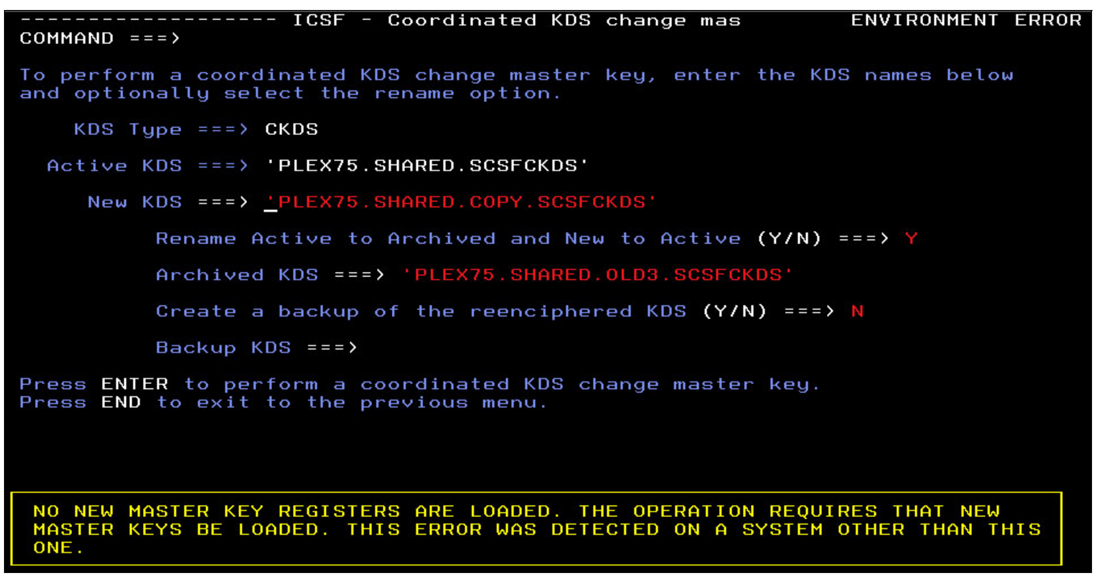


Figure 4-29 Environment error

The master keys incorrect error is shown in Example 4-9.

Example 4-9 New master key incorrect

```
CSFM615I COORDINATED CHANGE-MK FAILED.  NEW MASTER KEYS INCORRECT ON
SC75. RC = 12, RSN = 3098.
IEF196I CSFM615I COORDINATED CHANGE-MK FAILED.  NEW MASTER KEYS
```

```
IEF196I INCORRECT ON SC75. RC = 12, RSN = 3098.
CSFM636I SYSTEM SC75 FAILURE FOR COORDINATED CKDS ACTIVITY. MSGTYPE=00
0001 RC=12 RSN=3098.
CSFM616I COORDINATED CHANGE-MK FAILED, RC=0000000C RS= 00000C1A
SUPRC= 00000000 SUPRS= 00000000 FLAGS= 48000000.
CSFU006I CHANGE-MK FEEDBACK: RC=0000000C RS=00000C1A SUPRC=00000000
SUPRS=00000000 FLAGS=48000000.
IEF196I CSFU006I CHANGE-MK FEEDBACK: RC=0000000C RS=00000C1A
IEF196I SUPRC=00000000 SUPRS=00000000 FLAGS=48000000.
```

If your system is using multiple coprocessors, they must have the same master keys. When you load a new master key in one coprocessor, you load the same new master key in the other coprocessors. Therefore, to reencipher a key data set under a new master key, the new master key registers in all coprocessors must contain the same value.

Optionally, generating and loading the remaining master keys

Only the AES master key is required for z/OS data set encryption. If you need more master keys for other crypto applications, complete the following steps for the necessary master keys (such as DES, RSA, and ECC) and each key custodian or key part:

1. Generate a random key (and save the results) by using ICSF Option 5.3.
2. Generate a checksum, VP, HP (and save the results) by using ICSF Option 5.4.
3. Load the key part into the new master key register by using ICSF Option 1.e.
4. Repeat steps 1-3 for each key part.
5. Repeat steps 1-4 for each master key type.

When you start or reencipher a CKDS or PKDS, ICSF places the verification pattern for the master keys into the key data set header record.

Verifying the new master key registers

if you followed the previous steps, view the master key entry panel to verify that it is FULL for AES new master key register. If you repeated the steps for DES, RSA, and ECC, their status also is FULL.

4.3.6 Initializing the CKDS

CKDS initialization involves storing the master key verification pattern (MKVP) in the header record of the CKDS. The CKDS must be allocated before initialization (for more information, see 3.5, “ICSF administration considerations” on page 45). After the AES master key is loaded, the CKDS can be initialized.

CKDS initialization can be performed by using the CKDS Management ICSF panel (see Example 4-10 on page 101).

Example 4-10 CKDS Management Panel

```
---------------------------- ICSF - CKDS Management --------------------------
 OPTION ===> 1

 Enter the number of the desired option.

   1  CKDS OPERATIONS    -  Initialize a CKDS, activate a different CKDS,
                            (Refresh), or update the header of a CKDS and make
                            it active
   2  REENCIPHER CKDS    -  Reencipher the CKDS prior to changing a symmetric
                            master key
   3  CHANGE SYM MK      -  Change a symmetric master key and activate the
                            reenciphered CKDS
   4  COORDINATED CKDS REFRESH - Perform a coordinated CKDS refresh
   5  COORDINATED CKDS CHANGE MK - Perform a coordinated CKDS change master key
   6  COORDINATED CKDS CONVERSION - Convert the CKDS to use KDSR record format
   7  CKDS KEY CHECK     - Check key tokens in the active CKDS for format errors

 Press ENTER to go to the selected option.
 Press END   to exit to the previous menu.
```

For more information, see Setting up and maintaining the cryptographic key data set (CKDS).

4.3.7 Verifying the ICSF configuration

This section helps you determine whether the following building blocks are operational:

- ICSF Options
- CKDS Initialization
- Active Master Key

The *Cryptographic Services ICSF: System Programmer's Guide* provides helpful hints for ICSF first-time startup.

For more information about a checklist for first-time startup, see Checklist for first-time startup of ICSF.

Checking the ICSF job log

The ICSF address space job log shows which crypto hardware function is available.

Note: When you start ICSF with SUB=MSTR (as recommended), you cannot access the ICSF job log by using the System Display and Search Facility (SDSF) or vendor products, such as (E)JES. In this instance, you cannot browse the content of the ICSF job log.

When you display the ICSF address space job log, you can expect to see the messages that are shown in Example 4-11.

Example 4-11 ICSF address space job log

```
CSFM129I MASTER KEY DES ON CRYPTO EXPRESS6 COPROCESSOR 6C00, SERIAL NUMBER DV785
CSFM129I MASTER KEY AES ON CRYPTO EXPRESS6 COPROCESSOR 6C00, SERIAL NUMBER DV785
CSFM129I MASTER KEY RSA ON CRYPTO EXPRESS6 COPROCESSOR 6C00, SERIAL NUMBER DV785
CSFM129I MASTER KEY ECC ON CRYPTO EXPRESS6 COPROCESSOR 6C00, SERIAL NUMBER DV785
CSFM111I CRYPTOGRAPHIC FEATURE IS ACTIVE. CRYPTO EXPRESS6 COPROCESSOR 6C00, SERI
```

```
CSFM111I CRYPTOGRAPHIC FEATURE IS ACTIVE. CRYPTO EXPRESS6 ACCELERATOR 6A01, SERI
CSFM130I CRYPTOGRAPHY - RSA SERVICES ARE AVAILABLE.
CSFM130I CRYPTOGRAPHY - ECC SERVICES ARE AVAILABLE.
CSFM133I THERE ARE NO ACTIVE PKCS11 COPROCESSORS.
CSFM015I FIPS 140 SELF CHECKS FOR PKCS11 SERVICES SUCCESSFUL.
CSFM009I NO ACCESS CONTROL AVAILABLE FOR ICSF SERVICES OR KEYS
CSFM400I CRYPTOGRAPHY - SERVICES ARE NOW AVAILABLE.
CSFM130I CRYPTOGRAPHY - DES SERVICES ARE AVAILABLE.
CSFM127I CRYPTOGRAPHY - AES SERVICES ARE AVAILABLE.
CSFM126I CRYPTOGRAPHY - FULL CPU-BASED SERVICES ARE AVAILABLE.
CSFM001I ICSF INITIALIZATION COMPLETE
CSFM640I ICSF RELEASE FMID=HCR77C1.
```

The following messages are shown in Example 4-11 on page 101:

- A Master key was loaded (messages CSFM129I).
- The CCA Crypto Express is active (messages CSFM111I).
- AES services are available to generate AES DATA or CIPHER keys (message CSFM127I).
- CPACF services are available for protected keys that are used by data set encryption (message CSFM126I).

Viewing ICSF options

After ICSF is started, you can view the current usage settings from the ICSF Installation Option Display panel or the operator console.

STATS

STATS information can also be obtained by issuing the **Display ICSF** command from the system console. STATSFILTERS information is not displayed. All usage (engines, services, and algorithms) that is being tracked on SC60 (also known as PLEX60) is shown in Example 4-12.

Example 4-12 Command D ICSF,OPT

```
D ICSF,OPT
CSFM668I 11.06.42 ICSF OPTIONS 744
SYSNAME = SC60 ICSF LEVEL = HCR77C1
LATEST ICSF CODE CHANGE = 12/06/17
.
.
.
STATS:
SC60 ENG, SRV, ALG
```

Changing ICSF options

After ICSF is started, the **SETICSF** command can be used to dynamically change ICSF options.

SETICSF OPT,STATS

To change the cryptographic usage settings to monitor only crypto engine usage across a sysplex, issue the **SETICSF OPT,STATS=(ENG),SYSPLEX=YES** command.

To stop all cryptographic usage tracking, issue the **SETICSF OPT,STATS=NONE,SYSPLEX=YES** command.

> **Note:** STATSFILTERS is not supported. You can use **SETICSF OPT,REFRESH** to change the setting of STATSFILTERS

SETICSF OPT,REFRESH

To refresh common options that were updated in the CSFPRMxx PARMLIB member, issue the **SETICSF OPT,REFRESH** command. For more information about supported options for refresh, see SETICSF.

Displaying the CKDS state

You can use z/OS command **D ICSF,KDS** to obtain more information about your KDS status, as shown in Example 4-13.

Example 4-13 Using the D ICSF,KDS command

```
D ICSF,KDS
CSFM668I 13.36.58 ICSF KDS 551
  CKDS  SYS1.SC60NEW.SCSFCKDS
    FORMAT=KDSR                  SYSPLEX=N  MKVPs=DES AES
  PKDS  SYS1.SC60NEW.SCSFPKDS
    FORMAT=KDSR                  SYSPLEX=N  MKVPs=RSA ECC
  No TKDS was provided.
```

The system displays the following information (message CSFM668I) about the active key data sets (KDS) on the system or sysplex:

- The data set name for each active KDS (CKDS, PKDS, and TKDS).
- The format of the KDS (for example, KDSR is the recommended format to use).

  Possible values are KDSR, FIXED, and VARIABLE.
- The communication level in place for the KDS (for example, 3). This information is displayed in a sysplex environment only.
- Whether the KDS is being shared in a sysplex group (for example, Y/N).
- The MKVPs initialized in the KDS (for example, DES AES).

  The following values are used:

  – DES, AES, or both for CKDS
  – RSA, ECC, or both for PKDS
  – P11, RCS, or both for TKDS

Displaying the master key state

The use of the **D ICSF,MKS** command displays the status of the master key registers (see Example 4-14).

Example 4-14 D ICS,MKS command and results

```
D ICSF,MKS
CSFM668I 13.38.20 ICSF MKS 566
  SYSNAME: SC60     DOMAIN: 084 CPC Name: CETUS
    FEATURE SERIAL#  STATUS              AES DES ECC RSA P11
     6C00   DV785304 Active               A   A   A   A
```

The CCA feature device number (6C00), its serial number (DV785304), its status (active), and the master keys that are loaded (AES for our purpose) are shown in Example 4-14 on page 103.

Displaying the Crypto Express adapter status

The use of the **D ICSF,CARDS** command provides more information about the Crypto Express adapters (see Example 4-15).

Example 4-15 D ICSF, CARDS command and results

```
D ICSF,CARDS
CSFM668I 13.40.58 ICSF CARDS 568
  ACTIVE DOMAIN = 084
  CRYPTO EXPRESS6 COPROCESSOR 6C00
    STATUS=Active              SERIAL#=DV785304 LEVEL=6.0.6z
    REQUESTS=0000001185  ACTIVE=0000
  CRYPTO EXPRESS6 ACCELERATOR 6A01
    STATUS=Active
    REQUESTS=0000000005  ACTIVE=0000
```

The active domain (84), number of requests (1185 for the device), and firmware level of the device (for example, 6.0.6z) are shown in Example 4-15.

Viewing the Coprocessor Management Panel

You can also obtain information about your crypto configuration and usage by using the ICSF ISPF panels. The primary panel gives you the crypto domain in use (84), as shown in Example 4-16.

Example 4-16 ICSF ISPF primary panel

```
OPTION ===> 1
System Name:  SC60                          Crypto Domain: 84
Enter the number of the desired option.

  1  COPROCESSOR MGMT -  Management of Cryptographic Coprocessors
```

Select **option 1 COPROCESSOR MGMT** to see the results, as shown in Example 4-17.

Example 4-17 Option 1 COPROCESSOR MGMT selection results

```
Select the cryptographic features to be processed and press ENTER.
Action characters are: A, D, E, K, R, S, and V. See the help panel for details.

 CRYPTO    SERIAL
 FEATURE   NUMBER    STATUS                AES   DES   ECC   RSA   P11
 -------   --------  --------------------  ---   ---   ---   ---   ---
   6C00    DV785304  Active                 A     A     A     A
   6A01    N/A       Active
```

The COPROCESSOR MGMT selection results give the status of your crypto adapters, where “A” stands for active (which is expected).

If you select **S**, you see the information that is shown in Example 4-18 on page 105.

Example 4-18 Selection of an active coprocessor

```
REGISTER STATUS                  COPROCESSOR 6C00
                                                                   More:     +
 Crypto Serial Number          : DV785304
 Status                        : ACTIVE
 PCI-HSM Compliance Mode       : INACTIVE
 Compliance Migration Mode     : INACTIVE
 AES Master Key
   New Master Key register     : EMPTY
     Verification pattern      :
   Old Master Key register     : EMPTY
     Verification pattern      :
   Current Master Key register : VALID
     Verification pattern      : 49232659E5B39664
```

The Current Master Key register field that must be VALID and Status must be ACTIVE is shown in Example 4-18.

4.3.8 Reviewing messages and codes

When your installation is alerting and automating actions based on messages that are issued by ICSF, keep updated with any changing messages be seeing the following resources:

For more information about messages for ICSF, see *Cryptographic Services: Integrated Cryptographic Service Facility Messages*.

4.4 Audit configuration

Configure auditing for your environment to prove that data sets are being encrypted and encryption keys are being managed. For more information about the SMF record output, see Chapter 6, “Auditing z/OS data set encryption” on page 149.

4.4.1 Enabling SMF record types 14, 15, 42, 62, 70, 80, 82, and 113

This section describes the primary SMF records and subtypes that are used for data set encryption and CKDS auditing. It also provides a brief overview of the utilities that are used for data set encryption and audit requirements.

The SMF records, their subtypes, and descriptions are listed in Table 4-7.

Table 4-7 SMF record types

SMF record type	Sub type	Description
14 and 15		Encrypted sequential data sets (extended format, basic format, and large format) and PDSEs.
42	6	Used by zBNA. Contains information for encrypted sequential data sets (extended format, basic format, and large format) and PDSEs.
62		Encrypted VSAM data sets.

SMF record type	Sub type	Description
70	2	Cryptographic hardware activity that is measurements from the Resource Measurement Facility (RMF).
80		Security authorization attempts.
82	1	Records when ICSF is started or options are refreshed. Reports ICSF installation options.
82	9	Records when the CKDS is updated.
82	14	Records when clear master key parts are loaded.
82	28	Records when protected keys are created (creating protected keys requires the CSFKEYS CSF-PROTECTED-KEY-TOKEN profile).
82	31	Reports crypto engine usage. Enabled by way of the entry in CSFPRMxx: STATS(ENG,SRV,ALG) or dynamically by entering the `SETICSF OPT` command.
82	40	Reports changes in transitional phases in key lifecycle. Enabled by way of the entry in CSFPRMxx: AUDITKEYLIFECKDS(TOKEN(YES) LABEL(YES))
82	44	Reports usage of keys. Enabled by way of the entry in CSFPRMxx: AUDITKEYUSGCKDS(TOKEN(YES) LABEL(YES)INTERVAL(n)).
113		Required for zBNA.

4.4.2 Configuring SMF recording options in SMFPRMxx

The System Management Facility (SMF) collects and records system and job-related information in SMF records. Configuration of SMF recording involves updating the SMFPRMxx member of your PARMLIB.

The following important options must be considered for z/OS data set encryption:

- TYPE

  For z/OS data set encryption, ensure that the SMF types include 14 (more specifically, 14:9), 15, 42 (more specifically, 42:6), 62, 70 (more specifically, 70:2), 80, and 82 (more specifically, 82:31, 82:40, and 82:44).

- INTVAL

  The SMF global recording interval is used for SMF Type 82:31 and other record types. The system default is 30 minutes of SMF recording. At the end of the interval, SMF records are written to data sets or log streams.

- SYNCVAL

  The SMF global recording interval is used for SMF Type 82:31 and other record types. The system default is an offset of 0 from the top of the hour. From the top of the hour, SMF records for INTVAL number of minutes and then writes the SMF data to data sets or log streams.

SMFPRMxx example

You can set up SMF by using an SMF parameter member that is contained in the system PARMLIB. An SMF parameter member is shown in Example 4-19.

Example 4-19 SMF parameter member

```
DSNAME(HLQ.SMF.MANA,HLQ.SMF.MANB) /* SMF data sets */
INTVAL(15) /* INTERVAL = 15 MINUTES */
SYNCVAL(00) /* SYNCHRONIZATION = 00 MINUTES */
SYS(TYPE(0,3,82(13:23,31,40:42),83,134)) /* Include subtype 31 */
```

The following parameter members are shown in Example 4-19:

- INTVAL(xx) parameter determines how often usage statistics are recorded. For example, a value of 15 triggers ICSF to record the usage every 15 minutes.
- SYNCVAL(xx) parameter synchronizes the recording interval with the end of the hour on the TOD clock (for example, a value of 00 triggers ICSF to start recording the usage at the end of the hour).
- SYS(TYPE(*)) parameter specifies the type of records that are being recorded. These ICSF usage events are recorded in SMF type 82, subtype 31 records.

4.4.3 Enabling auditing for master key change operations

All ICSF administration services, utilities, and panels are access-controlled with SAF resources. They can be audited on the RACROUTE call that generates a RACF SMF record. RACF logs the execution time and associated user.

For critical services (for example, set master key) enable RACF to audit successes to produce an audit record for that event. If the record is not needed, use the default and audit only the failure.

The following CSFSERV profiles are accessed when performing a master key change, which are to be set to audit successes:

- CSFDKCS: Enter Master Key Part
- CSFCRC: Change Master Key
- CSFSMK: Set Master Key

We recommend turning on auditing by using the following commands before the start of the master key change process:

- `RALTER CSFSERV CSFDKCS UACC(NONE) AUDIT(ALL)` for entering master keys with option 1.E
- `RALTER CSFSERV CSFCRC UACC(NONE) AUDIT(ALL)` for changing master key with option 2.1.5 5 COORDINATED CKDS CHANGE MK
- `RALTER CSFSERV CSFSMK UACC(NONE) AUDIT(ALL)` for setting master key (option 2.4)

4.4.4 Enabling key part control for Master Key Entry

With z/OS 3.2 and ICSF HCR77F0, a new function to control who can enter what key part through the ICSF panels. If this function does not provide the level of security that the TKE provides, it can help in better controlling the key entry process and enforce dual-control.

Also see, Key part control for Master Key Entry utility.

4.4.5 RMF Crypto Hardware Activity report

The Resource Measurement Facility (RMF) provides performance monitoring of cryptographic coprocessor and accelerator usage in the RMF Crypto Hardware Activity report. This report is created from data in SMF record type 70, subtype 2.

The following Monitor gathering options on the REPORTS control statement are available:

- CRYPTO measures cryptographic hardware activity
- NOCRYPTO suppress the gathering

4.4.6 UKO auditing

UKO for z/OS and the EKMF Workstation will record all key management actions to the audit log in the UKO repository. The EKMF Workstation can further be configured to also log to SMF records.

The EKMF Workstation can add a MAC to each audit record for integrity validation.

> **Logging:** When enabled in the EKMF Workstation audit log settings, the EKMF Workstation will send its audit log to SMF via the UKO agent. By default, the UKO agent requires that SMF logging is enabled for any workstation that connects to the UKO agent task.
>
> In order for the SMF logging to be done, RACF must be setup for audit of the FACILITY class profile for resource KMG.EKMF.SMF.
>
> Audit must be for ALL(READ).

Example 4-20 illustrates how to define the resource with audit controls and grant the UKO Agent task user ID the necessary permissions.

Example 4-20 Define audit controls

```
RDEFINE FACILITY KMG.EKMF.SMF ACC(NONE) OWNER(SYS1) AUDIT(ALL(READ))
PERMIT KMG.EKMF.SMF CLASS(FACILITY) ID(<TASK_USERID>)
```

The RACF READ access check (RACROUTE REQUEST=AUTH) on the FACILITY class resource KMG.EKMF.SMF is done with a log-string where the content is the audit record from the EKMF Workstation.

When the EKMF Workstation logs to SMF, the records are written as type 80 (Authorized access) for FACILITY class resource KMG.EKMF.SMF where the audit data is placed in the relocate section variable data - data type 46 (X'2E') 1-255 bytes.

4.5 UKO configuration

The UKO key management system provides centralized key management across multiple z/OS LPARs. It can create keys and store them under specific key labels in the CKDS and PKDS keystores. For each keystore (or keystore sharing group) managed by UKO, a UKO agent must run on the same z/OS LPAR. When CKDS/PKDS sysplex sharing is used, only one UKO agent is needed per sysplex.

UKO must be configured with information about the location of UKO agents and the keys it creates.

Keys are defined by key templates that ensures consistent key creation process and distribution to predefined CKDS keystores with key labels that follow a predefined key label naming standard.

Also see Chapter 10, “IBM Unified Key Orchestrator” on page 223 for more information.

4.5.1 UKO agent configuration

The UKO agent must be configured to trust the connection from a UKO for z/OS instance or the EKMF Workstation.

To establish trust the following is needed:

- Obtain the ECC signature key fingerprint from UKO for z/OS and if available, the EKMF Workstation.
- Define a RACF resource profile in the XFACILIT class.
- Give the UKO agent task user ID READ access.

> **Tip:** Ensure that SMF logging is enabled before configuring UKO agent trust.

UKO for z/OS and the EKMF Workstation both provide unique key fingerprints for an ECC signature key. This fingerprint must be used to define a RACF resource profile in the XFACILIT class:

`KMG.WS.<64-character-hex-fingerprint>`

The UKO agent task user ID must have READ access to this resource.

For a UKO agent, with task user ID 'UKOA', the commands shown in Example 4-21 are used to allow connections to the UKO agent by UKO for z/OS and the EKMF Workstation having a signature key with fingerprint:

`‘8A7A87509C40A5ED228E05DED51DD690EEFAC4C49B83E0A9A288B515D6745102’`

Example 4-21 Establishing trust between UKO for z/OS or EKMF Workstation and UKO Agent

```
RDEFINE XFACILIT -
KMG.WS.8A7A87509C40A5ED228E05DED51DD690EEFAC4C49B83E0A9A288B515D6745102

PERMIT -
KMG.WS.8A7A87509C40A5ED228E05DED51DD690EEFAC4C49B83E0A9A288B515D6745102 -
CLASS(XFACILIT) ACC(READ) ID(UKOA)
```

4.5.2 UKO for z/OS AT-TLS setup

Application Transparent Transport Layer Security (AT-TLS) is a capability of z/OS Communications Server that can create a secure session on behalf of UKO (or other z/OS applications). AT-TLS provides encryption and decryption of data based on policy statements that are coded in the Policy Agent. When an application, such as UKO, sends and receives data, AT-TLS encrypts and decrypts data at the TCP transport layer.

AT-TLS is activated by specifying the TTLS option in the TCPCONFIG statement block in the TCP/IP profile data.

For more information on AT-TLS, see Application Transparent Transport Layer Security data protection.

Note: The use of AT-TLS is optional but recommended for the UKO Agent. The UKO for z/OS server has TLS capabilities provided by the IBM Websphere® Liberty application server and therefore does not need AT-TLS.

UKO agent AT-TLS setup

The UKO agent does not have native support for TLS. Security of the communication between UKO for z/OS and the UKO agent relies on a proprietary protocol that encrypts sensitive data. To increase the security of the communication, it is recommended to set up AT-TLS for the UKO agent, such that the communication is done through a channel secured by TLS.

Example 4-22 shows how an AT-TLS rule can be defined for the UKO agent. The policy is defined in the TLS policy file in the /etc/pagent/ USS directory.

Note that `ApplicationControlled` is set to `Off` and `HandShakeRole` is set to `Server`. There must be a server certificate with private key in the specified Keyring and the relevant CA certificates. The UKO server that connects to the UKO Agent must then have the corresponding CA certificate in its own trust key ring

When AT-TLS is configured for a UKO agent, the keystore connection in UKO for z/OS must be created using the CA certificate in PEM format. See "Defining keystores" on page 113.

Example 4-22 Configuration of AT-TLS for the UKO agent

```
## ----------------------------------------------------------------
# UKO Agent TLS setup
## ----------------------------------------------------------------
TTLSRule UKO_agent_50050
{
LocalPortRange 50050
LocalAddr ALL
Jobname UKOA
Userid UKOA
Direction INBOUND
TTLSGroupActionRef UKO-AGENT-TLS-GRPACTION
TTLSEnvironmentActionRef UKO-AGENT-TLS-ENVACTION
}

## ----------------------------------------------------------------
# UKO Agent group action
## ----------------------------------------------------------------
TTLSGroupAction UKO-AGENT-TLS-GRPACTION
{
TTLSEnabled On
Trace 255
}

## ---------------------------------------------------------------
# UKO Agent environment action
## ---------------------------------------------------------------
TTLSEnvironmentAction UKO-AGENT-TLS-ENVACTION
{
```

```
TTLSKeyRingParms
{
Keyring TLS-ACCESS
}
TTLSEnvironmentAdvancedParms
{
 ApplicationControlled  Off
 SecondaryMap Off
 TLSV1 Off
 TLSV1.1 Off
 TLSV1.2 Off
 TLSV1.3 On
 SSLV3 Off
}
TTLSCipherParms
{
V3CipherSuites TLS_AES_256_GCM_SHA384
}
HandShakeRole Server
}
```

UKO for z/OS server AT-TLS setup

Although it is possible to configure the UKO for z/OS server to use AT-TLS, it is recommended to use the TLS capabilities in IBM WebSphere Liberty.

If you use AT-TLS for the UKO for z/OS server, the UKO for z/OS server is an *unaware application*, which means it does not see the client certificate of the TLS connection. The effect is that mTLS needed for authentication to the UKO API by business applications is not possible.

Enabling AT-TLS for the UKO for z/OS server requires that UKO for z/OS is configured to use the plain HTTP port and AT-TLS is set up for that port, using a configuration similar to the one shown in Example 4-22 on page 110.

4.5.3 Automatic logon of UKO for z/OS to the UKO agent

Automatic logon of UKO for z/OS to the UKO agent is critical for secure and efficient key management operations for the following reasons:

- Eliminates manual authentication: Prevents the need for operators to enter credentials repeatedly, reducing human error and improving efficiency.
- Enables continuous key management: Supports automated tasks for key creation and distribution without interruption.
- Supports high availability and disaster recovery: Helps ensure seamless reconnection after failover or restart, avoiding downtime in key management services.
- Enforces secure, trusted communication: Uses ECC signature keys and RACF profiles to validate identity and prevent impersonation.
- Helps ensure compliance and audit: Maintains consistent identity for logging and audit trails, meeting regulatory requirements.

UKO for z/OS performs an automatic system logon to the UKO agent by using the ECC signature key. Therefore, the UKO agent must be told which client user ID to use for authorization. This is specified in the KMGPARM options data set with the &WEBCLIENT(client user ID) option.

To authorize the automatic logon, define a RACF resource profile in the XFACILIT class:

```
KMG.WEBCLIENT.<client user ID>
```

The UKO agent task user ID must have READ access to the resource.

For a specific UKO for z/OS key fingerprint there must be an authorization to use the &WEBCLIENT for logon. Define a RACF resource profile in the XFACILIT class:

```
KMG.LG. <64-character-hex-fingerprint>
```

The UKO agent task user ID must have READ access to the resource.

For a UKO agent, with task user ID 'UKOA', and parameter &WEBCLIENT(UKOCLT) specified in its KMGPARM option data set, the commands shown in Example 4-23 are used to allow log on to the UKO agent by UKO for z/OS having a signature key with fingerprint:

```
‘8A7A87509C40A5ED228E05DED51DD690EEFAC4C49B83E0A9A288B515D6745102’
```

Example 4-23 UKO for z/OS automatic logon configuration

```
RDEFINE XFACILIT KMG.WEBCLIENT.UKOCLT

PERMIT KMG.WEBCLIENT.UKOCLT CLASS(XFACILIT) ACC(READ) ID(UKOA)

RDEFINE XFACILIT -
KMG.LG.8A7A87509C40A5ED228E05DED51DD690EEFAC4C49B83E0A9A288B515D6745102

PERMIT -
KMG.LG.8A7A87509C40A5ED228E05DED51DD690EEFAC4C49B83E0A9A288B515D6745102 -
CLASS(XFACILIT) ACC(READ) ID(UKOA)
```

4.5.4 EKMF Workstation client user ID

An EKMF Workstation client user ID serves as the identity that authorizes the EKMF Workstation to interact with the UKO agent for key management operations. It is required for the following reasons:

- Access control: RACF uses this client user ID to validate that the EKMF Workstation has permission to perform key management tasks through the UKO agent.
- Separation of duties: Each EKMF Workstation user or process can have its own ID, ensuring granular control and auditability.
- Audit and compliance: When the EKMF Workstation logs actions to SMF, the client user ID is recorded, providing traceability for compliance and security audits.
- Secure communication: The UKO agent checks this ID against RACF profiles (KMG.EKMF.KMGPRACF.<task user ID>) before allowing operations like key creation or distribution.

Although UKO for z/OS has only a single client user ID, there can be multiple client user IDs for the EKMF Workstation.

An EKMF Workstation client user ID must be given permission to be used with an EKMF agent running under the specific task user ID. Define a RACF resource profile in the FACILITY class:

```
KMG.EKMF.KMGPRACF.<task user ID>
```

The client user ID(s) must have READ access to the resource. Example 4-24 shows the command used to permit client user 'EKMFCLT' to be used with an EKMF agent running as user 'UKOA'.

Example 4-24 Client user ID access

```
RDEFINE FACILITY KMG.EKMF.KMGPRACF.UKOA

PERMIT KMG.EKMF.KMGPRACF.UKOA CLASS(FACILITY) ACC(READ) ID(EKMFCLT)
```

4.5.5 UKO for z/OS

There are two essential steps in preparing UKO for z/OS to create data set encryption keys:

1. Defining the keystores so UKO for z/OS knows where to send keys and their associated key labels and which keystores are to receive them.
2. Defining key templates ensures a consistent key creation process and enforce naming standards for key labels.

   Key labels provide a unique identifier for each key, ensuring that keys can be accurately referenced, managed, and tracked throughout their lifecycle.

Defining keystores

Keystores act as the secure repository for cryptographic keys that are used in data set encryption.

A keystore in UKO for z/OS is defined by the network address of the z/OS system that holds the CKDS and the port number of the UKO agent running on that z/OS system. For details, see 10.3.2, “UKO keystores” on page 231.

Before UKO for z/OS can connect to a UKO agent, the UKO agent must be configured to trust UKO for z/OS.

A keystore belongs to a Vault in UKO (See 10.3.1, “UKO vaults” on page 231) and if multiple vaults send keys to the same LPAR, each vault must define its own connection to the keystore.

A keystore is defined in UKO for z/OS by selecting the vault in which the keystore belongs and the type of keystore, and specifying the information that is needed to connect to and use the keystore.

The first step in the keystore creation process is the selection of the vault for the keystore, as shown in Figure 4-30 on page 114.

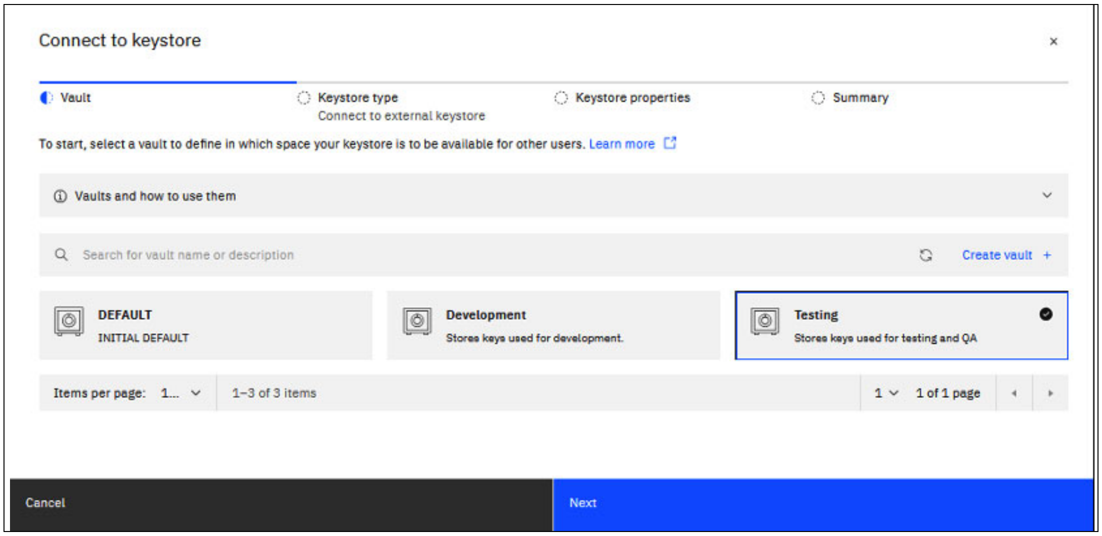


Figure 4-30 New keystore Vault selection

After selecting the vault, click **Next** to specify the keystore type, as shown in Figure 4-31.

Keystore type: For keys for dataset encryption and other ICSF keys, select the KMG Agent type, which puts keys into the CKDS on z/OS. The other available keystore types are the various cloud providers that are supported by UKO.

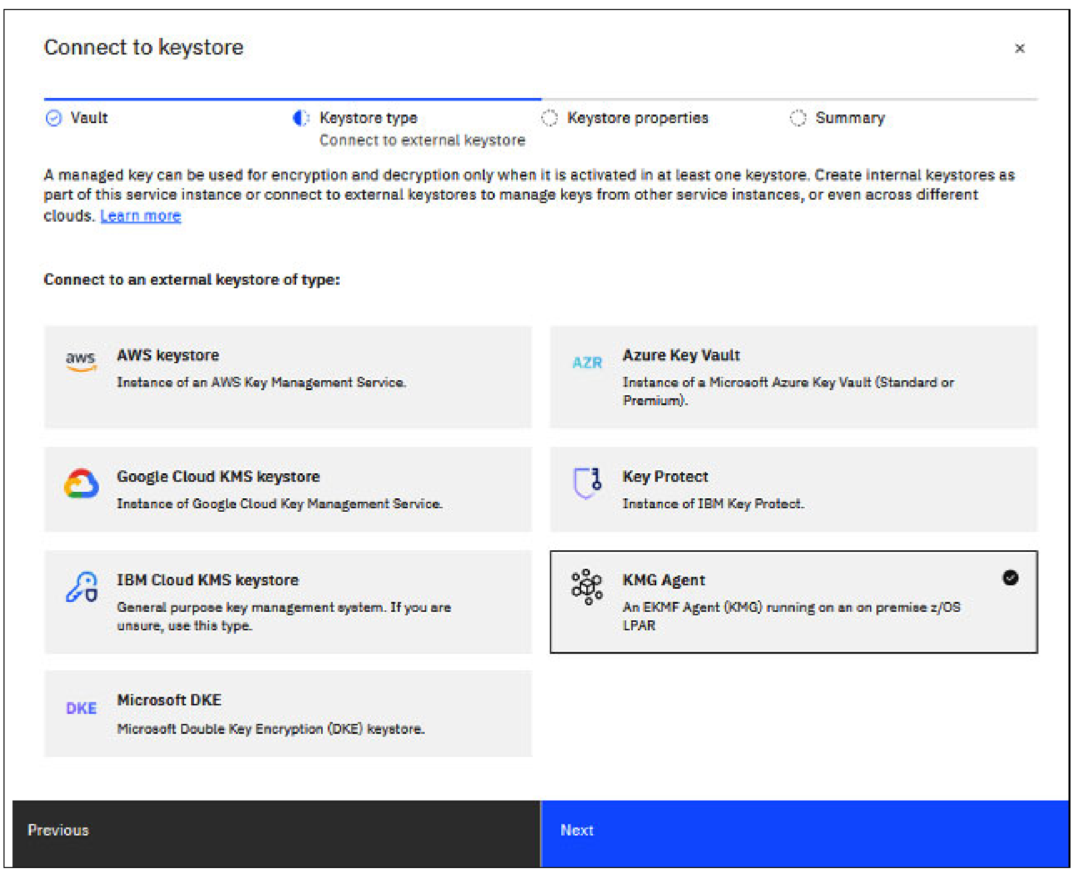


Figure 4-31 Selecting the keystore type

After selecting the keystore type, click **Next** (see Figure 4-32 on page 116) to specify the following keystore properties:

Name

The name is used only as an identification of the keystore in UKO for z/OS.

Description

Description of the keystore.

Keystore Groups

Comma-separated list of names and labels that identify the keystore. If the same label is used on multiple keystores, the label identifies all the keystores as a single group.

Address or host name

The network address of the z/OS system where the keystore and UKO agent reside. The z/OS system programmer who installed the UKO agent provides this information.

Port number

The port number is the port on which the UKO agent listens for connections. This information is provided by the z/OS system programmer who installed the UKO agent.

Public Key Hash

The public key hash is used to establish trust between UKO for z/OS and the UKO agent. This information is provided by the z/OS system programmer who installed the UKO agent.

PEM Certificate for TLS connection

Specify the PEM-format CA certificate of the TLS certificate used by the UKO agent only if the UKO agent is configured to use TLS.

Tip: The UKO agent can be set up by using AT-TLS for added connection security. See 4.5.2, “UKO for z/OS AT-TLS setup” on page 109.

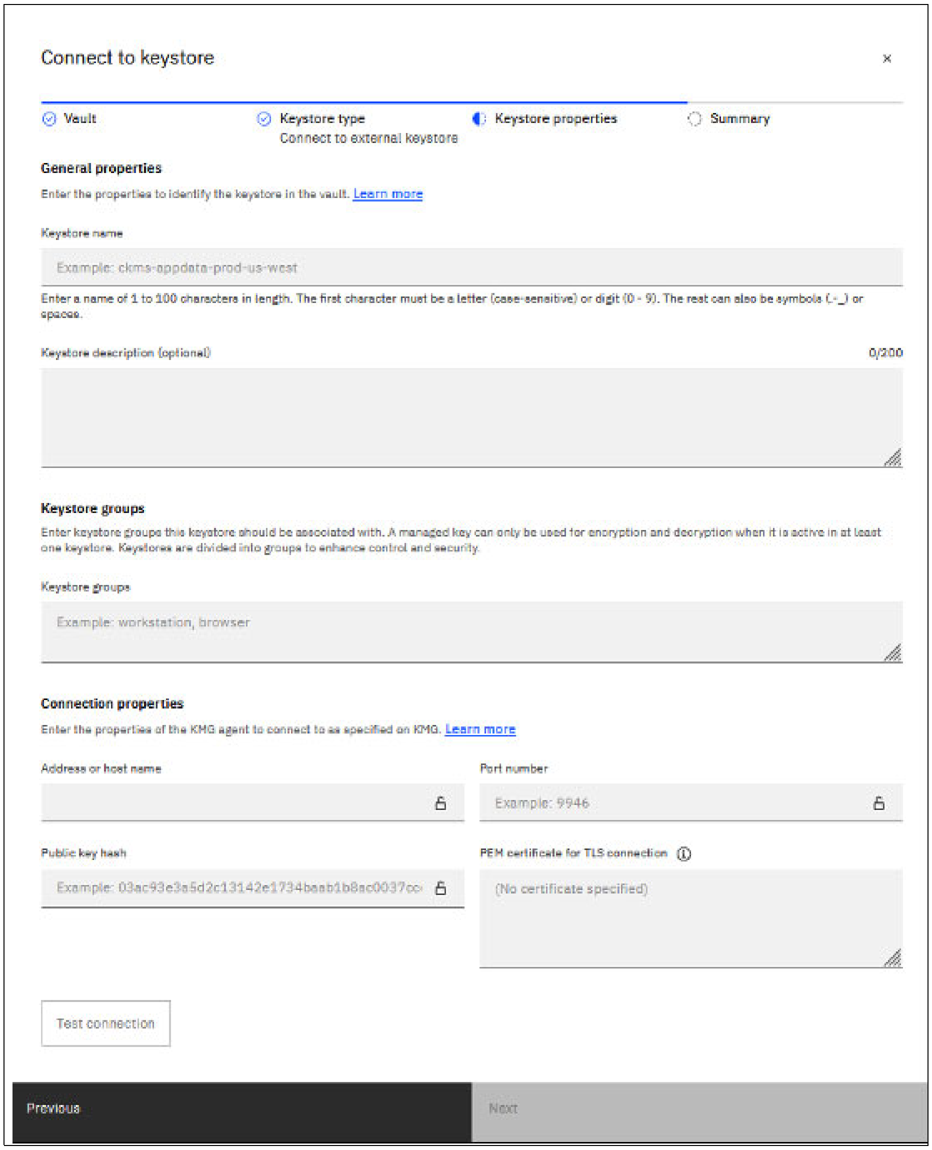


Figure 4-32 Providing keystore properties

Defining key templates

Keys are defined by key templates, which ensures a consistent key creation process. Each key template belongs to a vault in UKO (See 10.3.1, “UKO vaults” on page 231).

The first step in the key template creation process is the selection of the vault for the key template, as shown in Figure 4-33 on page 117.

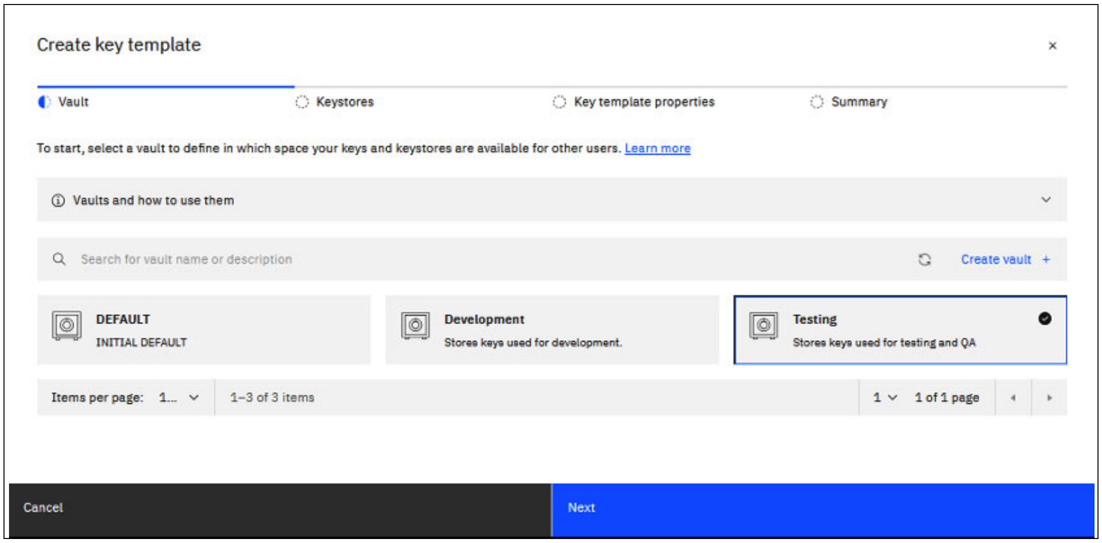


Figure 4-33 New Key Template Vault selection

After selecting the vault, click **Next** to progress to specifying the keystore type for the key template as well as the associated keystore groups, as shown in Figure 4-34. The properties to specify are the following:

- The **Keystore type** must be set to ‘KMG Agent’ to put keys on the IBM Z platform.
- Select the **Keystore group** that contains the keystores to which the key is to be distributed.

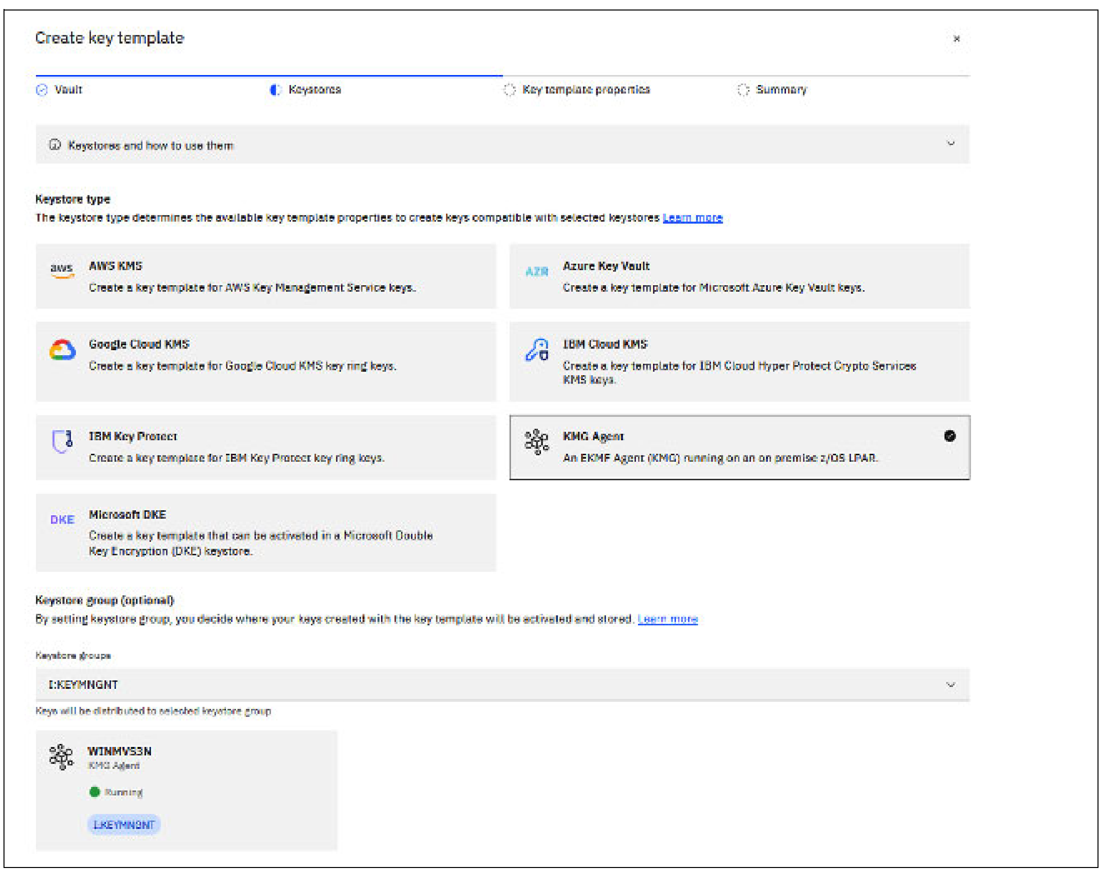


Figure 4-34 Selecting the keystore type

After selecting the keystores for the key template, click **Next** to specify the key properties of the key template, as shown in Figure 4-35 on page 119. Specify the following properties:

Key Template Name

Used as an identification of the key template within the UKO for z/OS. The Key Template Name can be between 1 and 100 characters.

Description

Provides a means to document the purpose and use of the key template.

Naming Scheme

Defines the ID of the key in UKO for z/OS and the keystore label in the CKDS. The naming scheme can contain tags, which are placeholders for values that can be specified at key generation time. Using a mix of literal text and tags means that a key template can enforce a given key label naming convention. See "Key label" on page 232.

Key instance naming scheme

Provides the means to specify a naming scheme for the key in the CKDS, which is different from the ID of the key.

Algorithm

Use the selection to specify the algorithm of the key to create. Supported algorithms are AES, HMAC, RSA, EC, ML-KEM, and ML-DSA. Select AES for a key for data set encryption.

Length

Also called Matrix Size is used to specify the key size of the key. Available values depend on the selected algorithm. The key size for an AES key for data set encryption is always 256 bits.

Key type

For AES keys, it can be either AES CIPHER keys or AES DATA keys. AES CIPHER keys are the recommended key type.

Key state

Refers to the UKO key state model (see 10.3.4, "Key lifecycle" on page 237) based on NIST recommendations.

Notes:

- A key that is created as *Active* is distributed to keystores as part of the key generation process.
- A key that is created as *pre-active is* not distributed. It can later be activated and distributed to keystores.
- Keystore group specifies the keystores (defined in "Defining keystores" on page 113) to where the key is distributed. Select the relevant keystore group in the drop-down menu.

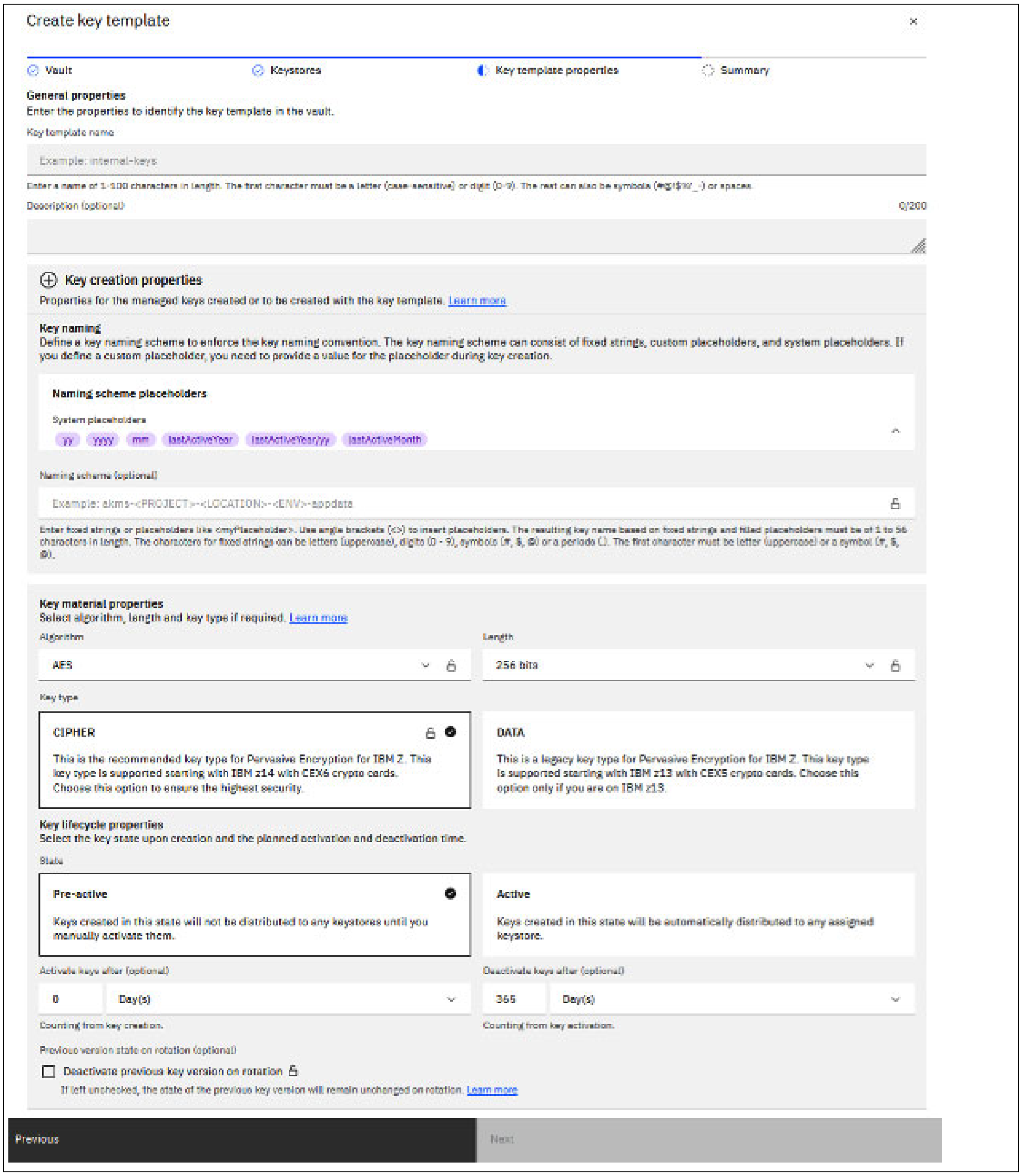


Figure 4-35 Create new key template

4.5.6 EKMF Workstation

There are two parts to preparing the EKMF Workstation for creating data set encryption keys. First, hosts must be defined to define how the EKMF Workstation sends keys to the CKDS keystores on those hosts. Second, key templates must be defined, which tells the EKMF Workstation the key label to use and which keystores the key is to be distributed.

Defining a host

A host in the EKMF Workstation is defined by the network address of the z/OS system that holds the CKDS and the port number of the UKO agent running on that z/OS system.

Before the EKMF Workstation can connect to an UKO agent, the UKO agent must be configured to trust the EKMF Workstation (see Figure 4-36 on page 121):

Host Identification: Unique name for the host.

Description: A short description of the host.

Host Type: Must be set to 'Z-SERIES' to identify the host where the UKO agent resides as z/OS. This information is provided by the z/OS system programmer who installed the UKO agent.

IP Name: Hostname of the z/OS host where the UKO Agent is running. If a DNS server is not defined, then an IP Address must be used.

IP Address: Updated based on the value entered in the IP Name field.

Port number: TCP port on which the UKO agent listens for connections. This information is provided by the z/OS system programmer who installed the UKO agent.

Timeout value: Amount of time that the EKMF Workstation waits for a response from the UKO agent. If the timeout is exceeded, the UKO agent is considered unavailable.

Codepage Name: Indicates which codepage the EKMF Workstation uses to communicate with the UKO agent. This information is provided by the z/OS system programmer who installed the UKO agent.

Link Encryption: Defaults to ECHD. This establishes an AES session encryption key by using an Elliptic-Curve Diffie-Hellman key exchange. The other link-encryption options will be removed in future versions of the EKMF Workstation.

TLS Mode: Tells the EKMF Workstation if the UKO agent is set up with AT-TLS (See "UKO agent AT-TLS setup" on page 110). Options are Auto, Disabled, and Enforced.

- **Auto:** The EKMF Workstation attempts TLS and falls back to no TLS if necessary. This mode is compatible with UKO agents, regardless whether AT-TLS is configured. With this mode, a non-AT-TLS enabled UKO agent can be reconfigured to use AT-TLS without affecting the EKMF Workstation.
- **Disabled:** The EKMF Workstation does not use TLS to communicate with the UKO agent.
- **Enforced:** The EKMF Workstation requires a TLS connection to be successful. When a UKO agent is configured with AT-TLS, change the host definition TLS Mode to Enforced such that TLS is required.

- **Logon group:** Used to group hosts into groups. The EKMF Workstation asks for credentials only once per logon group and reuses those credentials for all Hosts in the logon group.

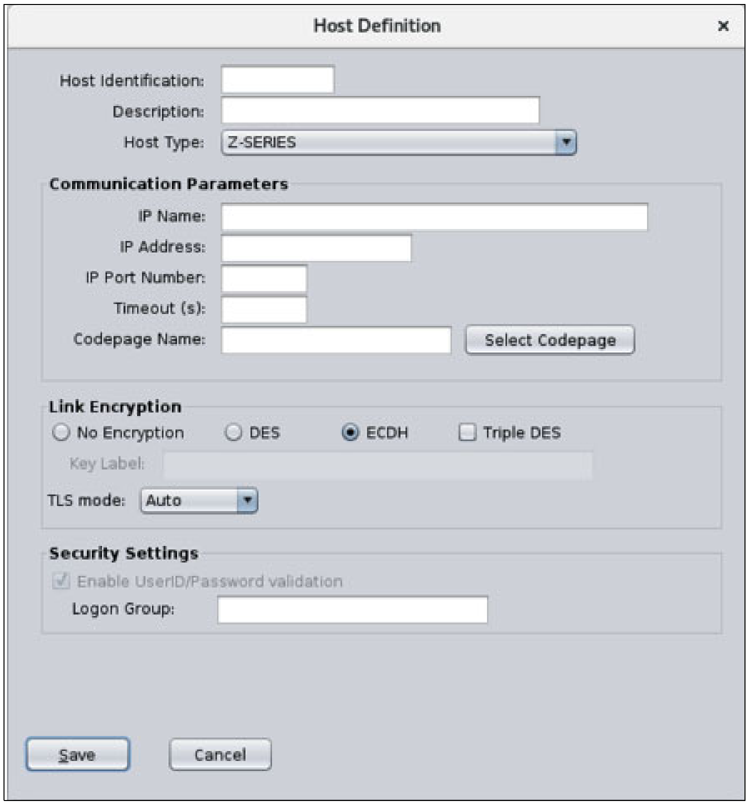


Figure 4-36 EKMF Workstation - host definition

Having defined all relevant hosts, update the EKMF Workstation device configuration to enable the EKMF Workstation to distribute keys to the CKDS keystores on the hosts.

Defining a key template

Keys are defined by key templates, which ensures a consistent key creation process.

The following paragraphs describe the fields required in a key template to create an AES CIPHER key or an AES DATA key that can be used for data set encryption. It is recommended to complete all fields of a key template. Additional descriptions of key template fields are in the EKMF Workstation documentation.

CIPHER KEYS

To create a key template for an AES CIPHER key, the fields in the key template (see Figure 4-37 on page 122), must be specified as follows:

Title

Short description of the purpose of the key template.

Number

Unique identifier for the key template. When set, it cannot be changed. Valid characters are a–z, 0–9 and underscore (_).

Status

When set to **Active,** uses the key template to generate keys.

Key label

Label that identifies the key in the EKMF Workstation. (The keystore label for the CKDS is defined in the key template instances.)

Key labels can contain tags, which are placeholders for values that can be specified at key generation time. Using a mix of literal text and tags means that a key template can enforce a given key label naming convention (see “Key label” on page 232).

Key state

The UKO key state model (see 10.3.4, “Key lifecycle” on page 237) based on NIST recommendations.

A key that is created as *active* is distributed to keystores as part of the key generation process.

A key that is created as *pre-active* is not distributed. It can later be activated and distributed to keystores.

- **Key size**

  For an AES key for data set, encryption is always 256 bits.

  The selected origins in a key template controls the origin of the key material for the key. Specify '**Generate**' to be able to generate a key using the key template.

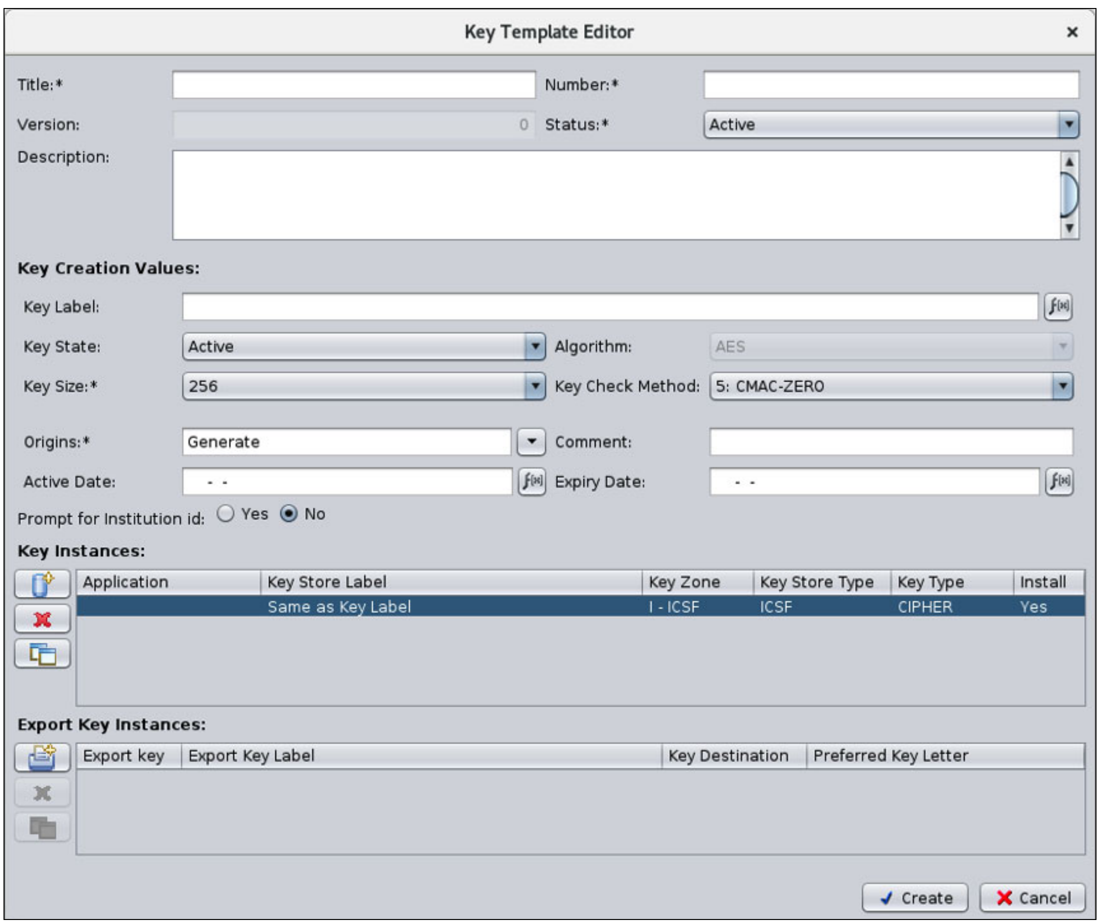


Figure 4-37 EKMF Workstation - key template editor

Key instances: Describe separate instances of a key in the CKDS keystore(s). See Figure 4-38.

For a PE key, select **Yes** for the **Install** button and set the keystore type as ICSF.

The **Key Store Label** field can be set to reference the previously specified key label, or a different key label can be defined.

The values to specify for application and key zone are specific to the particular EKMF Workstation setup because these two values combined determine the specific Hosts/CKDS keystores to which the key is distributed. Specifying KEYMNGNT for application and 'I - ICSF' for the key zone typically distributes a key to all configured hosts/CKDS keystores.

Select the KEK algorithm to be AES and select an AES KEK used to protect the AES CIPHER key when stored in the UKO repository and during transport from the EKMF Workstation to the CKDS.

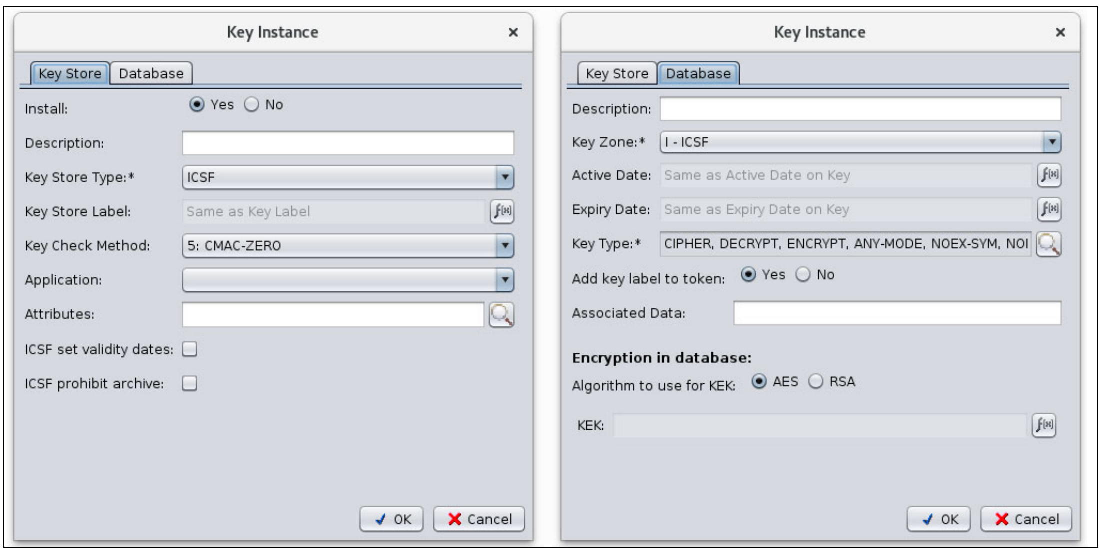


Figure 4-38 EKMF Workstation - key instance

The key type has a separate editor (see Figure 4-39 on page 124), which is opened by clicking the 'magnifying-glass' icon to the right of the key type field.

The key type defines the type and attributes of the key in the CKDS. For the key to be usable as a PE key, select the following values:

- **Key type**: CIPHER
- **CPACF export control**: XPRTCPAC.
- **Key-usage control**: DECRYPT and ENCRYPT
- **Key-usage mode**: ANY-MODE

The additional settings control key exportability and its key token format. The selection doesn't affect the usability of the key, but it is recommended to configure the key to be nonexportable, because it prevents unauthorized copying of the key.

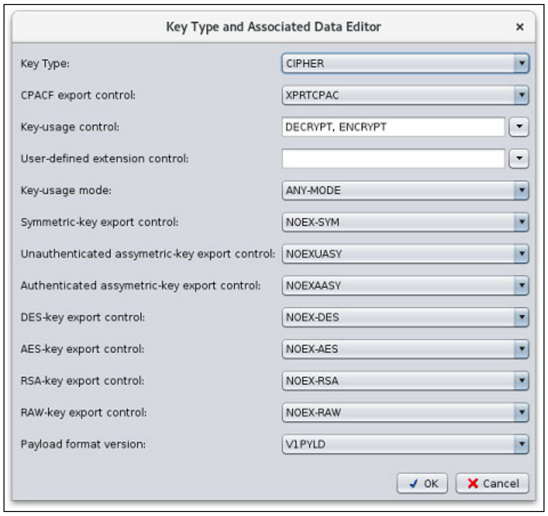


Figure 4-39 EKMF Workstation - key editor (CIPHER key)

Defining a data key

It is recommended to not use AES data keys on systems that support AES Cipher keys.

If a data key is required, create a key template as for CIPHER key (see Figure 4-38 on page 123), with the notable difference that the key type selection is not used, and the KEK algorithm is set to RSA.

For the DATA key specific fields (shown when selecting RSA as the KEK algorithm), set 'KEK location' to UKDS7 and Token Format to PKCSOAEP (see Figure 4-40).

Select an RSA KEK used to protect the AES DATA key when stored in the UKO repository and during transport from the EKMF Workstation to the CKDS.

Figure 4-40 EKMF Workstation - Key instance - DATA key

5

Deploying z/OS data set encryption

This chapter provides information about how to deploy z/OS data set encryption. It also includes checklists to help you determine whether your environment is ready for deployment.

This chapter includes the following topics:

5.1 Readiness checklists for deployment

Questions to help you determine what is needed for your z/OS data set encryption implementation are listed in Table 5-1. Answer any questions that are relevant to your environment to provide a complete understanding of the requirements.

Table 5-1 Checklist for determining deployment readiness for z/OS data set encryption

Checklist item	Comments	More information
Have you installed the required hardware and software components and prerequisites?	Data set encryption is supported by all in-service z/OS releases. CPACF and Crypto Express encryption rates for AES on the current IBM Z generation are faster than previous generations according to IBM tests.	2.2, "Required and optional hardware features" on page 26 2.3, "Required and optional software features" on page 29
Have you determined the performance effect that z/OS data set encryption might have on the CPUs?	Use the IBM Z Batch Network Analyzer (zBNA) and zCP3000 to determine potential performance effects.	2.3.7, "IBM zBNA" on page 33
Have you determined which data sets will be encrypted?	Supported data set types are: Virtual Storage Access Method (VSAM) extended format, sequential extended format, sequential basic and large format, and PDSE.	3.3.1, "Supported data set types" on page 38
Will you use data compression[a]?	Encrypted data is not compressible. Therefore, where possible, consider the use of access method compression with data set encryption. When used together, the access methods compress the data at the server before it is encrypted and written out. Access method compression is only supported with extended format data sets. For compressing extended format sequential data sets, zEDC offers a solution for compression.	3.3.2, "Data set compression" on page 40
Have you defined the naming standards that you will use?	Consider defining naming conventions to group data sets logically. Proper naming standards facilitate the ease of migrating key labels and keys to another machine or Business Partner.	3.3.3, "Data set naming conventions" on page 40 3.6.7, "Creating a key label naming convention" on page 52
Have you identified how to supply the key label to create an encrypted data set?	To create an encrypted data set, a key label must be supplied at the time the data set is initially allocated (created).	"Starting z/OS data set encryption deployment" on page 130
Have you determined the owners of the new tasks for administering and maintaining encrypted data sets?	Fine or course-grained access controls can limit access to data content by personnel that can otherwise pose a possible exposure point.	3.4.2, "Separating duties of data owners and administrators" on page 44
Have you determined the Resource Access Control Facility (RACF) or System Authorization Facility (SAF) controls that you will be using?	Assess access controls based on your security policies and how they apply to data set encryption.	4.2, "RACF configuration" on page 71 Table 5-2 on page 128

Checklist item	Comments	More information
Have you determined which key management tools you will use?	You can create master keys by using a TKE workstation, which includes smart cards and smart card readers. You can create operational keys by using UKO or an alternative product. ICSF is a panel-driven operational key entry and management tool. **Note:** UKO is useful in managing secure keys in large or multi-site organizations.	3.6, "Key management considerations" on page 49
Have you defined your key lifecycle management process?	Track changes to key's parameters during its lifecycle.	3.6.14, "Determining key availability needs" on page 62
Does your PARMLIB contain the required ICSF CSFPRMxx and Installation Options Data Set?	Parameters for ICSF setup.	4.3.3, "CSFPRMxx and installation options" on page 83
Have you determined your ICSF storage requirements?	Determine the allocation size and format of the CKDS.	3.5.3, "Using the Common Record Format (KDSRL) cryptographic key data set" on page 46
Have you determined the best time to start ICSF during the IPL process?	ICSF must always be up and running because all access to encrypted data sets depends on calls that are made to ICSF.	3.5.2, "Starting ICSF early in the IPL process" on page 45
Have you considered your backup/recovery planning scenarios?	The disaster plan includes a mirrored implementation of data set encryption at the backup site with the appropriate master key, ICSF release, and key data sets.	3.6.14, "Determining key availability needs" on page 62 3.7, "General considerations" on page 64
Have you considered sysplex sharing and remote site usage, if applicable?	Consider whether systems in the parallel sysplex must share the master key and CKDS.	3.5.6, "Sharing the CKDS in a sysplex" on page 48
Have you considered what is required to fall back?	Any implementation requires a plan to fall back.	3.7.3, "Backing out of z/OS data set encryption" on page 66
Have you reviewed your security, audit, and compliance practices?	In support of z/OS data set encryption, identify if any gaps exist in your current practices.	3.7, "General considerations" on page 64
Are you collecting applicable System Management Facility (SMF) records for data set encryption?	Ensures that you are compliant and ready for an audit by collecting SMF Record types 14, 15, 62, and 82.	Chapter 6, "Auditing z/OS data set encryption" on page 149
Have you considered the use of security tools for your Z environment?	Ensure that your security policies and keys are managed and monitored. Auditors can quickly determine whether files were encrypted with IBM Security zSecure Suite.	Chapter 2, "Identifying components and release levels" on page 25

Checklist item	Comments	More information
Have you planned for test scenarios and education for potential users?	This IBM Redbooks publication can be used as a reference for building test scenarios and education.	Review all chapters of this publication.

a. IBM z15® and later provide significant compression performance improvements with the (on-chip) IBM Integrated Accelerator for zEnterprise Data Compression.

A checklist that can be used for implementing access controls as part of data set encryption is included in Table 5-2.

Table 5-2 Access controls and policies checklist

Checklist item	Comments	More information
Have you determined your data set access criteria?	This issue includes restricting access to the ICSF data sets, which contain the encryption keys and associated key labels for your installation.	4.2, “RACF configuration” on page 71
Have you defined your RACF profiles, including CSFSERV, CSFKEYS, and XFACILIT?	CSFSERV profiles control access to ICSF services, such as defining encryption keys. CSFKEYS profiles control access to cryptographic keys in the ICSF Key Data Sets (CKDS and PKDS) and enables or disables the use of protected keys. XFACILIT class defines rules for the user of encrypted key tokens that are stored in the CKDS and PKDS.	
Have you created entries in OPERCMDS?	To ensure that operators do not enter STOP ICSF or SET ICSF commands without authorization.	
Have you reviewed the Catalog LISTCAT access command?	Restricted usage of LISTCAT display of data sets.	

5.2 Deploying z/OS data set encryption

This IBM Redbooks publication assumes the z/OS data set encryption configuration use the CPACF and Crypto Express features on the Z platform, and secure keys and protected keys.

With the planning complete and the prerequisites in place, you are now ready to deploy z/OS data set encryption. For the tasks that are described in the remainder of this chapter, assume that the following prerequisites are met:

- ICSF is up and running with the CKDS initialized
- Crypto Express adapters are configured in CCA mode with assigned domains
- AES master keys are active

If ICSF is not yet installed in your environment, see the following resources:

- For more information about planning and configuration, see *z/OS Cryptographic Services ICSF Administrator's Guide*.

- For more information about for installation, initialization, and customization, see *Abstract for Cryptographic Services ICSF: System Programmer's Guide*.

Note: If you plan to share encrypted data sets across multiple systems, ensure that all z/OS data set encryption prerequisites were met on all systems before shared encrypted data sets are created.

The IBM Z hardware and software environment that is used in this chapter is shown in Figure 5-1. It consists of the following components:

- IBM Z LPAR with:
  - CPACF enabled
  - Crypto Express adapter (with a domain value of 084)
- z/OS is running with:
  - Resource Access Control Facility (RACF).
  - Integrated Cryptographic Service Facility (ICSF) with the Advanced Encryption Standard (AES) master key loaded.
  - Cryptographic key data set (CKDS) in common record format (KDSR) to allow for reference date tracking.

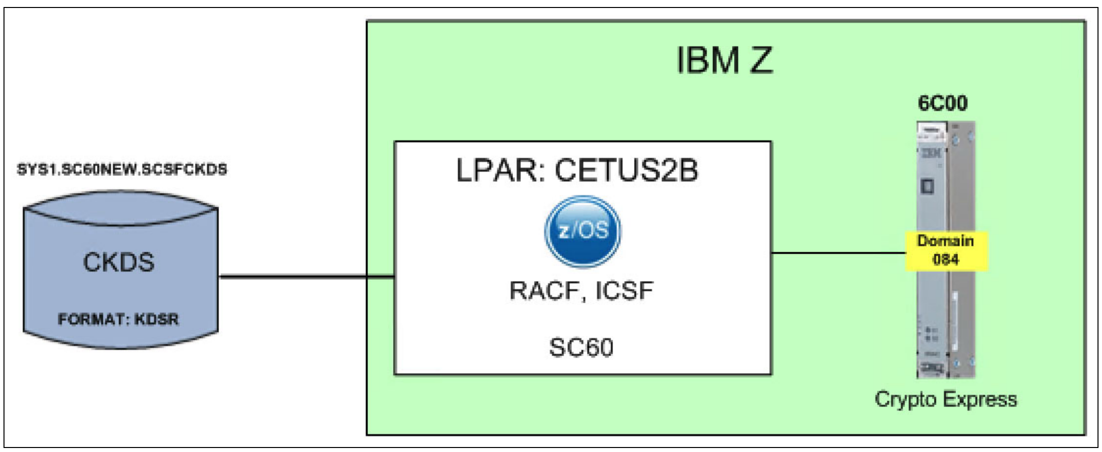


Figure 5-1 Our IBM Z hardware and software environment

Because this scenario was built as a monoplex environment, the CKDS is not shared with other systems. For more information about the steps to share the CKDS with other systems, see 4.3.3, “CSFPRMxx and installation options” on page 83.

Starting z/OS data set encryption deployment

The high-level steps in the deployment of data set encryption are shown in Figure 5-2.

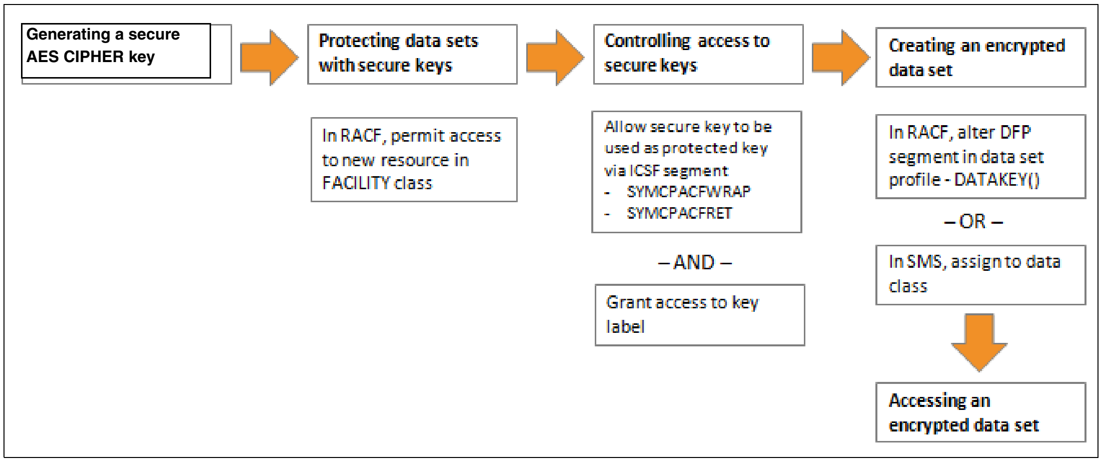


Figure 5-2 High-level steps for deployment of data set encryption

A data set is defined as encrypted when a key label is supplied on allocation of a new data set of a data set encryption supported data set type. A key label is supplied by way of new keywords in any of the following sources (using order of precedence):

- The Data Facility Product (DFP) segment in the RACF data set profile
- JCL, Dynamic Allocation, TSO Allocate, and IDCAMS DEFINE
- Storage Management Subsystem (SMS) Construct: Data Class

Choosing the method to request data set encryption

For a specific high-level qualifier (HLQ), you might have many different RACF profiles. You might think that to encrypt these data sets, you must update all the RACF profiles, a process that is burdensome. You might also think that a better approach is to update one or a few data classes, according to the SMS rules. The use of DFSMS can be viewed as more generic and an easier method to implement.

You might find it easier to use the DATACLAS keyword in your environment. However, a side effect of the use of this method is that it takes control of creating encrypted data sets out of the hands of a security administrator. Instead it puts control into the hands of the Db2 administrators, storage administrators, or other users. One important reason to use data set encryption is to prevent administrators from accessing sensitive data and removing them from compliance scope.

z/OS data set encryption as a whole is intended to be a security capability. As a security capability, use the security profiles (that is, DATASET class) along with defining the following resource profile in the FACILITY class:

```
RDEFINE FACILITY STGADMIN.SMS.ALLOW.DATASET.ENCRYPT UACC(NONE)
```

Specifying `UACC(NONE)` prevents unauthorized users from encrypting data sets without oversight by the security administrator.

From a security perspective, the security administrator controls, determines, and oversees who can encrypt data sets. It might be said that enough security oversight is provided because the security administrator gives users the authorization to a key label within the CSFKEYS class. However, the recommended way to manage encrypted data sets is to use the RACF profile. Therefore, it takes precedence over all other means of setting a key label.

A tradeoff exists between ease of setup (that is, SMS DATACLAS) and security control (that is, the RACF DATASET class).

> **Important:** Use of the RACF profile is the recommended method for z/OS data set encryption to ensure the most secure environment.

5.3 Generating a secure 256-bit data set encryption key

Secure or protected keys are recommended for use with z/OS data set encryption. For more information about secure and protected keys, see 1.6.3, “IBM Z cryptographic system” on page 16 and 3.6.2, “Reviewing industry regulations” on page 50.

Generating a secure 256-bit data set encryption key can be done by using one of the following methods:

- Unified Key Orchestrator (UKO)
- ICSF panels
- ICSF APIs
- CSFKGUP

5.3.1 Using Unified Key Orchestrator

Unified Key Orchestrator (UKO) can generate and manage keys in multiple keystores across multiple platforms. This includes the ICSF CKDS on z/OS. UKO creates keys centrally and then distributes the generated keys to the relevant keystores.

Keys are created based on the key templates prepared in “Defining key templates” on page 116 (for UKO for z/OS) or “Defining a key template” on page 121 (for the EKMF Workstation).

Key generation with UKO for z/OS

Key generation in UKO for z/OS is done using key templates. When starting a key generation, first select the Vault in which the key belongs. Then select the key template, which describes the key to be generated.

The content of the key template is shown (see Figure 5-3 on page 132), and input fields are available for the tags used in the key label. Do the following actions:

- Specify values for the tags to finalize the value of the key label.
- When you use the <seqno> tag, UKO for z/OS calculates the appropriate next sequence number during key generation. Until then 00000 is displayed as a placeholder.
- Click **Next** and then click **Create key** to create the key.

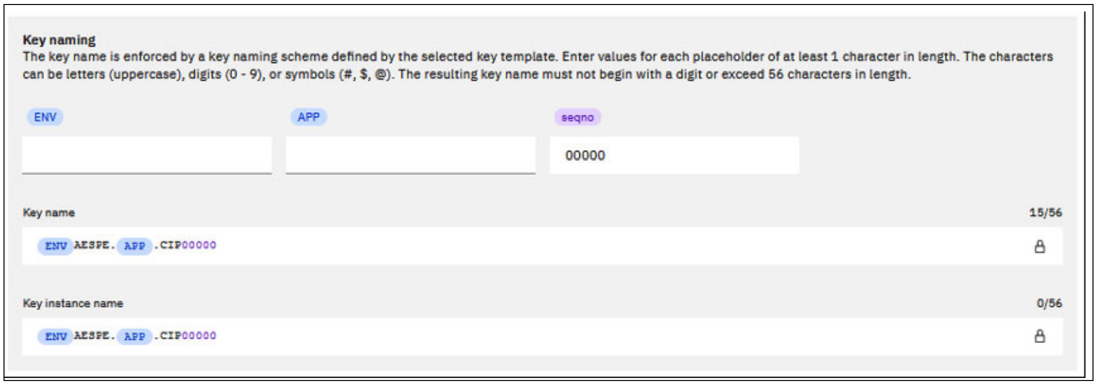


Figure 5-3 UKO for z/OS create CIPHER key

If the key template is specified to create the key is in the active state, the key generation process includes distribution of the key to keystores. The keystores are those specified by the key template that was used to generate the key.

If the key template specifies to create the key in the pre-active state, the key is generated but is not distributed.

Distribution to keystores happens when the key is activated. This activation can be done from the key list overview (see Figure 5-4 on page 133).

At the end of each row of keys, there is a menu symbolized by three dots, arranged vertically. Open the menu for the key to be activated. Under the **Change state to** heading, select the new state, such as **Active**.

A dialog opens, asking for confirmation of the state change. Click **Activate key** to perform the state change. The key state changes to active, and the key is distributed to the keystores specified by the key template that was used to generate the key.

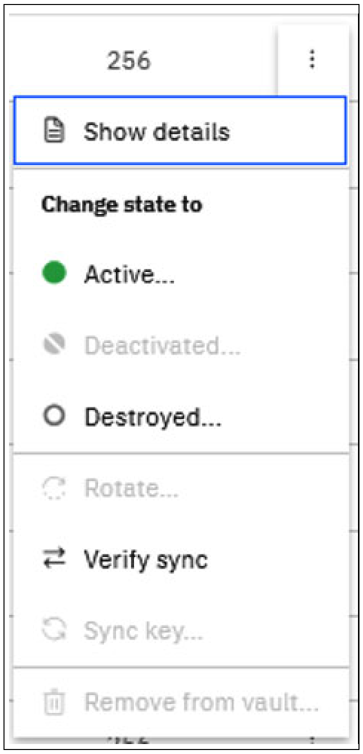


Figure 5-4 UKO for z/OS changing key state

Key generation with EKMF Workstation

Key generation in the EKMF Workstation is done by using key templates. When starting a key generation, a dialog opens as shown in Figure 5-5 on page 134, which guides you through the key generation process:

1. Select the key template that describes the key to be generated.
2. The content of the key template is shown, and input fields are available for the tags that are used in the key label. When you use the <seqno> tag, the EKMF Workstation automatically determines the next available sequence number.
3. Enter the tags and any other required information.
4. The UI shows a summary of what the EKMF Workstation does in the key generation. Confirm the summary to start the key generation process.
5. The system confirms that the key was generated and also confirms whether the key was distributed to keystores.

If the key template specified to create the key is in the active state, the key generation process includes distribution of the key to keystores. The keystores are those specified by the key template that was used to generate the key.

If the key template specified to create the key is in the pre-active state, the key is generated but is not distributed.

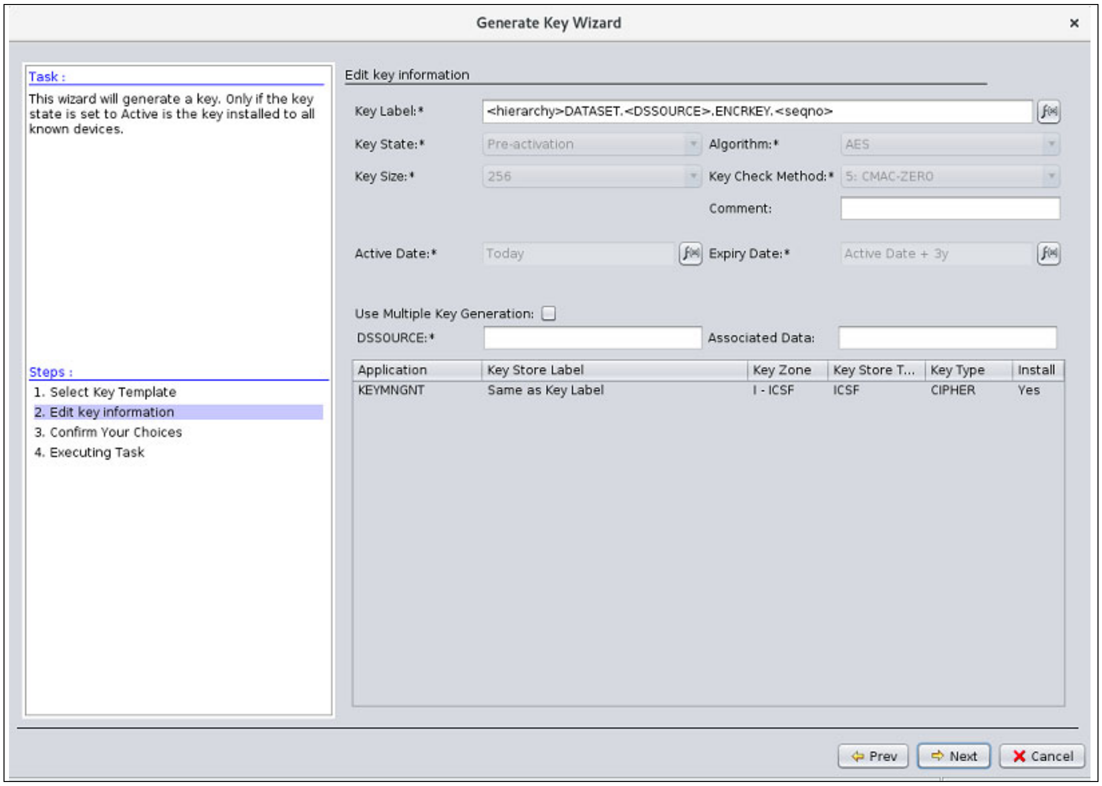


Figure 5-5 EKMF Workstation Generate Key Wizard

Distribution to keystores happens when the key is activated.

The activation is done from the (right-click) context menu of the key (see Figure 5-6), when it is displayed in the list overview.

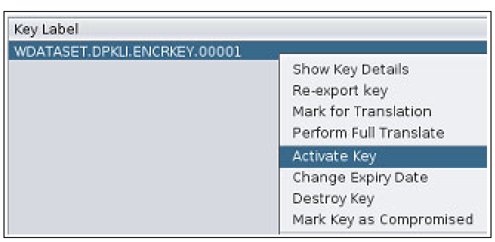


Figure 5-6 EKMF Workstation - key activation

Selecting **Activate Key** from this menu changes the key state from pre-active to active. The key is distributed to the keystores specified by the key template that was used to generate the key.

5.3.2 Using ICSF panels

ICSF gives users the option of generating AES DATA keys by using the ICSF Utility panel. From the panel, you specify a key label and the key length to generate a key.

Authorization

Users must have READ authority to the following resources in the CSFSERV class to perform key generation by using the ICSF panels:

- CSFKRR2
- CSFKGN
- CSFKRC2

For more information about the RACF commands to authorize users to the CSFSERV class, see 4.2.2, “Defining DATASET, CSFSERV, CSFKEYS, and other resources” on page 71.

Generating a 256-bit AES DATA key with ICSF panels

From the ICSF primary menu (see Figure 5-7), select **5 UTILITY - ICSF Utilities** → **5 CKDS KEYS - Manage keys in the CKDS** → **7 Generate AES DATA keys**.

```
HCR77C1  -------------- Integrated Cryptographic Service Facility -------------
OPTION ===>
System Name:  SC60                            Crypto Domain: 84
Enter the number of the desired option.

  1  COPROCESSOR MGMT -  Management of Cryptographic Coprocessors
  2  KDS MANAGEMENT   -  Master key set or change, KDS Processing
  3  OPSTAT           -  Installation options
  4  ADMINCNTL        -  Administrative Control Functions
  5  UTILITY          -  ICSF Utilities
  6  PPINIT           -  Pass Phrase Master Key/KDS Initialization
  7  TKE              -  TKE PKA Direct Key Load
  8  KGUP             -  Key Generator Utility processes
  9  UDX MGMT         -  Management of User Defined Extensions

     Licensed Materials - Property of IBM
     5650-ZOS Copyright IBM Corp. 1989, 2017.
     US Government Users Restricted Rights - Use, duplication or
     disclosure restricted by GSA ADP Schedule Contract with IBM Corp.

Press ENTER to go to the selected option.
Press END   to exit to the previous menu.
```

Figure 5-7 ICSF for System SC60

In the CKDS Generate Key panel (see Figure 5-8), enter the key (record) label and select the AES key bit length. Press Enter to generate the key.

```
--------------------------- ICSF - CKDS Generate Key ---------------------------
COMMAND ===>

Active CKDS: SYS1.SC60NEW.SCSFCKDS

Enter the CKDS record label for the new AES DATA key
==> DATASET.ENCRYPTKEY.001

AES key bit length:  _ 128  _ 192  / 256






Press ENTER to process
Press END to return to the previous menu
```

Figure 5-8 CKDS Generate Key panel

If the record exists, the Record Replace Confirmation panel appears (see Figure 5-9 on page 136).

Important: By replacing the key, applications cannot decrypt the associated data sets.

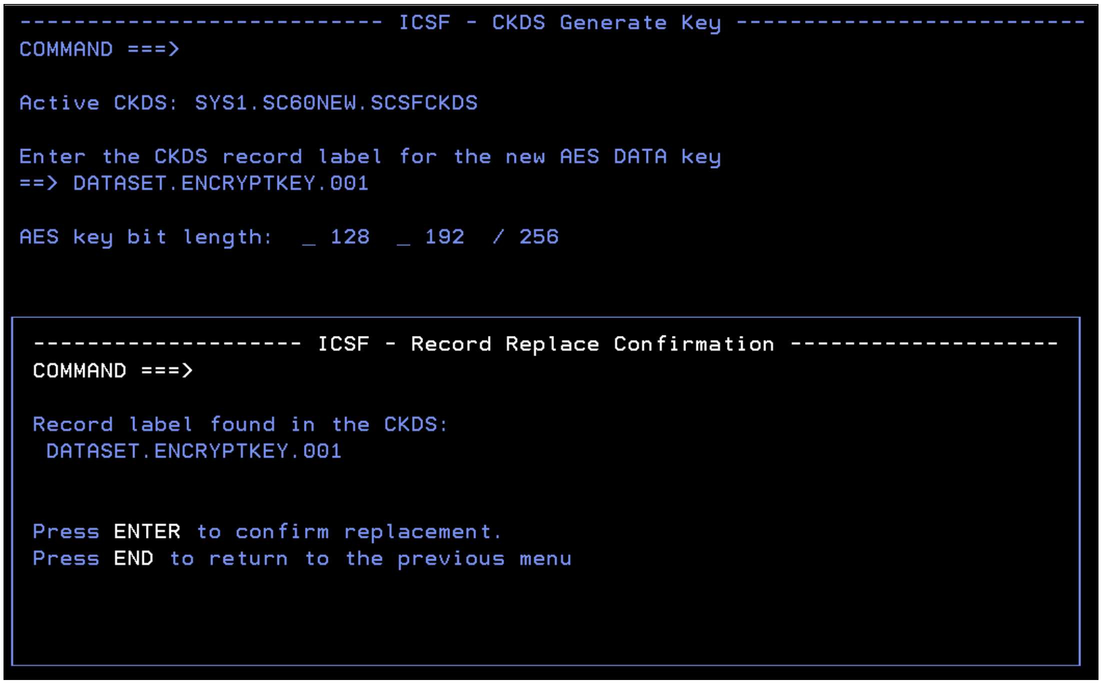


Figure 5-9   Record Replace Confirmation

You see a message that indicates a 256-bit AES DATA key was successfully generated (see Figure 5-10).

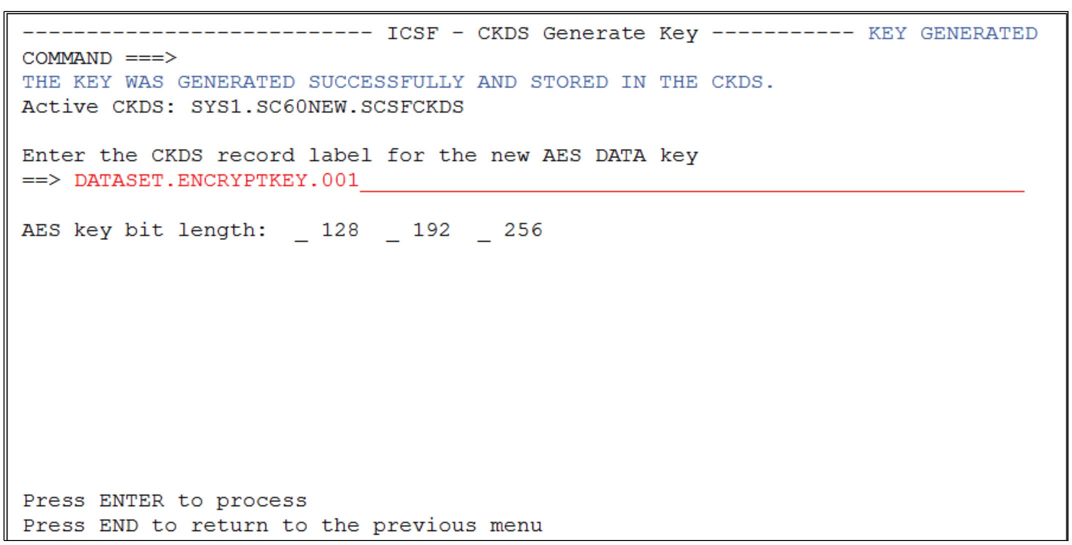


Figure 5-10   Key generated successfully message

No refresh of the CKDS is required when the key is generated by using this method.

5.3.3 Using ICSF APIs

ICSF-callable services support the programmatic generation of AES DATA or CIPHER keys. After a key is generated, the key can be stored in the CKDS and associated with a key label.

Authorization

The authorization required depends on which key type is being created. Users must have READ authority to the following resources in the CSFSERV class to perform key generation with the associated ICSF callable service:

- CSFKGN (DATA keys)
- CSFKGN2 (CIPHER keys)
- CSFKRC2

For more information about the RACF commands to authorize users to the CSFSERV class, see 4.2.2, “Defining DATASET, CSFSERV, CSFKEYS, and other resources” on page 71.

> **Note:** The sample scripts shown in Appendix B, “Sample REXX scripts for creating DATA and CIPHER keys” on page 253 also use CSFKRR or CSFKRR2, as such the user must be authorized to those resources in the CSFSERV class (unless the sample is modified).

Generating a 256-bit AES CIPHER Key with ICSF APIs

For more information about a REXX sample to create a 256-bit AES CIPHER type key, see Example B-1 on page 253.

Generating a 256-bit AES DATA Key with ICSF APIs

For more information about a Rexx sample to create a 256-bit AES DATA type key, see Rexx Sample: Secure Key Generate (256-bit AES DATA Key). See also Example B-2 on page 258.

5.3.4 Using CSFKGUP

The key generator utility program (KGUP) generates and maintains keys in the CKDS. To run KGUP, ICSF must be active, and the user must have access to KGUP (by way of the CSFKGUP[1] profile in the CSFSERV class) and UPDATE authority to the profile that is covering the CKDS in the DATASET class.

In addition, if key lifecycle auditing of tokens or labels is enabled, the option `AUDITKEYLIFECKDS(TOKEN(YES))` or `AUDITKEYLIFECKDS(LABEL(YES))` is specified in CSFPRMxx. The user must also have access to the CSFGKF profile in the CSFSERV class to generate the key fingerprint that is used in auditing.

When KGUP generates a secure AES DATA key value, the program adds an entry in the CKDS that is enciphered under your system’s master key. For z/OS data set encryption, a 256-bit AES DATA type key must be generated.

Authorization

Users must have READ authority to the following resources in the CSFSERV class to perform key generation with KGUP:

- ICSF running with CHECKAUTH(YES) - CSFKGUP, CSFKGN
- ICSF running with CHECKAUTH(NO) - CSFKGUP

[1] Can be used for DATA keys at this time.

Users must also have UPDATE authority to the CKDS in the DATASET class to perform key generation with KGUP.

For more information about the RACF commands to authorize users to the CSFSERV and DATASET classes, see 4.2.2, “Defining DATASET, CSFSERV, CSFKEYS, and other resources” on page 71.

Generating a 256-bit AES DATA Key with KGUP

A JCL sample to create a 256-bit AES DATA type key with key label of DATASET.PE08.ENCRYPT.DS.ENCRKEY.00000001 is shown in Example 5-1.

Example 5-1 JCL example to create a 256-bit AES DATA type key

```
//STEP10  EXEC PGM=CSFKGUP
//CSFCKDS DD DISP=OLD,DSN=SYS1.SC60NEW.SCSFCKDS
//CSFIN   DD *,LRECL=80
ADD TYPE(DATA) ALGORITHM(AES),
 LABEL(DATASET.PE08.ENCRYPT.DS.ENCRKEY.00000001) LENGTH(32)
/*
//CSFDIAG  DD SYSOUT=*,LRECL=133
//CSFKEYS  DD SYSOUT=*,LRECL=1044
//CSFSTMNT DD SYSOUT=*,LRECL=80
```

Multiple keys can be generated with each run of the KGUP.

A JCL sample to create three 256-bit AES DATA type keys with a key label of DATASET.PE08.ENCRYPT.DS.ENCRKEY.00000001/2/3 is shown in Example 5-2.

Example 5-2 JCL example create three 256-bit AES DATA type key

```
//STEP10  EXEC PGM=CSFKGUP
//CSFCKDS DD DISP=OLD,DSN=SYS1.SC60NEW.SCSFCKDS
//CSFIN   DD *,LRECL=80
ADD TYPE(DATA) ALGORITHM(AES),
LABEL(DATASET.PE08.ENCRYPT.DS.ENCRKEY.00000001,
DATASET.PE08.ENCRYPT.DS.ENCRKEY.00000002,
DATASET.PE08.ENCRYPT.DS.ENCRKEY.00000003) LENGTH(32)
/*
//CSFDIAG  DD SYSOUT=*,LRECL=133
//CSFKEYS  DD SYSOUT=*,LRECL=1044
//CSFSTMNT DD SYSOUT=*,LRECL=80
```

Consider the following points regarding Example 5-1 and Example 5-2:

- TYPE must be DATA.
- ALGORITHM must be AES.
- A key LABEL can consist of up to 64 characters.

  The first character must be alphabetic or a national character (#, $, or @). The remaining characters can be alphanumeric, a national character (#, $, or @), or a period (.). The key label is used to identify the key in the ICSF keystore (CKDS).
- LENGTH specified as 32, which generates a 256-bit key.

Key generation output is shown in Example 5-3 on page 139.

Example 5-3 Key generation output

```
KEY GENERATION DIAGNOSTIC REPORT
ADD TYPE(DATA) ALGORITHM(AES),
 LABEL(DATASET.PE08.ENCRYPT.DS.ENCRKEY.00000001) LENGTH(32)
>>>CSFG0321 STATEMENT SUCCESSFULLY PROCESSED.
>>>CSFG0780 A REFRESH OF THE IN-STORAGE CKDS IS NECESSARY TO ACTIVATE CHANGES MADE BY KGUP.
>>>CSFG0002 CRYPTOGRAPHIC KEY GENERATION - END OF JOB.  RETURN CODE = 0.
```

Refreshing in-storage copy of CKDS

ICSF references an in-storage copy of the CKDS for key label lookup. However, when KGUP is used to generate the AES data-encrypting key, change the CKDS that is stored on disk rather than the in-storage copy. ICSF does not recognize the changes that were made to disk unless the CKDS is refreshed such that the in-storage copy of the CKDS is updated.

When you update the CKDS on disk and you are sharing the CKDS in a sysplex, use the Coordinated CKDS Refresh panel utility to refresh the CKDS so that all members of the sysplex can see the new keys. Otherwise, the other members of the sysplex cannot decrypt data that was encrypted by the system on which the refresh was issued.

When you update the CKDS on disk and you are not in a sysplex, you can use the Refresh option on the Key Administration panel to replace the in-storage copy with the disk copy. You can also start a utility program to refresh the CKDS.

> **Note:** This refresh is only necessary when KGUP is used to generate the key as it writes directly to the CKDS on disk. No refresh is required for keys that are generated by the ICSF panel or callable services.

A JCL sample to refresh the in-storage copy of CKDS by using ICSF utility program CSFEUTIL is shown in Example 5-4.

Example 5-4 Refresh in-storage copy of CKDS by using ICSF utility program CSFEUTIL

```
//STEP20  EXEC PGM=CSFEUTIL,
//       PARM='SYS1.SC60NEW.SCSFCKDS,REFRESH'
```

The messages from successful refresh by using CSFEUTIL are shown in Example 5-5.

Example 5-5 Messages from successful refresh

```
CSFM653I CKDS LOADED 12 RECORDS WITH AVERAGE SIZE 249
CSFU002I CSFEUTIL COMPLETED, RETURN CODE = 0, REASON CODE = 0.
```

To refresh the in-storage copy of the CKDS, from the ICSF Primary menu, select **Option 8 KGUP (Key Generator Utility processes)** → **Option 4 Refresh (Activate an existing cryptographic key data set)**.

Refresh in-storage CKDS panel is shown in Figure 5-11 on page 140.

```
------------------------ ICSF - Refresh in-storage CKDS -----------------------
COMMAND ===>

Enter the name of the CKDS to become active.

  New CKDS ===> 'SYS1.SC60NEW.SCSFCKDS'



Press ENTER to refresh the in-storage CKDS
Press END   to exit to previous panel
```

Figure 5-11   ICSF Refresh in-storage CKDS

Press Enter to perform the refresh. A successful refresh results in message CSFM653I (see Example 5-6).

Example 5-6   Message CSFM6531

```
CSFM653I CKDS LOADED 10 RECORDS WITH AVERAGE SIZE 252
```

The Refresh in-storage CKDS panel displays a message to indicate a successful refresh (see Figure 5-12).

```
------------------------ ICSF - Refresh in-storage CKDS --- REFRESH SUCCESSFUL
COMMAND ===>
THE SPECIFIED CKDS IS NOW THE CURRENT CKDS
Enter the name of the CKDS to become active.

  New CKDS ===> 'SYS1.SC60NEW.SCSFCKDS'



Press ENTER to refresh the in-storage CKDS
Press END   to exit to previous panel
```

Figure 5-12   ICSF Refresh Successful message

The following refresh can also be performed from Option 2.1; 1.2 from the ICSF Primary menu:

- Option 2 KDS MANAGEMENT: Master key set or change, KDS Processing
- Option 1 CKDS MANAGEMENT: Perform Cryptographic Key Data Set (CKDS) functions including master key management
- Option 1 CKDS OPERATIONS: Initialize a CKDS, activate a different CKDS (Refresh), or update the header of a CKDS and make it active
- Option 2 REFRESH: Activate an updated CKDS

5.4 Protecting data sets with secure keys

> **Note:** This procedure applied regardless of whether the data-encrypting key is a DATA key or a CIPHER key.

A key label can be supplied in any of the following options (using order of precedence):

- The Data Facility Product (DFP) segment in the RACF data set profile
- JCL, Dynamic Allocation, TSO Allocate, and IDCAMS DEFINE
- Storage Management Subsystem (SMS) Construct: Data Class

To make data set encryption unavailable to users who are not authorized to use it, complete the following tasks:

- Define the STGADMIN.SMS.ALLOW.DATASET.ENCRYPT profile in the FACILITY class, and set the universal access to NONE, as shown in the following example:

  ```
  RDEFINE FACILITY STGADMIN.SMS.ALLOW.DATASET.ENCRYPT UACC(NONE)
  ```

- If the FIELD class is active, check for any profile that allows any user without the SPECIAL attribute access to the DATASET.DFP.DATAKEY. If no profile exists, no other action is needed. If any profile allows access to DATASET.DFP.DATAKEY, create a DATASET.DFP.DATAKEY profile in the FIELD class with a UACC of NONE, as shown in the following example:

  ```
  RDEFINE FIELD DATASET.DFP.DATAKEY UACC(NONE)
  ```

Taking these steps helps ensure that only authorized users are allowed to use data set encryption. Users must have at least READ authority to resource `STGADMIN.SMS.ALLOW.DATASET.ENCRYPT` in the FACILITY class to create encrypted data sets when the key label is specified by way of a method other than the DFP segment in the RACF data set profile.

The system does not require the user to have authority to resource `STGADMIN.SMS.ALLOW.DATASET.ENCRYPT` when the key label is specified in the DFP segment in the RACF data set profile.

To create a set of SAF resources to protect new data sets (PE08.ENCRYPT.DS.*) with encryption, complete the following steps:

1. Create a generic DATASET resource to protect a set of data sets, as shown in the following example:

   ```
   ADDSD 'PE08.ENCRYPT.DS.*' UACC(NONE)
   ```

2. Specify the encryption key label in the DFP segment (the recommended way to supply a key label is for a security administrator to add the key label to the DFP segment), as shown in the following example:

   ```
   ALTDSD 'PE08.ENCRYPT.DS.*'
   DFP(DATAKEY(DATASET.PE08.ENCRYPT.DS.ENCRKEY.00000001))
   ```

3. Nonencrypted data sets must be copied over to these new (PE08.ENCRYPT.DS.*) data sets to be protected by encryption.

5.5 Encrypting a data set with a secure key

> **Note:** This procedure applied regardless of whether the data-encrypting key is a DATA key or a CIPHER key.

To create an encrypted data set, you must assign a key label to the data set when it is newly allocated (data set create). In addition, an encrypted data set must be allocated as an extended format data set in one of the following ways:

- JCL, by way of DSNTYPE=EXTPREF or DSNTYPE=EXTREQ
- TSO allocate, by way of DSNTYPE(EXTPREF) or DSNTYPE(EXTREQ)
- IDCAMS DEFINE parameter DATACLASS references an extended format data class.
- Dynamic allocation text unit DALDSNT

A key label can be specified by using any of the following methods:

- RACF data set profile

  To specify a key label by using the DFP segment in the RACF data set profile, use keyword `DATAKEY(Key-Label)`. The system uses this key label for data set encryption supported data set types that are created after DATAKEY is added to the data set profile. Use new keyword `NODATAKEY` to remove a key label (if previously defined) from the DFP segment.

  The key label is ignored for a data set that is not a DASD data set. The key label is also ignored for a data set that is not extended format unless the user has READ access to the `STGADMIN.SMS.FAIL.INVALID.DSNTYPE.ENC` resource in the FACILITY class.

  > **Note:** A key label that is specified in the DFP segment in the RACF data set profile is used regardless of the setting of ACSDEFAULTS that is specified in `SYS1.PARMLIB(IGDSMSxx)`.

- To specify a key label by using JCL, dynamic allocation, and TSO, allocate the use the following components:
  - JCL keyword `DSKEYLBL='key-label'`
  - Dynamic allocation text unit `DALDKYL`
  - TSO allocate DSKEYLBL(label-name)

    DSKEYLBL is effective only if the new data set is on DASD. The key label is ignored for a data set that is not a DASD data set.

    For more information about the DSKEYLBL(key-label) keyword on the JCL DD statement, see Abstract for MVS JCL Reference.

- SMS data class

  To specify a key label by using SMS data class, use the Data Set Key Label field in the Interactive Storage Management Facility (ISMF) DEFINE/ALTER panel.

  The system uses this key label for data set encryption supported data set types that are created after the data set key label is added to the data class. The key label is ignored for a data set that is not a DASD data set.

  For more information about the use of the new Data Set Key Label field in the ISMF panels, see Data Set Encryption.

- **`IDCAMS DEFINE`** command

  To specify a key label by using the **`IDCAMS DEFINE`** command for a Virtual Storage Access Method (VSAM) data set, use the KEYLABEL parameter, as shown in the following example:

  `KEYLABEL(MYLABEL)`

  Any alternative index that is associated with the VSAM base cluster is encrypted and uses the same key label as specified for the base cluster. The key label is ignored for a data set that is not a DASD data set.

  For more information about the use of the **`IDCAMS DEFINE`** command for a VSAM data set, see Abstract for DFSMS Access Method Services Commands.

When a key label is specified on more than one source, the key label is derived from one of these sources only on the first data set allocation (on data set create). The key label is derived in the following order of precedence:

1. From the DFP segment in the RACF data set profile

2. Explicitly specified on the DD statement, dynamic allocation text unit, TSO ALLOCATE command, or IDCAMS DEFINE control statement
3. From the data class that applies to the current DD statement

> **Note:** The `REFDD` and `LIKE JCL DD` statement keywords do not cause a key label from which the data set is referred to be used when allocating a new data set.

On successful allocation of an encrypted data set, the successful allocation message that is shown in Example 5-7 is issued.

Example 5-7 Successful allocation message

```
IGD17150I DATA SET dsname IS ELIGIBLE FOR ACCESS METHOD ENCRYPTION
KEY LABEL IS (key_label)
```

> **Note:** When a key label is specified for a DASD data set that is not extended-format, allocation fails and the message IGD17151I is issued. When IDCAMS DEFINE is used, message IDC3009I is issued with RC48 and RSN82.

5.6 Verifying that the data set is encrypted

To verify that a sequential dataset is encrypted, use ISPF dataset list (option 3.4) with the Data Set information line command (I) (see Figure 5-13).

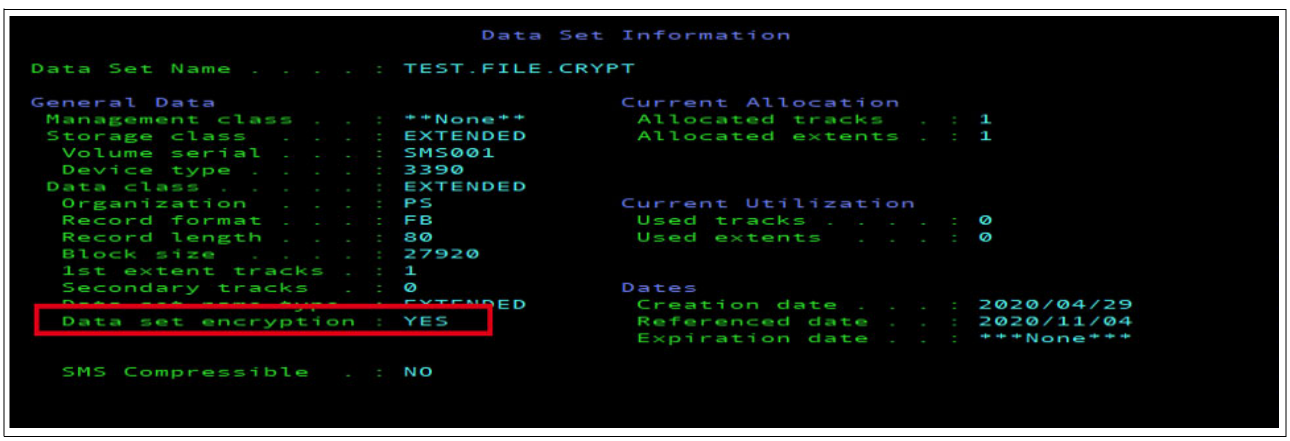


Figure 5-13 Data Set information

Alternatively, the **IDCAMS LISTCAT** command can be run to verify that a data set is enabled for encryption, as shown in the following example:

```
LISTCAT ENTRIES('PE08.ENCRYPT.DS.D01') ALL
```

The output of LISTCAT for a data set is shown in Example 5-8. The encryption data section indicates that the data set is encrypted and displays the key label. The attributes section shows that the data set is in EXTENDED format.

Example 5-8 IDCAMS LISTCAT for the data set

```
NONVSAM ------- PE08.ENCRYPT.DS.D01
     IN-CAT --- UCAT.CONCAT
     HISTORY
```

```
      DATASET-OWNER-----(NULL)     CREATION--------2017.317
      RELEASE----------------2     EXPIRATION------0000.000
      ACCOUNT-INFO-----------------------------------(NULL)
    SMSDATA
      STORAGECLASS ----DEFAULT     MANAGEMENTCLASS---(NULL)
      DATACLASS ----------EFEA     LBACKUP ---0000.000.0000
    ENCRYPTIONDATA
      DATA SET ENCRYPTION----(YES)
      DATA SET KEY LABEL-----DATASET.PE08.ENCRYPT.DS.ENCRKEY.00000001
    VOLUMES
      VOLSER------------CONSM1     DEVTYPE------X'3010200F'     FSEQN---------
---------0
    ASSOCIATIONS--------(NULL)
    ATTRIBUTES
      VERSION-NUMBER---------2
      STRIPE-COUNT-----------1
      EXTENDED
```

5.7 Granting access to encrypted data sets

The CSFKEYS class controls access to cryptographic keys that are identified by the key label. Any user that needs access to data that is encrypted with a data key that is associated with a key label must have access to that key label.

Complete the following steps to set up a key label:

1. If the CSFKEYS class was not enabled for generic usage, run the following command:

   ```
   SETROPTS GENERIC(CSFKEYS)
   ```

2. Define a profile so no user can access the key label (DATASET.PE08.ENCRYPT.DS.ENCRKEY.00000001) and then use the key label for a protected key by using the following command:

   ```
   RDEFINE CSFKEYS DATASET.PE08.ENCRYPT.DS.ENCRKEY.* UACC(NONE)
   ICSF(SYMCPACFWRAP(YES) SYMCPACFRET(YES))
   ```

   Consider the following points:

   – UACC(NONE) indicates that no universal access to the key label exists.

   – SYMCPACFWRAP(YES) indicates that ICSF can rewrap the encrypted key by using the Central Processor Assist for Cryptographic Function (CPACF) wrapping key.

   – SYMCPACFRET(YES) indicates that keys that are covered by the profile can be returned to an authorized caller in the protected-key CPACF form.

   Any of the following situations cause a IEC143I 213-85 message:

   – Attempting to use a nonexistent key label.
   – No RACF CSFKEYS profile is defined for the key label.
   – A CSFKEYS profile is defined incorrectly.

   The resulting IEC143I when attempt to use a key label that is not defined is shown in Example 5-9.

   Example 5-9 IEC143I 213-85 message

   ```
   IEC143I 213-85,mod,jjj,sss, ddname[-#],dev,volser,dsname(member),
   ```

```
RC=X'00000008',RSN=X'0000271C'
```

The resulting IEC143I 213-85 message when an attempt to use a key label that includes a RACF CSFKEYS profile that is defined with SYMCPACFWRAP(YES), SYMCPACFRET(NO) is shown in Example 5-10.

Example 5-10 IEC143I 213-85 message when attempting to use a key label

```
IEC143I 213-85,mod,jjj,sss, ddname[-#],dev,volser,dsname(member),
RC=X'00000008',RSN=X'00000C04'
```

The resulting IEC143I 213-85 message (see Example 5-11) when attempting to use a key label that includes the following components:

- RACF CSFKEYS profile that is defined without ICSF data
- RACF CSFKEYS profile that is defined with ICSF data SYMCPACFWRAP(NO), SYMCPACFRET(YES)
- RACF CSFKEYS profile that is defined with ICSF data SYMCPACFWRAP(NO), SYMCPACFRET(NO)

Example 5-11 IEC1431 213-85 message

```
IEC143I 213-85,mod,jjj,sss, ddname[-#],dev,volser,dsname(member),
RC=X'00000008',RSN=X'00000BFB'
```

3. Define a profile to allow access to key label:

   a. To allow DATASET.PE08.ENCRYPT.DS.ENCRKEY.00000001 to be used by John (in this example), issue the following command:

   ```
   PERMIT DATASET.PE08.ENCRYPT.DS.ENCRKEY.* CLASS(CSFKEYS) ID(JOHN)
   ACCESS(READ)
   ```

   b. To allow key label DATASET.PE08.ENCRYPT.DS.ENCRKEY.00000001 to be used by Mike (in this example) only when accessed by DFSMS, issue the following command:

   ```
   PERMIT DATASET.PE08.ENCRYPT.DS.ENCRKEY.* CLASS(CSFKEYS) ID(MIKE)
   ACCESS(READ) WHEN(CRITERIA(SMS(DSENCRYPTION)))
   ```

4. Set the following RACF options as needed:

   - If the CSFKEYS class is not active, issue the following command:

     ```
     SETROPTS CLASSACT(CSFKEYS)
     ```

   - If the CSFKEYS class was not RACLISTed, issue the following command:

     ```
     SETROPTS RACLIST(CSFKEYS)
     ```

   - If the CSFKEYS class is RACLISTed, issue the following command:

     ```
     SETROPTS RACLIST(CSFKEYS) REFRESH
     ```

   If the user does not have sufficient access to the key label, the messages IEC150I and ICH408I are issued (see Example 5-12).

Example 5-12 Message IEC1501 and ICH4081

```
IEC150I 913-84,mod,jjj,sss, ddname[-#],dev,ser,dsname(member)
ICH408I USER(userid) GROUP(group-name) NAME(user-name)
  DATASET.PE08.ENCRYPT.DS.ENCRKEY.00000001 CL(CSFKEYS )
  INSUFFICIENT ACCESS AUTHORITY
  ACCESS INTENT(READ   )  ACCESS ALLOWED(NONE   )
```

Note: Authorization to the key label is checked only when the data set is opened. At data set creation (for example, allocate by way of IEFBR14), key label authorization is not checked if an open operation is not performed.

5.8 Accessing encrypted data sets

With z/OS data sets encryption, applications that use standard basic sequential access method (BSAM), queued sequential access method (QSAM), and VSAM access methods do not require changes to access encrypted data sets. Data set encryption is flexible in allowing as much granularity as wanted when key labels are identified for data sets.

The data is encrypted when it is written to DASD and decrypted when it is read from DASD. For encrypted compressed format data sets, the access methods compress the data before the data is encrypted on output. On input, the access methods first decrypt the data before the data is decompressed.

The following system functions process data in the encrypted form. Users who are performing these functions do not require authorization to the key labels that are associated with the encrypted data sets being processed:

- DFSMSdss functions: COPY, DUMP, RESTORE, and PRINT
- DFSMShsm functions: Migrate/Recall, Backup/Recover, abackup/arecover, dump/data set restore, and FRBACKUP/FRRECOV DSNAME
- Track-based copy (PPRC, XRC, FlashCopy, and concurrent copy) operations

To access the content of an encrypted data set, a user needs authorization to access the key label that is associated with the data set in addition to the normal access that is required by way of RACF Data Set Profiles.

If the user does not have proper authority, message ICH408I is issued (see Example 5-13), which indicates insufficient authority for a key label when attempting to access the data set.

Example 5-13 ICH4081 message

```
ICH408I USER(userid) GROUP(group-name) NAME(user-name)
  key.label.name CL(CSFKEYS )
  INSUFFICIENT ACCESS AUTHORITY
  ACCESS INTENT(READ   )  ACCESS ALLOWED(NONE   )
```

This message indicates which profile is stopping this user from reading the key that is associated with the particular key label. The RACF administrator must give the user read access to the resource profile that is listed in the message.

For more information about how to set up access to a key label, see 5.4, “Protecting data sets with secure keys” on page 140.

Sample JCL to create an encrypted sequential data set is shown in Example 5-14.

Example 5-14 JCL to create an encrypted sequential data set

```
//STEP1    EXEC PGM=IEBDG
//SYSPRINT DD SYSOUT=*
//OUTDATA  DD DISP=(,CATLG),DSN=PE08.ENCRYPT.DS.D01,
//         UNIT=SYSDA,SPACE=(TRK,(5,1),RLSE),
//         DSKEYLBL='DATASET.PE08.ENCRYPT.DS.ENCRKEY.00000001',
```

```
//         DCB=(LRECL=80,RECFM=FB,BLKSIZE=0,DSORG=PS),
//         DSNTYPE=EXTREQ
//SYSIN    DD *
   DSD     OUTPUT=(OUTDATA)
   FD      NAME=HELLO,LENGTH=11,PICTURE=11,'HELLO WORLD'
   CREATE  QUANTITY=1,NAME=(HELLO),FILL=X'40'
   END
/*
```

If access to the key label DATASET.PE08.ENCRYPT.DS.ENCRKEY.00000001 is granted, a browse of the data set displays as normal, unencrypted data (see Example 5-15).

Example 5-15 Browse of the data set PE08.ENCRYPT.DS.D01

```
BROWSE    PE08.ENCRYPT.DS.D01                      Line 0000000000 Col 001 080
 Command ===>                                                  Scroll ===> PAGE
********************************* Top of Data **********************************
HELLO WORLD
******************************** Bottom of Data ********************************
```

If access to the key label DATASET.PE08.ENCRYPT.DS.ENCRKEY.00000001 is not granted, messages IEC150I and ICH408I are issued (see Example 5-16).

Example 5-16 Message IEC1501 and ICH4081

```
IEC150I 913-84,IGG0193V,PE10,TSOPROC,ISP09390,95C3,CONSM2,
ICH408I USER(PE10    ) GROUP(SYS1    ) NAME(PERVASIVE ENCRYPTION)
  DATASET.PE08.ENCRYPT.DS.ENCRKEY.00000001 CL(CSFKEYS )
  INSUFFICIENT ACCESS AUTHORITY
  ACCESS INTENT(READ   )  ACCESS ALLOWED(NONE   )
PE08.ENCRYPT.DS.D01,
RC=X'00000008',RSN=X'00000000'
```

5.9 Viewing the encrypted text

The DFSMSdss **PRINT** command can be used to view the encrypted (or unencrypted) data that is stored on track.

Example

To demonstrate that system functions, such as DFSMSdss COPY, DUMP, RESTORE, and PRINT, do not require access to the key label, we ran a PRINT of the encrypted data set (see Example 5-17).

Example 5-17 PRINT encrypted data set

```
//STEP1 EXEC PGM=ADRDSSU,REGION=0M
//SYSPRINT DD SYSOUT=*
//SYSIN    DD *
 PRINT DATASET(PE08.ENCRYPT.DS.D01) INDYNAM(CONSM3)
/*
```

The output of the PRINT showed garbled (encrypted) data (see Example 5-18).

Example 5-18 Output of the PRINT

```
0000  2FB4DC87 845BC185 1D399D25 ... *...gd$Ae....T.w.k.../.1.....,N8.*
0020  88C412C3 BC3292AD 6528E12A ... *hD.C..k......EO@....9.kA#,.3....*
0040  0540667D 8B1B2454 95AAE035 ... *...'....n...?..u................*
```

We attempted to access an encrypted data set that was created on another system (by way of DFSMSdss DUMP/RESTORE) for which the data-encrypting key and key label is not available. Message IEC143I was issued, as shown in Example 5-19.

Example 5-19 Message IEC1431

```
IEC143I 213-85,IGG0193V,PE08,TSOPROC,ISP10848,95C3,CONSM2,
DATASET.PE08.ENCRYPT.DS.ENCRKEY.00000002,
RC=X'00000008',RSN=X'0000271C'
```

For more information about how to transport AES data-encrypting key between systems to allow access to encrypted data in data sets that were created on one system and shared (by way of DFSMSdss DUMP/RESTORE) on another system, see Chapter 3, “Planning for z/OS data set encryption” on page 35.

6

Auditing z/OS data set encryption

This chapter focuses on the various system management facilities (SMF)[1] records that can aid in monitoring the z/OS data set encryption environment.

Brief descriptions and references are provided to demonstrate specific uses and tasks that are related to auditing the z/OS data set encryption environment. If a reporting process or workflow for auditing purposes is not yet established, another option to create reports is IBM Security zSecure (for more information, see 2.3.6, “IBM Security zSecure Suite” on page 32).

This chapter includes the following topics:

[1] For more information about a functional overview of SMF, see Abstract for MVS System Management Facility (SMF).

6.1 Auditing encrypted sequential data sets and PDSEs

The z/OS Data Facility Storage Management Subsystem (DFSMS) creates SMF Type 14 and Type 15 records to audit data set activity for sequential data sets and PDSEs.

SMF Type 14, Subtype 9 and SMF Type 15, and Subtype 9 records provide DASD data set encryption information. They indicate the following information:

- If the data set is encrypted
- The data set encryption type
- The data set encryption key label

For more information, see DASD Data Set Encryption Information (Type 9).

6.2 Auditing encrypted VSAM data sets

z/OS DFSMS creates SMF Type 62 records to audit data set activity for VSAM data sets.

SMF Type 62 records indicate the following information about the data set encryption:

- Type
- Key label

For more information, see Record Type 62 (3E) — VSAM Component or Cluster Opened.

6.3 Auditing crypto hardware activity

The Resource Measurement Facility (RMF) writes SMF Type 70 and Subtype 2 records, which show cryptographic coprocessor and accelerator usage, such as the following examples:

- Cryptographic CCA Coprocessor data
- Cryptographic Accelerator data
- ICSF Services data
- Cryptographic PKCS 11 Coprocessor data

For more information, see Subtype 2 — Cryptographic Hardware Activity.

In addition, overview criteria are shown for the Postprocessor in the Postprocessor Workload Activity Report - Goal Mode (WLMGL) report. For more information, see z/OS RMF publications.

6.4 Auditing security authorization attempts

The Resource Access Control Facility (RACF) writes SMF Type 80 records for scenarios, such as the following examples:

- Authorized or unauthorized attempts to access RACF-protected resources.
- Authorized or unauthorized attempts to modify profiles on a RACF database.

SMF Type 80 records can be examined to determine which users attempted to access the following information:

- Key labels that are protected by the CSFKEYS class
- Data sets that are protected by the DATASET class
- Crypto services that are protected by the CSFSERV class

For more information about SMF Type 80 records, see Record type 80: RACF processing record.

For processing RACF SMF records, the RACF SMF Unload Utility is a good choice. Samples are available in SYS1.SAMPLIB(IRRICE).

Post-processing of the output can be done by using DFSORT. For more information about examples, see Using the RACFICE procedure to generate reports.

6.5 Auditing crypto engine, service, and algorithm usage

The z/OS Integrated Cryptographic Services Facility (ICSF) provides a means for security administrators and capacity planners to monitor the use of cryptographic resources with Crypto Usage Statistics. ICSF writes SMF Type 82, Subtype 31 records when cryptographic usage tracking is enabled.

Note: This feature is optional with ICSF and usage tracking algorithms can be turned on or off, depending on your needs. For more information about enabling crypto usage tracking, see 4.3.3, “CSFPRMxx and installation options” on page 83.

Crypto usage tracking helps users determine the following information:

- Which jobs or tasks use the various crypto engines
- Which crypto card types are receiving the most requests
- If any crypto requests are being handled in software
- The peak periods of crypto usage
- ICSF services that are started by other z/OS components
- Which jobs or tasks use out-of-date algorithms or key sizes

Cryptographic usage statistics are recorded in SMF data sets. Statistics are recorded for each SMF recording interval. Use the usage and interval recording to determine usage over various time periods. For more information, see 4.4.2, “Configuring SMF recording options in SMFPRMxx” on page 106.

Each ICSF instance can track the usage of cryptographic engines (ENG), cryptographic services (SRV), and cryptographic algorithms (ALG) for the LPAR in which it runs.

SMF Type 82 Subtype 31 contains information about the cryptographic user’s HOME address space job ID, SECONDARY address space job name, HOME address space user ID, HOME task level user ID, and ASID.

By using Crypto Usage Statistics, you can assess your cryptographic usage and determine any areas that might need attention. By determining which applications are using which cryptographic engines, services, and algorithms, you can ensure that you are operating in the most secure manner. The use of Crypto Usage Statistics can also help you tune your systems for optimal performance.

6.6 Auditing key lifecycle transitions

Some regulations, such as PCI-DSS, require that specific key management activities are performed regularly. ICSF provides the capability for auditing the lifecycle of keys.

For z/OS data set encryption, which uses Common Cryptographic Architecture (CCA) symmetric data keys, ICSF writes SMF Type 82, Subtype 40 records to track key lifecycle transitions.

> **Note:** This feature is optional with ICSF and key lifecycle tracking can be turned on or off, depending on your needs. For more information about enabling key lifecycle tracking, see 4.3.3, "CSFPRMxx and installation options" on page 83.

A subset of the SMF Type 82, Subtype 40 fields include the following information:

- Key event, such as the key token that is:
  - Added to KDS
  - Updated in KDS
  - Deleted from KDS
  - Archived
  - Restored
  - Metadata changed
  - Pre-activated
  - Activated
  - Deactivated
  - Exported
  - Generated
  - Imported
- Key label
- Key data set
- Service that is associated with the event
- Key token format
- Key security
- Key algorithm
- Key length

6.7 Auditing key usage operations

Regulations can specify limitations on which key types are allowed for use in crypto operations or if a single key type is disallowed for multiple crypto operations. ICSF provides the key usage tracking to audit the use of keys.

Key usage data is recorded in SMF data sets. Data is recorded within key usage intervals, as defined in the CSFPRMxx member. Use the interval recording to analyze key usage over various time periods. For more information, see 4.3.3, "CSFPRMxx and installation options" on page 83.

> **Note:** This feature is optional with ICSF and key use tracking can be turned on or off, depending on your needs. For more information about enabling key usage tracking, see 4.3.3, "CSFPRMxx and installation options" on page 83.

For z/OS data set encryption, which uses CCA symmetric data keys, ICSF writes SMF type 82, subtype 44 records to track key usage. Usage counts are accumulated during each key usage recording interval.

A subset of the SMF Type 82, Subtype 44 fields includes the following information:

- Key label
- Service that is associated with the event
- Key token format
- Key security
- Key algorithm
- Key length
- Usage count

6.8 Formatting SMF Type 82 records

SMF Type 82 formatters for ICSF are available in SYS1.SAMPLIB members CSFSMFJ (JCL) and CSFSMFR (REXX). Consider the following points:

- CSFSMFJ is the JCL to submit the job.
- CSFSMFR is the REXX exec to run the report against the SMF records.

CSFSMFJ (shown in Example 6-1) reads Type 82 SMF records and formats them in a report.

Example 6-1 Sample JCL to unload type 82 SMF records

```
//*-----------------------------------------------------------------*
//*   UNLOAD SMF 82 RECORDS FROM VSAM TO VBS                        *
//*-----------------------------------------------------------------*
//SMFDMP   EXEC  PGM=IFASMFDP
//DUMPIN   DD    DISP=SHR,DSN=PRICHAR.SMFRECS
//DUMPOUT  DD    DISP=(NEW,PASS),DSN=&&VBS,UNIT=3390,
//         SPACE=(CYL,(1,1)),DCB=(LRECL=32760,RECFM=VBS,BLKSIZE=4096)
//SYSPRINT DD    SYSOUT=*
//SYSIN    DD    *
    INDD(DUMPIN,OPTIONS(DUMP))
    OUTDD(DUMPOUT,TYPE(82))
//*
//*-----------------------------------------------------------------*
//*   COPY VBS TO SHORTER VB AND SORT ON DATE/TIME                  *
//*-----------------------------------------------------------------*
//COPYSORT EXEC PGM=SORT,REGION=6000K
//*TEPLIB  DD DISP=SHR,DSN=SYS1.SORTLPA,VOL=SER=ttttt1,UNIT=3390
//*        DD DISP=SHR,DSN=SYS1.SICELINK,VOL=SER=ttttt2,UNIT=3390
//SYSOUT   DD SYSOUT=*
//SORTWK01 DD UNIT=3390,SPACE=(CYL,10)
//SORTIN   DD DISP=(OLD,DELETE),DSN=&&VBS
//SORTOUT  DD    DISP=(NEW,PASS),DSN=&&VB,UNIT=3390,
//         SPACE=(CYL,(1,1)),DCB=(LRECL=32752,RECFM=VB)
//SYSIN    DD *
 SORT FIELDS=(11,4,A,7,4,A),FORMAT=BI,SIZE=E4000
//*
//*-----------------------------------------------------------------*
//*   FORMAT TYPE 82 RECORDS                                        *
//*-----------------------------------------------------------------*
```

```
//FMT     EXEC PGM=IKJEFT01,REGION=5128K,DYNAMNBR=100
//SYSPROC  DD DISP=SHR,DSN=SYS1.SAMPLIB
//SYSTSPRT DD SYSOUT=*
//INDD     DD DISP=(OLD,DELETE),DSN=&&VB
//OUTDD    DD SYSOUT=*
//SYSTSIN DD *
  %CSFSMFR
```

An excerpt of the Crypto Usage Statistics for SMF record type 82, subtype 31 is shown in Example 6-2.

Example 6-2 Excerpt from Crypto Usage Statistics

```
Subtype=001F Crypto Usage Statistics
 Written periodically to record crypto usage counts
 7 Nov 2017 17:10:30.00
    TME... 005E5858 DTE... 0117311F SID... SC60    SSI... 00000000 STY... 001F
    INTVAL_START.. 11/07/2017 22:02:24.247495
    INTVAL_END.... 11/07/2017 22:10:30.001940
    USERID_AS..... NET
    USERID_TK.....
    JOBID.........
    JOBNAME....... NET
    JOBNAME2......
    PLEXNAME...... PLEX60
    DOMAIN........ 84
    SRV...CSFKGN..... 12
 **************************************************
 Subtype=001F Crypto Usage Statistics
 Written periodically to record crypto usage counts
 7 Nov 2017 17:10:30.00
    TME... 005E5858 DTE... 0117311F SID... SC60    SSI... 00000000 STY... 001F
    INTVAL_START.. 11/07/2017 22:02:24.247495
    INTVAL_END.... 11/07/2017 22:10:30.001940
    USERID_AS..... PE08
    USERID_TK.....
    JOBID......... TSU05881
    JOBNAME....... PE08
    JOBNAME2......
    PLEXNAME...... PLEX60
    DOMAIN........ 84
    ENG...CARD...6C00/DV785304... 2
```

Example 6-2 on page 126 shows that the first usage event is recorded for `jobname=NET`. It occurred on system PLEX60 and used crypto domain 84.

The time interval for the event is 22:02 - 22:10 on 7 November, 2017. During the event, the CSFKGN (key generate) service was called 12 times. In the second usage event, two calls were made to the cryptographic card (6C00) by `jobname=PE08`.

6.9 Auditing Key management in UKO

The UKO key management system can simplify the process of showing compliance, by automating the logging of actions and by ensuring/enforcing that certain procedures are followed. For example: a financial institution must use dual-control/separation-of-privileges as specified by PCI, which they must comply with to be allowed to process payments.

All key management actions performed in UKO are recorded in the UKO audit log. The audit log includes the following information:

- Key generation, including the key label for the generated key
- Key state changes (activation, deactivation, mark compromised, deletion, and so on). See 10.3.4, “Key lifecycle” on page 237 for more information.
- Key distribution to CKDS
- Key removal from CKDS
- Changes to the UKO configuration

UKO key templates are also relevant for audit purposes, as they document the type and attributes of keys as well as the systems to which the keys are distributed.

For audit purposes, key templates can be used to document compliance with a process/policy with specific requirements for the keys.

For more information about UKO logging and key templates, see Chapter 10, “IBM Unified Key Orchestrator” on page 223.

7

Maintaining encrypted data sets

Encrypted data sets can be retained for the entire life of the data. This chapter briefly explains how to determine which data sets are encrypted. It also provides step-by-step procedures for rotating the keys that are associated with data set encryption.

This chapter includes the following topics:

7.1 Identifying encrypted data sets

Data sets become encrypted when key labels are provided at data set allocation. Those key labels might be provided by the following sources:

- DATASET class resources
- SMS DATACLAS
- TSO ALLOCATE
- JCL

Although you might identify the source for the key label, that process does not always ensure that the data sets that are associated with the source are encrypted. For example, you can set a key label in a DATASET class resource that encrypts new data sets, but the old data sets remain unencrypted. Therefore, you need more tools to determine which data sets are encrypted.

7.1.1 Using IBM zSecure

You can use IBM zSecure to identify all encrypted data set names, along with their associated key labels. For more information about zSecure, see 2.3.6, "IBM Security zSecure Suite" on page 32.

7.1.2 Using UKO for z/OS

UKO for z/OS has a data set dashboard, which provides an overview of data sets, their encryption status (encrypted, not encrypted, not encryptable), and the key (label) used to encrypt them. For details, see 10.4, "UKO for z/OS view of data sets" on page 243.

7.2 Rekeying encrypted data sets

Your security policy might dictate the rotation of keys in your environment. On z/OS, ICSF supports the rotation of master keys and operational keys.

7.2.1 Rotating the AES master key

IBM recommends that the master keys be rotated periodically. However, the frequency of master key rotation is at the organization's discretion. Master key rotation is nondisruptive and does not affect the data that is encrypted by keys in the Key Data Sets.

Master key rotation involves reenciphering secure, operational keys that are in the Key Data Sets. Reencipherment occurs in the secure boundary of the Crypto Express adapter.

Master key rotation involves decrypting secure, operational key values that are in the Key Data Sets from under the current master key to encryption under the new master key. This reencipherment occurs in the secure boundary of the Crypto Express adapter. After reencipherment, the new master key is set to become the current master key.

The process for master key rotation is automated and can be run on a single system or synchronized across a sysplex.

Rotating the AES master key includes the following steps:

1. Allocate a new CKDS.
2. Load the new AES master key.
3. Start the coordinated CKDS Change MK operation.
4. Verify the new AES master key.

These steps are described next.

Step 1: Allocating a new CKDS

To allocate a new key data set for the CKDS that can store variable-length records, we used the JCL that is shown in Example 7-1. This JCL was based on the sample job from SYS1.SAMPLIB(CSFCKD3).

Example 7-1 JCL to allocate a new CKDS data set

```
//DEFINE  EXEC PGM=IDCAMS,REGION=4M
//SYSPRINT DD SYSOUT=*
//SYSIN DD *
  DEFINE CLUSTER (NAME(PLEX75.SHARED3.SCSFCKDS)  -
                 VOLUMES(BH5CAT)            -
                 RECORDS(100 50)            -
                 RECORDSIZE(372,2048)       -
                 KEYS(72 0)                 -
                 FREESPACE(10,10)           -
                 SHAREOPTIONS(2,3))         -
            DATA (NAME(PLEX75.SHARED3.SCSFCKDS.DATA)     -
                 BUFFERSPACE(100000)        -
                 ERASE)                     -
            INDEX (NAME(PLEX75.SHARED3.SCSFCKDS.INDEX))
/*
```

You can optionally allocate a backup data set for the reenciphered keys, as shown in Example 7-2.

Example 7-2 JCL to allocate a backup CKDS data set

```
//DEFINE  EXEC PGM=IDCAMS,REGION=4M
//SYSPRINT DD SYSOUT=*
//SYSIN DD *
  DEFINE CLUSTER (NAME(PLEX75.SHARED3.COPY.SCSFCKDS)  -
                 VOLUMES(BH5CAT)            -
                 RECORDS(100 50)            -
                 RECORDSIZE(372,2048)       -
                 KEYS(72 0)                 -
                 FREESPACE(10,10)           -
                 SHAREOPTIONS(2,3))         -
            DATA (NAME(PLEX75.SHARED3.COPY.SCSFCKDS.DATA) -
                 BUFFERSPACE(100000)        -
                 ERASE)                     -
            INDEX (NAME(PLEX75.SHARED3.COPY.SCSFCKDS.INDEX))
/*
```

Step 2: Loading the new AES master key

Loading the master key for key rotation uses the same steps that are required to load a new master key register before CKDS initialization.

For more information about the steps to load the AES master key, see 4.3.5, “Loading the AES master key” on page 86.

Step 3 Starting the Coordinated Change MK operation

The Coordinated Change Master Key operation can be started from the TKE workstation (for TKE 8.0 and later) or by using z/OS ICSF.

The following steps show how to start a coordinated CKDS master key change by using ICSF:

1. In the main ICSF panel, select **Option 2 -KDS Management** (see Figure 7-1). Press Enter.

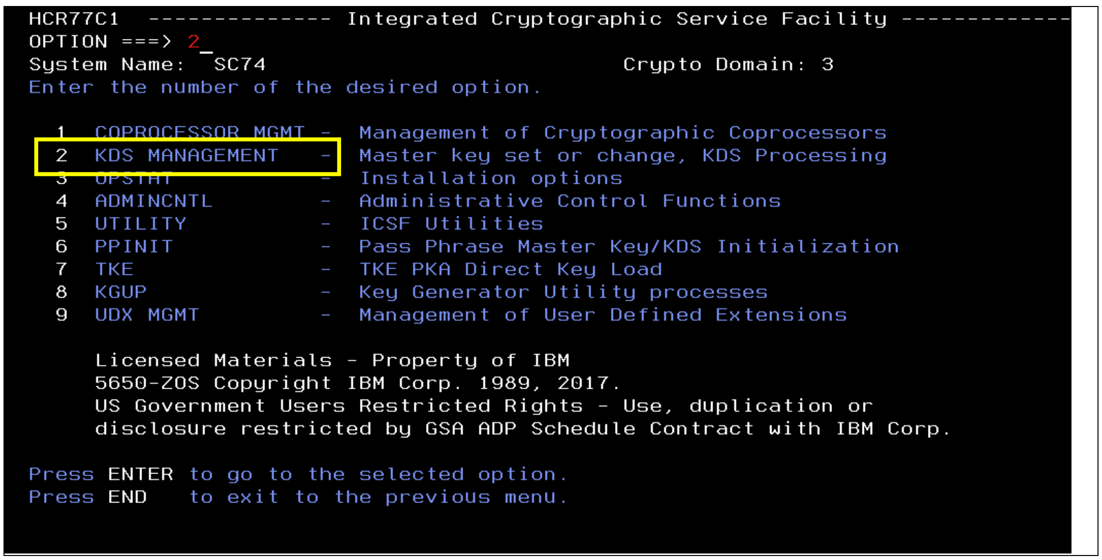


Figure 7-1 ICSF - KDS Management

2. In the ICSF - Key Data Set Management panel, select **Option 1-CKDS Management** (see Figure 7-2 on page 161). Press Enter.

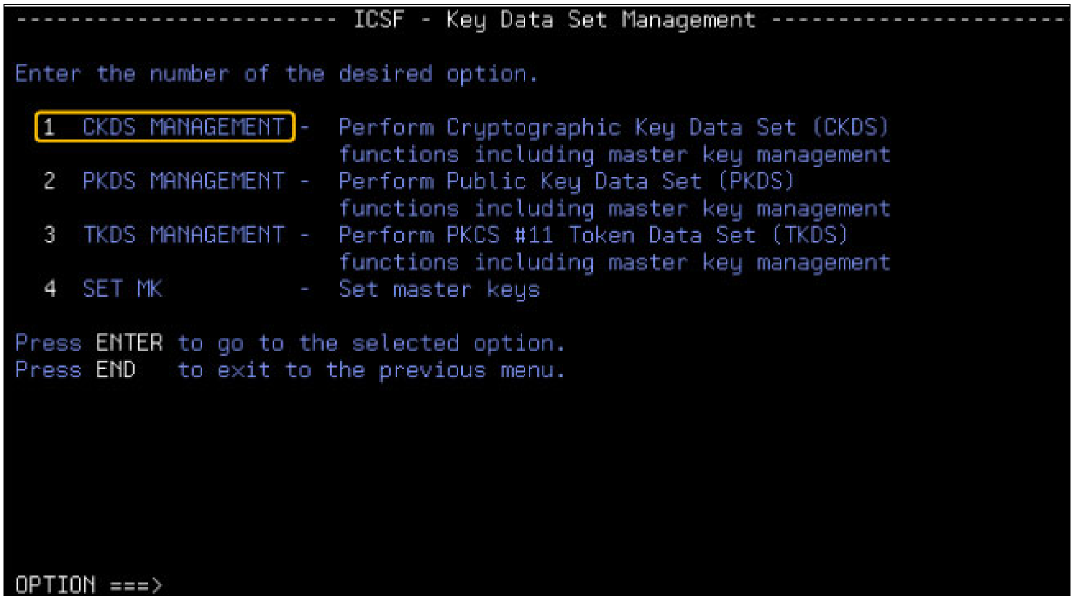


Figure 7-2 ICSF- Key Data Set Management Option 1

3. In production, it is recommended to run a CKDS KEY CHECK (option 7) before the coordinated master key change.

   In the ICSF - CKDS Management panel, select **Option 5-Perform a coordinated CKDS change master key** (see Figure 7-3). Press Enter.

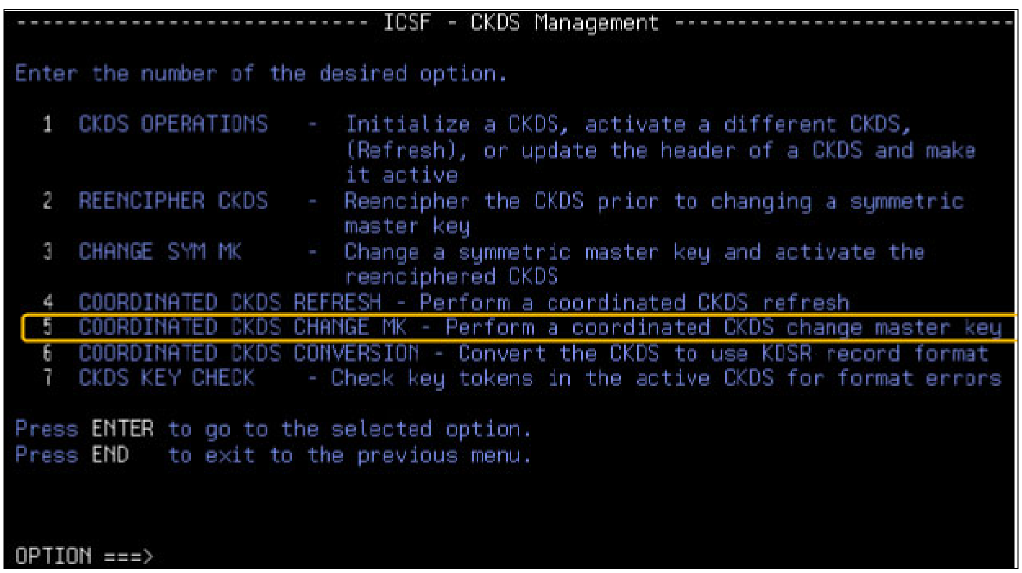


Figure 7-3 ICSF - CKDS Management Option 5

4. Review the Coordinated KDS change master key panel (see Figure 7-4). Press Enter.

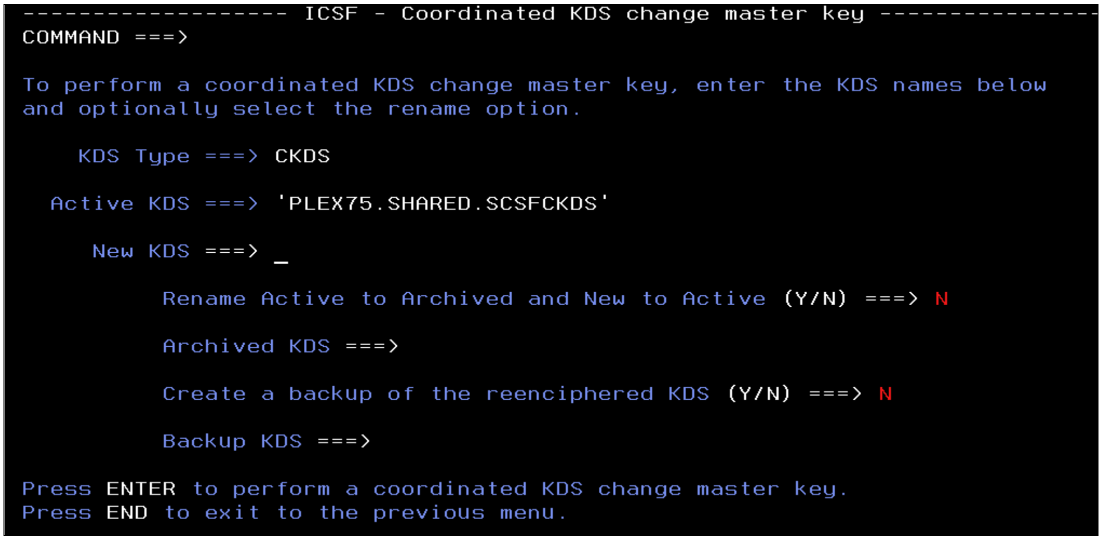


Figure 7-4 Coordinated KDS change master key panel

5. Perform the following changes on the ICSF - Coordinated KDS change master key (see Figure 7-5):

   a. Enter `Y` for fields Rename Active to Archive and New to Active [Y/N} and Create a backup of the reenciphered KDS [Y/N].

   b. Update the New KDS, Archived KDS, Backup KDS names.

      In our example:

      - The New KDS (allocated and empty) was named `PLEX75.SHARED3.SCSFCKDS`
      - The Archive KDS (not allocated) was named `PLEX75.SHARED3.OLD.SCSFCKDS`
      - The Backup KDS (allocated but empty) was named `PLEX75.SHARED3.COPY.SCSFCKDS`

   c. Press Enter.

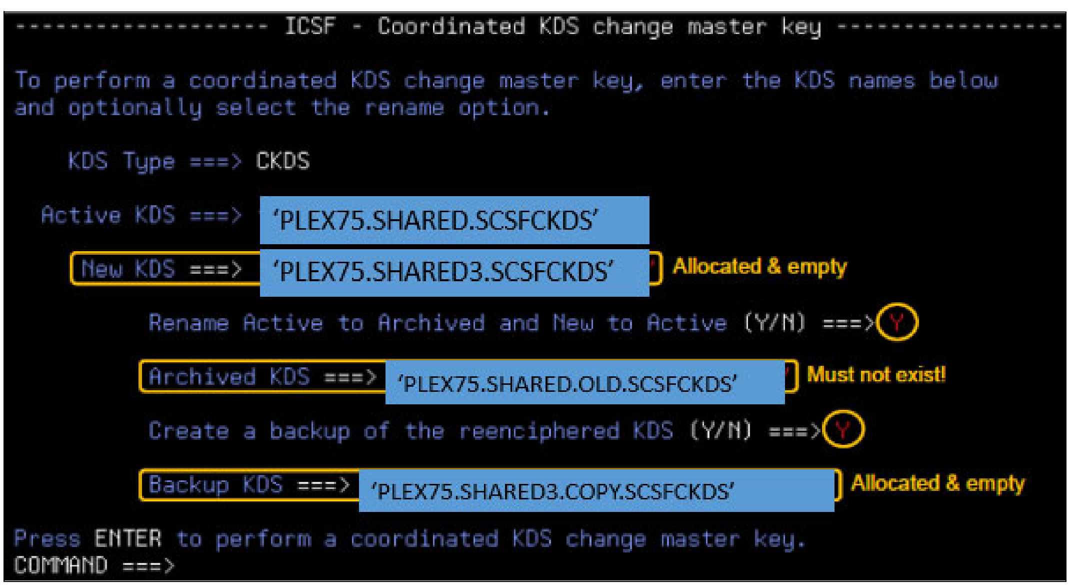


Figure 7-5 Archive KDS

6. Enter `Y` to confirm the operation (see Figure 7-6). Press Enter.

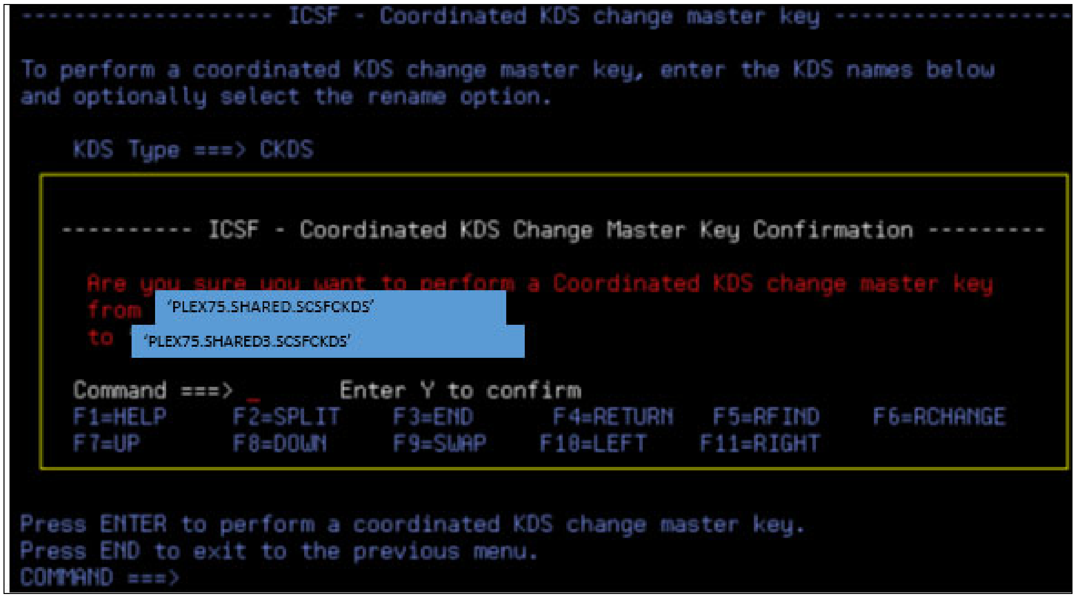


Figure 7-6 Confirm KDS change

7. Check the status message on the ICSF - Coordinated KDS change master key panel (see Figure 7-7).

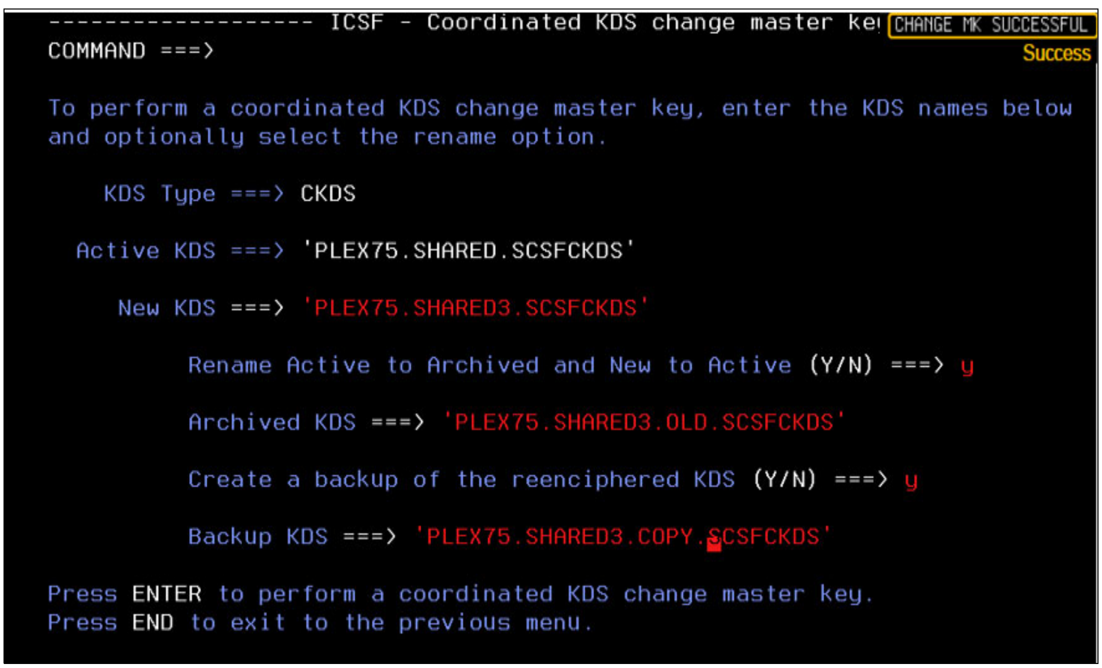


Figure 7-7 Coordinated KDS change: Change successful

8. Check for the MVS Console messages. SYSLOG includes the messages that are shown in Example 7-3.

Example 7-3 SYSLOG messages

```
CSFM618I CKDS DATA SET PLEX75.SHARED3.SCSFCKDS.INDEX RENAMED TO
PLEX75.SHARED.SCSFCKDS.INDEX
```

```
IEF196I CSFM618I CKDS DATA SET PLEX75.SHARED3.SCSFCKDS.INDEX RENAMED
TO
IEF196I PLEX75.SHARED.SCSFCKDS.INDEX
CSFM622I COORDINATED CHANGE-MK PROGRESS: DATA SET RENAMING COMPLETE.
IEF196I CSFM622I COORDINATED CHANGE-MK PROGRESS: DATA SET RENAMING
IEF196I COMPLETE.
IEF196I IEF237I 9788 ALLOCATED TO SYS00005
CSFM653I CKDS LOADED 9 RECORDS WITH AVERAGE SIZE 253
IEF196I CSFM653I CKDS LOADED 9 RECORDS WITH AVERAGE SIZE 253
IEF196I IGD104I PLEX75.SHARED.SCSFCKDS                      RETAINED,
IEF196I DDNAME=SYS00005
CSFM622I COORDINATED CHANGE-MK PROGRESS: NEW IN-STORAGE KDS LOADED ON
REMOTE SYSTEMS.
IEF196I CSFM622I COORDINATED CHANGE-MK PROGRESS: NEW IN-STORAGE KDS
IEF196I LOADED ON REMOTE SYSTEMS.
CSFM622I COORDINATED CHANGE-MK PROGRESS: OPERATION TERMINATION IS
TEMPORARILY INHIBITED.
IEF196I CSFM622I COORDINATED CHANGE-MK PROGRESS: OPERATION TERMINATION
IEF196I IS TEMPORARILY INHIBITED.
CSFM622I COORDINATED CHANGE-MK PROGRESS: A NEW CKDS HASH TABLE WAS
CONSTRUCTED.
IEF196I CSFM622I COORDINATED CHANGE-MK PROGRESS: A NEW CKDS HASH TABLE
IEF196I WAS CONSTRUCTED.
IXC121I CFRM ENCRYPT CHANGE-MK PROGRESS: CHANGE MASTER KEY PROCESSING COMPLETED.
IEF196I CSFM622I COORDINATED CHANGE-MK PROGRESS: CHANGE MASTER KEY
IEF196I PROCESSING COMPLETED.
CSFM622I COORDINATED CHANGE-MK PROGRESS: ALL FINAL CKDS DSN REFERENCES
 UPDATED.
IEF196I CSFM622I COORDINATED CHANGE-MK PROGRESS: ALL FINAL CKDS DSN
IEF196I REFERENCES UPDATED.
CSFM622I COORDINATED CHANGE-MK PROGRESS: SWITCHED THE ACTIVE CKDS
HASH TABLE TO NEW.
IEF196I CSFM622I COORDINATED CHANGE-MK PROGRESS: SWITCHED THE ACTIVE
IEF196I CKDS HASH TABLE TO NEW.
CSFM622I COORDINATED CHANGE-MK PROGRESS: OPERATION TERMINATION IS NOW
REENABLED.
IEF196I CSFM622I COORDINATED CHANGE-MK PROGRESS: OPERATION TERMINATION
IEF196I IS NOW REENABLED.
CSFM622I COORDINATED CHANGE-MK PROGRESS: COMPLETING CORE WORK.
IEF196I CSFM622I COORDINATED CHANGE-MK PROGRESS: COMPLETING CORE WORK.
IXC121I CFRM ENCRYPT CHANGE-MK PROGRESS: REENCIPHERING KEYS IN CFRM
CDS.
CSFM622I COORDINATED CHANGE-MK PROGRESS: A NEW CKDS HASH TABLE WAS
CONSTRUCTED.
IEF196I CSFM622I COORDINATED CHANGE-MK PROGRESS: A NEW CKDS HASH TABLE
IEF196I WAS CONSTRUCTED.
IXC121I CFRM ENCRYPT CHANGE-MK PROGRESS: ADMINISTRATIVE DATA KEYS
IXC121I CFRM ENCRYPT CHANGE-MK PROGRESS: CHANGE MASTER KEY PROCESSING
STARTED.
IXC121I CFRM ENCRYPT CHANGE-MK PROGRESS: ACTIVE POLICY DATA HAS BEEN
UPDATED.
IXC121I CFRM ENCRYPT CHANGE-MK PROGRESS: CF STRUCTURE UPDATES PENDING.
CSFM622I COORDINATED CHANGE-MK PROGRESS: CHANGE MASTER KEY PROCESSING
COMPLETED.
IEF196I CSFM622I COORDINATED CHANGE-MK PROGRESS: CHANGE MASTER KEY
```

```
IEF196I PROCESSING COMPLETED.
CSFM622I COORDINATED CHANGE-MK PROGRESS: ALL FINAL CKDS DSN REFERENCES
 UPDATED.
IEF196I CSFM622I COORDINATED CHANGE-MK PROGRESS: ALL FINAL CKDS DSN
IEF196I REFERENCES UPDATED.
CSFM622I COORDINATED CHANGE-MK PROGRESS: SWITCHED THE ACTIVE CKDS
HASH TABLE TO NEW.
IEF196I CSFM622I COORDINATED CHANGE-MK PROGRESS: SWITCHED THE ACTIVE
IEF196I CKDS HASH TABLE TO NEW.
CSFM617I COORDINATED CHANGE-MK ACTION COMPLETED SUCCESSFULLY.
CSFU006I CHANGE-MK FEEDBACK: RC=00000000 RS=00000000 SUPRC=00000000
SUPRS=00000000 FLAGS=40000000.
IEF196I CSFU006I CHANGE-MK FEEDBACK: RC=00000000 RS=00000000
IEF196I SUPRC=00000000 SUPRS=00000000 FLAGS=40000000.
IXC121I CFRM ENCRYPT CHANGE-MK PROGRESS: CF STRUCTURE UPDATES PENDING.
IXC121I CFRM ENCRYPT CHANGE-MK PROGRESS: CF STRUCTURE UPDATES
COMPLETED.
```

Note: All of the IXC messages are sent by Sysplex Services for Data Sharing (XES) in this scenario because we enabled CF structure encryption in our environment. XES detected a change in the AES master key so that it drove a CF structure change in the CFRM couple data set.

9. Press F3 to return to the CKDS Management panel. Press F3 to return to the KDS Management panel.

Step 4: Verifying the AES master key is active

To verify that the master keys are active, complete the following steps:

1. In the ICSF panel, select **Option 1 - Coprocessor Mgmt** (see Figure 7-8) and press Enter.

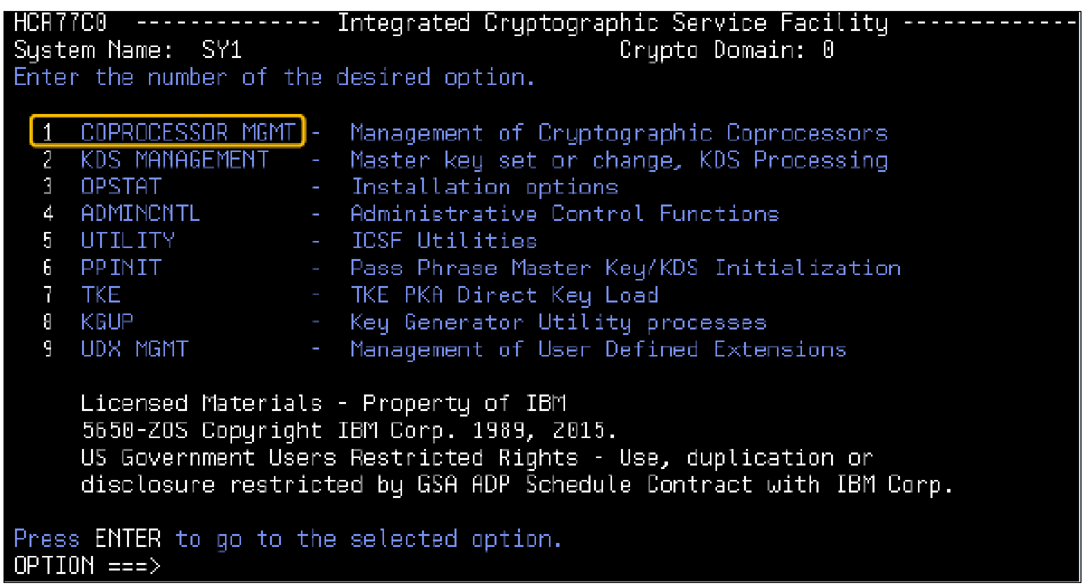
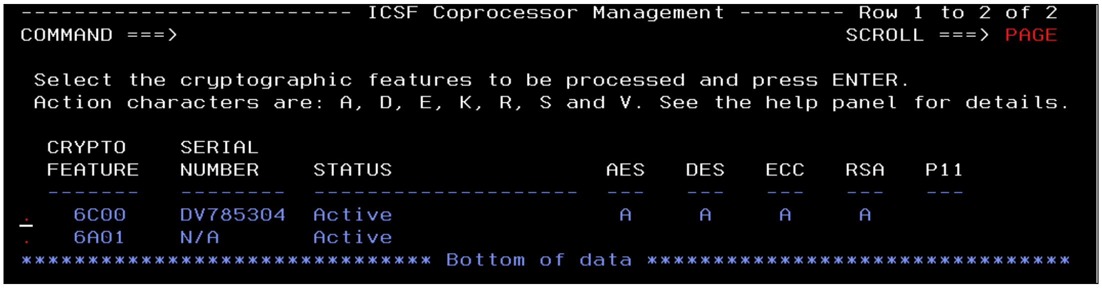


Figure 7-8 ICSF panel select Option 1

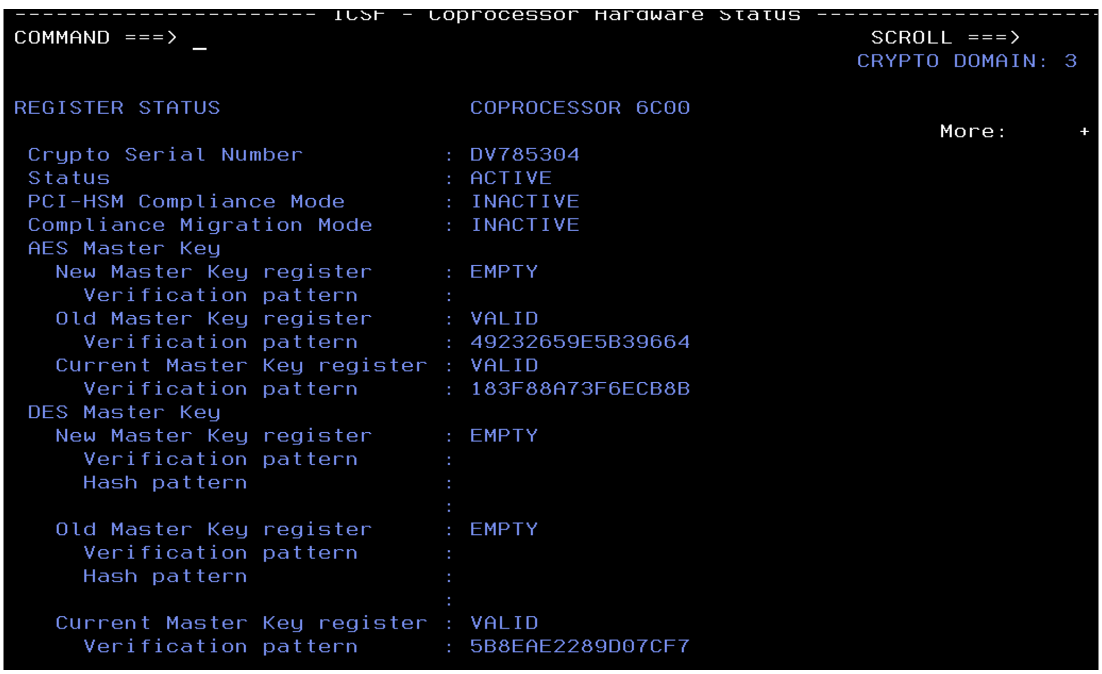
2. The ICSF Coprocessor Management panel is shown in Figure 7-9 on page 166. Enter s next to the crypto feature to view its status. Press Enter.



```
------------------------- ICSF Coprocessor Management -------- Row 1 to 2 of 2
COMMAND ===>                                                  SCROLL ===> PAGE

 Select the cryptographic features to be processed and press ENTER.
 Action characters are: A, D, E, K, R, S and V. See the help panel for details.

  CRYPTO     SERIAL
  FEATURE    NUMBER     STATUS                AES   DES   ECC   RSA   P11
  -------    --------   --------------------  ---   ---   ---   ---   ---
_   6C00     DV785304   Active                 A     A     A     A
.   6A01     N/A        Active
******************************* Bottom of data ********************************
```

Figure 7-9 ICSF Coprocessor Management for the crypto features

3. View the Coprocessor Hardware Status panel (see Figure 7-10). Review the following Master keys information:
   - New Master Key register is EMPTY.
   - Current Master Key register is VALID with a new Verification Pattern.
   - Old Master Key register is VALID with the Verification Pattern from the previous Master Key.



```
---------------------- ICSF - Coprocessor Hardware Status ---------------------
COMMAND ===> _                                                  SCROLL ===>
                                                             CRYPTO DOMAIN: 3

REGISTER STATUS                   COPROCESSOR 6C00
                                                                 More:     +
 Crypto Serial Number           : DV785304
 Status                         : ACTIVE
 PCI-HSM Compliance Mode        : INACTIVE
 Compliance Migration Mode      : INACTIVE
 AES Master Key
   New Master Key register      : EMPTY
     Verification pattern       :
   Old Master Key register      : VALID
     Verification pattern       : 49232659E5B39664
   Current Master Key register  : VALID
     Verification pattern       : 183F88A73F6ECB8B
 DES Master Key
   New Master Key register      : EMPTY
     Verification pattern       :
     Hash pattern               :
                                :
   Old Master Key register      : EMPTY
     Verification pattern       :
     Hash pattern               :
                                :
   Current Master Key register  : VALID
     Verification pattern       : 5B8EAE2289D07CF7
```

Figure 7-10 Coprocessor hardware status

7.2.2 Rotating data set encryption keys

Deciding to rotate your data set encryption keys is determined by your security policy or compliance mandates. It is also done for a compromised key.

Two common approaches to rotating data set encryption keys are listed in Table 7-1 on page 167. Each approach features pros and cons.

Table 7-1 Data set encryption key rotation approaches

Approach	Description	Considerations
Aging Out	Current data is encrypted with current key. After a specific period, a new key is generated and assigned. New data is encrypted with the new key.	▶ Nondisruptive ▶ Not sufficient when a key is compromised ▶ Affects new data only ▶ Existing data is not reencrypted ▶ Old keys must remain in the CKDS ▶ More keys to manage ▶ Key versioning is recommended
Re-Encrypt All Data	A new key is generated and assigned. Existing data is reencrypted with the new key. New data is encrypted with the new key.	▶ Disruptive in most cases[1] ▶ Recommended when a key is compromised ▶ Affects all data (new and old) ▶ Must identify all data encrypted with the old key ▶ Archiving the old key is recommended rather than deleting the old key ▶ Crypto-periods can be established to restrict key usage ▶ Key versioning is recommended

1 For Db2 data sets, you can start an online reorganization to make the reencryption process nondisruptive.

The first three steps for aging out and reencrypting all data are the same for both approaches. The later steps apply to re-encrypting all data only.

Step 1: Generating an operational key

To rotate a data set encryption key, you must generate a key and associated key label.

As described in 3.6.7, "Creating a key label naming convention" on page 52, you must consider your key label naming convention in determining the key label for the new key. Consider the following questions:

- Is there an existing naming convention?
- Is there a new sequence number?
- What is the next sequence number?
- Will the key label be protected under an existing data set profile?

After you determine your key label name, you can choose one of several methods to generate a key label, as described in 5.3, "Generating a secure 256-bit data set encryption key" on page 131.

Step 2: Locating all key label sources

Identify the DATASET class resources, SMS DATACLAS resource, TSO ALLOCATE CLISTs, and JCL that include the key label.

DATASET class

The DATASET class is the primary method that is used to supply a key label for data set encryption. You can create an ICETOOL to identify which DATASET class resources include a DATAKEY segment. Sample JCL to create such a report is shown in Example 7-4 on page 168.

In Example 7-4 on page 168, ISFPP.ITSO.RACF is the RACF database name in our environment. Also, the INCLUDE statement can be modified to filter the results.

Example 7-4 Report for DATASET class profiles

```
//* ----------------------------------------------------------------
//* DISPLAYS A REPORT OF ALL DATASET CLASS PROFILES WITH A
//* DATAKEY LABEL IN THE DFP SEGMENT.
//* ----------------------------------------------------------------
//* UNLOAD THE RACF DB IN QUESTION TO A FLAT DATASET.
//* ----------------------------------------------------------------
//IRRDBU00 EXEC PGM=IRRDBU00,PARM='NOLOCK'
//SYSPRINT DD  SYSOUT=*
//INDD1    DD  DISP=SHR,DSN=ISFPP.ITSO.RACF
//OUTDD    DD  DSN=&&RACFLAT,DISP=(NEW,PASS,DELETE),
//             SPACE=(CYL,(1,1,0)),
//             DCB=(LRECL=4096,RECFM=VB)
//SYSUDUMP DD  SYSOUT=*
//* ----------------------------------------------------------------
//* USE THE OUTPUT FOR THE ICE REPORT
//* ----------------------------------------------------------------
//ICETOOL EXEC PGM=ICETOOL,PARM='MSGPRT=ALL'
//COUNTMSG DD SYSOUT=*
//TOOLMSG  DD SYSOUT=*
//PRINT    DD SYSOUT=*
//DFSMSG   DD SYSOUT=*
//DBUDATA  DD DSN=&&RACFLAT,DISP=(OLD,DELETE,DELETE)
//TEMP0001 DD UNIT=SYSALLDA,SPACE=(CYL,(10,5))
//TOOLIN   DD *
 SORT     FROM(DBUDATA)     TO(TEMP0001) USING(DFP$)
 DISPLAY  FROM(TEMP0001) LIST(PRINT) -
          PAGE -
          TITLE('DATA SET PROFILES WITH A DFP DATAKEY')          -
          DATE(YMD/) -
          TIME(12:)  -
          BLANK -
          ON(10,44,CH)                HEADER('DATA SET NAME')     -
          ON(55,06,CH)                HEADER('VOLUME')            -
          ON(62,08,CH)                HEADER('RESOWNER')          -
          ON(71,64,CH)                HEADER('DATAKEY')
//DFP$CNTL  DD *
  SORT    FIELDS=COPY
  OPTION  VLSHRT
  INCLUDE COND=(05,04,CH,EQ,C'0410',AND,
                71,08,CH,NE,C' ')
/*
```

Example 7-5 shows the output from the JCL in Example 7-4 with the following INCLUDE statement:

```
INCLUDE COND=(05,04,CH,EQ,C'0410',AND,
                71,08,CH,NE,C' ')
```

The output lists the DATASET profiles that include a DFP segment.

Example 7-5 DFP segment with key labels

```
DATA SET NAME                VOLUME   RESOWNER   DATAKEY
---------------------------  ------   --------   ------------------------------------------
DB12D.DSNDBC.DSN8D11A.**                         DATASET.PE06.ICSF.ENCRYPT.DB12D
```

```
DB12D.DSNDBC.DSN8*.*                                       DATASET.PE06.ICSF.ENCRYPT.DB12D
DB12D.DSNDBC.LIBD.*                                        DATASET.PE06.ICSF.ENCRYPT.DB12D
DB12D.DSNDBC.LIBDB.**                                      DATASET.PE06.ICSF.ENCRYPT.DB12D
DB12D.DSNDBD.DSN8D11A.**                                   DATASET.PE06.ICSF.ENCRYPT.DB12D
DB12D.DSNDBD.DSN8*.*                                       DATASET.PE06.ICSF.ENCRYPT.DB12D
DB12D.DSNDBD.LIBD.*                                        DATASET.PE06.ICSF.ENCRYPT.DB12D
DB12D.DSNDBD.LIBDB.*                                       DATASET.PE06.ICSF.ENCRYPT.DB12D
DB12D.DSNDBD.LIB                                           DATASET.PE06.ICSF.ENCRYPT.DB12D
PE01.ENCRYB2.*                          PE01               DATASET.PE01.NOT
PE01.ICSF.ENCRYPT.ME.*                                     DATASET.PE01.ICSF.ENCRYPT.ME.ENCRKEY.00000001
PE01.ENCRYB2.DATASET           CONSM2   PE01               DATASET.PE01.TEST
PE03.PE03.ENC.*                         PE03               DATASET.PE03.AC01
PE03.SECURE.**                          PE03               PE03.SECURE.KEY
PE06.ICSF.ENCRYPT.ME.*                                     DATASET.PE06.ICSF.ENCRYPT.ME.ENCRKEY.00000005
PE06.ICSF.ENCRYPT.ME06.*                                   DATASET.PE06.ICSF.ENCRYPT.ME06.ENCRKEY.00000006
PE06.ICSF.ENCRYPT.ME07.*                                   DATASET.PE06.ICSF.ENCRYPT.ME07.ENCRKEY.00000007
PE08.SC60.ENCRYPT.*                                        DATASET.ENCRYPTKEY.001
```

Example 7-6 shows the output from the JCL in Example 7-4 on page 168 with the following INCLUDE statement:

```
INCLUDE COND=(05,04,CH,EQ,C'0410',AND,
              71,17,CH,EQ,C'DATASET.PE01.TEST')
```

The output lists the DATASET profiles that include a key label DATASET.PE01.TEST.

Example 7-6   Data sets with a specific key label

```
DATA SET NAME                  VOLUME   RESOWNER   DATAKEY
-----------------------------  ------   --------   -----------------------------------------------
PE01.ENCRYB2.*                          PE01       DATASET.PE01.NOT
PE01.ICSF.ENCRYPT.ME.*                             DATASET.PE01.ICSF.ENCRYPT.ME.ENCRKEY.00000001
PE01.ENCRYB2.DATASET           CONSM2   PE01       DATASET.PE01.TEST
```

Step 3: Updating the key label sources with the new key label

Now that the new key is generated and the key label sources are identified, you can update the key label sources with the new key label. From that point forward, all newly allocated data sets are encrypted with the new key. Any data sets remain encrypted by the original key.

> **Note:** Because the old data sets remain encrypted by the old key, the old key must remain in the CKDS.

Step 4: Identifying data sets encrypted with the old key label

When you must reencrypt all data that is associated with a particular key, you must identify which data sets are associated with the original key label.

You must consider every location where the data is stored, such as data that is:

- Active on DASD
- Migrated to tape
- Replicated to DR

You can use IBM zSecure (2.3.6, "IBM Security zSecure Suite" on page 32) or the UKO for z/OS data set dashboard (10.4, "UKO for z/OS view of data sets" on page 243) to identify encrypted data sets that are associated with a particular key label.

When the data sets are located, you can verify the key label that is associated with the data set by issuing the `LISTCAT ENTRIES(<data set name>) ALL` command on the data sets.

Step 5: Allocating new data sets to replace the old data sets

Key labels are associated with data sets at data set allocation. Therefore, every data set must include an associated data set that was allocated with the new key.

Step 6: Copying the contents of the old data sets to the new data sets

After the new data sets are created, the data set contents can be copied from the old data set to the new data set.

> **Note:** To perform the copy operation, the user who is performing the copy must have READ access to the old key label and the new key label.

Standard utilities can be used to perform the copy operation, including the following standard utilities:

- ISPF 3.3 Copy data set
- IDCAMS REPRO
- IEBGENER

> **Note:** Db2 and IMS provide nondisruptive migration to encryption with their Database Online Reorganization function. For high availability, Db2 and IMS provide nondisruptive migration to encryption with Database Online Reorganization function.

After the copy operation completes, you have the old data set encrypted with the old key and the new data set encrypted with the new key.

Step 7: Deleting the old data sets

You can choose to delete the old data sets and rename the new data sets to the old so that applications do not need to change.

Alternatively, you can choose to keep or rename the old data sets. In this case, you might want to consider setting the old key label cryptoperiod to end on the day the data was copied into the new data set. This process prevents applications from using the old data set. For more information about cryptoperiods, see 9.6, “Setting key expiration dates” on page 219.

Step 8: Archiving the old key

We do not recommend deleting old data set keys. If the key is deleted, the associated encrypted data sets cannot be opened. For example, if you missed a few data sets during the identification process and the key was deleted, those data sets cannot be decrypted. If you have a backup of the CKDS and the corresponding master key at the time of the backup, you can attempt to recover the old key.

Rather than deleting the old data set encryption key, you can archive the key. The archived key remains in the active CKDS and is disabled for active use by default. When you want to use an archived key, you can recall it by using ICSF or you can define the `CSF.KDS.KEY.ARCHIVE.USE` resource in the `XFACILIT` class to allow all archived keys to be used.

For more information about key archiving, see 9.4, “Archiving data set encryption keys” on page 217.

Tip: With UKO, the old key can be deactivated. Deactivating a key in UKO removes the key from the CKDS but UKO retains a backup of the key. UKO can use the backup to restore the key if necessary.

The effect is similar to archiving but reduces the overall number of keys in the CKDS.

An alternative to key archiving is setting a key expiration date. When a key expires, it can no longer be used in cryptographic operations. The expiration date must be changed to allow use of the key.

For more information, see 9.6, “Setting key expiration dates” on page 219.

8

Maintaining the ICSF environment

With ICSF installed and configured for data set encryption, it is important to maintain the ICSF environment. This process includes several tasks, such as verifying the master keys, verifying the ICSF installation options, ensuring the CKDS has sufficient space, and validating the CKDS keys.

This chapter includes the following topics:

8.1 Viewing master key information

Master key information can be verified at ICSF initialization, during a master key rotation, and any time a Crypto Express adapter is enabled. You can check the status and state of a master key by using the ICSF utility panels and z/OS operator commands.

8.1.1 ICSF Coprocessor Management panel

You can obtain information about your Crypto Express adapter configuration and usage by using the ICSF ISPF panels. The primary panel indicates the crypto domain that is in use (84), as shown in Example 8-1.

Example 8-1 ICSF ISPF primary panel

```
OPTION ===> 1
System Name:  SC60                          Crypto Domain: 84
Enter the number of the desired option.

  1  COPROCESSOR MGMT -  Management of Cryptographic Coprocessors
```

Select **option 1 COPROCESSOR MGMT** to see the results, as shown in Example 8-2.

Example 8-2 Option 1 COPROCESSOR MGMT selection results

```
Select the cryptographic features to be processed and press ENTER.
Action characters are: A, D, E, K, R, S, and V. See the help panel for details.

 CRYPTO    SERIAL
 FEATURE   NUMBER    STATUS               AES   DES   ECC   RSA   P11
 -------   --------  -------------------- ---   ---   ---   ---   ---
   6C00    DV785304  Active                A     A     A     A
   6A01    N/A       Active
```

Selecting COPROCESSOR MGMT displays the state of your crypto adapters. The “A” stands for “active” (which is expected). When key data sets are not initialized, the state of the crypto adapter is marked “I” for “ignored”.

For more information about activating a master key, see 4.3.5, “Loading the AES master key” on page 86.

Enter s on the line next to a cryptographic feature to see more information about its status (see Example 8-3).

Example 8-3 Status of a crypto adapter

```
REGISTER STATUS                   COPROCESSOR 6C00
                                                                  More:     +
 Crypto Serial Number          : DV785304
 Status                        : ACTIVE
 PCI-HSM Compliance Mode       : INACTIVE
 Compliance Migration Mode     : INACTIVE
 AES Master Key
   New Master Key register     : EMPTY
     Verification pattern      :
   Old Master Key register     : EMPTY
     Verification pattern      :
   Current Master Key register : VALID
     Verification pattern      : 49232659E5B39664
```

As shown in Example 8-3, the Current Master Key register is VALID and the Status is ACTIVE.

8.1.2 Using the command Display ICSF (D ICSF,MKS and D ICSF,CARDS)

You can use Display ICSF operator commands to obtain master key information and Crypto Express adapter status.

Notes:

- ICSF FMID HCR77C0 and newer versions are supported for Data Set Encryption with Crypto Express5S and later.
- z/OS V2R3 and later are supported.

The **D ICSF,MKS** command and the status of the master key registers is shown in Example 8-4.

Example 8-4 D ICSF,MKS command and results

```
D ICSF,MKS
CSFM668I 13.38.20 ICSF MKS 566
  SYSNAME: SC60     DOMAIN: 084 CPC Name: CETUS
    FEATURE SERIAL#  STATUS              AES DES ECC RSA P11
     6C00   DV785304 Active               A   A   A   A
```

The CCA feature device number (6C00), its serial number (DV785304), its status (active), and the master keys that are loaded (AES for our purpose) are shown in Example 8-4.

Command D ICSF,CARDS provides more information (see Example 8-5).

Example 8-5 D ICSF, CARDS command and results

```
D ICSF,CARDS
CSFM668I 13.40.58 ICSF CARDS 568
  ACTIVE DOMAIN = 084
  CRYPTO EXPRESS6 COPROCESSOR 6C00
    STATUS=Active               SERIAL#=DV785304 LEVEL=6.0.6z
```

```
    REQUESTS=0000001185  ACTIVE=0000
  CRYPTO EXPRESS6 ACCELERATOR 6A01
    STATUS=Active
    REQUESTS=0000000005  ACTIVE=0000
```

The active domain (84), the number of requests that are sent to the adapter since ICSF initialization (1185 for the device), and the firmware level of the device (for example, 6.0.6z) are also shown in Example 8-5 on page 175.

8.2 Viewing ICSF options

ICSF installation options can be verified by using the ICSF utility panels and MVS operator commands.

8.2.1 ICSF OPSTAT utility panel

You can display your ICSF configuration options by using the ICSF ISPF panels (see Example 8-6).

Example 8-6 ICSF selecting Option 3

```
HCR77C1  -------------- Integrated Cryptographic Service Facility ----
OPTION ===> 3.1
System Name:  SC74                          Crypto Domain: 3
Enter the number of the desired option.

  1  COPROCESSOR MGMT -  Management of Cryptographic Coprocessors
  2  KDS MANAGEMENT   -  Master key set or change, KDS Processing
  3  OPSTAT           -  Installation options
  4  ADMINCNTL        -  Administrative Control Functions
  5  UTILITY          -  ICSF Utilities
  6  PPINIT           -  Pass Phrase Master Key/KDS Initialization
  7  TKE              -  TKE PKA Direct Key Load
  8  KGUP             -  Key Generator Utility processes
  9  UDX MGMT         -  Management of User-Defined Extensions

------------------------ ICSF - Key Data Set Management ------------
```

Select **option 3.1 OPSTAT** to see the results, as shown in Example 8-7.

Example 8-7 iCSF configuration options

```
---------------------- ICSF - Installation Option Display -- Row 1 to 23 of 33
 COMMAND ===>                                                 SCROLL ===> PAGE
          Active CKDS: PLEX75.SHARED.SCSFCKDS
          Active PKDS: PLEX75.SHARED.SCSFPKDS
          Active TKDS: PLEX75.SHARED.SCSFTKDS
   OPTION                                                     CURRENT VALUE
   ------                                                     -------------
   AUDITKEYLIFECKDS Audit CCA symmetric key lifecycle events   TOKEN(N),LABEL(N)
   AUDITKEYLIFEPKDS Audit CCA asymmetric key lifecycle events  TOKEN(N),LABEL(N)
   AUDITKEYLIFETKDS Audit PKCS #11 key lifecycle events        TOKO(N),SESSO(N)
   AUDITKEYUSGCKDS  Audit CCA symmetric key usage events       TOK(N),LAB(N),
```

```
                                                      INT(001/00.00.00)
  AUDITKEYUSGPKDS  Audit CCA asymmetric key usage events     TOK(N),LAB(N),
                                                      INT(001/00.00.00)
  AUDITPKCS11USG   Audit PKCS #11 usage events               TOKO(N),SESSO(N),
                                                      NOKEY(N),
                                                      INT(001/00.00.00)
  CHECKAUTH        RACF check authorized callers             NO
  CICSAUDIT        Audit CICS client identity                NO
  COMPAT           Allow CUSP/PCF compatibility              NO
  COMPLIANCEWARN   Compliance Warn mode                      NOT SPECIFIED
  CTRACE           CTRACE parmlib used at ICSF startup       CTICSF00
  DEFAULTWRAP      Default symmetric key wrapping - internal  ORIGINAL
  DEFAULTWRAP      Default symmetric key wrapping - external  ORIGINAL
  DOMAIN           Current domain index or usage domain index 3
  FIPSMODE         Operate PKCS #11 in FIPS 140-2 mode       NO,FAIL(NO)
  KDSREFDAYS       Number of days between reference updates  1
  KEYARCHMSG       JOBLOG message for archived key use       NO
  MASTERKCVLEN     Length of master key verification patterns ALL
  MAXSESSOBJECTS   Max non-auth pgm PKCS #11 session objects  65535
 F1=HELP      F2=SPLIT     F3=END       F4=RETURN    F5=RFIND     F6=RCHANGE
 F7=UP        F8=DOWN      F9=SWAP     F10=LEFT     F11=RIGHT    F12=RETRIEVE
```

8.2.2 Using the command Display ICSF (D ICSF,OPT)

On systems that are running ICSF FMID HCR77B1 or later, and running z/OS V2R1 or later, you can use the **`Display ICSF (D ICSF,OPT)`** command to display ICSF options.

The **`D ICSF,OPT`** command and the ICSF options are shown in Example 8-8.

Example 8-8 Display ICSF options

```
D ICSF,OPT
 CSFM668I 11.21.40 ICSF OPTIONS 108
   SYSNAME = SC74          ICSF LEVEL = HCR77C1
     LATEST ICSF CODE CHANGE = 04/03/18
     Refdate update interval in Days/HH.MM.SS = 001/00.00.00
     Refdate update period   in Days/HH.MM.SS = 000/01.00.00
     MASTERKCVLEN = display ALL digits
     AUDITKEYLIFECKDS: Audit CCA symmetric key lifecycle events
       SYSNAME   LABEL    TOKEN
       SC74       No       No
     AUDITKEYLIFEPKDS: Audit CCA asymmetric key lifecycle events
       SYSNAME   LABEL    TOKEN
       SC74       No       No
     AUDITKEYLIFETKDS: Audit PKCS #11 key lifecycle events
       SYSNAME   TOKOBJ   SESSOBJ
       SC74       No       No
     AUDITKEYUSGCKDS: Audit CCA symmetric key usage events
       SYSNAME   LABEL    TOKEN     Interval Days/HH.MM.SS
       SC74       No       No                 001/00.00.00
     AUDITKEYUSGPKDS: Audit CCA asymmetric key usage events
       SYSNAME   LABEL    TOKEN     Interval Days/HH.MM.SS
       SC74       No       No                 001/00.00.00
     AUDITPKCS11USG: Audit PKCS #11 usage events
       SYSNAME   TOKOBJ   SESSOBJ   NOKEY  Interval Days/HH.MM.SS
```

```
  SC74       No       No        No              001/00.00.00
STATS:
  SC74      NONE
COMPLIANCEWARN: Compliance warning events
SC74      NOT SPECIFIED STATS:
```

8.3 Refreshing the CKDS

ICSF references an in-storage copy of the CKDS for key label lookup. However, when utilities, such as KGUP or IDCAMS, are used to read and write directly to the key data sets, changes are made to the CKDS that is stored on disk rather than the in-storage copy. ICSF does not recognize the changes that were made to disk unless the CKDS is refreshed such that the in-storage copy of the CKDS is updated.

Note: No refresh is required for keys that are generated by the ICSF panel or callable services.

8.3.1 Refreshing a CKDS shared in a sysplex

If you update the CKDS on disk and are sharing the CKDS in a sysplex, use the Coordinated CKDS Refresh panel utility to refresh the local CKDS and alert all members of the sysplex that are sharing the CKDS to refresh their CKDS. Coordinated CKDS Refresh can be run on a single system.

While the coordinated refresh is in progress, all active systems in the sysplex that are sharing the active KDS or the new KDS are affected. Updates are also suspended. For more information about rejecting update requests, see “Disabling CKDS Updates” on page 183 and “Re-enable CKDS Updates” on page 185.

Complete the following steps to perform a coordinated CKDS refresh:

1. From the ICSF utility panels, select **option 2 KDS management** (see Example 8-9).

Example 8-9 ICSF selecting Option 2

```
HCR77C1  -------------- Integrated Cryptographic Service Facility ----
OPTION ===> 2
System Name:  SC74                           Crypto Domain: 3
Enter the number of the desired option.

  1  COPROCESSOR MGMT -  Management of Cryptographic Coprocessors
  2  KDS MANAGEMENT   -  Master key set or change, KDS Processing
  3  OPSTAT           -  Installation options
  4  ADMINCNTL        -  Administrative Control Functions
  5  UTILITY          -  ICSF Utilities
  6  PPINIT           -  Pass Phrase Master Key/KDS Initialization
  7  TKE              -  TKE PKA Direct Key Load
  8  KGUP             -  Key Generator Utility processes
  9  UDX MGMT         -  Management of User-Defined Extensions

------------------------ ICSF - Key Data Set Management ------------
```

2. Select **option 1 CKDS management** (see Example 8-10).

Example 8-10 ICSF Key Data Set Management with Option 1

```
------------------------ ICSF - Key Data Set Management ------------
OPTION ===> 1

Enter the number of the desired option.

  1  CKDS MANAGEMENT -  Perform Cryptographic Key Data Set (CKDS)
                        functions including master key management
```

3. Select **option 4 COORDINATED CKDS REFRESH** (see Example 8-11).

Example 8-11 ICSF CKDS Management Option 4

```
---------------------------- ICSF - CKDS Management ---------------------------
OPTION ===> 4

Enter the number of the desired option.

  1  CKDS OPERATIONS   -  Initialize a CKDS, activate a different CKDS,
                          (Refresh), or update the header of a CKDS and make
                          it active
  2  REENCIPHER CKDS   -  Reencipher the CKDS prior to changing a symmetric
                          master key
  3  CHANGE SYM MK     -  Change a symmetric master key and activate the
                          reenciphered CKDS
  4  COORDINATED CKDS REFRESH - Perform a coordinated CKDS refresh
```

You see a panel that is similar to the panel that is shown in Example 8-12.

Example 8-12 ICSF - Coordinated KDS Refresh

```
------------------------ ICSF - Coordinated KDS Refresh --------------------
COMMAND ===>

To perform a coordinated KDS refresh to a new KDS, enter the KDS names
below and optionally select the rename option. To perform a coordinated KDS
refresh of the active KDS, simply press enter without entering anything on
this panel.

    KDS Type ===> CKDS

  Active KDS ===> 'PLEX75.SHARED.SCSFCKDS'

     New KDS ===> 'PLEX75.SHARED.LARGER.SCSFCKDS'

          Rename Active to Archived and New to Active (Y/N) ===> y

          Archived KDS ===> 'PLEX75.SHARED.SMALL.SCSFCKDS'
```

In the new KDS field, specify the name of the new KDS to which you want to refresh.

In the archived KDS field, specify the data set name to which you want the active KDS renamed. If the rename option is specified, the active KDS is renamed to the archived KDS name and the new KDS is renamed to the active KDS name. This action removes the necessity to modify the ICSF startup options because the data set remains the same.

> **Note:** The archived KDS name cannot be the same name as the active KDS name or new KDS name.

4. Press Enter to view a confirmation panel (see Example 8-13).

Example 8-13 Confirmation panel

```
--------------- ICSF - Coordinated KDS Refresh Confirmation --------------

   Are you sure you want to perform a Coordinated KDS Refresh
   from 'PLEX75.SHARED.SCSFCKDS'
   to   'PLEX75.SHARED.LARGER.SCSFCKDS'?

  Command ===> _      Enter Y to confirm
```

5. Enter Y to confirm the action.

   The Refresh successful message in the upper right corner displays (see Example 8-14).

Example 8-14 Refresh successful message

```
------------------------ ICSF - Coordinated KDS Refresh --- REFRESH SUCCESSFUL
COMMAND ===>

To perform a coordinated KDS refresh to a new KDS, enter the KDS names
below and optionally select the rename option. To perform a coordinated KDS
refresh of the active KDS, simply press enter without entering anything on
this panel.

    KDS Type ===> CKDS

  Active KDS ===> 'PLEX75.SHARED.SCSFCKDS'

     New KDS ===> 'PLEX75.SHARED.LARGER.SCSFCKDS'

          Rename Active to Archived and New to Active (Y/N) ===> Y

          Archived KDS ===> 'PLEX75.SHARED.SMALL.SCSFCKDS'
```

If you review the SYSLOG or OPERLOG, you see a sequence of messages that were sent by ICSF. These messages confirm the successful run of the coordinated refresh (see Example 8-15).

Example 8-15 Messages confirming success

```
CSFM653I CKDS LOADED 9 RECORDS WITH AVERAGE SIZE 253
CSFM622I COORDINATED REFRESH PROGRESS: NEW IN-STORAGE KDS CONSTRUCTED.
CSFM622I COORDINATED REFRESH PROGRESS: MKVPS VERIFIED BETWEEN CURRENT ACTIVE AND TARGET
DATA SETS.
CSFM618I CKDS DATA SET PLEX75.SHARED.SCSFCKDS RENAMED TO PLEX75.SHARED.SMALL.SCSFCKDS
CSFM618I CKDS DATA SET PLEX75.SHARED.SCSFCKDS.DATA RENAMED TO
PLEX75.SHARED.SMALL.SCSFCKDS.DATA
CSFM618I CKDS DATA SET PLEX75.SHARED.SCSFCKDS.INDEX RENAMED TO
PLEX75.SHARED.SMALL.SCSFCKDS.INDEX
CSFM618I CKDS DATA SET PLEX75.SHARED.LARGER.SCSFCKDS RENAMED TO
PLEX75.SHARED.SCSFCKDS
CSFM618I CKDS DATA SET PLEX75.SHARED.LARGER.SCSFCKDS.DATA RENAMED TO
PLEX75.SHARED.SCSFCKDS.DATA
```

```
CSFM618I CKDS DATA SET PLEX75.SHARED.LARGER.SCSFCKDS.INDEX RENAMED TO
PLEX75.SHARED.SCSFCKDS.INDEX
CSFM622I COORDINATED REFRESH PROGRESS: DATA SET RENAMING COMPLETE.
IEF196I IEF237I 9788 ALLOCATED TO SYS00006
CSFM653I CKDS LOADED 9 RECORDS WITH AVERAGE SIZE 253
IEF196I CSFM653I CKDS LOADED 9 RECORDS WITH AVERAGE SIZE 253
IEF196I IGD104I PLEX75.SHARED.SCSFCKDS                       RETAINED,
IEF196I DDNAME=SYS00006
CSFM622I COORDINATED REFRESH PROGRESS: NEW IN-STORAGE KDS LOADED ON
REMOTE SYSTEMS.
CSFM622I COORDINATED REFRESH PROGRESS: OPERATION TERMINATION IS
TEMPORARILY INHIBITED.
CSFM622I COORDINATED REFRESH PROGRESS: ALL FINAL CKDS DSN REFERENCES
UPDATED.
CSFM622I COORDINATED REFRESH PROGRESS: SWITCHED THE ACTIVE CKDS HASH
TABLE TO NEW.
CSFM622I COORDINATED REFRESH PROGRESS: OPERATION TERMINATION IS NOW
REENABLED.
CSFM622I COORDINATED REFRESH PROGRESS: COMPLETING CORE WORK.
CSFM622I COORDINATED REFRESH PROGRESS: ALL FINAL CKDS DSN REFERENCES
UPDATED.
IEF196I CSFM622I COORDINATED REFRESH PROGRESS: ALL FINAL CKDS DSN
IEF196I REFERENCES UPDATED.
CSFM622I COORDINATED REFRESH PROGRESS: SWITCHED THE ACTIVE CKDS HASH
TABLE TO NEW.
IEF196I CSFM622I COORDINATED REFRESH PROGRESS: SWITCHED THE ACTIVE CKDS
IEF196I HASH TABLE TO NEW.
CSFM617I COORDINATED REFRESH ACTION COMPLETED SUCCESSFULLY.
CSFU006I REFRESH FEEDBACK: RC=00000000 RS=00000000 SUPRC=00000000 SUPRS=00000000
FLAGS=00000000.
```

8.3.2 Refreshing a single CKDS

To update the CKDS on disk, use one of the following methods:

- Use the Refresh option on the Key Administration panel to replace the in-storage copy with the disk copy.
- Start a utility program to refresh the CKDS.

Refresh by using CSFEUTIL JCL

A JCL sample to refresh the in-storage copy of CKDS by using ICSF utility program CSFEUTIL is shown in Example 8-16.

Example 8-16 Refresh in-storage copy of CKDS by using ICSF utility program CSFEUTIL

```
//STEP20  EXEC PGM=CSFEUTIL,
//       PARM='SYS1.SC60NEW.SCSFCKDS,REFRESH'
```

The messages from successful refresh by using CSFEUTIL are shown in Example 8-17.

Example 8-17 Messages from successful refresh

```
CSFM653I CKDS LOADED 12 RECORDS WITH AVERAGE SIZE 249
CSFU002I CSFEUTIL COMPLETED, RETURN CODE = 0, REASON CODE = 0.
```

Refresh from the KGUP panels

From the ICSF Primary menu, select **Option 8 KGUP (Key Generator Utility processes)** → **Option 4 Refresh (Activate an existing cryptographic key data set)** to refresh the in-storage copy of the CKDS.

The refresh in-storage CKDS panel is shown in Figure 8-1.

```
------------------------ ICSF - Refresh in-storage CKDS ------------------
COMMAND ===>

Enter the name of the CKDS to become active.

  New CKDS ===> 'SYS1.SC60NEW.SCSFCKDS'



Press ENTER to refresh the in-storage CKDS
Press END   to exit to previous panel
```

Figure 8-1 ICSF Refresh in-storage CKDS

Press Enter to perform the refresh. A successful refresh results in message CSFM653I, as shown in Example 8-18.

Example 8-18 Message CSFM653I

```
CSFM653I CKDS LOADED 10 RECORDS WITH AVERAGE SIZE 252
```

The Refresh in-storage CKDS panel displays a message to indicate a successful refresh, as shown in Figure 8-2.

```
----------------------- ICSF - Refresh in-storage CKDS --- REFRESH SUCCESSFUL
COMMAND ===>
THE SPECIFIED CKDS IS NOW THE CURRENT CKDS
Enter the name of the CKDS to become active.

  New CKDS ===> 'SYS1.SC60NEW.SCSFCKDS'



Press ENTER to refresh the in-storage CKDS
Press END   to exit to previous panel
```

Figure 8-2 ICSF Refresh Successful message

Refresh by using the ICSF utility panels

Refresh can also be performed from Option 2.1;1.2 from the ICSF Primary menu. The following options are available:

- Option 2 KDS MANAGEMENT: Master key set or change, KDS Processing
- Option 1 CKDS MANAGEMENT: Perform Cryptographic Key Data Set (CKDS) functions, including master key management
- Option 1 CKDS OPERATIONS: Initialize a CKDS, activate a different CKDS, (Refresh), or update the header of a CKDS and make it active
- Option 2 REFRESH: Activate an updated CKDS

8.4 Increasing the CKDS size

You might want to increase the size of your CKDS if your current CKDS is out of space or you determine that you need a larger CKDS.

> **Note:** This method cannot be used to change the CKDS format (for example, non-KDSR to KDSR).

The process to increase the CKDS size includes the following steps:

1. Disabling CKDS updates.
2. Allocating a new, larger CKDS.
3. Copying data from the existing CKDS to the new CKDS.
4. Verifying a successful copy.
5. Refreshing ICSF with the new CKDS.
6. Reenabling CKDS updates.

Determining the current CKDS space allocation

Issue the **`LISTCAT ENTRIES(<CKDSNAME>) ALL`** command to see space allocation values for the CKDS (see Example 8-19). Next, compare the HI-A-RBA (where “A” is allocated) with the HI-U-RBA (where “U” is used) to determine whether the CKDS must be enlarged.

Example 8-19 Results of the LISTCAT ENT(<CKDSNAME>) ALL

```
ALLOCATION
    SPACE-TYPE---------TRACK     HI-A-RBA----------221184
    SPACE-PRI--------------4     HI-U-RBA-----------55296
    SPACE-SEC--------------1
```

Disabling CKDS Updates

Updates to the CKDS should be disabled before the CKDS is enlarged. This update ensures that changes are not made to the CKDS while the copy in is process.

The easiest way to disable updates to the CKDS is to use the **`SETICSF DISABLE,CKDS,SYSPLEX=YES`** operator command. The use of this command disables updates to the CKDS across all members of the sysplex.

Allocating a new, larger CKDS

Determine how large to allocate your new CKDS by using the formula that is described in 3.5.3, “Using the Common Record Format (KDSRL) cryptographic key data set” on page 46.

Allocate the CKDS based on the sample as described in 4.3.2, “Creating a Common Record Format (KDSRL) CKDS” on page 81. Update the primary and secondary values in the RECORDS field based on your calculations.

Copying the CKDS to the new CKDS

Use the IDCAMS utility to copy the data from the current active CKDS into a new, larger CKDS, similar to the JCL that is shown in Example 8-20 on page 184.

Example 8-20 REPRO data from active CKDS to larger CKDS

```
// EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=A
//SYSIN DD *
       REPRO -
       INDATASET(PLEX75.SHARED.SCSFCKDS) -
       OUTDATASET(PLEX75.SHARED.LARGER.SCSFCKDS)
```

Verifying the contents

Use the IDCAMS utility to list the number of records in the current CKDS. The JCL is shown in Example 8-21.

Example 8-21 REPRO to list keys

```
//JS010  EXEC PGM=IDCAMS
//CKDS    DD DISP=SHR,DSN=PLEX75.SHARED.SCSFCKDS
//SYSPRINT DD SYSOUT=*
//OUTPUT  DD SYSOUT=*,LRECL=2048
//SYSIN   DD *,LRECL=80
 REPRO INFILE(CKDS) OUTFILE(OUTPUT)
```

The results of the REPRO command, which includes nine records that consist of eight cryptographic key records and one header record, is shown in Example 8-22.

Example 8-22 Results of REPRO

```
REPRO INFILE(CKDS) OUTFILE(OUTPUT)
IDCAMS  SYSTEM SERVICES                                            TIME: 16:36:56

 REPRO INFILE(CKDS) OUTFILE(OUTPUT)
IDC0005I NUMBER OF RECORDS PROCESSED WAS 9
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0

IDC0002I IDCAMS PROCESSING COMPLETE. MAXIMUM CONDITION CODE WAS 0
                                                                         20170731
ICSF.SECRET.AES256.KEY001                                        DATA
LABEL.01.CLEAR                                                   DATA
LABEL.01.TEST                                                    DATA
LABEL.02.TEST                                                    DATA
LABEL.05.CLEAR                                                   DATA
LABEL.06.TEST                                                    DATA
SAMPLE.DERIVED.AES.IMPORTER.KEY                                  IMPORTER
SAMPLE.RECEIVED.AES.DATA.KEY                                     DATA
```

Use the IDCAMS utility to list the number of records in the new, larger CKDS. The JCL is shown in Example 8-23.

Example 8-23 Printing REPRO

```
//JS010  EXEC PGM=IDCAMS
//CKDS    DD DISP=SHR,DSN=PLEX75.SHARED.LARGER.SCSFCKDS
//SYSPRINT DD SYSOUT=*
//OUTPUT  DD SYSOUT=*,LRECL=2048
//SYSIN   DD *,LRECL=80
 REPRO INFILE(CKDS) OUTFILE(OUTPUT)
```

Verify that the new, larger CKDS contains the same number of records and labels as the original data set.

Refreshing ICSF

After the new CKDS is verified, ICSF must be refreshed to process the new CKDS. For more information, see 8.3, "Refreshing the CKDS" on page 178.

Re-enable CKDS Updates

After the CKDS is refreshed, CKDS updates can be reenabled.

Issue the **`SETICSF ENABLE,CKDS,SYSPLEX=YES`** operator command to enable updates to the CKDS across all members of the sysplex.[1]

8.5 Validating CKDS keys

If a CKDS exists, you can check the key tokens for format errors.

Select ICSF option **2.1.7 CKDS KEY CHECK** (see Figure 8-3).

```
---------------------------- ICSF - CKDS Management ----------------------------
OPTION ===> 7

Enter the number of the desired option.

  1  CKDS OPERATIONS   -  Initialize a CKDS, activate a different CKDS,
                          (Refresh), or update the header of a CKDS and make
                          it active
  2  REENCIPHER CKDS   -  Reencipher the CKDS prior to changing a symmetric
                          master key
  3  CHANGE SYM MK     -  Change a symmetric master key and activate the
                          reenciphered CKDS
  4  COORDINATED CKDS REFRESH - Perform a coordinated CKDS refresh
  5  COORDINATED CKDS CHANGE MK - Perform a coordinated CKDS change master key
  6  COORDINATED CKDS CONVERSION - Convert the CKDS to use KDSR record format
  7  CKDS KEY CHECK    - Check key tokens in the active CKDS for format errors
```

Figure 8-3 ICSF CKDS Management with Option 7 selected

The message "CHECK SUCCESSFUL" is displayed in the upper right corner if no error was detected, as shown in Figure 8-4 on page 186.

[1] Modify your CSFPRMxx to specify the new CKDS name (or redo the operation to make the original CKDS bigger using the same process).

```
---------------------------- ICSF - CKDS Management --------- CHECK SUCCESSFUL
OPTION ===>

Enter the number of the desired option.

  1  CKDS OPERATIONS   -  Initialize a CKDS, activate a different CKDS,
```

Figure 8-4 ICSF CKDS Management with Check Successful message

8.6 Verifying the CKDS format

You can use z/OS command **D ICSF,KDS** to obtain more information about your KDS status (see Example 8-24).

Example 8-24 Use of the D ICSF,KDS command

```
D ICSF,KDS
CSFM668I 13.36.58 ICSF KDS 551
  CKDS  SYS1.SC6ONEW.SCSFCKDS
    FORMAT=KDSRL                 SYSPLEX=N  MKVPs=DES AES
  PKDS  SYS1.SC6ONEW.SCSFPKDS
    FORMAT=KDSR                  SYSPLEX=N  MKVPs=RSA ECC
  No TKDS was provided.
```

The system displays (message CSFM668I) the following information about the active key data sets (KDS) on the system or sysplex:

- The data set name for each active KDS (CKDS, PKDS, and TKDS).
- The format of the KDS (for example, KDSRL is the recommended format to use). The following values are available:
  - KDSR
  - FIXED
  - VARIABLE
- The communication level that is in place for the KDS (for example, 3). This information is displayed in a sysplex environment only.
- Whether the KDS is being shared in a sysplex group (for example, Y/N).
- The MKVPs that were started in the KDS (for example, DES AES). The following values are available:
  - DES, AES, or both for CKDS
  - RSA, ECC, or both for PKDS
  - P11, RCS, or both for TKDS

8.7 Dumping CKDS contents

You can use the IDCAMS utility to dump the contents of the CKDS to a sequential file.

Note: If clear keys are in your CKDS, the clear keys are made available in the dumped data set. If you secure (encrypted) keys are in your CKDS, the keys remain encrypted in the dumped data set. Protected keys are not applicable to the CKDS because they are stored in ICSF protected memory only.

After you allocate your sequential file (for example, PLEX75.SHARED.SCSFCKDS.DUMP), run JCL similar to the JCL that is shown in Example 8-25.

Example 8-25 REPRO to list keys

```
//JS010  EXEC PGM=IDCAMS
//CKDS    DD DISP=SHR,DSN=PLEX75.SHARED.SCSFCKDS
//SYSPRINT DD SYSOUT=*
//OUTPUT  DD DISP=OLD,DSN=PLEX75.SHARED.SCSFCKDS.DUMP
//SYSIN   DD *,LRECL=80
 REPRO INFILE(CKDS) OUTFILE(OUTPUT)
```

The results of running the **REPRO** command, which includes nine records that consist of eight cryptographic key records and one header record, is shown in Example 8-26.

Example 8-26 Results of REPRO

```
                                                                        20170731
ICSF.SECRET.AES256.KEY001                                       DATA
LABEL.01.CLEAR                                                  DATA
LABEL.01.TEST                                                   DATA
LABEL.02.TEST                                                   DATA
LABEL.05.CLEAR                                                  DATA
LABEL.06.TEST                                                   DATA
SAMPLE.DERIVED.AES.IMPORTER.KEY                                 IMPORTER
SAMPLE.RECEIVED.AES.DATA.KEY                                    DATA
```

Note: Clear keys are never returned from the CKDS by way of any of the ICSF callable services. Only secure (encrypted) keys are returned. This result is by design to prevent disclosure of clear key material.

Another option to list the records in a CKDS is to run a REXX script from TSO. For more information about a REXX script, see Rexx Sample: List Records in the CKDS.

8.8 Browsing the CKDS

On systems that are running ICSF, you can browse the contents of the CKDS by completing the following steps:

1. Select option **5** (see Figure 8-5 on page 188).

```
HCR77C1  -------------- Integrated Cryptographic Service Facility ------
OPTION ===> 5
System Name:  SC74                           Crypto Domain: 3
Enter the number of the desired option.

  1  COPROCESSOR MGMT -  Management of Cryptographic Coprocessors
  2  KDS MANAGEMENT   -  Master key set or change, KDS Processing
  3  OPSTAT           -  Installation options
  4  ADMINCNTL        -  Administrative Control Functions
  5  UTILITY          -  ICSF Utilities
```

Figure 8-5 ICSF panel selection Option 5

2. Select option **5** CKDS keys (see Figure 8-6).

```
------------------------------- ICSF - Utilities -----------------
OPTION ===> 5

Enter the number of the desired option.

  1  ENCODE       -  Encode data
  2  DECODE       -  Decode data
  3  RANDOM       -  Generate a random number
  4  CHECKSUM     -  Generate a checksum and verification and
                     hash pattern
  5  CKDS KEYS    -  Manage keys in the CKDS
  6  PKDS KEYS    -  Manage keys in the PKDS
  7  PKCS11 TOKEN -  Management of PKCS11 tokens
```

Figure 8-6 ICSF Utilities

You can generate any list that you want: full or partial record labels, use wildcard characters, or use option 1 List and manage all records (see Figure 8-7).

Tip: Ensure that you have READ access to CSFBRCK CL(CSFSERV).

```
------------------------------- ICSF - CKDS KEYS ----------------------------
 OPTION ===> 1

 Active CKDS: SYS1.SC60NEW.SCSFCKDS                              Keys: 19

 Enter the number of the desired option.
   1  List and manage all records
   2  List and manage records with label key type            leave blank for
```

Figure 8-7 CKDS KEYS panel to list keys

A list that is similar to the list that is shown in our environment in Figure 8-8 is displayed.

```
---------------------------- ICSF - CKDS KEYS List --------- Row 1 to 12 of 1
COMMAND ===>                                                   SCROLL ===> PAG

Active CKDS: SYS1.SC60NEW.SCSFCKDS                             Keys: 12

Action characters: A, D, K, M, P, R  See the help panel for details.
Status characters: - Active   A Archived   I Inactive

Select the records to be processed and press ENTER
When the list is incomplete and you want to see more labels, press ENTER
Press END to return to the previous menu

A S Label     Displaying 1      to 12     of 12                   Key Type
------------------------------------------------------------------------------
_ - AAAA.FIRST.KEY                                                DATA
_ - DATASET.ENCRYPTKEY.001                                        DATA
_ - DATASET.PE01.TEST                                             DATA
_ A DATASET.PE03.AC01                                             DATA
_ - KEYLABEL.ANDY.COU1                                            DATA
_ - KEYLABEL.THOMAS.LIU                                           DATA
_ - NOSTANDARD.NAMING.CONVENTION                                  DATA
_ - PE01.TEST                                                     DATA
_ I PE01.TEST.ACCESS.KEY                                          DATA
_ A PE01.TEST.KEY                                                 DATA
_ - PE03.SECURE.KEY                                               DATA
_ - SC60.TL.AES.EXPORTER.KEY                                      EXPORTER
```

Figure 8-8 CKDS Keys list

3. When the list is incomplete and you want to see more labels (see Figure 8-9 on page 190), press Enter. Press End to return to the previous menu.

   The following Status characters can be displayed in the 'S' column:

   – Active is represented by a hyphen (-).
   – Archived is represented by an A.
   – Inactive is represented by an I.

   Note: Ensure that data set encryption keys are always defined as data-encrypting keys.

   The following action characters are available:

   – K: Display information about the record metadata and the key attributes
   – M: Display record metadata
   – D: Delete the record from the CKDS
   – A: Archive the record (marks the record as archived)
   – R: Recall the record (marks an archived record as available for use)
   – P: Prohibit archive (marks the record so that it cannot be archived)

   Note: The ability to archive, recall, and prohibit archive require the KDSR CKDS format.

4. Select option **K** to display information about the record metadata and the key attributes.

   You can also verify that the keys are secured keys and protected by the AES master key (not clear keys). If the key is not encrypted, the Key Attributes field displays the message “Key value is not encrypted”, as shown in Figure 8-9.

```
------------------- ICSF - CKDS Key Attributes and Metadata -------------------
 COMMAND ===>                                                  SCROLL ===> PAGE

 Active CKDS: SYS1.SC60NEW.SCSFCKDS

 Label: DATASET.PE03.AC01                                                DATA

  Record status: Active         (Archived, Active, Pre-active, Deactivated)

  Select an action:
    1  Modify one or more fields with the new values specified
    2  Delete the record

-------------------------------------------------------------------------------
                                                                    More:   -

 Key Attributes   Key value is not encrypted
  Algorithm:      AES           Key type:        DATA
  Length (bits):  256           Key check value: 16124C    ENC-ZERO
  Key Usage:      ENCIPHER DECIPHER
```

Figure 8-9 CKDS Key Attributes and Metadata panel

5. Select option **M** to display record metadata (see Figure 8-10).

```
Active CKDS: SYS1.SC60NEW.SCSFCKDS

Label: DATASET.PE03.AC01                                                      DATA

 Record status: Active         (Archived, Active, Pre-active, Deactivated)

 Select an action:
   1  Modify one or more fields with the new values specified
   2  Delete the record
--------------------------------------------------------------------------------

                               YYYYMMDD                YYYYMMDD
 Record creation date:         20171120
 Update date:                  00000000
 Cryptoperiod start date:      00000000     New value:
 Cryptoperiod end date:        00000000     New value:
 Date the record was last used: 20171120    New value:
 Service called when last used: CSFSAE
 Date the record was recalled:  00000000
 Date the record was archived:  00000000
 Archived flag:                FALSE        New value:
 Prohibit archive flag:        FALSE        New value:
```

Figure 8-10 Modifying the metadata

The following metadata information is available:

- Date the record was last used: 20171120
- Service called when last used: CSFSAE

Some of the metadata values can be modified. To modify a value, enter a new value in the provided field. A value of all zeros can be used to remove a date field.

9

Maintaining data set encryption keys with ICSF

Properly maintaining data set encryption keys is vital to ensuring that encrypted data sets remain protected and can be successfully decrypted. Data set encryption keys must be available and properly managed.

This chapter includes the following topics:

9.1 Backing up and restoring data set encryption keys

Backing up the CKDS is a critical task for z/OS data set encryption. This process helps ensure that if the CKDS becomes corrupted or a key is mistakenly overwritten, it can be recovered.

9.1.1 Manual backup and restore

A manual backup of the CKDS is recommended in several scenarios. Typically, manual backups are suggested before a critical or manual key update or exchange. After the key update or exchange is complete, verify the key changes.

For a rekeying operation, verification includes checking that the old and new keys are working. You can attempt to open one or more data sets that are encrypted under the old key and new key to ensure that the old and new keys are valid. For more information about rotating data set encryption keys, see 7.2.2, “Rotating data set encryption keys” on page 166.

For a manual key transport operation, verification includes checking that any data sets that are referencing the new key label can be opened successfully. If a key label was intentionally overwritten, verification includes ensuring that any data that is protected by the original key is deleted. For more information about transporting encryption keys, see 9.2, “Transporting data set encryption keys” on page 199.

Several tools are available to manually back up a CKDS, such as DFSMShsm and DFSMSdss. These tools are described next.

Using DFSMShsm

DFSMShsm provides TSO commands and an ISMF utility panel. The following TSO commands can be used for backup and recovery:

- HBACKDS

  Creates a backup of an SMS-managed or non-SMS-managed data set. The command options differ based on how the data set is managed. For more information, see Syntax for SMS-managed data sets.

- HRECOVER

  Recovers a backup data set in by replacing the original or renaming the back up to a different name such that two versions exist. For more information, see Recovering a backup version or a dump copy of a data set.

- HBDELETE

  Deletes a backup version of an SMS-managed or non-SMS-managed data set. For more information, see Using TSO.

Using DFSMSdss

DFSMSdss provides the ADRDSSU z/OS utility, which can be used to dump and restore data sets. The following commands are available for dump and restore:

DFSMSdss Dump	Dumps DASD data to basic, large format, or extended format sequential data sets. For more information, see DUMP command for DFSMSdss.
DFSMSdss Restore	Restores data from dump volumes to DASD volumes. For more information, see RESTORE command for DFSMSdss.

9.1.2 Automated backup and restore

Automated backups can be created at the data set level or volume level. Several solutions and options are available to create automated backups, such as IBM Tivoli® Workload Scheduler for z/OS and DFSMShsm. These tools make help to schedule regular backups during a backup window.

Using DFSMShsm

DFSMShsm can perform automated backups of SMS-managed and non-SMS managed data sets and volumes. For more information about configuration and setup, see DFSMShsm Storage Administration Guide.

9.1.3 Refreshing the CKDS

If a CKDS was recovered from a backup, you must perform a CKDS refresh operation to update the in-memory copy of the CKDS for all ICSF instances that are sharing the CKDS.

You can refresh the CKDS by using the ICSF utility panels. Complete the following steps:

1. In the ICSF ISPF application, select **option 2 KDS management** (see Example 9-1).

Example 9-1 ICSF selecting Option 2

```
HCR77C1  -------------- Integrated Cryptographic Service Facility ----
OPTION ===> 2
System Name:  SC74                           Crypto Domain: 3
Enter the number of the desired option.

  1  COPROCESSOR MGMT -  Management of Cryptographic Coprocessors
  2  KDS MANAGEMENT   -  Master key set or change, KDS Processing
  3  OPSTAT           -  Installation options
  4  ADMINCNTL        -  Administrative Control Functions
  5  UTILITY          -  ICSF Utilities
  6  PPINIT           -  Pass Phrase Master Key/KDS Initialization
  7  TKE              -  TKE PKA Direct Key Load
  8  KGUP             -  Key Generator Utility processes
  9  UDX MGMT         -  Management of User Defined Extensions

----------------------- ICSF - Key Data Set Management ------------
```

2. Select **option 1 CKDS management** (see Example 9-2).

Example 9-2 ICSF Key Data Set Management with Option 1

```
------------------------ ICSF - Key Data Set Management ------------
OPTION ===> 1

Enter the number of the desired option.

  1  CKDS MANAGEMENT -  Perform Cryptographic Key Data Set (CKDS)
                        functions including master key management
```

3. Select **option 4 COORDINATED CKDS REFRESH** (see Example 9-3 on page 196).

Example 9-3 ICSF CKDS Management Option 4

```
---------------------------- ICSF - CKDS Management ---------------------------
OPTION ===> 4

Enter the number of the desired option.

  1  CKDS OPERATIONS   -  Initialize a CKDS, activate a different CKDS,
                          (Refresh), or update the header of a CKDS and make
                          it active
  2  REENCIPHER CKDS   -  Reencipher the CKDS prior to changing a symmetric
                          master key
  3  CHANGE SYM MK     -  Change a symmetric master key and activate the
                          reenciphered CKDS
  4  COORDINATED CKDS REFRESH - Perform a coordinated CKDS refresh
```

While the coordinated refresh is in progress, all active systems in the sysplex that are sharing the active KDS or sharing the new KDS are affected.

> **Note:** Consider temporarily disabling dynamic CKDS updates on all sysplex members for the CKDS that you are processing (ICSF main panel option 4, ADMINCNTL) before the coordinated refresh.

The Coordinated KDS refresh panel is shown in Example 9-4.

Example 9-4 ICSF - Coordinated KDS Refresh

```
------------------------ ICSF - Coordinated KDS Refresh --------------------
COMMAND ===>

To perform a coordinated KDS refresh to a new KDS, enter the KDS names
below and optionally select the rename option. To perform a coordinated KDS
refresh of the active KDS, press enter without entering anything on
this panel.

    KDS Type ===> CKDS

  Active KDS ===> 'PLEX75.SHARED.SCSFCKDS'

     New KDS ===> 'PLEX75.SHARED.SCSFCKDS.BACKUP'

          Rename Active to Archived and New to Active (Y/N) ===> y

          Archived KDS ===> 'PLEX75.SHARED.SCSFCKDS.CORRUPT'
```

In the new KDS field, specify the name of the backup KDS to which to refresh. The following process occurs while a coordinated refresh is processed on the New KDS:

1. The specified new KDS is read into memory.
2. The in-memory copy is distributed to all systems that are sharing the active KDS or new KDS.
3. All systems are switched over to the new in-memory copy and the new KDS.

In the archived KDS field, specify the data set name to which you want the active KDS renamed. If the rename option is specified, the currently active KDS is renamed to the archived KDS name and the new KDS is renamed to the current active KDS name. This process removes the necessity to modify the ICSF startup options in CSFPRMxx because the data set remains the same.

Note: The archived KDS name cannot be the same name as the active KDS name or new KDS name.

4. Press Enter to see a confirmation panel (see Example 9-5).

Example 9-5 Confirmation panel

```
--------------- ICSF - Coordinated KDS Refresh Confirmation --------------

   Are you sure you want to perform a Coordinated KDS Refresh
   from 'PLEX75.SHARED.SCSFCKDS'
   to   'PLEX75.SHARED.SCSFCKDS.BACKUP'?

  Command ===> _      Enter Y to confirm
```

5. Enter Y to confirm the action. The Refresh successful message at the upper right corner is shown (see Example 9-6).

Example 9-6 Refresh successful message

```
------------------------ ICSF - Coordinated KDS Refresh --- REFRESH SUCCESSFUL
COMMAND ===>

To perform a coordinated KDS refresh to a new KDS, enter the KDS names
below and optionally select the rename option. To perform a coordinated KDS
refresh of the active KDS, simply press enter without entering anything on
this panel.

    KDS Type ===> CKDS

  Active KDS ===> 'PLEX75.SHARED.SCSFCKDS'

     New KDS ===> 'PLEX75.LARGER.SCSFCKDS.BACKUP'

          Rename Active to Archived and New to Active (Y/N) ===> Y

          Archived KDS ===> 'PLEX75.SHARED.SCSFCKDS.CORRUPT'
```

If you review the SYSLOG or OPERLOG, you see a sequence of messages that was sent by ICSF that confirms the successful execution of the coordinated refresh (see Example 9-7).

Example 9-7 Messages confirming success

```
CSFM653I CKDS LOADED 9 RECORDS WITH AVERAGE SIZE 253
CSFM622I COORDINATED REFRESH PROGRESS: NEW IN-STORAGE KDS CONSTRUCTED.
CSFM622I COORDINATED REFRESH PROGRESS: MKVPS VERIFIED BETWEEN CURRENT ACTIVE AND TARGET
DATA SETS.
CSFM618I CKDS DATA SET PLEX75.SHARED.SCSFCKDS RENAMED TO PLEX75.SHARED.SCSFCKDS.CORRUPT
```

```
CSFM618I CKDS DATA SET PLEX75.SHARED.SCSFCKDS.DATA RENAMED TO
PLEX75.SHARED.SCSFCKDS.CORRUPT.DATA
CSFM618I CKDS DATA SET PLEX75.SHARED.SCSFCKDS.INDEX RENAMED TO
PLEX75.SHARED.SCSFCKDS.CORRUPT.INDEX
CSFM618I CKDS DATA SET PLEX75.SHARED.SCSFCKDS.BACKUP RENAMED TO
PLEX75.SHARED.SCSFCKDS
CSFM618I CKDS DATA SET PLEX75.SHARED.SCSFCKDS.BACKUP.DATA RENAMED TO
PLEX75.SHARED.SCSFCKDS.DATA
CSFM618I CKDS DATA SET PLEX75.SHARED.SCSFCKDS.BACKUP.INDEX RENAMED TO
PLEX75.SHARED.SCSFCKDS.INDEX
CSFM622I COORDINATED REFRESH PROGRESS: DATA SET RENAMING COMPLETE.
IEF196I IEF237I 9788 ALLOCATED TO SYS00006
CSFM653I CKDS LOADED 9 RECORDS WITH AVERAGE SIZE 253
IEF196I CSFM653I CKDS LOADED 9 RECORDS WITH AVERAGE SIZE 253
IEF196I IGD104I PLEX75.SHARED.SCSFCKDS                       RETAINED,
IEF196I DDNAME=SYS00006
CSFM622I COORDINATED REFRESH PROGRESS: NEW IN-STORAGE KDS LOADED ON
REMOTE SYSTEMS.
CSFM622I COORDINATED REFRESH PROGRESS: OPERATION TERMINATION IS
TEMPORARILY INHIBITED.
CSFM622I COORDINATED REFRESH PROGRESS: ALL FINAL CKDS DSN REFERENCES
UPDATED.
CSFM622I COORDINATED REFRESH PROGRESS: SWITCHED THE ACTIVE CKDS HASH
TABLE TO NEW.
CSFM622I COORDINATED REFRESH PROGRESS: OPERATION TERMINATION IS NOW
REENABLED.
CSFM622I COORDINATED REFRESH PROGRESS: COMPLETING CORE WORK.
CSFM622I COORDINATED REFRESH PROGRESS: ALL FINAL CKDS DSN REFERENCES
UPDATED.
IEF196I CSFM622I COORDINATED REFRESH PROGRESS: ALL FINAL CKDS DSN
IEF196I REFERENCES UPDATED.
CSFM622I COORDINATED REFRESH PROGRESS: SWITCHED THE ACTIVE CKDS HASH
TABLE TO NEW.
IEF196I CSFM622I COORDINATED REFRESH PROGRESS: SWITCHED THE ACTIVE CKDS
IEF196I HASH TABLE TO NEW.
CSFM617I COORDINATED REFRESH ACTION COMPLETED SUCCESSFULLY.
CSFU006I REFRESH FEEDBACK: RC=00000000 RS=00000000 SUPRC=00000000 SUPRS=00000000
FLAGS=00000000.
```

The command **D ICSF,KDS** that is used to confirm the refresh is shown in Example 9-8.

Example 9-8 D ICSF,KDS to confirm migration

```
D ICSF,KDS
CSFM668I 16.49.31 ICSF KDS 045
  CKDS  PLEX75.SHARED.SCSFCKDS
    FORMAT=KDSR      COMM LVL=3  SYSPLEX=Y  MKVPs=DES AES
  PKDS  PLEX75.SHARED.SCSFPKDS
    FORMAT=KDSR      COMM LVL=3  SYSPLEX=Y  MKVPs=RSA ECC
  TKDS  PLEX75.SHARED.SCSFTKDS
    FORMAT=VARIABLE  COMM LVL=3  SYSPLEX=Y  MKVPs=None
```

The current CKDS name is still the same (PLEX75.SHARED.SCSFCKDS), but the actual CKDS was switched over during the refresh, as confirmed by a LISTCAT or ISPF display that shows four tracks are allocated for the data part (see Example 9-9 on page 199).

Example 9-9 Results of refresh

```
PLEX75.SHARED.SCSFCKDS
PLEX75.SHARED.SCSFCKDS.DATA                                 4    ?     1
PLEX75.SHARED.SCSFCKDS.INDEX                                1    ?     1
```

9.1.4 UKO backup and restore

UKO stores a copy of every managed key in the UKO repository database. If a key is removed from a CKDS by accident, UKO can retrieve the relevant key from the UKO repository and restore it to the CKDS.

Effectively, the UKO repository contains a copy of all keys managed by UKO. A backup of UKO includes a backup of the UKO repository, which should be backed up as part of the regular database backup procedures.

Keys in the UKO repository are stored encrypted in a form that is independent from any ICSF master key. UKO for z/OS implements a key hierarchy (described in 10.2.2, "Key hierarchy" on page 230), which means that data set encryption keys are stored, in the repository, encrypted under a dedicated key encrypting key (KEK). The KEKs are also stored in the UKO repository. The KEKs are encrypted by the UKO for *z/OS Disaster Recovery Key* (DRK). To be able to retrieve any key from the UKO repository, the DRK must be in the CKDS on the z/OS system where UKO for z/OS is running.

The DRK is critical to being able to access the keys in the UKO repository and any backup of the UKO repository is useless without it.

The DRK should be backed up to ensure that it can always be restored. It is recommended that the DRK is created from key parts. The use of TKE is recommended for this purpose because TKE can store key parts securely on smart cards. For more information about creating the DRK, see: UKO key hierarchy setup.

Regardless of how the DRK is created, the key parts must be stored securely, such that no single individual knows or has access to both key parts.

It is recommended to also have a backup of the CKDS. If a full CKDS with thousands of keys must be restored, it might be more efficient to restore the backed up CKDS, rather than using UKO.

If only a subset of keys in the CKDS must be restored, UKO can identify the specific keys that are missing and can be used to restore the keys. This is likely more efficient than attempting to extract individual key records out of a CKDS backup.

9.2 Transporting data set encryption keys

Data set encryption keys might need to be transported to systems that have a different CKDS that might not include the same Master Key. Regardless of the environment, transport the keys such that the keys remain protected while they are transported.

When the sending and receiving systems share the Master Key, the AES data-encrypting key that is transported remains protected by the Master Key during transfer.

When the sending and receiving systems have different Master Keys, the AES data-encrypting key must be protected by a key-encrypting key that is called a transporter key.

Note: The following scenarios apply when UKO is not used for key management. With UKO, keys are created outside of any CKDS and are subsequently distributed to the relevant CKDS keystores across multiple systems. Keys are stored in the UKO repository from where they can be restored to the CKDS keystores on demand. Manual transfer of keys between systems is therefore not relevant with UKO. See 10.3, “Key management with UKO for z/OS” on page 231 for more details.

9.2.1 Overview of scenarios

Three potential scenarios were investigated and implemented. For each scenario, the procedures and steps that were used to perform the transfer are described in this section.

The scenarios involve two sites: Site A and Site B. They focus on common situations that must be handled in the exchange and transport of encrypted data sets and keys. The following scenarios were used:

- Scenario 1: Site A and Site B feature the same Master Key. The data set key (KeyA) that Site A wants to send to Site B is not used or defined in Site B. The data set that is transferred in this scenario is Data setA (at Site A) with encrypted key KeyA.
- Scenario 2: Site A and Site B feature different Master Keys. Site A wants to send its key KeyA and Data setA to Site B. The key is not used or defined in Site B.
- Scenario 3: Site A and Site B feature the same key label that is defined for different key material.

These scenarios are described next.

9.2.2 Scenario 1: Same Master Key

In this scenario, Site A and Site B feature the same Master Key (Mkab). The data set key (KeyA) that Site A wants to send to Site B is not used or defined in Site B. The data set that is transferred in this scenario is Data setA (at Site A on system SC60) with encrypted key KeyA.

Our IT environment to run this scenario consists of the following systems, as shown in Figure 9-1 on page 201:

Site A features system SC60.

Site B features system environment PLEX75 (with SC74 and SC75).

Consider the following points regarding Figure 9-1 on page 201:

- Arrow 1 shows sending Data setA from SC60 (Site A) to the PLEX75 (at Site B).
- Arrow 2 shows exporting KeyA (from Site A) to the PLEX75 (at Site B).

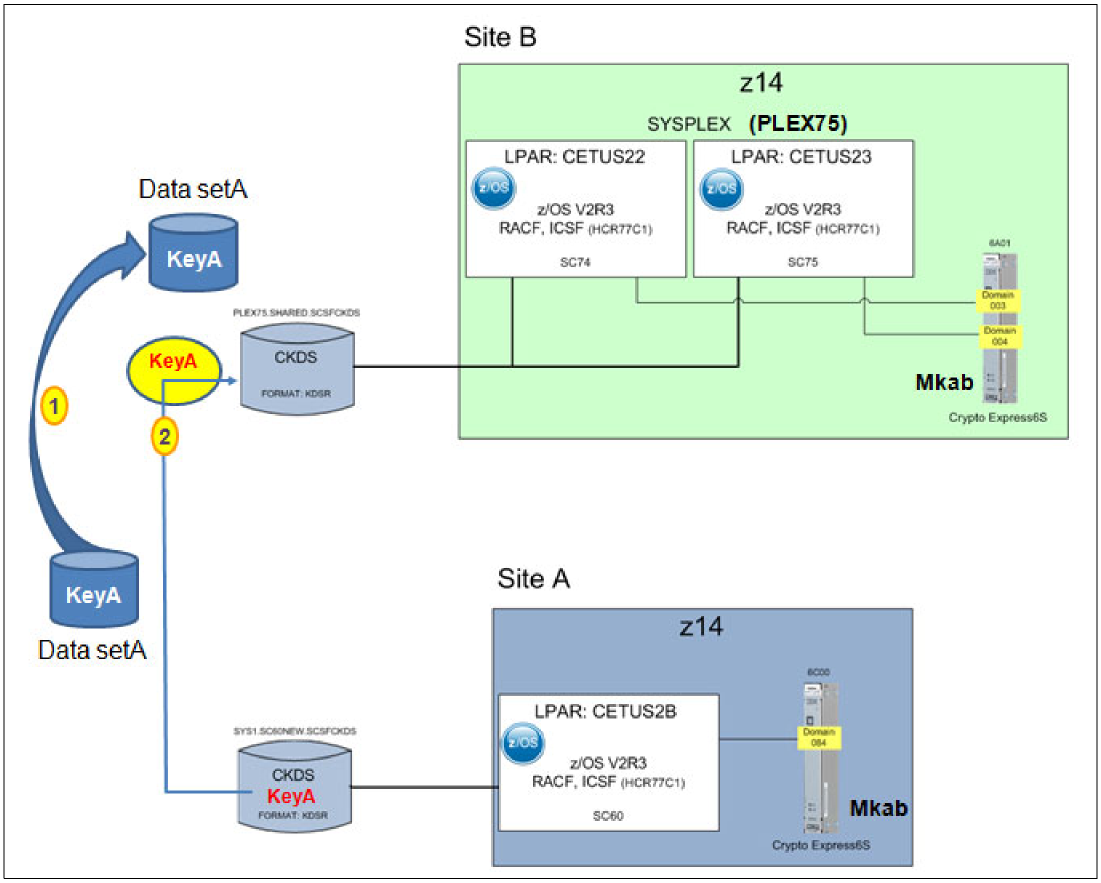


Figure 9-1 Scenario 1: Same Master Key, Data setA is encrypted with KeyA

Using the KEYXFER key transfer tool

The KEYXFER tool can be used to copy a secure key from one CKDS to another CKDS that features the same Master Key. KEYXFER is a REXX exec that runs on MVS. This tool is available for download from the ftp server `ftp://ftp.software.ibm.com/s390/zos/tools/keyxfer/`.

The KEYXFER tool assumes that the following conditions exist:

- ICSF is running on the systems that are involved in the key transfer.
- ICSF includes an active Key Data Set (CKDS or PKDS).

KEYXFER starts the following ICSF callable services to perform key transfer:

- CSNBKRC: Key record create
- CSNBKRR: Key record read
- CSNBKRW: Key record write

CSFSERV authorization is required for the CSFKRC, CSFKRR, and CSFKRW resources.

To transport a secure AES data-encrypting key, the tool reads the key token from the active CKDS and writes it to a data set. The data set can then be transmitted to any number of systems. On each system, the tool can be used again to read the key token from the transmitted file and store it into the active CKDS. The key tokens are referenced by key label.

The receiving system must include the same ICSF Master Key as the sending system. A secure key in the CKDS is protected by the Master Key and cannot leave the CKDS in clear form. When it is transferred to another CKDS, it remains encrypted by the Master Key on the sending system. For the target CKDS to accept the key, the target system must include the same Master Key.

Complete the following steps to transfer a key by using the KEYXFER tool:

> **Important:** Create a backup of each CKDS before transporting a key by using this method. For more information, see 9.1, "Backing up and restoring data set encryption keys" on page 194.

1. On the sending and receiving systems, copy the KEYXFER REXX utility into a PDS in the SYSPROC/SYSEXEC concatenation.
2. On the sending system, allocate a data set to contain the encryption key for transfer (see Example 9-10).

Example 9-10 Data set PE06.ICSF.CSFKEYS

```
Data Set Name . . . . : PE06.ICSF.CSFKEYS

General Data                           Current Allocation
 Management class . . : **None**        Allocated cylinders : 5
 Storage class  . . . : **None**        Allocated extents . : 1
  Volume serial . . . : CONTS4
  Device type . . . . : 3390
 Data class . . . . . : **None**
  Organization  . . . : PS             Current Utilization
  Record format . . . : FB              Used cylinders  . . : 1
  Record length . . . : 512             Used extents  . . . : 1
  Block size  . . . . : 5120
  1st extent cylinders: 5
  Secondary cylinders : 5              Dates
```

3. On the sending and receiving systems, allocate a PDS (or choose an existing PDS) to contain your code libraries for the KEYXFER operations.
4. On the sending system, add a member ($KEYXFWR) to the PDS from Step 3 to perform the WRITE_CKDS operation that reads a key token from the CKDS and writes it to the data set that you allocated in Step 2 (see Example 9-11).

Example 9-11 Member $KEYXFWR

```
   "BROWSE    PE06.JCL($KEYXFWR) - 01.02               Line 0000000000 Col
   " Command ===>                                                 Scroll =
   "******************************** Top of Data ************************
   "KEYXFER WRITE_CKDS, DATASET.PE06.ICSF.ENCRYPT.ME.ENCRKEY.00000005   ,+
   "PE06.ICSF.CSFKEYS
```

   This member (see Example 9-11) indicates that you want to retrieve the key that is associated with key label `DATASET.PE06.ICSF.ENCRYPT.ME.ENCRKEY.00000005`.

5. On the sending system, run the $KEYXFWR member to write the key. You see a message in the TSO panel (see Example 9-12 on page 203).

Example 9-12 Messages in TSO panel

```
> 11/13/17  9:58am
 > DATASET.PE06.ICSF.ENCRYPT.ME.ENCRKEY.00000005   written to  'PE06.ICSF.CSFKE
YS'
***
```

6. On the sending system, browse the data set to verify that the key was added (see Example 9-13).

Example 9-13 Contents of PE06.ICSF.CSFKEYS

```
BROWSE    PE06.ICSF.CSFKEYS                        Line 000000000
Command ===>                                                  Scr
******************************** Top of Data ********************
 > 11/13/17  10:00am
DATASET.PE06.ICSF.ENCRYPT.ME.ENCRKEY.00000005
010000000400C01E49232659E5B39664D67CF764BE9E97A6E084588633B11F1C
CA9B482C96F07867AE13357F627E23D40000000000000000100002050459D8D
```

7. From the sending system, transmit the data set that contains the key to the receiving system.
8. On the receiving system, add a member ($KEYXFRD) to the PDS from Step 3 to perform the READ_CKDS operation that reads the key token from the data set you transmitted in Step 7 and writes it to the CKDS (see Example 9-14).

Example 9-14 Edit PE06.JCL($KEYXFRD)

```
EDIT       PE06.JCL($KEYXFRD) - 01.01                      Columns 00001 0007
Command ===>                                                  Scroll ===> DAT
****** ***************************** Top of Data ***************************
000001 KEYXFER READ_CKDS, DATASET.PE06.ICSF.ENCRYPT.ME.ENCRKEY.00000005   ,+
000002 PE06.ICSF.CSFKEYS
```

This action reads the key and key label from the transmitted data set and starts ICSF to create the corresponding entry in the active CKDS. Start the member by entering `EX` before the member name.

9. On the receiving system, run the $KEYXFRD member to read the key. You see a message in the TSO panel (see Example 9-15).

Example 9-15 Messages from read action

```
> 11/13/17  10:09am
 > DATASET.PE06.ICSF.ENCRYPT.ME.ENCRKEY.00000005  created
 > DATASET.PE06.ICSF.ENCRYPT.ME.ENCRKEY.00000005  overwritten
 > 'PE06.ICSF.CSFKEYS'  processed successfully.
***
```

10. If the key label exists, the tool fails the operation (see Example 9-16).

Example 9-16 KEYXFER fail message

```
> 11/13/17  10:29am
*ERROR*  DATASET.PE06.ICSF.ENCRYPT.ME.ENCRKEY.00000005  exists
          - overwrite option has not been specified
***
```

11. You must specify the overwrite option to force the existing key label to be replaced (overwritten) by using the following command:

```
KEYXFER READ_CKDS, DATASET.PE06.ICSF.ENCRYPT.ME.ENCRKEY.00000005   ,+
PE06.ICSF.CSFKEYS , OVERWRITE
```

The results are shown in Example 9-17.

Example 9-17 Results of overwrite

```
> 11/13/17  10:32am
 > DATASET.PE06.ICSF.ENCRYPT.ME.ENCRKEY.00000005  overwritten
 > 'PE06.ICSF.CSFKEYS'  processed successfully.
***
```

12. On the receiving system, you can use the CKDS KEYS panel utility to verify that the key label was created (see Example 9-18). For more information about the CKDS KEYS panel utility, see 8.6, "Verifying the CKDS format" on page 186.

Example 9-18 New key label created

```
Active CKDS: SYS1.SC60NEW.SCSFCKDS                          Keys: 14

 Action characters: A, D, K, M, P, R  See the help panel for details.
 Status characters: - Active   A Archived   I Inactive

 Select the records to be processed and press ENTER
 When the list is incomplete and you want to see more labels, press ENTER
 Press END to return to the previous menu

 A S Label     Displaying 1       to 14     of 14                   Key Type
 ---------------------------------------------------------------------------
 _ - AAAA.FIRST.KEY                                                 DATA
 _ - DATASET.ENCRYPTKEY.001                                         DATA
 _ - DATASET.PE01.TEST                                              DATA
 _ - DATASET.PE01.TESTNEWGEN                                        DATA
 _ - DATASET.PE01.TESTNEWKEY                                        DATA
 _ - DATASET.PE03.AC01                                              DATA
 _ - DATASET.PE06.ICSF.ENCRYPT.ME.ENCRKEY.00000005                  DATA
```

9.2.3 Scenario 2: Different Master Key

In this scenario, Site A and Site B feature different Master Keys (Mk75 and Mk60). Site A wants to send its data set key (KeyA) and encrypted data set (Data setA) to Site B. KeyA is not used or defined in Site B.

Our IT environment to run this scenario consists of the following systems that are shown in Figure 9-2 on page 205:

- Site A features system SC60.
- Site B features system environment PLEX75 (with SC74 and SC75).

Consider the following points regarding Figure 9-2 on page 205:

- Arrow 1 shows sending Data setA from SC60 (Site A) to the PLEX75 (at Site B).
- Arrow 2 shows exporting KeyA to the PLEX75 (at Site B).

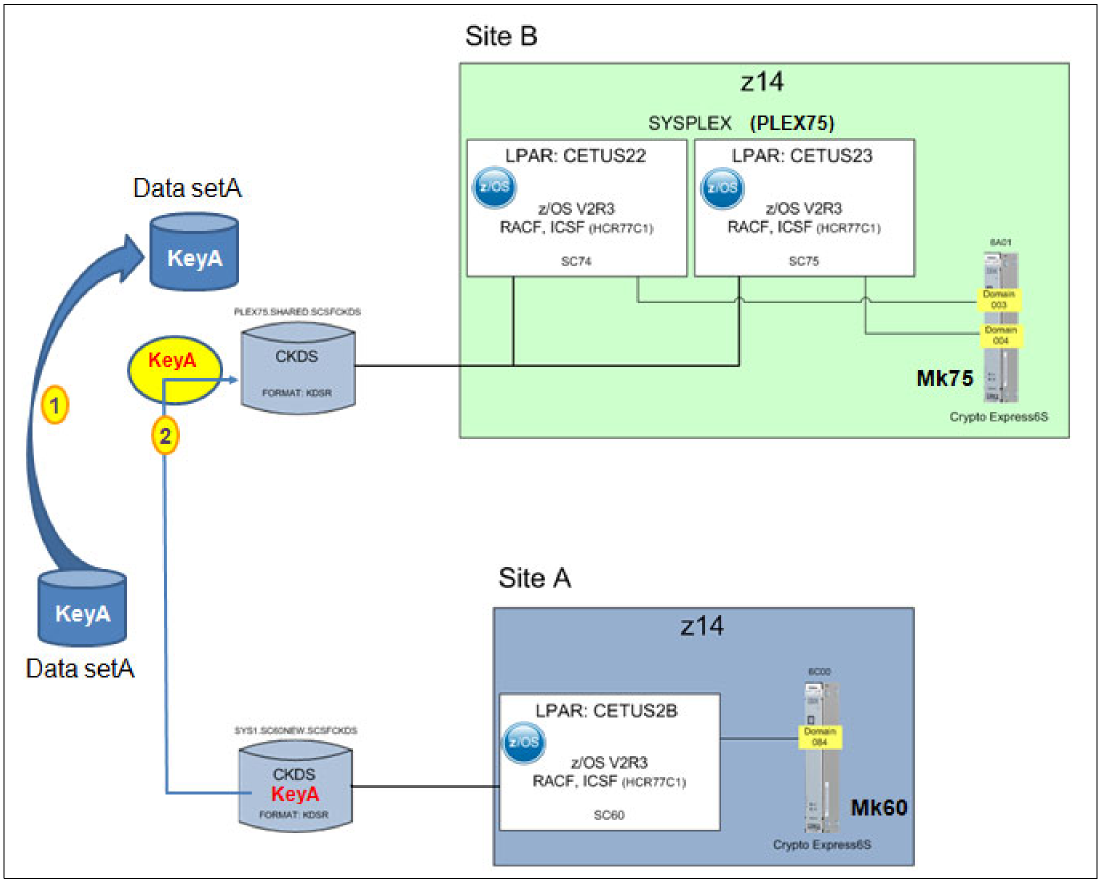


Figure 9-2 Scenario 2: Different Master Keys, Data setA is encrypted with KeyA

Using Transporter Keys

Secure keys that are generated and wrapped with a master key of one domain cannot be used directly in another domain that uses a different master key. To access encrypted data on a system that includes a different Master Key to the system on which the encrypted data sets are created, the secure AES data-encrypting key must be transported from the sending system to the receiving system.

To protect the AES data-encrypting key material from being disclosed, a transporter key can be used to encrypt the AES data-encrypting key. Transporter keys that are used to encrypt a data-encrypting key must have equal or greater key strength than the key that is protected.

Figure 9-1 on page 205 is the National Institute of Standards and Technology (NIST) table of comparable key strengths.

Table 9-1 NIST SP800-57 Table of Comparable Key Strengths

Security Strength	**Symmetric key algorithms**	**FFC (for example, DSA, D-H)**	**IFC (for example, RSA)**	**ECC (for example, ECDSA)**
≤ 80	2TDEA	L = 1024 N = 160	k = 1024	f = 160-223
112	3TDEA	L = 2048 N = 224	k = 2048	f = 224-255

Security Strength	Symmetric key algorithms	FFC (for example, DSA, D-H)	IFC (for example, RSA)	ECC (for example, ECDSA)
128	AES-128	L = 3072 N = 256	k = 3072	f = 256-383
192	AES-192	L = 7680 N = 384	k = 7680	f = 384-511
256	AES-256	L = 15360 N = 512	k = 15360	f = 512+

DFSMS uses 256-bit AES keys for z/OS data set encryption. Therefore, comparable strength transporter keys must be at least 256-bit AES, 15360-bit DSA or RSA, or 512-bit ECC. Because IBM Z does not support 15360-bit RSA keys, 256-bit AES or 512-bit ECC keys can be used for the secure transport of a 256-bit AES key.

REXX samples

REXX scripts can start z/OS ICSF callable services to transport AES data-encrypting keys between LPARs at your site or between your site and target sites that include different Master Keys.

For more information about REXX samples that are used to perform AES data-encrypting key transportation between LPARs at your site or between your site and target sites that include different Master Keys, see IBM Crypto Education.

The following REXX samples must be downloaded with their descriptions:

- GENECC2.rexx: Generate ECC private or public key pair.
- IMPRTEC2.rexx: Store partner's ECC public key in PKDS.
- DRVAESXP.rexx: Derive AES EXPORTER key on sending system from private key and receiving system's public key.
- DRVAESMP.rexx: Derive AES IMPORTER key on receiving system from private key and sending system's public key.
- EXPAES32.rexx: Translate the AES data-encrypting key to an AES Cipher key then, export the AES Cipher key under the AES EXPORTER key.
- IMPAES32.rexx: Import the AES Cipher key by using the AES IMPORTER key and translate the AES Cipher key back to an AES data-encrypting key.

Required RACF CSFSERV profiles for REXX samples

The RACF CSFSERV resource profiles that are required (with READ access) for successful execution of the REXX samples are listed in Figure 9-2 on page 206.

Table 9-2 RACF CSFSERV resource profiles

REXX Exec	Resource name	Callable service description
GENECC2	CSFPKG CSFPKRC CSFPKRD CSFPKX	PKDA Key Generate PKDS Record Create PKDS Record Delete PKA Public Key Extract
IMPRTEC2	CSFPKRC CSFPKRD CSFPKRR	PKDS Record Create PKDS Record Delete PKDS Record Read

REXX Exec	Resource name	Callable service description
DRVAESXP	CSFEDH CSFKRC2 CSFKRD CSFKRR2	ECC Diffie-Hellman Key Record Create2 Key Record Delete Key Record Read2
DRVAESMP	CSFEDH CSFKRC2 CSFKRD CSFRR2	ECC Diffie-Hellman Key Record Create2 Key Record Delete Key Record Read2
EXPAES32	CSFKRR2 CSFKTR2 CSFKYT2 CSFSYX	Key Record Read2 Key Translate2 Key Test 2 Symmetric Key Export
IMPAES32	CSFKRC2 CSFKRD CSFKTR2 CSFKYT2 CSFSY12	Key Record Create2 Key Record Delete Key Translate2 Key Test 2 Symmetric Key Import2

The process that is used to transfer data sets to a separate site that have different master keys includes the steps that are described next.

Step 1: Generating ECC key pairs (GENECC2)

To securely transport the AES data-encrypting key, each sending and receiving system must have their own ECC key pair that can be used to derive the AES IMPORTER and EXPORTER keys.

Note: The GENECC2 sample assumes that an active PKDS is available on both systems to store the public and private key pairs.

Complete the following steps to generate the ECC private and public key pairs:

Note: These steps are on the *sending* system.

1. Update `ecc_key_label` in GENECC2 with a label that represents the sending system's key pair.

   For example, update `ecc_key_label` to `SC60.ECC.KEYPAIR`.

2. Run GENECC2 on the sending system.

   For example, on Site A - SC60, we ran GENECC2, as shown in the following example:

   ```
   EX 'PE08.EXEC(GENECC2)'
   ```

   The output from GENECC2 on Site A - SC60 is shown in Example 9-19.

Example 9-19 Output from GENECC2 (on SC60)

```
ecc public key length (hex) 0000009Bx
ecc public key
1E00009B0000000021000093000000000000002090085040016087F6C5E91EC16
6FAAEC904652CEC3A58AE51ABC825FFCB4745D392F0899D9A874957487D4AF4A
087D69E13520AF20A7E50C669D2A617A450DD8FB86300C7F280147D0F57ED8E7
FE2DC33B93ED9BC2C7FF1154383D1518FDD2BFA37EB680339CD1900CF5519CF8
```

```
B297088D0516400676F9B14092DFFFC383DF79625790D534904867
-----------------------------------------------------------------
End of Sample
-----------------------------------------------------------------
```

Note: The remaining steps are run on the *receiving* system.

3. Update `ecc_key_label` in GENECC2 with a label that represents the receiving system's key pair.

   For example, we updated `ecc_key_label` to `PLEX75.ECC.KEYPAIR`.

4. Run GENECC2 on the receiving system.

   For example, on Site B - PLEX75, run the GENECC2, as shown in the following example:

   `EX 'PE08.EXEC(GENECC2)`

   The output from GENECC2 on Site B - PLEX75 is shown in Example 9-20.

Example 9-20 Output from GENECC2 (on PLEX75)

```
ecc public key length (hex) 0000009Bx
ecc public key
1E00009B000000002100009300000000000000209008504002DB3764961C22132
7A540F282FE6CF0473BE51F835216426C6133D67C3307BF6704B764B120D9BB5
6648B6D1DB8BA4D81130C26C8F8DAE8709EEC200FFCC21538C014C821DADA329
D2C6A7879D89245119E1FC2E2BF5F6DCFD3F9BAE940F018830C191AFDD8C7302
97552C6BDBC6C3BE2628B455876D847B3A8DFF5F36A76B57BE5F7A
-----------------------------------------------------------------
End of Sample
-----------------------------------------------------------------
```

Step 2: Importing partner's public key (IMPRTEC2)

Now that each participant features its own ECC private and public key pair, the public keys must be exchanged among each system. Complete the following steps to import a partner's public key (IMPRTEC2):

Note: The following steps are on the *sending* system.

1. Update `ecc_pubkey_label` in IMPRTEC2 with a label representing the receiving system's public key.

   For example, we updated `ecc_pubkey_label` to `PLEX75.ECC.PUBLIC.KEY`

2. Update `ecc_pubkey` and `ecc_pubkey_length` in IMPRTEC2 of the sending system with the output from running GENECC2 on the receiving system.

   For example, we updated `ecc_pubkey` in IMPRTEC2 on Site A - SC60 with the output from running GENECC2 on Site B - PLEX75 (see Example 9-20).

Example 9-21 ECC_PUBKEY on SC60

```
ecc_pubkey = ,
'1E00009B000000002100009300000000000000209008504002DB3764961C22132'x||,
'7A540F282FE6CF0473BE51F835216426C6133D67C3307BF6704B764B120D9BB5'x||,
'6648B6D1DB8BA4D81130C26C8F8DAE8709EEC200FFCC21538C014C821DADA329'x||,
'D2C6A7879D89245119E1FC2E2BF5F6DCFD3F9BAE940F018830C191AFDD8C7302'x||,
```

```
'97552C6BDBC6C3BE2628B455876D847B3A8DFF5F36A76B57BE5F7A'x
```

3. Run IMPRTEC2.rexx on the sending system.

   For example, on Site A - SC60, we ran IMPRTEC2, as shown in the following example:

   `EX 'PE08.EXEC(IMPRTEC2)'`

   The resulting output on Site A - SC60 is shown in Example 9-22.

Example 9-22 Results of IMPRTEC2 on SC60

```
ecc key label  PLEX75.ECC.PUBLIC.KEY
ecc public key
1E00009B000000002100009300000000000000209008504002DB3764961C22132
7A540F282FE6CF0473BE51F835216426C6133D67C3307BF6704B764B120D9BB5
6648B6D1DB8BA4D81130C26C8F8DAE8709EEC200FFCC21538C014C821DADA329
D2C6A7879D89245119E1FC2E2BF5F6DCFD3F9BAE940F018830C191AFDD8C7302
97552C6BDBC6C3BE2628B455876D847B3A8DFF5F36A76B57BE5F7A
ecc public key length 155
ecc public key length (hex) 0000009Bx
-----------------------------------------------------------------
End of Sample
-----------------------------------------------------------------
```

Note: The remaining steps are run on the *receiving* system.

4. Update `ecc_pubkey` and `ecc_pubkey_length` in IMPRTEC2 of the receiving system with the output from running GENECC2 on the sending system.

   For example, we updated `ecc_pubkey` in IMPRTEC2 on Site B - PLEX75 with the output from running GENECC2 on Site A - SC60 (see Example 9-23).

Example 9-23 ECC_PUBKEY on PLEX75

```
ecc_pubkey = ,
'1E00009B00000000210000930000000000000020900850400160 87F6C5E91EC16'x||,
'6FAAEC904652CEC3A58AE51ABC825FFCB4745D392F0899D9A874957487D4AF4A'x||,
'087D69E13520AF20A7E50C669D2A617A450DD8FB86300C7F280147D0F57ED8E7'x||,
'FE2DC33B93ED9BC2C7FF1154383D1518FDD2BFA37EB680339CD1900CF5519CF8'x||,
'B297088D0516400676F9B14092DFFFC383DF79625790D534904867'x
```

5. Run IMPRTEC2 on the receiving system.

   For example, on Site B - PLEX75, we ran IMPRTEC2, as shown in the following example:

   `EX 'PE08.EXEC(IMPRTEC2)'`

   The resulting output on Site B - PLEX75 is shown in Example 9-24.

Example 9-24 Results of IMPRTEC2 on PLEX75

```
ecc key label  SC60.ECC.PUBLIC.KEY
ecc public key
1E00009B000000002100009300000000000000209008504001608 7F6C5E91EC16
6FAAEC904652CEC3A58AE51ABC825FFCB4745D392F0899D9A874957487D4AF4A
087D69E13520AF20A7E50C669D2A617A450DD8FB86300C7F280147D0F57ED8E7
FE2DC33B93ED9BC2C7FF1154383D1518FDD2BFA37EB680339CD1900CF5519CF8
B297088D0516400676F9B14092DFFFC383DF79625790D534904867
ecc public key length 155
ecc public key length (hex) 0000009Bx
```

```
----------------------------------------------------------------
End of Sample
----------------------------------------------------------------
```

The ECC public keys are now stored in each system's PKDS.

Step 3: Deriving an AES exporter key (DRVAESXP)

Now that each participant features its own ECC private and public key pair and the public keys of the other systems, they can independently derive a common AES transporter key. Complete the following steps to derive an AES exporter key (DRVAESXP):

> **Note:** The following steps are run on the *sending* system.

1. Update `sender_key_label` in DRVAESXP with the sending system's key pair label.

   For example, we updated `sender_key_label` to `SC60.ECC.KEYPAIR`.
2. Update `receiver_key_label` in DRVAESXP with the receiving system's public key label.

   For example, we updated `receiver_key_label` to `PLEX75.ECC.PUBLIC.KEY`.
3. Update `exporter_key_label` in DRVAESXP with a label that represents the AES exporter key.

   For example, we updated `exporter_key_label` to `SC60.AES.EXPORTER.KEY`.
4. Run DRVAESXP on the sending system.

   For example, on Site A - SC60, we ran DRVAESXP, as shown in the following example:

   `EX 'PE08.EXEC(DRVAESXP)'`

   The resulting output on Site A - SC60 is shown in Example 9-25.

Example 9-25 Output from DRVAESXP

```
derived AES EXPORTER key label: SC60.AES.EXPORTER.KEY

----------------------------------------------------------------
End of Sample
----------------------------------------------------------------
```

The EXPORTER key, SC60.AES.EXPORTER.KEY, is stored in the CKDS.

Step 4: Deriving an AES importer key (DRVAESMP)

Because each participant features its own ECC private and public key pair and the public keys of the other systems, they can independently derive a common AES transporter key. Complete the following steps to derive an AES importer key (DRVAESMP):

> **Note:** The following steps are run on the *receiving* system.

1. Update `sender_key_label` in DRVAESMP with the sending system's public keylabel.

   For example, we updated `sender_key_label` to `SC60.ECC.PUBLIC.KEY`.
2. Update `receiver_key_label` in DRVAESMP with the receiving system's key pair label.

   For example, we updated `receiver_key_label` to `PLEX75.ECC.KEYPAIR`.

3. Update `importer_key_label` in DRVAESMP with a label that represents the AES importer key.

   For example, we updated `importer_key_label` to `PLEX75.AES.IMPORTER.KEY`.

4. Run DRVAESMP on the receiving system.

   For example, on Site B - PLEX75, we ran DRVAESMP, as shown in the following example:

   `EX 'PE08.EXEC(DRVAESMP)'`

   The resulting output on Site B - PLEX75 is shown in Example 9-26.

Example 9-26 Output from DRVAESMP

```
derived AES IMPORTER key label: PLEX75.AES.IMPORTER.KEY

----------------------------------------------------------------
End of Sample
----------------------------------------------------------------
```

The IMPORTER key, PLEX75.AES.IMPORTER.KEY, is stored in the CKDS.

Step 5: Exporting the AES data-encrypting key (EXPAES32)

Now, each participant features a common transporter key that is derived from a combination of their private key and their partner's public key. This transporter key can be used repeatedly to export and import keys between the two parties. Complete the following steps to export the AES data-encrypting key (EXPAES32):

> **Note:** The following steps are run on the *sending* system.

1. Update `exporter_key_label` in EXPAES32 with the exporter key label.

   For example, we updated `exporter_key_label` to `SC60.AES.EXPORTER.KEY`.

2. Update `aes_data_key_label` in EXPAES32 with the AES data-encrypting key label to be generated and exported.

   > **Note:** The REXX sample assumes that the AES data-encrypting key is being generated on each invocation. If you plan to use an existing AES data-encrypting key, you *must* comment out the calls to CSNBKRD, CSNBKGN, and CSNBKRC2 in the REXX sample that delete (and generate) the AES data-encrypting key.

   For example, we updated `aes_data_key_label` to `DATASET.ENCRYPTKEY.001`.

3. Run EXPAES32 on the sending system.

   For example, on Site A - SC60, we ran EXPAES32, as shown in the following example:

   `EX 'PE08.EXEC(EXPAES32)'`

   The resulting output on Site A - SC60 is shown in Example 9-27.

Example 9-27 Output from EXPAES32

```
verification pattern: 6E27120A7165770B
exported key:
020000880500000002021303 3A016F0896190000000000000000020201000100
```

```
F46F0BF786F6A139F55EED7760EB883EDDE4FA608DC5C5510FCA36381B6ADA21
A0849C886F9DD316
exported key LENGTH:  136
exported key length (hex):  00000088x
-----------------------------------------------------------------
End of Sample
-----------------------------------------------------------------
```

Step 6: Importing the AES data-encrypting key (IMPAES32)

Now, the AES data-encrypting key is protected by a transporter key and can be transmitted securely to the receiving system. The receiving system can import the key into its keystore and rewrap the key with its Master Key. Complete the following steps to import the AES data-encrypting key (IMPAES32):

Note: The following steps are on the *receiving* system.

1. Update `importer_key_label` in IMPAES32 with the importer key label.

   For example, we updated `importer_key_label` to `PLEX75.AES.IMPORTER.KEY`.

2. Update `verification_pattern` in IMPAES32 with the verification pattern output from EXPAES32.

   For example, we updated `verification_pattern` to `6E27120A7165770B`.

3. Update `encrypted_key` and `encrypted_key_length` in IMPAES32 with the encrypted key material from EXPAES32.

   For example, we updated encrypted key as shown in the following example:

   ```
   02000088050000000020213033A016F0896190000000000000000000020201000100
   001A00000000002800002000102C000000003E0000000001409611F5998BBE716
   3D625EE8C861E5611B73FE05F2250428B1E6304365DD8B2C5F588FF2AB0F231C
   F46F0BF786F6A139F55EED7760EB883EDDE4FA608DC5C5510FCA36381B6ADA21
   A0849C886F9DD316
   ```

4. Run IMPAES32 on the receiving system.

   For example, on Site B - PLEX75, we ran IMPAES32, as shown in the following example:

   ```
   EX 'PE08.EXEC(IMPAES32)'
   ```

   The resulting output on Site B - PLEX75 is shown in Example 9-28.

Example 9-28 Results of running IMPAES32

```
Verification of the AES DATA KEY key succeeded
-----------------------------------------------------------------
End of Sample
-----------------------------------------------------------------
```

Verifying the AES data-encrypting key in the CKDS

You can verify the importer key and data key in the ICSF CKDS panel utility. Select **ICSF Option 5.5**, then, **option 1** on the receiving system (PLEX75) to see the CKDS KEYS List (see Example 9-29).

Example 9-29 CKDS KEYS List for PLEX75

```
--------------------------- ICSF - CKDS KEYS List ----------- Row 1 to 5 of 5
COMMAND ===>                                                SCROLL ===> PAGE
```

```
Active CKDS: PLEX75.SHARED.SCSFCKDS                         Keys: 5

Action characters: A, D, K, M, P, R  See the help panel for details.
Status characters: - Active   A Archived   I Inactive

Select the records to be processed and press ENTER
When the list is incomplete and you want to see more labels, press ENTER
Press END to return to the previous menu

A S Label     Displaying 1       to 5     of 5                  Key Type
-------------------------------------------------------------------------------
_ - DATASET.ENCRYPTKEY.001                                      DATA
_ - ICSF.SECRET.AES256.KEY001                                   DATA
_ - PLEX75.AES.IMPORTER.KEY                                     IMPORTER
_ - SAMPLE.DERIVED.AES.IMPORTER.KEY                             IMPORTER
_ - SAMPLE.RECEIVED.AES.DATA.KEY                                DATA
******************************* Bottom of data ********************************
```

The AES data-encrypting key, DATASET.ENCRYPTKEY.001, and the IMPORTER key, PLEX75.AES.IMPORTER.KEY, that were created with DRVAESMP are shown in Example 9-29 on page 212.

9.2.4 Scenario 3: Duplicate Key Label

In this scenario, Site A and Site B feature the same Master Key or different Master Keys. The key label that is used to encrypt DatasetA is in Site B with a different key value.

Our IT environment that was used to run this scenario consists of the systems that are shown in Figure 9-3 on page 214:

- Site A features system SC60.
- Site B features system environment PLEX75 (with SC74 and SC75).

Consider the following points regarding Figure 9-3 on page 214:

- Arrow 1 shows sending Data setA from SC60 (Site A) to the PLEX75 (at Site B).
- Arrow 2 shows exporting KeyB (from Site A) to the PLEX75 (at Site B).

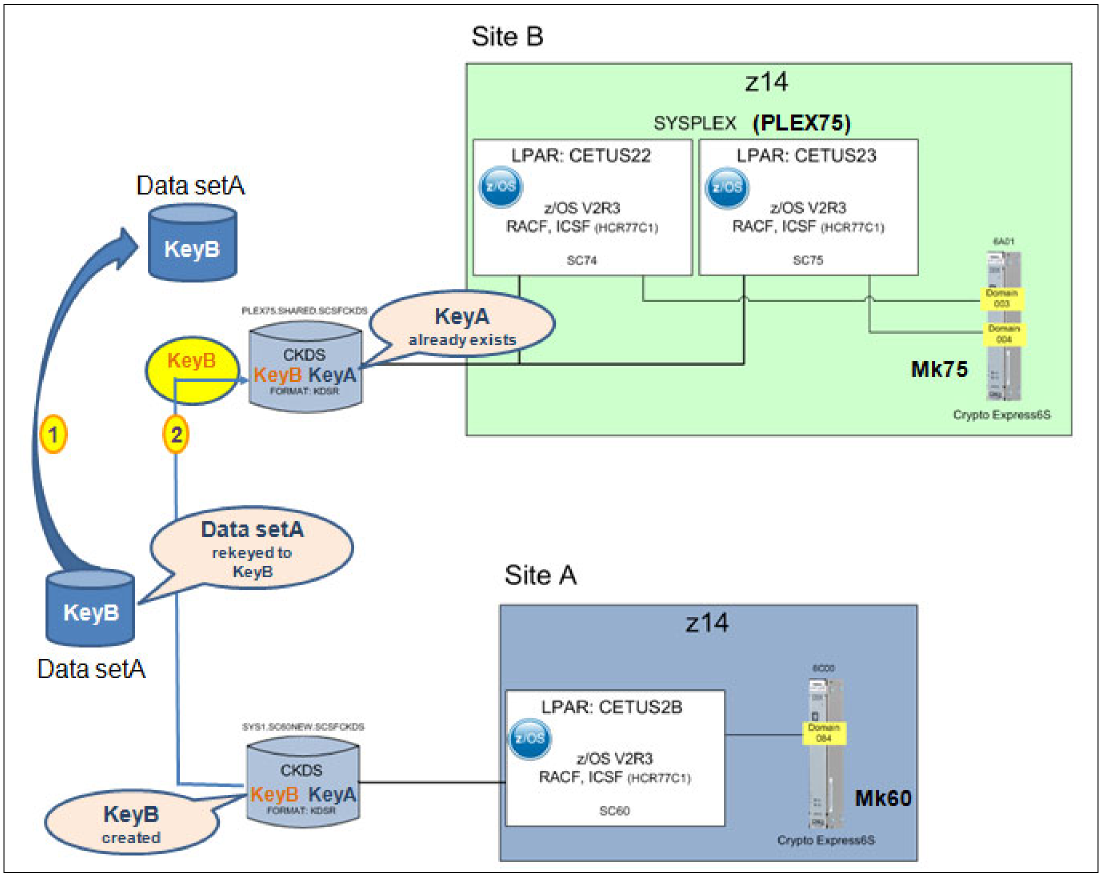


Figure 9-3 Scenario 3: Changing the key label

Option 1: Overwrite the existing key

You can decide to overwrite the existing key in Site B. However, if you overwrite the existing key in Site B, other encrypted data sets on Site B might be affected.

Option 2: Change the key label name

If you import the key on Site B with a different key label than what is specified in the data set catalog, the original data set becomes inaccessible. DFSMS requires that the key label in the data set catalog is in the CKDS with the appropriate key value.

An imported key for which the key label was changed on Site B can be used only to encrypt new data sets on Site B. That is, it can never be used to work with data sets that were created on Site A and then transferred to Site B. This fact is true for exchanging keys between sites that include the same Master Key or different Master Keys.

Option 3: Rekey the data set

The best (but most complex) option is to rekey DatasetA with a new key label on Site A that is unused on Site B. For more information about rekeying data sets, see 7.2.2, “Rotating data set encryption keys” on page 166.

The following process is used:

- A new key that is stored under a new key label is generated.
- Data setA at Site A is rekeyed from key label KEYA to KEYB.
- The rekeyed data set (which is now using key label KEYB) is sent from Site A to Site B.

- Key label KEYB is transported from Site A to Site B.

9.3 Viewing the last reference date

ICSF can track when encryption keys were last referenced in a cryptographic operation that was performed by ICSF callable services or read from the CKDS for use in a cryptographic operation elsewhere. ICSF stores the last reference date and the associated callable service name in the key record metadata at user-defined intervals.

> **Note:** The ability to track last reference dates requires a Common Record Format Key Data Set. For more information, see 4.3.2, "Creating a Common Record Format (KDSRL) CKDS" on page 81.

Last reference date tracking must be enabled in the ICSF Installation Options (CSFPRMxx). For more information about enablement, see 4.3.3, "CSFPRMxx and installation options" on page 83.

After tracking is enabled, last reference dates can be viewed in one of the following ways:

- By using the ICSF CKDS keys panel utility
- By using the CSFKDMR callable service

9.3.1 Using the CKDS Keys panel utility

The CKDS Keys panel utility is accessible from the ISPF ICSF panel option 5.5. From the panel, you can specify a full or partial record label and choose one of the options to list and manage records (see Example 9-30).

Example 9-30 ICSF CKDS Keys panel

```
------------------------------- ICSF - CKDS KEYS -----------------------------

Active CKDS: SYS1.SC60NEW.SCSFCKDS                            Keys: 18

Enter the number of the desired option.
  1  List and manage all records
  2  List and manage records with label key type             leave blank for
                                                             list, see help
  3  List and manage records that are            (ACTIVE, INACTIVE, ARCHIVED)
  4  List and manage records that contain unsupported CCA keys
  5  Display the key attributes and record metadata for a record
  6  Delete a record
  7  Generate AES DATA keys

Full or partial record label
  ==> DATASET.PE06.ICSF.ENCRYPT.ME06.ENCRKEY.00000006
  The label may contain up to seven wild cards (*)

Number of labels to display ==> 100  (Maximum 100)

Press ENTER to go to the selected option.
OPTION ===> 1
```

In the next panel, you see a list of key entries that match the record label criteria that you specified (see Example 9-31).

Example 9-31 ICSF CKDS Keys panel

```
COMMAND ===>                                                   SCROLL ===> PAGE

 Active CKDS: SYS1.SC60NEW.SCSFCKDS                             Keys: 18

 Action characters: A, D, K, M, P, R  See the help panel for details.
 Status characters: - Active   A Archived   I Inactive

 Select the records to be processed and press ENTER
 When the list is incomplete and you want to see more labels, press ENTER
 Press END to return to the previous menu

 A S Label     Displaying 1       to 18     of 18                  Key Type
 -----------------------------------------------------------------------------
_ - DATASET.PE06.ICSF.ENCRYPT.ME06.ENCRKEY.00000006                DATA
```

If you enter `M` (metadata) or `K` (key attributes and metadata) in the action column before the key entry, you see the date that the record was last used and the service name (see Example 9-32).

Example 9-32 Browse a key entry

```
------------------- ICSF - CKDS Key Attributes and Metadata ------ Top of data
 COMMAND ===>                                                   SCROLL ===> PAGE

 Active CKDS: SYS1.SC60NEW.SCSFCKDS

 Label: DATASET.PE06.ICSF.ENCRYPT.ME06.ENCRKEY.00000006                DATA

  Record status: Deactivated     (Archived, Active, Pre-active, Deactivated)

  Select an action:
    1  Modify one or more fields with the new values specified
    2  Delete the record
 -------------------------------------------------------------------------------
                                                                   More:     +
 Metadata                       YYYYMMDD              YYYYMMDD
  Record creation date:         20171114
  Update date:                  20171114
  Cryptoperiod start date:      00000000     New value:
  Cryptoperiod end date:        20171114     New value:
  Date the record was last used: 20171114     New value:
  Service called when last used: CSFKRR
```

Note: If you see a last used date but do not see the service name in the panel, you might need to wait for the KDSREFDAYS interval to end (which is 1 hour by default). To modify the interval, use the SETICSF OPT,RPSEC option. For more information, see the *ICSF System Programmer's Guide*.

9.3.2 Using the CSFKDMR callable service

You can programmatically view last reference dates by using the CSFKDMR callable service with the CSFKDSL callable service.

The CSFKDSL callable service can list all key labels that match specified criteria.

For more information about a REXX sample that shows how to start CSFKDSL, see IBM Crypto Education.

The CSFKDMR callable service can read the metadata of a record that is associated with a key label.

For more information about a REXX sample that shows how to read the last reference date, see IBM Crypto Education.

These two services can be combined to find and view a set of key records that matches specific criteria.

> **Note:** If you can read the last used date but cannot read the service name, you might need to wait for the KDSREFDAYS interval to end (which is 1 hour by default). To modify the interval, use the SETICSF OPT,RPSEC option. For more information, see the *ICSF System Programmer's Guide*.

9.4 Archiving data set encryption keys

Encryption keys that are stored as key records in key data sets can be archived. The key remains in the data set, but it might not be used in cryptographic operations. When a key is archived, an archive date is set in the key record.

By default, any attempts to use an archived key fail. However, if the key archive use control is enabled, ICSF allows the request to complete successfully. For more information, see 3.6.11, "Establishing a process for handling compromised operational keys" on page 56. In both cases, an SMF record is logged.

Archived keys can be recalled from the archived state. When an archived key is recalled, a recall date is set in the key record.

Key archival and key recall can be performed in one of the following ways:

- By using the ICSF CKDS Keys panel utility
- By using the CSFKDMW callable service

> **Note:** The ability to archive keys requires a Common Record Format Key Data Set. For more information, see 4.3.2, "Creating a Common Record Format (KDSRL) CKDS" on page 81.

Using the CKDS Keys panel utility

The CKDS Keys panel utility is accessible from the ISPF ICSF panel option 5.5. From the panel, you can specify a full or partial record label and choose one of the options to list and manage records, as shown in Example 9-33 on page 218.

Example 9-33 ICSF CKDS Keys panel

```
------------------------------- ICSF - CKDS KEYS -----------------------------

Active CKDS: SYS1.SC60NEW.SCSFCKDS                          Keys: 18

Enter the number of the desired option.
  1  List and manage all records
  2  List and manage records with label key type           leave blank for
                                                            list, see help
  3  List and manage records that are           (ACTIVE, INACTIVE, ARCHIVED)
  4  List and manage records that contain unsupported CCA keys
  5  Display the key attributes and record metadata for a record
  6  Delete a record
  7  Generate AES DATA keys

Full or partial record label
  ==> DATASET.PE06.ICSF.ENCRYPT.ME06.ENCRKEY.00000006
  The label may contain up to seven wild cards (*)

Number of labels to display ==> 100  (Maximum 100)

Press ENTER to go to the selected option.
OPTION ===> 1
```

In the next panel, you see a list of key entries that match the record label criteria that you specified (see Example 9-34).

Example 9-34 ICSF CKDS Keys panel showing status I

```
COMMAND ===>                                                SCROLL ===> PAGE

 Active CKDS: SYS1.SC60NEW.SCSFCKDS                          Keys: 18

 Action characters: A, D, K, M, P, R  See the help panel for details.
 Status characters: - Active   A Archived   I Inactive

 Select the records to be processed and press ENTER
 When the list is incomplete and you want to see more labels, press ENTER
 Press END to return to the previous menu

 A S Label     Displaying 1      to 18     of 18                  Key Type
 -----------------------------------------------------------------------------
_ - DATASET.PE06.ICSF.ENCRYPT.ME06.ENCRKEY.00000006                 DATA
```

You can enter `A` in the action column before the key entry to archive a key. Archived keys show an "A" in the status column to indicate that the key is archived (see Example 9-35).

Example 9-35 View key archival status

```
_ A DATASET.PE06.ICSF.ENCRYPT.ME06.ENCRKEY.00000006                 DATA
```

You can enter `R` in the action column in front of the key entry to recall a previously archived key and mark it available for use (see Example 9-36 on page 219).

Example 9-36 View key archival status

```
_ - DATASET.PE06.ICSF.ENCRYPT.ME06.ENCRKEY.00000006                 DATA
```

Using the CSFKDMW callable service

You can programmatically archive or recall keys by using the CSFKDMW callable service with the CSFKDSL callable service.

The CSFKDSL callable service can list all key labels that match specified criteria. For example, you can list all key labels that were not referenced since 2018.01.01.

For more information about a REXX sample that shows how to start CSFKDSL, see IBM Crypto Education.

The CSFKDMW callable service can write the metadata to a record that is associated with a key label.

For more information about a REXX sample that shows how to archive a key, see IBM Crypto Education.

These two services can be combined to find and update a set of key records that matches specific criteria.

9.5 Deactivating UKO-managed data set encryption keys

With UKO, keys can be deactivated. When a key is deactivated, the key is removed from the CKDS but is kept in the UKO repository. For keys that exist in multiple CKDS keystores, the key is removed from all of them.

Deactivating keys in UKO instead of archiving them in the CKDS reduces the number of keys in the CKDS.

A deactivated key can be reactivated, which restores the key to the CKDS keystores it was removed from when the key was deactivated. After it is restored, the key can be used as any other data set encryption key.

It is possible to combine archiving and deactivation by initially archiving the key and then deactivating it. Perform the deactivation when no attempts to access the key are made for a given period of time.

9.6 Setting key expiration dates

Establishing a cryptoperiod can be useful to control the time frame in which an encryption key can be used for encryption and decryption operations. For more information about cryptoperiods, see 3.6.10, “Establishing cryptoperiods” on page 56.

With z/OS ICSF, cryptoperiods can be established by setting a key validity start date and key validity end date in the key record. Key validity dates can be set in one of the following ways:

- By using the ICSF CKDS Keys panel utility
- By using the CSFKDMW callable service

Note: The ability to set cryptoperiods requires a Common Record Format Key Data Set. For more information, see 4.3.2, “Creating a Common Record Format (KDSRL) CKDS” on page 81.

Using the CKDS Keys panel utility

The CKDS Keys panel utility is accessible from the ISPF ICSF panel option 5.5. From the panel, you can specify a full or partial record label and choose one of the options to list and manage records (see Example 9-37).

Example 9-37 ICSF CKDS Keys panel

```
------------------------------- ICSF - CKDS KEYS -----------------------------

Active CKDS: SYS1.SC60NEW.SCSFCKDS                            Keys: 18

Enter the number of the desired option.
  1  List and manage all records
  2  List and manage records with label key type            leave blank for
                                                             list, see help
  3  List and manage records that are           (ACTIVE, INACTIVE, ARCHIVED)
  4  List and manage records that contain unsupported CCA keys
  5  Display the key attributes and record metadata for a record
  6  Delete a record
  7  Generate AES DATA keys

Full or partial record label
  ==> DATASET.PE06.ICSF.ENCRYPT.ME06.ENCRKEY.00000006
  The label may contain up to seven wild cards (*)

Number of labels to display ==> 100  (Maximum 100)

Press ENTER to go to the selected option.
OPTION ===> 1
```

In the next panel, you see a list of key entries that match the record label criteria you specified. When you view a key that is out of range for its cryptoperiod, the panel shows a status of “I” for inactive (see Example 9-38).

Example 9-38 ICSF CKDS Keys panel showing status I

```
COMMAND ===>                                                  SCROLL ===> PAGE

 Active CKDS: SYS1.SC60NEW.SCSFCKDS                            Keys: 18

 Action characters: A, D, K, M, P, R  See the help panel for details.
 Status characters: - Active   A Archived   I Inactive

 Select the records to be processed and press ENTER
 When the list is incomplete and you want to see more labels, press ENTER
 Press END to return to the previous menu

 A S Label     Displaying 1       to 18     of 18                    Key Type
 -----------------------------------------------------------------------------
_ I DATASET.PE06.ICSF.ENCRYPT.ME06.ENCRKEY.00000006                 DATA
```

If you enter `M` (metadata) or `K` (key attributes and metadata) in the action column before the key entry, you see the information that is shown in Example 9-39.

Example 9-39 Browse a key entry

```
------------------- ICSF - CKDS Key Attributes and Metadata ------ Top of data
 COMMAND ===>                                                 SCROLL ===> PAGE

 Active CKDS: SYS1.SC60NEW.SCSFCKDS

 Label: DATASET.PE06.ICSF.ENCRYPT.ME06.ENCRKEY.00000006                DATA

  Record status: Deactivated     (Archived, Active, Pre-active, Deactivated)

  Select an action:
    1  Modify one or more fields with the new values specified
    2  Delete the record
 -------------------------------------------------------------------------------
                                                                  More:     +
 Metadata                        YYYYMMDD                YYYYMMDD
  Record creation date:          20171114
  Update date:                   20171114
  Cryptoperiod start date:       00000000     New value:
  Cryptoperiod end date:         20171114     New value:
  Date the record was last used: 20171114     New value:
  Service called when last used: CSFKRR
```

You can select action 1 to Modify then enter a new expiration date in the future (if needed) to reactivate the key. After you refresh your ICSF browser panel, you see that the key is no longer Inactive (I), as shown in Example 9-40.

Example 9-40 Refresh ICSF browser to see the key is no longer inactive

```
_ - DATASET.PE06.ICSF.ENCRYPT.ME06.ENCRKEY.00000006                 DATA
```

You can use these actions to set or update key validity dates in the CKDS.

Using the CSFKDMW callable service

You can programmatically set key validity dates by using the CSFKDMW callable service with the CSFKDSL callable service.

The CSFKDSL callable service can list all key labels that match specified criteria.

For more information about a REXX sample that shows how to start CSFKDSL, see IBM Crypto Education.

The CSFKDMW callable service can write the metadata to a record that is associated with a key label.

For more information about a REXX sample that shows how to set a key validity date, see IBM Crypto Education.

These two services can be combined to find and update a set of key records that matches specific criteria.

10

IBM Unified Key Orchestrator

z/OS data set encryption relies on cryptographic keys stored in ICSF CKDS. Without IBM Unified Key Orchestrator (UKO), managing these keys across multiple LPARs and sysplex environments can become complex and error-prone.

IBM UKO ensures that key naming conventions, key templates, and security policies are applied uniformly across all systems. It automates key creation, distribution, rotation, and archival, reducing manual intervention and minimizing human error.

This chapter covers the basics of UKO for z/OS and the capabilities that support the key management lifecycle:

> **Note:** The tasks described in this chapter are based on UKO for z/OS version 3.1. Always check the latest product documentation for the current version of UKO for z/OS.

10.1 Introduction to IBM Unified Key Orchestrator

IBM Unified Key Orchestrator (UKO) is a flexible and highly secure key management system for the enterprise. It provides centralized key management on IBM Z and distributed platforms for streamlined, efficient and secure key and certificate management operations.

UKO serves as the foundation on which remote cryptographic solutions and analytics for the cryptographic infrastructure can be provided. The UKO family consists of UKO for z/OS, UKO for Containers, and the EKMF Workstation:

- UKO for z/OS

  Released as a product in April 2020. It is developed at the IBM Crypto Competence Center in Copenhagen, Denmark, which has also developed EKMF Workstation. Both UKO for z/OS and EKMF Workstation use the same UKO Agent that communicates with ICSF through APIs.

  UKO for z/OS runs in a z/OS LPAR and is accessible through a Web browser. The UKO repository is a Db2 database, and keys are distributed to keystores through UKO agents, which are used to manage keys in the agents' local keystores.

  Currently, UKO for z/OS supports generation and management of keys for the Mainframe, including those used for data set encryption, and for setting up keys for cloud deployments.

  Note: UKO for z/OS is an orderable software product.

- UKO for Containers

  UKO for Containers provides the same functionality as UKO for z/OS but is deployed as containers in OCP on LinuxONE or IBM Z (IFL's on IBM Z). It uses the Crypto Express capabilities provided by the LinuxONE platform to create keys and can manage keys on z/OS through the use of UKO agents on z/OS.

  Statements made in this IBM Redbooks publication about UKO for z/OS generally apply to UKO for Containers as well.

  Note: UKO for Containers is an orderable software product.

- EKMF Workstation

  This is implemented as a self-contained hardware appliance (including a dedicated workstation and Linux operating system and an embedded crypto card). It is feature rich, supporting many key types, with authentication done by using smart cards with PINs. The security controls enable EKMF Workstation to have a higher security rating and administer more complex key types. The EKMF Workstation integrates with and operates on the same database as UKO for z/OS. The EKMF Workstation is an addition to UKO for z/OS.

  Note: EKMF Workstation is a service-based offering provided as an addition to UKO for z/OS.

Note that the EKMF Workstation, UKO for z/OS, and UKO for Containers provide the following key management services:

- Generate operational keys
- Facilitate backup and recovery of key material
- Provide monitoring, auditing, and planning capabilities

For more information about IBM Unified Key Orchestrator see the products page Unified Key Orchestrator for IBM z/OS in the IBM and Unified Key Orchestrator for IBM z/OS in IBM documentation.

10.1.1 UKO for z/OS overview

UKO for z/OS provides a centralized key management solution for file and dataset encryption keystores. The GUI for UKO for z/OS is provided by Websphere Liberty application server deployment running on z/OS (see Figure 10-1).

With UKO for z/OS, users can log in using a RACF ID or an ID from LDAP/SSO. Enterprise JavaBeans (EJB) roles define the authorization to UKO for z/OS functionality. Keys generated by UKO for z/OS are stored in the UKO key repository. Keys are secured by a key hierarchy protected by the master key, which resides in the HSM, which means that keys are never in the clear outside of the Crypto Express HSM.

The UKO key repository back end is provided by Db2 database running on z/OS. Use normal resiliency, backup and recovery processes and procedures for z/OS Db2 to protect the repository.

UKO for z/OS can generate keys and assign them to specific key labels within ICSF keystores (CKDS/PKDS) via a UKO agent.

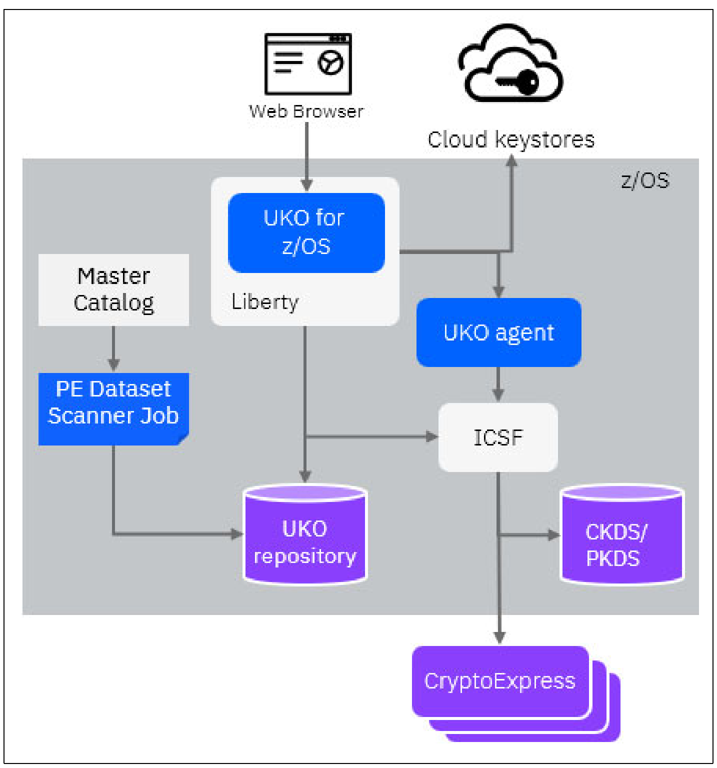


Figure 10-1 UKO for z/OS architecture

Based on policies defined in key templates, keys are distributed to the keystores in which they are needed. If a key is lost or deleted from a particular keystore, UKO for z/OS can restore the key by redistributing it. If a key needs to be archived or deleted, UKO for z/OS can remove the key from the connected keystores.

In addition to the capabilities related to data set encryption, UKO for z/OS also provides the following features:

- Key management for ICSF

  In addition to AES keys for data set encryption, UKO for z/OS can create and manage classical HMAC, RSA and ECC keys and the quantum-safe ML-KEM and ML-DSA. UKO for z/OS is continuously being enhanced with new capabilities. The general purpose capabilities of UKO for z/OS does not extend to the full capabilities of the EKMF Workstation, which has full key management support for all key types supported by ICSF.

  Refer to the current UKO for z/OS documentation for the up-to-date list of supported key types.

- Key management for Cloud

  With applications and data in the cloud, it is important to be able to protect them using the available cryptographic functions provided by the cloud providers. UKO for z/OS can create keys and distribute them to several cloud providers. With UKO for z/OS, keys that have been disabled in the cloud can be restored. Using UKO for z/OS, keys can also be deactivated and reactivated as needed.

  UKO for z/OS can create and manage keys for the following cloud providers:

  - Amazon Web Services (AWS)
  - Google Cloud Platform (GCP)
  - IBM Cloud®
  - Microsoft Azure

- Key management for double key encryption

  As a special feature of UKO cloud support, UKO for z/OS has support for creating and managing keys for Microsoft Azure Double Key Encryption (MSDKE). MSDKE is an advanced data protection feature that enables organizations to maintain exclusive control over their most sensitive data, even when stored in Microsoft cloud services such as Microsoft 365. This encryption model uses two keys: one managed by Microsoft (typically through Azure Key Vault) and the other controlled by the organization, usually hosted on-premises or in a trusted environment separate from Azure.

  Double key encryption is crucial for protecting highly sensitive data, such as intellectual property, strategic business plans, and personal or financial information. It is particularly relevant for organizations with data residency, sovereignty, and compliance requirements, helping ensure that data remains under organizational control and external access is not permitted, not even by cloud providers.

  In addition to UKO for z/OS to do the key management, the support for MSDKE also requires the Crypto Connect component of UKO to provide the on-premises double key encryption service that provides encryption keys to Microsoft 365 applications.

- Application Programming Interface (API) for automation

  Because it is a web-based application, UKO for z/OS is split into a front-end user interface and a back-end. The user interface, shown in the browser, uses the API provided by the back end to perform all operations that the user initiates.

  The API is available for use by other applications, to automate key management actions through integration with other business applications. By accessing the same API that the UKO for z/OS user interface uses, it means that all actions available in the user interface can be automated.

By scheduling jobs on z/OS (or other platforms) to run applications that call the API, it is possible to run regular key management actions, like automated key rotation. It is the scheduled job's responsibility to perform the additional non-key-management actions that are required to complete the key rotation, such as update RACF to change data set encryption key labels and to reencipher encrypted data sets.

- UKO Crypto Connect Advanced Crypto Service Provider

  UKO contains the Crypto Connect (CC) Advanced Crypto Service Provider (ACSP) component, which can be used to provide an Enterprise Crypto as a Service architecture by using ICSF and Crypto Express. ACSP exposes the ICSF crypto API to non-mainframe systems and allows business applications to consume hardware cryptography services, including quantum-safe cryptography.

  The usage of Crypto Express adapters is typically low single digits for many organizations. By using ACSP to provide an Enterprise Crypto Service, the usage of the available CEX can be raised significantly (better ROI) and reduces the need (and cost) for other HSMs in the organization.

10.1.2 EKMF Workstation overview

The EKMF Workstation, which provides additional key and certificate management capabilities, is an addition to UKO for z/OS. The EKMF Workstation is a physical workstation designed for key management of highly sensitive keys, including keys in scope of PCI-DSS and PCI-PIN.

Authentication to the EKMF Workstation requires 2FA by using smart cards with PIN. Authorization requires dual control by having two (or more) key managers log on at the same time.

The EKMF Workstation uses the same UKO repository Db2 database as UKO for z/OS.

The EKMF Workstation and UKO for z/OS manage the same set of keys and use the same set of key templates to define keys.

In addition to the capabilities related to data set encryption, the EKMF Workstation provides the following features:

- General purpose key management

  In addition to AES keys for data set encryption, the EKMF Workstation can create and manage all keys supported by ICSF. This includes the classical AES, DES, HMAC, RSA and ECC keys as well as the quantum-safe ML-KEM and ML-DSA. The EKMF Workstation is continuously being enhanced with new capabilities.

  The EKMF Workstations key management capabilities extend beyond ICSF. The EKMF Workstation is capable of managing keys in non-IBM HSMs to provide keys to all HSMs within an organization.

- Key Management for PCI environments

  The general PCI-level security of the EKMF Workstation makes it the preferred key management system for management of keys in scope of PCI-DSS and PCI-PIN. Dual control, role-based access control, and extensive audit logging ensures that keys can be created securely and key management actions can be audited and demonstrated to be in compliance with PCI requirements.

- Key management for Cloud

  With applications and data in the cloud it is important to be able to protect these using the available cryptographic functions provided by the cloud providers. The EKMF Workstation is able to create keys and distribute them to several cloud providers. With the EKMF Workstation, keys that have been disabled in the cloud can be restored. By using the EKMF Workstation, keys can also be deactivated and reactivated as needed.

The EKMF Workstation can create and manage keys for these cloud providers:

- Amazon Web Services (AWS)
- Google Cloud Platform (GCP)
- IBM Cloud
- Microsoft Azure

- Certificate Management

  The EKMF Workstation has support for management of x509 certificates, including creation of certificate signing requests (CSR), receiving signed certificates, and importing foreign (CA) certificates. The EKMF Workstation provides a structured view of certificates to simplify the management of certificates for individual key holders (such as, applications or other owners of certificates). The EKMF Workstation integrates with RACF for automatic installation of certificates in key rings and private keys in RACF or the PKDS keystore.

  It is possible to export private keys and certificates to manually provide these to other (distributed) applications.

- Remote Key Loading using TR-31 and TR-34 key exchange

  Since January 2025, PCI has required all key exchanges to be done using PCI compliant key exchange formats. When managing keys for ATMs and similar sensitive devices, this key exchange requires a specific protocol using the TR-31 and TR-34 key exchange formats. Features in the EKMF Workstation combined with APIs on z/OS allow efficient generation, management, and exchange of keys for ATMs and similar devices, in accordance with PCI requirements.

- Key Management for AWS Payment Cryptography

  Amazon has implemented support for payment processing in their cloud services. This requires special PCI compliant key exchange formats and processes as well as management of related certificates.

  By using the PCI-level security of the EKMF Workstation, combined with its support for PCI-compliant TR-31 and TR-34 key blocks for exchange, certificate management, and its cloud key management support for AWS, the EKMF Workstation is able to create and manage Payment Cryptography keys in AWS.

10.2 Foundations of UKO for z/OS

UKO for z/OS is a native IBM z/OS solution that uses the core capabilities of the z/OS operating system and IBM Z hardware.

UKO for z/OS runs in the Websphere Liberty application server on z/OS. It uses a Db2 database (also on z/OS) to store all its data. The database with its tables and views must have been created based on DDLs provided by the UKO for z/OS installation.

To perform all of the sensitive key management operations, it uses ICSF and Crypto Express adapters. At least one adapter must be available and assigned to the LPAR on which UKO for z/OS is running. UKO for z/OS requires that the Crypto Express adapter has its AES, RSA, and ECC master keys set. The DES master key is not required by UKO for z/OS.

After the keys are created, they are distributed to the CKDS or PKDS keystores on the target systems by using the UKO agent tasks on the target systems.

The recommended formats for the CKDS and PKDS keystores are the KDSR and KDSRL formats. The PKDS must be in KDSRL format to store PQC keys (ML-KEM and ML-DSA). The fixed-length CKDS format does not support AES keys and therefore cannot be used with UKO for z/OS.

The program requirements for UKO for z/OS is described in the UKO documentation at: Unified Key Orchestrator for IBM z/OS: Requirements.

For an up-to-date list of requirements for any specific version of UKO for z/OS, see the Software Product Compatibility Reports. Search for the product name “Unified Key Orchestrator for IBM z/OS”, select the relevant version, and view the “Detailed system requirements”. The “Prerequisites” tab is in the list of requirements.

10.2.1 UKO for z/OS graphical user interface

Access UKO for z/OS through a web browser. Users must log in using credentials, which can be authenticated using RACF or LDAP. The authorizations are handled by EJBROLES which are discussed further in 10.2.3, “UKO for z/OS authorization roles” on page 230.

After logging in, you can use the navigation menu on the left side to navigate between functions. The Overview page for UKO for z/OS is shown in Figure 10-2.

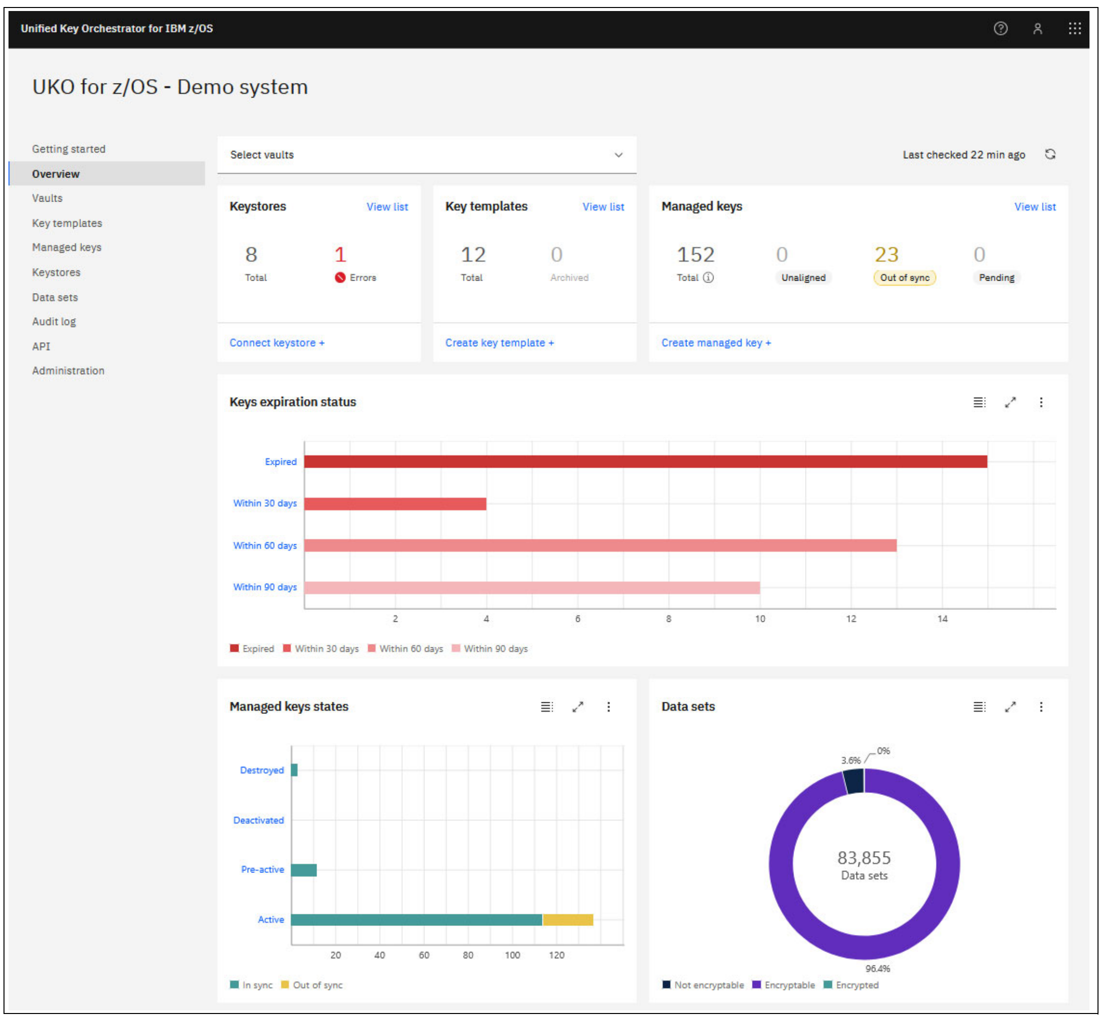


Figure 10-2 UKO for z/OS overview page

10.2.2 Key hierarchy

Security and integrity of UKO for z/OS generation and distribution of keys is based on a key hierarchy (see Figure 10-3). If the master keys and UKO for z/OS disaster recovery keys are generated securely, there is never a time any of the keys are in the clear.

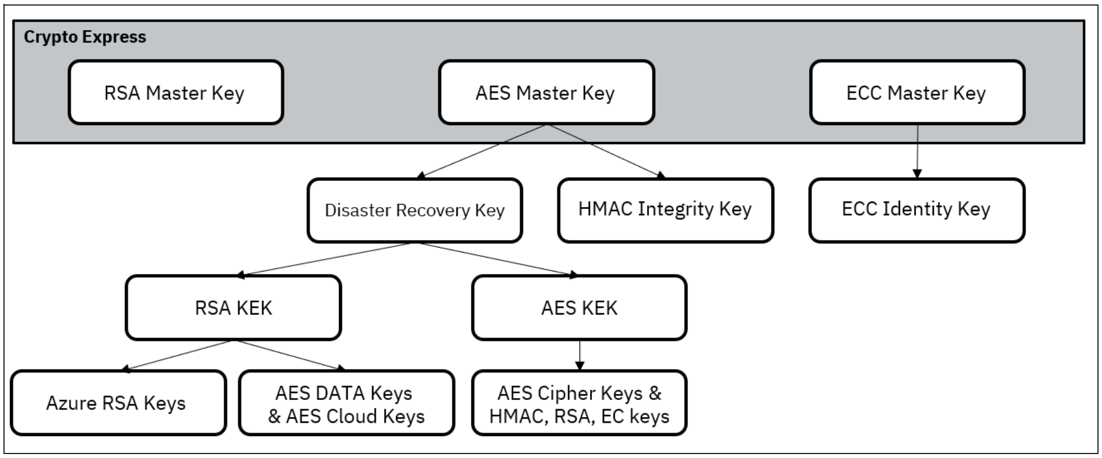


Figure 10-3 Key hierarchy

Trusted Key Entry Workstation (TKE) is strongly recommended because you can generate keys securely that are never in the clear. UKO for z/OS requires master keys to be loaded into the AES, RSA and ECC registries for each cryptographic processor for each LPAR.

UKO for z/OS Disaster Recovery Key

UKO for z/OS also requires the secure creation of the UKO for z/OS Disaster Recovery Key (DRK). The DRK must be created in such a way that it is recoverable. The recommended approach is to store the DRK in parts, such that it can be reconstructed from these key parts.

Options for creating the DRK includes:

- Use TKE: When using TKE (recommended, especially for production systems), these key parts can be securely generated and stored on smart cards. Subsequently, the key parts can be loaded from the smart cards and can be installed in the CKDS keystore using ICSF. Restoring the DRK follows the same key loading process using the existing key parts on smart cards.
- Use ICSF and the ICSF API: If TKE is not available, key parts and their corresponding key check values can be created and calculated using ICSF and then recorded on paper. Note that key part creation for the UKO for z/OS DRK using ICSF is not considered secure.

  For more information about creating the DRK, see: UKO key hierarchy setup.

Keeping track of the paper copies of the key parts and storing them securely will be critical to prevent data loss in case the CKDS is corrupted. Each key part should be stored separately, such that no single individual has access to all parts.

Entry of these key parts to build the UKO for z/OS DRK will require the use of the ICSF API.

10.2.3 UKO for z/OS authorization roles

UKO for z/OS uses EJBROLES defined in RACF to authorize users to perform different functions in UKO for z/OS. These roles are defined during setup. UKO for z/OS provides a sample of four different types of user groups to be created based on these roles.

These are: *Key Administrator*, *Key Custodian 1*, *Key Custodian 2,* and *Auditor*. The function of each role is introduced in the following list:

- Key Admin: sets up key hierarchy, controls key stores, creates key templates, as well as performs special key state actions
- Key Custodian 1: can generate keys in Pre-Activation state but cannot activate and distribute them. Can also destroy keys and mark keys as compromised.
- Key Custodian 2: cannot generate keys but can activate and distribute them into key stores. Can also destroy keys and mark keys as compromised.
- Auditor: can view the audit log as well as keystores, templates, key list, data set dashboard

10.3 Key management with UKO for z/OS

Key management with UKO for z/OS means securely creating, storing, and distributing cryptographic keys across IBM Z systems, using ICSF, CKDS/PKDS keystores, and Crypto Express hardware for enterprise-grade security.

This section provides an overview of the key management process using UKO for z/OS.

10.3.1 UKO vaults

A central concept in UKO for z/OS is the concept of a vault. A vault is an isolated container in which keys can be managed independently of keys in other vaults.

Some organizations have a central key management team and prefer to have all key management done in a single vault. Other organizations might decide to (partially) delegate key management responsibilities to their application teams.

Vaults in UKO for z/OS enables these organizations to give individual application teams access to application-specific vaults and allow application teams to manage keys for their applications without being able to see or interfere with keys belonging to other vaults and by extension to other applications.

It is possible to permit users to access multiple vaults. Although specific application teams might have access to only their specific vault, a supervising central key management team might be given access to all defined vaults for audit and support of the application teams.

Access to vaults can be granular and individual users might be permitted to perform all operations or might be limited to key generation only, based on predefined key templates, created by a central key management team. The level of access is controlled by the organization in accordance with the organization’s security policies.

10.3.2 UKO keystores

UKO for z/OS is a centralized key manager. Keys are generated by UKO for z/OS using ICSF and the Crypto Express adapters and stored in the UKO repository (Db2). Based on the policies and GUI requests, keys are then installed into the local keystores. For this to work, UKO for z/OS must connect to z/OS CKDS keystores through UKO agents.

On each LPAR, UKO for z/OS has an agent that listens for request and issues the underlying ICSF API calls to load the keys into the CKDS (see Figure 10-4 on page 232). This requires a

UKO agent to be installed on each system that has a unique CKDS. If you are sharing a CKDS in a sysplex, you need one agent per sysplex.

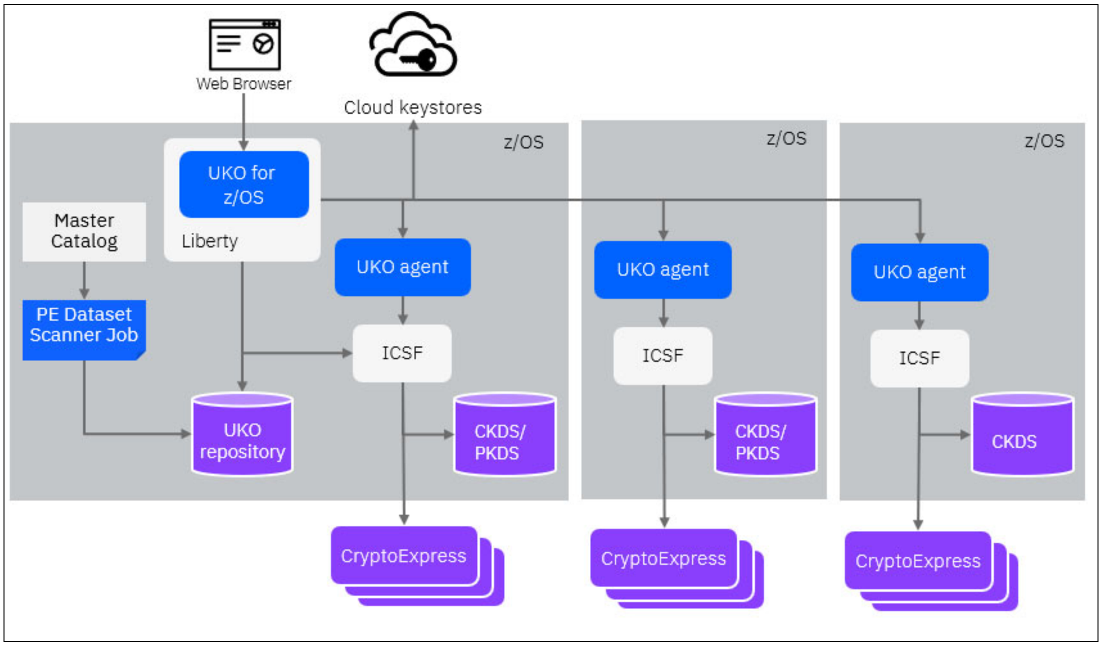


Figure 10-4 UKO for z/OS deployment - multiple z/OS images

10.3.3 Key template

All keys that are generated by UKO for z/OS must be generated from a key template. The key template defines the key label name, keystores where keys are installed, and key characteristics. Typically, the key administrator role has access to create the key template, whereas the key custodians have access to use the key template. The auditor role would have read only access to key templates.

Key label

When creating the key label, it is important to have in mind the key usage and meaningful label parts (see 3.6.7, “Creating a key label naming convention” on page 52). The key label typically describes the type of key, area where the key would be used, application key that would be used, and the sequence number.

It is considered best practice to have a sequence number as a tag. UKO for z/OS determines the next sequence number during key generation, if instructed to do so.

This key label naming scheme sets the key label to a fixed text, with only the sequence number being variable for successive key generations:

AES256.TEC2MVS.DB2.DB1A.CIP<sqn>

Although this works well for the specific purpose, it leaves little flexibility for reuse of the key template for other purposes. To achieve flexibility, the key label naming scheme can be defined with custom tags that are placeholders for values specified at key generation time.

The following key label naming scheme allows the enforcement of a key label naming convention starting with the fixed text of ‘AES256’, followed by a system identifier, application identifier, environment identifier, another fixed text ‘CIP’ to indicate the key is a cipher key, and finally an auto-generated sequence number:

AES256.<SYS>.<APPID>.<ENV>.CIP<sqn>

In the key label naming scheme, <SYS>, <APPID>, and <ENV> are the custom tags that are replaced during key generation. In addition, UKO for z/OS defines the tag <sqn> (and <seqno>), which tells UKO for z/OS to calculate the next available sequence number and to insert that number in the key label. By default, the sequence number is expanded to five digits with leading zeros.

For a key used on system TEC2MVS for the Db2 subsystem DB1A, the resulting key label for the 14th key generation is AES256.TEC2MVS.DB2.DB1A.CIP00014.

Keystores

The key template has a drop-down menu that lists all available keystore groups (see Figure 10-5). When you select a keystore group, UKO for z/OS installs the keys generated from the key template into all keystores that are contained within the specified keystore group. Only one group can be selected.

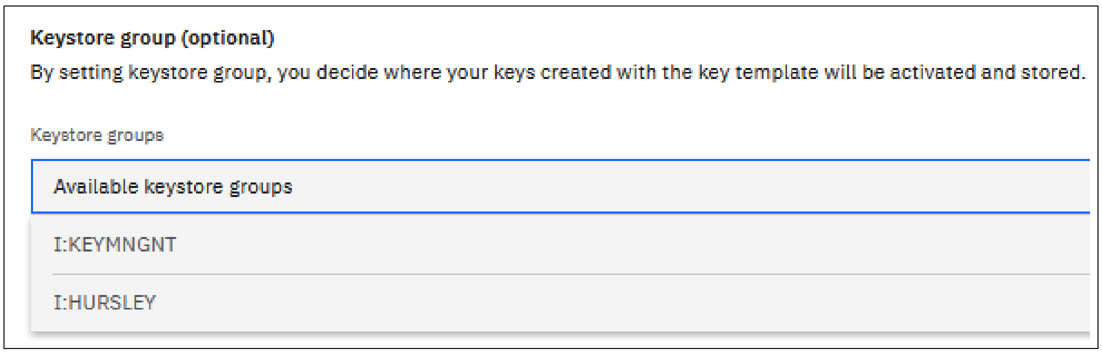


Figure 10-5 Keystore group in UKO for z/OS

Characteristics of the key template

There are a number of key characteristics to consider when creating the key template.

For Figure 10-6, select the Vault in which the key template should be. The key template can be used only within the selected Vault and can reference only keystores that belong to the same Vault.

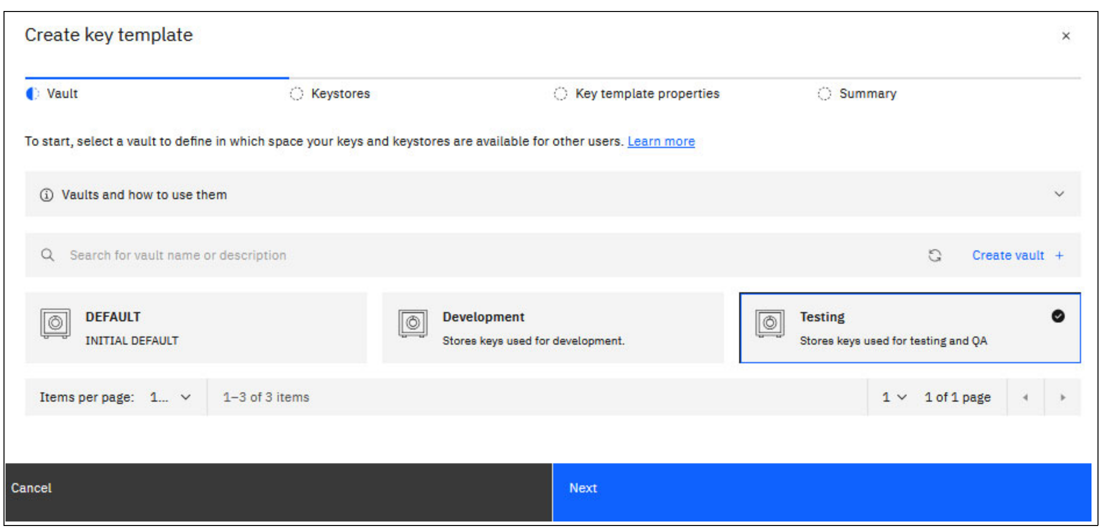


Figure 10-6 Key template Vault selection

The following characteristics are in listed in Figure 10-7:

- Keystore Type: Select the type of keystore to which the key should be distributed. UKO supports several different cloud keystores and the KMG Agent keystore, which is used when creating keys for ICSF on z/OS.
- Keystore Group: Select the group of keystores to which the keys, which are generated from the key template, should be distributed.

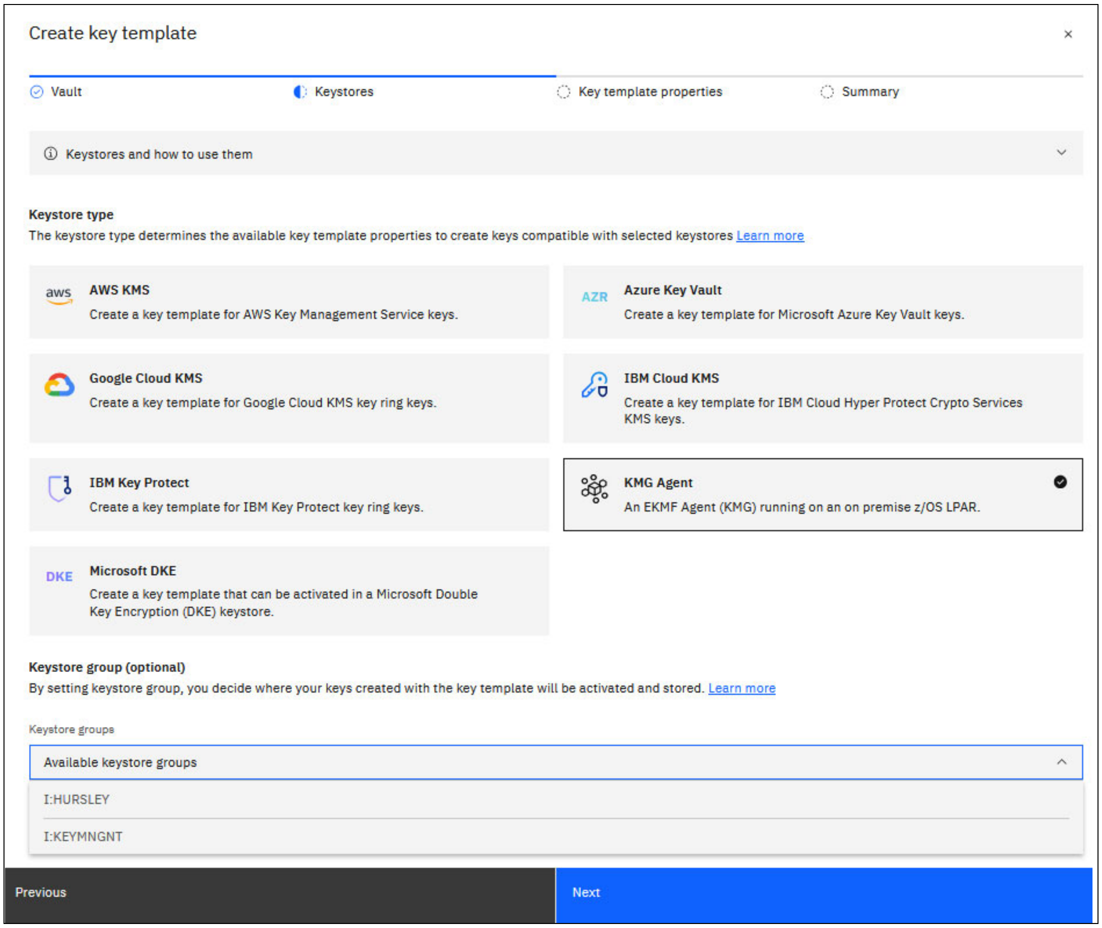


Figure 10-7 Key template keystore selection

The following fields are included Figure 10-8 on page 235:

- **Key Template Name**: The name of the key template, which identifies the key template in the key template overview.
- **Description**: The description of the key template. This is a free-text field that can be used to document the purpose of the key template. The description is displayed in the key template details.
- **Naming Scheme**: The naming scheme consists of fixed text and tags that are replaced at key generation time. The resulting string is the ID of the key in UKO. Use the naming scheme to encode the key label naming convention that the generated keys must follow.
- **Key Instance Naming Scheme**: The naming scheme consists of fixed text and tags that are replaced at key generation time. The resulting string is the key label of the key in the keystore. Use the naming scheme to encode the key label naming convention that the generated keys must follow. In most cases, the Naming Scheme and Key Instance Naming Scheme should be identical.

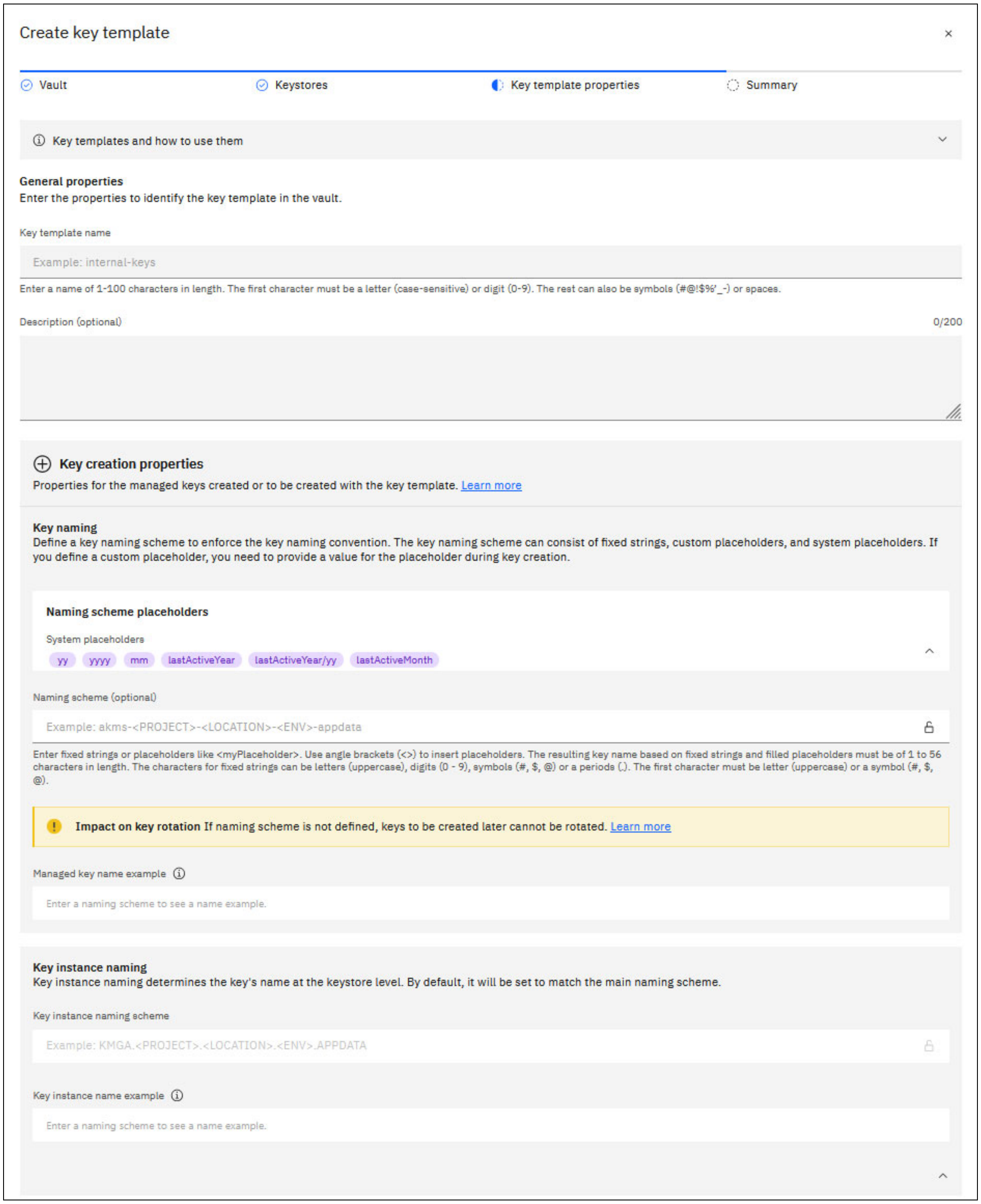


Figure 10-8 Key template properties (1/2)

Figure 10-9 on page 236 includes the following fields:

- **Algorithm**: Select the algorithm of the keys that are generated from the key template.
- **Length** (or Matrix Size): Select the size of the key.
- **Key Type** (AES Keys only): Select the type of key to generate. In general, CIPHER Keys are considered more secure than DATA keys. For keys used for data set encryption, the Cipher key type is recommended.
- **Hash Methods** (HMAC keys only): Select the allowed hash methods for the keys that are generated from the key template.

- **State**: If you select Active, the key is distributed to all keystores (specified by the keystore group of the key template) as part of the key generation process. If you select Pre-activation, the key is not distributed automatically until the key is activated.
- **Activate Keys After** (activation date): Select the amount of time after key generation to set as the key's activation date.
- **Deactivate Keys After** (expiration date): Select the amount of time after key generation to set as the key's expiration date. The difference between the Activation and Expiration dates defines the lifetime of the generated key. Define the lifetime of the key according to policies but be careful of implications. Encrypted data can be archived. If you retrieve the data years later, you might have to use an expired key. For this reason it is never recommended to delete but to archive them instead.
- **Deactivate previous key version on rotation**: Selecting this checkbox results in the previously generated key being deactivated and removed from keystore when the key rotation function is used. For a data set encryption key, removing the old key as part of the key rotation makes data set reencryption impossible and immediately removes access to encrypted data sets. For data set encryption keys, always leave the checkbox unchecked. For other keys, similar considerations likely applies.

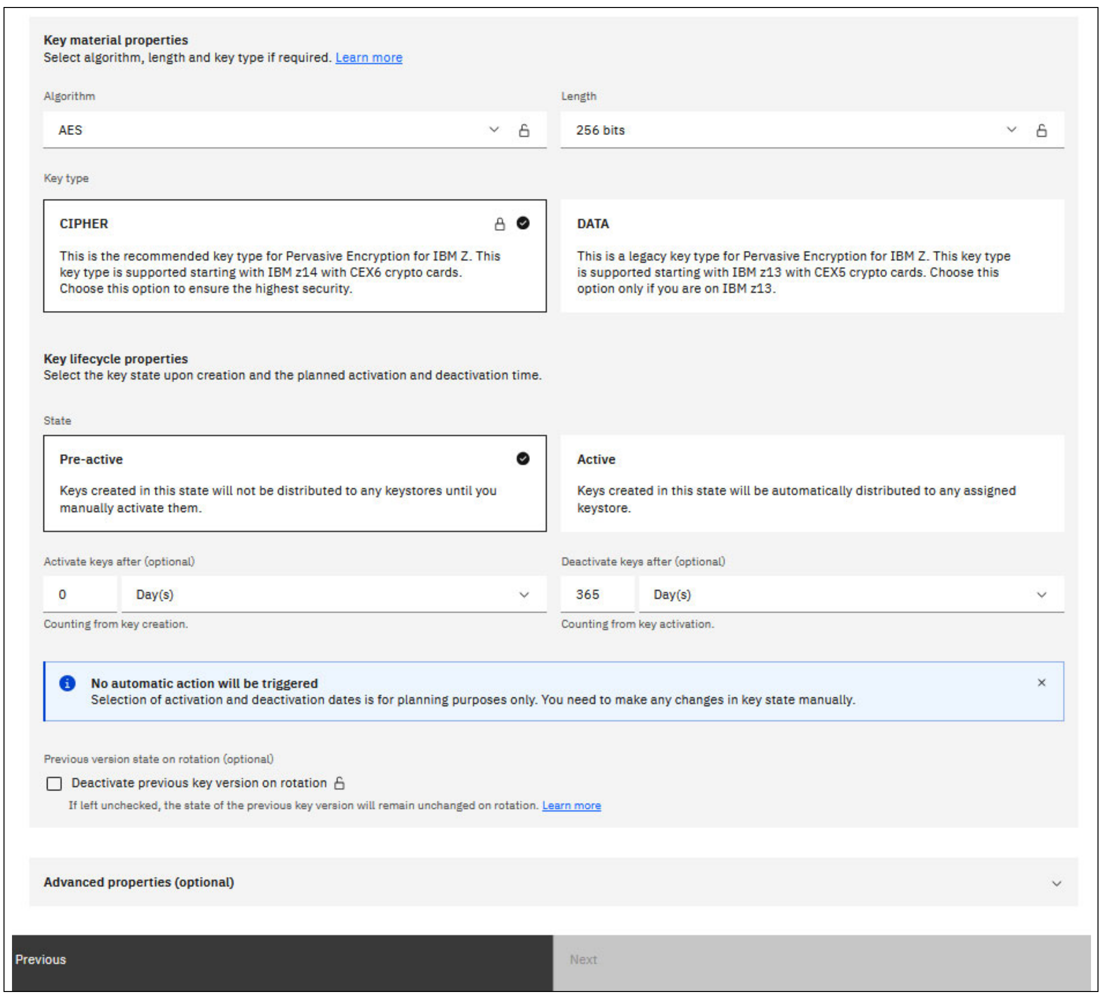


Figure 10-9 Key template properties (2/2)

10.3.4 Key lifecycle

The lifecycle of keys is one of the most important aspects of key management.

The National Institute of Standards and Technology (NIST) provides a standardized framework for managing cryptographic keys, which is described in publications such as NIST SP 800-57. The lifecycle ensures that keys are properly controlled throughout their existence. NIST has a well-defined process for the key lifecycle, which defines flows for key generation (*Generate*), activation (*Activate*), deactivation (*Deactivate*), and destroying a key (*Destroy*).

UKO for z/OS provides the functionality to manage the key lifecycle. Figure 10-10 depicts the key lifecycle as part of UKO for z/OS, which includes the following key actions:

- Activate: install the key in the keystore first time.
- Deactivate: removes the key from all keystores.
- Reactivate: Will restore the key into the relevant keystores, as defined by the associated key template.
- Destroy: removes the key permanently from the UKO repository.

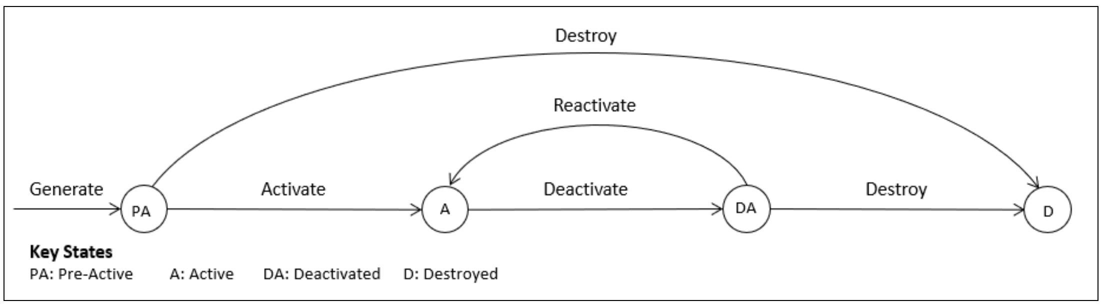


Figure 10-10 Key lifecycle

Generating a key

The key template is the basis for any key generation in UKO for z/OS. Key generation is started from the managed key overview by using the 'Create Key' button (see Figure 10-11).

When generating a key, the base naming convention and key characteristics are retrieved from the key template and applied to the key generation. When using custom tags in the key label naming scheme, the key generation process prompts for the values to use instead of the tags.

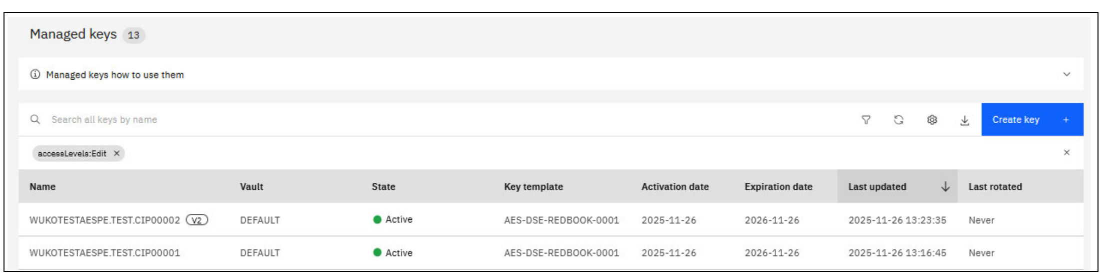


Figure 10-11 Start key generation

Key generation starts with selecting the Vault in which the key should be placed. This is similar to the Vault selection of key templates as shown in Figure 10-6 on page 233.

After selecting the Vault, UKO for z/OS displays the key templates available in the Vault from which the key can be generated. After the key template has been selected, UKO for z/OS prompts for input for the custom tags that are part of the key label naming scheme specified in the key template.

The example in Figure 10-12 shows the custom tag inputs for a key template that has specified the key label with the following naming scheme:

<ENV>.AESPE.<APP>.CIP<seqno>

Before proceeding to the next step, values must be specified for ENV and APP. The seqno is shown as five zeros, which is just a placeholder until the sequence number can be determined during key generation.

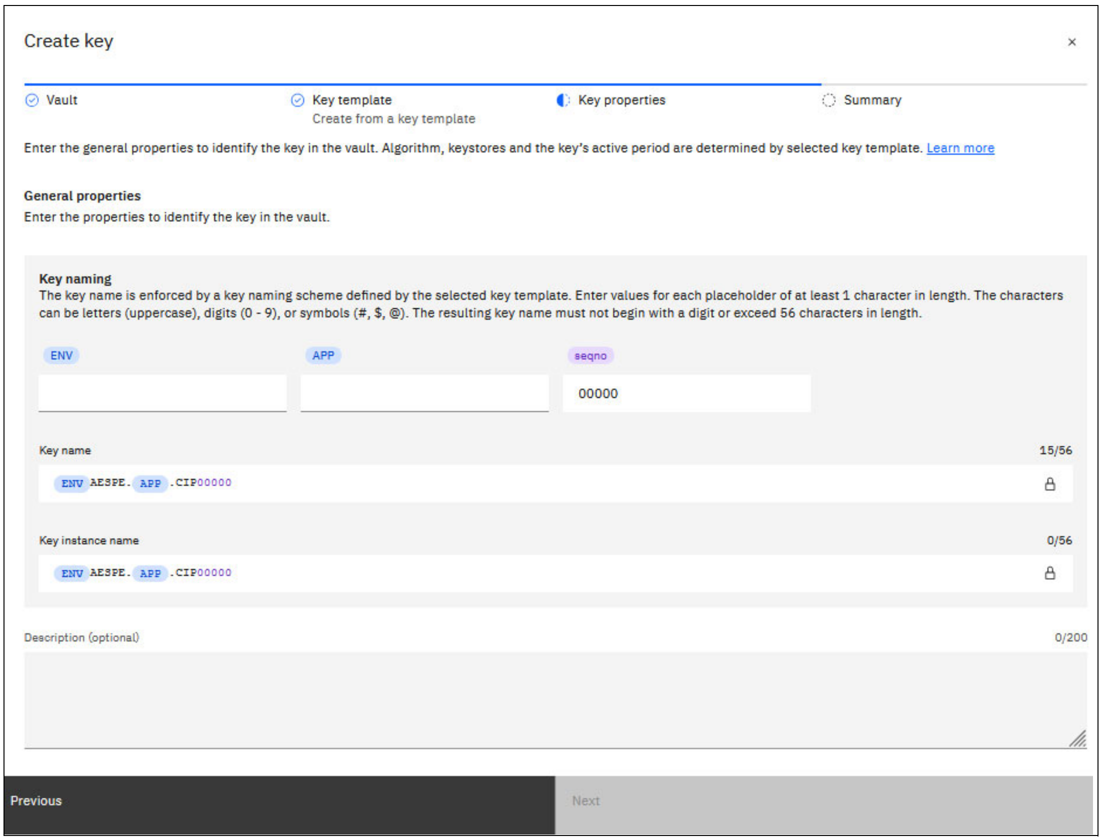


Figure 10-12 Key creation custom tag input

Activating a key

Key activation means that the key is added to the key stores that are defined by the template. This can be done upon generation, or, if a key is generated in the pre-active state, it can be done in the key view.

The pre-active state provides the ability to have separation of duties where a key custodian can have the ability to generate keys but is not allowed to activate keys or install keys in keystores. Figure 10-13 on page 239 shows a screen shot of a key in pre-active state, and how to install it into the keystore(s).

To activate a key and then install it into keystores, select the action menu by selecting the three vertical dots at the end of the key record in the key overview. From there select ‘Active…’.

Then confirm the activation in the subsequent confirmation dialog.

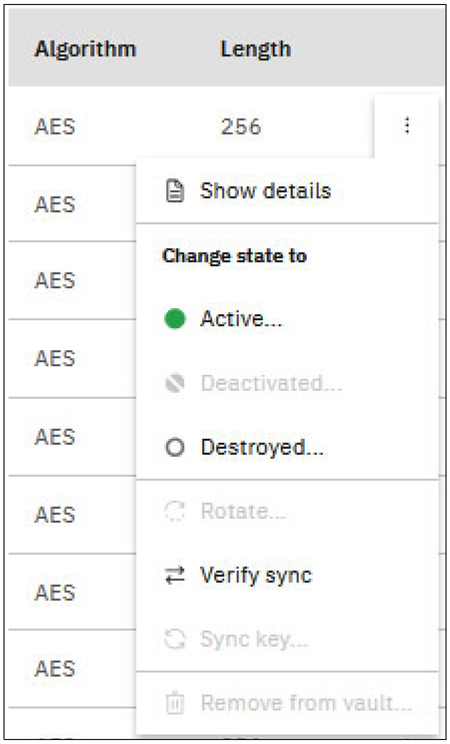


Figure 10-13 Key activation from key overview

An alternate method of activating a key is to edit its lifecycle properties in the key details. Use the Edit button shown in Figure 10-14 to enable the editing.

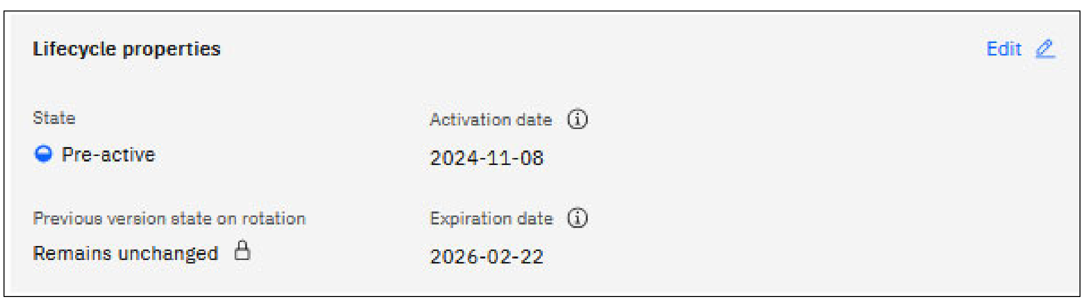


Figure 10-14 Key activation from key details

Open the drop-down menu shown in Figure 10-15 on page 240 and select the **Active** state, and save the changes.

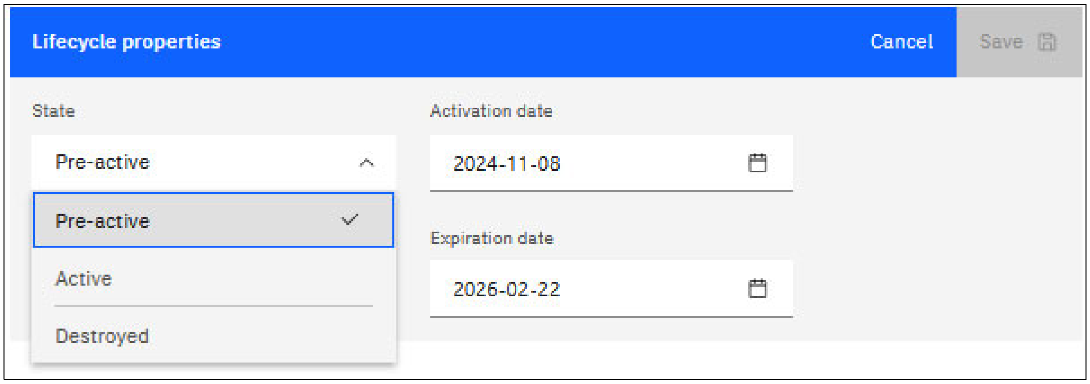


Figure 10-15 Key activation from key details

Deactivating a key

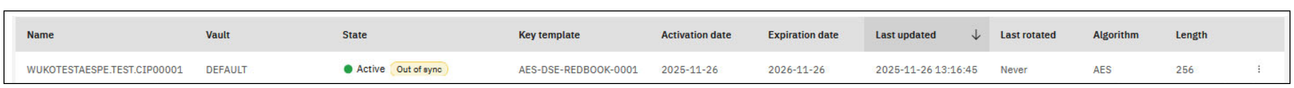
When a key is deactivated it is removed from the keystore. The key, however, is not deleted from the UKO repository, so it can be reactivated or reinstalled at a later time if necessary.

The steps to deactivate a key is similar to activation. To deactivate a key and remove it from all keystores, select the action menu using the three vertical dots at the end of the key record in the key overview, as shown in Figure 10-13 on page 239. Select 'Deactivated…'. Confirm the deactivation in the subsequent confirmation dialog.

An alternate method of deactivating a key is to edit its lifecycle properties in the key details. Use the **Edit** button shown in Figure 10-14 on page 239 to enable the editing. Open the drop-down menu and select the desired (Deactivated) state. After it is selected, save the changes.

Restoring a key

If UKO for z/OS determines that a key is missing from one or more of the keystores that UKO for z/OS expects the key to be in, it labels the key as `Out of sync` in the key overview, as shown in Figure 10-16.



Name	Vault	State	Key template	Activation date	Expiration date	Last updated	Last rotated	Algorithm	Length
WUKOTESTAESPE.TEST.CIP00001	DEFAULT	Active Out of sync	AES-DSE-REDBOOK-0001	2025-11-26	2026-11-26	2025-11-26 13:16:45	Never	AES	256

Figure 10-16 Key overview: Key Out of sync

To restore the key back to its keystores, click the key record to open the key details shown in Figure 10-17 on page 241.

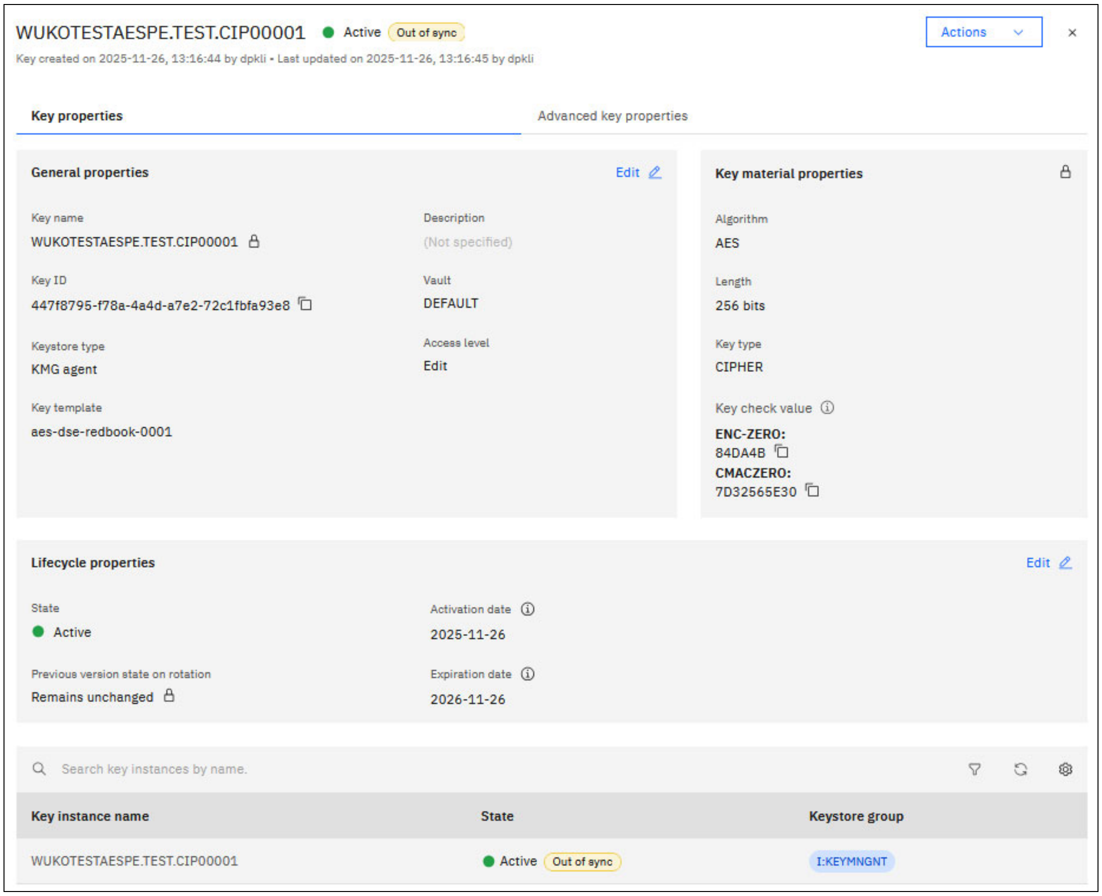


Figure 10-17 Key details: Key out of sync

UKO for z/OS shows the instances of the key and highlights the instance that is `Out of sync`. Click the instance to open the keystore details shown in Figure 10-18.

To restore the key, click the **Sync Key** button and confirm the synchronization in the subsequent confirmation dialog.

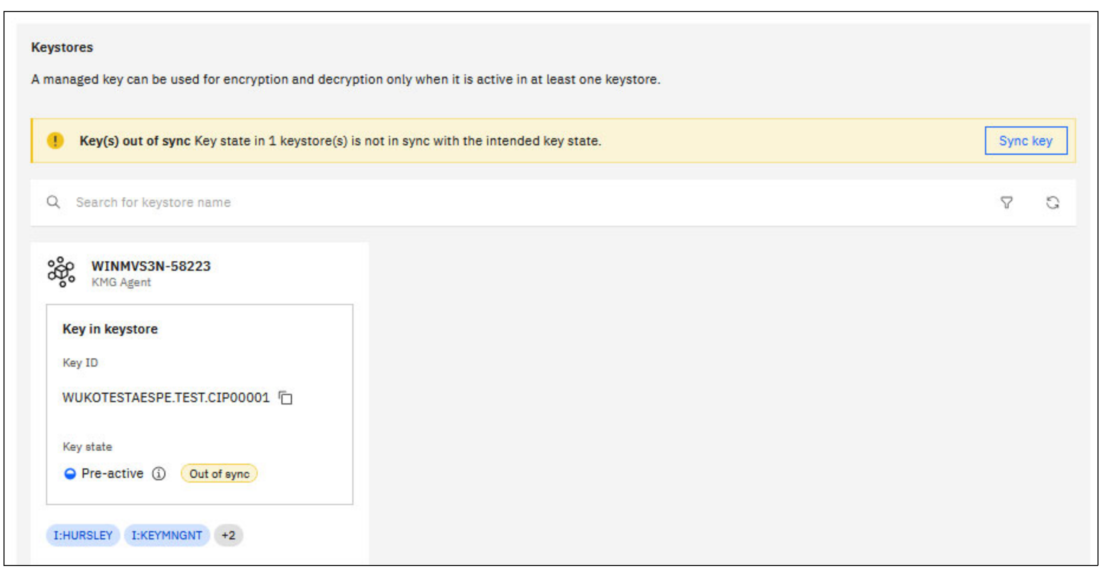


Figure 10-18 Keystore details: Key Out of sync

Destroying a key

When a key has reached its end of life and it is no longer used for dataset encryption, the key can be destroyed. It might be preferable to delay destruction for a time to ensure that the key is truly no longer needed. After a key is destroyed, it is removed from the UKO repository and *cannot be recovered*. As such, all data encrypted with a key that has been destroyed is considered to be cryptographically erased. UKO for z/OS retains metadata about the key, making it possible to prove to auditors that the key was destroyed.

10.3.5 Key rotation

Rotating a data set encryption key starts with creating a new key to replace the old key.

UKO for z/OS has a function called **Rotate**, which allows generation of a new key based on the parameters of an existing key. At the end of each row of keys in the key list overview there is a menu symbolized by three dots, arranged vertically.

To generate a key again, find the key in the key list overview and open the menu for the key and select **Rotate** (as shown in Figure 10-19).

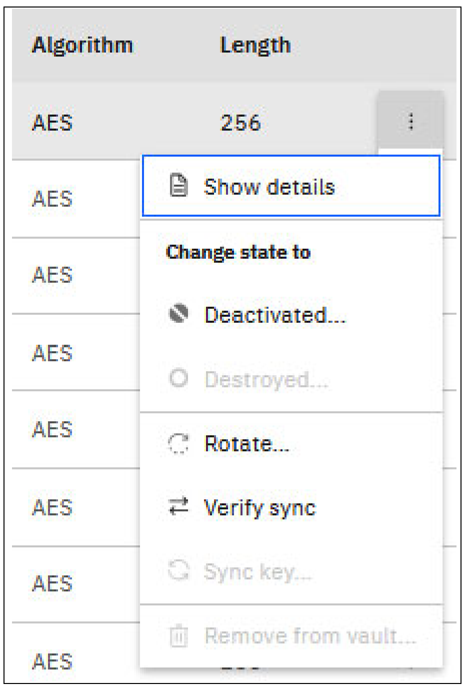


Figure 10-19 Rotating a key

Selecting **Rotate** starts a key generation (see “Key generation with EKMF Workstation” on page 133). The key template and values for key label tags are pre-filled based on the values used for the selected key.

Complete the key generation, then return to z/OS to complete the key rotation, as described in 7.2.2, “Rotating data set encryption keys” on page 166.

> **Note:** For users of the EKMF Workstation, the corresponding function is called 'renew key', which is available from functions menu and then right-click context menu for a key in the key list overview.

10.4 UKO for z/OS view of data sets

UKO for z/OS provides a dashboard view of data sets on the system. To achieve this, two JCL jobs are provided:

- The first job reads the master catalog to create a detailed extract of datasets on a system.
- The second job takes the extracts and loads them into Db2.

Run the job that creates extracts on each LPAR. The frequency is determined by the organization and depends on how often the organization wants make changes. When actively deploying data set encryption, a daily schedule can be relevant to see up-to-date changes. When it is in a more steady state, a weekly or monthly extract might be more appropriate. Reporting requirements related to compliance with regulations can influence the frequency of extracting data.

When all extracts are transferred to the same system and merged, the load-job can input everything into the UKO repository database. Subsequently, UKO for z/OS can display the result in the data set dashboard, accessible from the menu. Datasets are viewed at sysplex level. The search can be refined by specifying the start of a dataset name (use * as a wildcard) or a key label by prefixing the label with the qualifier `label:`.

The following list provides other examples of searches:

- To search for datasets starting with ‘HLQ.APP’ enter `HLQ.APP` in the search field.
- To search for data sets encrypted by keys with key labels starting with UKO.PE, enter `label:UKO.PE` in the search field.
- To search for datasets matching both criteria, enter `HLQ.APP label:UKO.PE` in the search field.
- To select the sysplex to view data sets from, and to limit the search to data sets that are encrypted, not encrypted (encryptable), or not encryptable, click the filter icon to the right of the search field to expand the search criteria section. Select the relevant options and click **Apply**.

Figure 10-20 shows a view of the data sets and their encryption status.

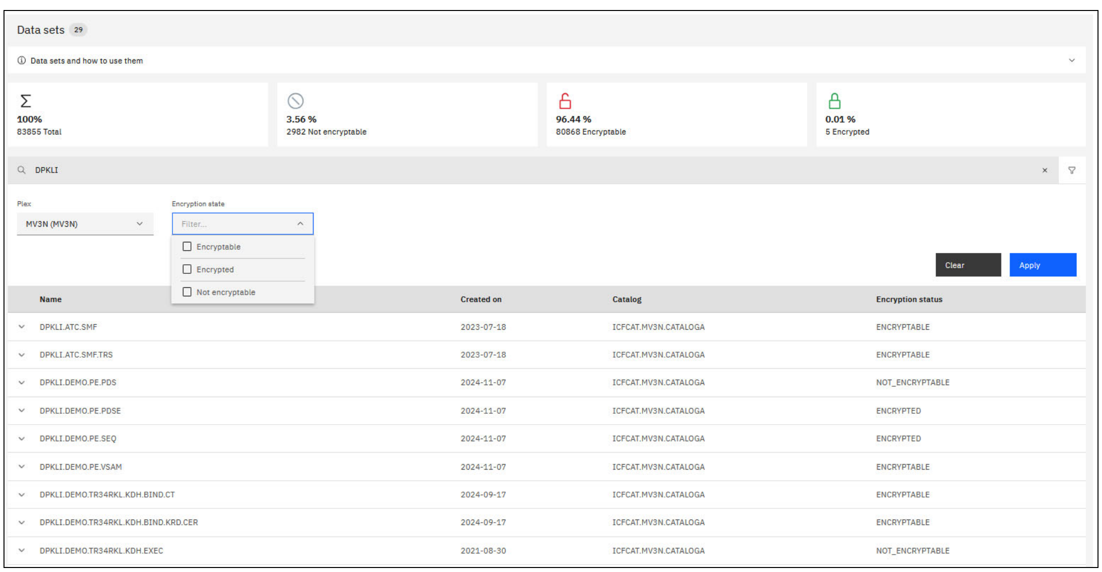


Figure 10-20 Data set encryption status

Additional details for individual data sets can be viewed by clicking on the data set name.

Figure 10-21 provides detailed view of a data set (DPKLI.DEMO.PE.PDSE). The data set is identified as `ENCRYPTABLE`. However, the `Reason` provided at the bottom of the screen states that, to enable encryption, this data set must be changed to extended format before allocating a key label.

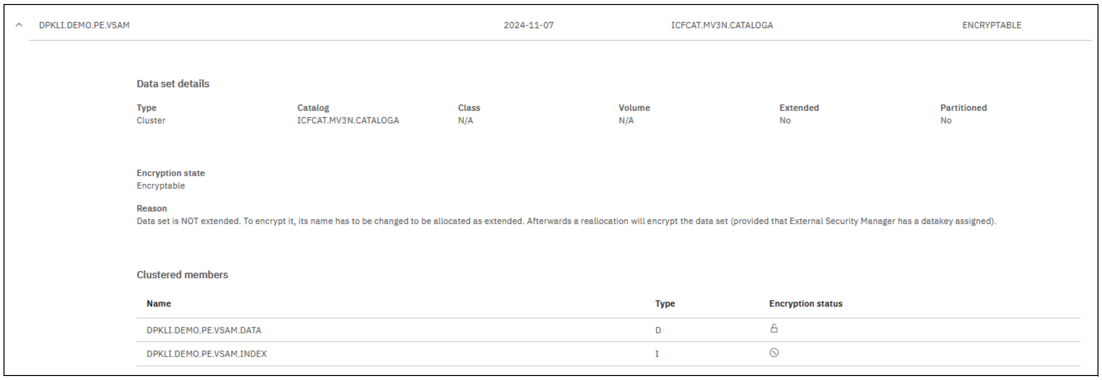


Figure 10-21 Unencrypted data set view

Figure 10-19 shows an encrypted data set (DPKLI.DEMO.PE.PDSE). For an encrypted data set, the associated key label is also shown. You can also click the encryption key label. If the corresponding key is under UKO for z/OS control, you can see further details.

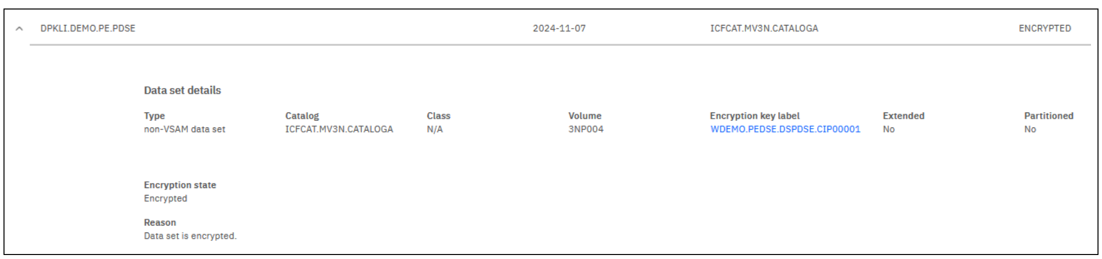


Figure 10-22 Encrypted data set view

A

Troubleshooting

This appendix describes some of the common error situations you might encounter when working with data set encryption. Errors and their symptoms (error messages, unexpected result, or behavior) also are described, including how to remedy or bypass the problem.

> **Note:** The errors that are described in this appendix include attempting to access an encrypted data set with specific characteristics.

This appendix includes the following topics:

A.1 Accessing data sets

This section describes issues that you might encounter while attempting to access data sets.

A.1.1 Accessing a data set without proper access to the key label

Error messages that can occur when attempting to access a data set without proper access to the key label are shown in Example A-1.

Example: A-1 Attempting to access a data set without proper key label access

```
IEC150I 913-84,IGG0193V,PE02,TSOPROC,ISP14455,9642,CONSM3,
ICH408I USER(PE02    ) GROUP(SYS1    ) NAME(PERVASIVE ENCRYPTION)
  DATASET.PE06.ICSF.ENCRYPT.ME.ENCRKEY.00000005
  CL(CSFKEYS )
  INSUFFICIENT ACCESS AUTHORITY
  ACCESS INTENT(READ   )  ACCESS ALLOWED(NONE   )
PE06.ICSF.ENCRYPT.ME.DATA5,
RC=X'00000008',RSN=X'00000000'
***
```

The remedy to perform is to permit the user READ access to the profile DATASET.PE06.ICSF.ENCRYPT.ME.ENCRKEY.00000005 in CLASS(CSFKEYS). For more information, see 5.7, “Granting access to encrypted data sets” on page 144.

A.1.2 Accessing data set with a key but key label is missing in the CKDS

Attempting to access a data set encrypted with a key, but the key label is not available in your CKDS, is shown in Example A-2.

Example: A-2 Attempting to access a data set encrypted with a key without the key label in CKDS

```
IEC143I 213-85,IGG0193V,PRICHAR,IKJACCNT,ISP08467,9642,CONSM3,
 PE06.ICSF.ENCRYPT.ME.DATA5,
 RC=X'00000008',RSN=X'0000271C'
 ***
```

The messages that you receive for a VSAM data set are shown in Example A-3.

Example: A-3 Error for a VSAM data set

```
IEC161I 069(00000008,0000271C)-162,PE061,IDCPRINT,SYS00001,,, 911
IEC161I PE06.ICSF.ENCRYPT.ME.VSAM,,UCAT.CONCAT
IEC161I 069(00000008,0000271C)-162,PE061,IDCPRINT,SYS00002,,, 912
IEC161I PE06.ICSF.ENCRYPT.ME.VSAM,,UCAT.CONCAT

IDC3300I  ERROR OPENING PE06.ICSF.ENCRYPT.ME.VSAM
IDC3351I ** VSAM OPEN RETURN CODE IS 186
IDC0005I NUMBER OF RECORDS PROCESSED WAS 0
IDC3003I FUNCTION TERMINATED. CONDITION CODE IS 12
```

The remedy to take is to define the key label in your CKDS or import the missing key.

For more information about generating a key and defining the key label in the CKDS, see 5.3, “Generating a secure 256-bit data set encryption key” on page 131.

For more information about importing a missing key, see 9.2, “Transporting data set encryption keys” on page 199.

A.1.3 Accessing a data set but key label is not defined in the CSFKEYS class

Attempting to access an encrypted data set with a key where the key label is in CKDS, but the key is not defined in the CSFKEYS class, is shown in Example A-4.

Example: A-4 Accessing encrypted data set with key

```
IEC143I 213-85,IGG0193V,PRICHAR,IKJACCNT,ISP08484,96C2,CONSM4,
 PE06.ICSF.ENCRYPT.ME06.DATA,
 RC=X'00000008',RSN=X'00000BFB'
 ***
```

The remedy is to define the CSFKEYS profile for your key, as shown in Example A-5.

Example: A-5 JCL to add key label to CSFKEYS

```
RDEFINE CSFKEYS DATASET.PE06.ICSF.ENCRYPT.ME.ENCRKEY.00000005         +
          UACC(NONE)
PERMIT DATASET.PE06.ICSF.ENCRYPT.ME.ENCRKEY.00000005                  +
  CLASS(CSFKEYS) ID(PE06)   +
  ACCESS(READ)
/*-------------------------------------------------------------------*/
/* The resource must specify the ICSF segment keywords to be able to */
/* use the key label for protected key.                              */
/*-------------------------------------------------------------------*/
RALTER CSFKEYS DATASET.PE06.ICSF.ENCRYPT.ME.ENCRKEY.00000005          +
  ICSF(SYMCPACFWRAP(YES) SYMCPACFRET(YES))
```

For more information, see 5.7, “Granting access to encrypted data sets” on page 144.

A.1.4 Accessing a data set with a key but attributes are missing

Attempting to access an encrypted data set that includes a key and key label that is in CKDS is shown in Example A-6. Also, the key is defined in the CSFKEYS class, but is missing attributes.

Example: A-6 Error where CSFKEYS class is missing attributes

```
SYMCPACFWRAP = NO
SYMCPACFRET = NO
IEC143I 213-85,IGG0193V,PRICHAR,IKJACCNT,ISP08488,96C2,CONSM4,
PE06.ICSF.ENCRYPT.ME06.DATA,
RC=X'00000008',RSN=X'00000BFB'
***

SYMCPACFWRAP = YES
SYMCPACFRET = NO
IEC143I 213-85,IGG0193V,PRICHAR,IKJACCNT,ISP08490,96C2,CONSM4,
PE06.ICSF.ENCRYPT.ME06.DATA,
```

```
RC=X'00000008',RSN=X'00000C04'
***
SYMCPACFWRAP = NO
SYMCPACFRET = YES
IEC143I 213-85,IGG0193V,PRICHAR,IKJACCNT,ISP08496,96C2,CONSM4,
PE06.ICSF.ENCRYPT.ME06.DATA,
RC=X'00000008',RSN=X'00000BFB'
***
```

The action is to ensure that the key is defined in the CSFKEYS class with attributes (see Example A-7).

Example: A-7 Defining attributes in CSFKEYS class

```
SYMCPACFWRAP = YES
SYMCPACFRET = YES
```

For more information, see 5.7, “Granting access to encrypted data sets” on page 144.

A.1.5 ICSF not active

If you attempt to access any sequential encrypted data set while ICSF is stopped or is not available, you receive the error messages that are shown in Example A-8.

Example: A-8 ICSF not active

```
IEC143I 213-85,IGG0193V,PRICHAR,IKJACCNT,ISP08455,9642,CONSM3,
 PE06.ICSF.ENCRYPT.ME.DATA5,
 RC=X'0000000C',RSN=X'00000000'
 ***
```

This situation for a VSAM data set is shown in Example A-9.

Example: A-9 ICSF not active for a VSAM data set

```
IEC161I 069(0000000C,00000000)-162,PE061,IDCPRINT,SYS00001,,,  607
IEC161I PE06.ICSF.ENCRYPT.ME.VSAM,,UCAT.CONCAT
IEC161I 069(0000000C,00000000)-162,PE061,IDCPRINT,SYS00002,,,  608
IEC161I PE06.ICSF.ENCRYPT.ME.VSAM,,UCAT.CONCAT
```

A PRINT of an encrypted VSAM data set is shown in Example A-10.

Example: A-10 PRINT encrypted VSAM data set

```
PRINT INDATASET(PE06.ICSF.ENCRYPT.ME.VSAM) DUMP
0IDC3300I  ERROR OPENING PE06.ICSF.ENCRYPT.ME.VSAM
 IDC3351I ** VSAM OPEN RETURN CODE IS 186
0IDC0005I NUMBER OF RECORDS PROCESSED WAS 0
0IDC3003I FUNCTION TERMINATED. CONDITION CODE IS 12
```

The message `CSFM401I CRYPTOGRAPHY - SERVICES ARE NO LONGER AVAILABLE` indicates that a problem exists because the ICSF address space was ended. Your automation should monitor this message and take any necessary action to remedy the problem.

The action is to ensure that the ICSF address space is always started. If necessary, start by using the command **S ICSF,SUB=MSTR**. For more information, see 4.3.4, “Starting and stopping ICSF” on page 85.

A.1.6 Processing a Basic and Large dataset with a small LRECL

Attempting to allocate and process a new dataset with a small LRECL (FB < 16, VB < 12) triggers an error as shown in Example A-11.

Example A-11 Small LRECL processing error

```
IEC143I 213-9A,IGG0191B,DANDRE$,PROCESS,SMALLDS,A262,LS0026,  195
NEW.DATASET.SMALL
```

To solve this issue, you have three options:

- Exclude this specific dataset from encryption by using a new dataset profile with ENCRYPTTYPES(EXSEQ)
- Change the JCL so that this particular dataset is allocated in Extended Format (using DSNTYPE=EXTREQ) if the application is compatible with extended format.
- Change the application so that the output dataset is at least 16 bytes FB or VB.

A.1.7 Processing a Basic and Large dataset with EXCP without encryption support

Attempting to process an encrypted Basic and Large dataset with an application using its own EXCP without implementing encryption triggers an error as shown in Example A-12.

Example A-12 EXCP application without encryption support

```
IEC143I 213-9A,IGG0191B,DANDRE$,PROCESS,EXCPDS,A262,LS0026,  195
NEW.DATASET.EXCP
```

To solve this issue, contact the programmer of the application or the vendor if the program is a vendor program product.

A.2 Invalid keys in CKDS

After starting ICSF or refreshing the in-storage CKDS, the message that is shown in Example A-13 might be issued if ICSF detects any invalid keys in the CKDS.

Example: A-13 ICSF detects an invalid key in the CKDS

```
CSFM533I CKDS RECORD 8 UNUSABLE AND SKIPPED, LABEL DATASET.PE06.ICSF.E
NCRYPT.ME.ENCRKEY.00000001.
CSFM533I CKDS RECORD 9 UNUSABLE AND SKIPPED, LABEL DATASET.PE06.ICSF.E
NCRYPT.ME.ENCRKEY.00000002.
```

If the key is required, you can attempt to recover the key from a backup. (For more information, see 9.1, “Backing up and restoring data set encryption keys” on page 194.) Otherwise, the message can be ignored.

You can access a data set that is associated with an invalid key. The error that occurs when attempting to access a data set that is associated with an invalid key is shown in Example A-14. For example, you define the key as CIPHER instead of DATA.

Example: A-14 Error when accessing a data set that is associated with an invalid key

```
IEC143I 213-85,IGG0193V,PRICHAR,IKJACCNT,ISP09234,9642,CONSM3,
PE06.ICSF.ENCRYPT.ME07.DATA,
RC=X'00000008',RSN=X'0000085E'
***
```

The ICSF CKDS browser also shows you at a quick glance the key type and for data set encryption (see Example A-15). You must define them as DATA keys (Data-encrypting key for the CSNBDEC, CSNBENC, CSNBSAD, CSNBSAE, CSNBSYD, and CSNBSYE services).

Example: A-15 ICSF CKDS browser

```
A S Label     Displaying 1       to 19     of 19                       Key Type
 -------------------------------------------------------------------------------
 _ - AAAA.FIRST.KEY                                                     DATA
 _ - DATASET.ENCRYPTKEY.001                                             DATA
 _ - DATASET.PE01.TEST                                                  DATA
 _ A DATASET.PE01.TESTNEWGEN                                            DATA
 _ I DATASET.PE01.TESTNEWKEY                                            DATA
 _ I DATASET.PE03.AC01                                                  DATA
 _ - DATASET.PE06.ICSF.ENCRYPT.ME.ENCRKEY.00000005                      DATA
 _ - DATASET.PE06.ICSF.ENCRYPT.ME.ENCRKEY.00000006                      DATA
 _ - DATASET.PE06.ICSF.ENCRYPT.ME06.ENCRKEY.00000006                    DATA
 _ - DATASET.PE06.ICSF.ENCRYPT.ME07.ENCRKEY.00000007                    CIPHER
 _ - KEYLABEL.ANDY.COU1                                                 DATA
 _ - KEYLABEL.THOMAS.LIU                                                DATA
 _ - NOSTANDARD.NAMING.CONVENTION                                       DATA
```

The remedy is to delete the key and redefine with KGUP or ICSF services as a DATA key. For more information, see 5.3, "Generating a secure 256-bit data set encryption key" on page 131.

A.3 Keys

In this section, we describe issues you might encounter with expiring, expired, and archived keys.

A.3.1 Expiring keys

The health checker sends the message that is shown in Example A-16 if it detects that one or more keys are soon to be expired.

Example: A-16 Error records expiring

```
CSFH0031E Records were detected that will expire within the next 60 days.
```

If you browse the check record, you receive the information that is shown in Example A-17 on page 251.

Example: A-17 Browsing check record

```
CHECK(IBMICSF,ICSF_KEY_EXPIRATION)
SYSPLEX:    PLEX60    SYSTEM: SC60
START TIME: 11/14/2017 17:55:42.374895
CHECK DATE: 20140101  CHECK SEVERITY: MEDIUM
CHECK PARM: DAYS(60)

CSFH0030I Cryptographic records expiring in 60 days.

Active CKDS: SYS1.SC60NEW.SCSFCKDS
---------------------------------------------------------

Records expiring on 20171114
DATASET.PE06.ICSF.ENCRYPT.ME06.ENCRKEY.00000006                 DATA

Records expiring on 20171118
PE01.TEST.ACCESS.KEY                                            DATA

Active PKDS: SYS1.SC60NEW.SCSFPKDS
…
…
END TIME: 11/14/2017 17:55:42.375335  STATUS: EXCEPTION-MED
```

If your key is expected to expire soon, consider whether the key validity date should be extended or if the key should be rotated. For more information, see 3.6.10, “Establishing cryptoperiods” on page 56.

A.3.2 Expired keys

If you attempt to access an encrypted data set while its encryption key is expired, you receive the error messages that are shown in Example A-18.

Example: A-18 Error expired key

```
IEC143I 213-85,IGG0193V,PRICHAR,IKJACCNT,ISP09203,96C2,CONSM4,
PE06.ICSF.ENCRYPT.ME06.DATA,
RC=X'00000008',RSN=X'00000D0F'
***
```

Consider whether the key validity date should be extended or the key should be rotated. For more information, see 9.6, “Setting key expiration dates” on page 219.

A.3.3 Archived keys

If you attempt to access an encrypted data set for which the encryption key is archived, you receive the error messages that are shown in Example A-19.

Example: A-19 Error encrypted key archived

```
CSFM655I AN ARCHIVED RECORD DATASET.PE06.ICSF.ENCRYPT.ME.ENCRKEY.00000005 IN TH
E ACTIVE CKDS WAS REFERENCED.
 ***
```

For more information about recalling an archived key, see 9.4, “Archiving data set encryption keys” on page 217.

B

Sample REXX scripts for creating DATA and CIPHER keys

The scripts in this appendix will create random, unknown keys in a single CKDS.

Important:

- To avoid losing the generated keys, it is important to always keep an up-to-date backup of the CKDS and the corresponding master keys.
- Without a backup, there is a risk of losing encrypted data as a result of a lost key, e.g. as a result of accidental key deletion.

CIPHER key creation sample REXX script is shown in Example B-1.

Example B-1 CIPHER key creation sample

```
/* REXX */

/* Pervasive (Data Set) Encryption: Step 6 of 10                    */
/*------------------------------------------------------------------*/
/* Generate a secure AES CIPHER key, store the key in the CKDS and  */
/* display the key label and key. If the key label already exists,  */
/* return the existing key label.                                   */
/*------------------------------------------------------------------*/
/* Instructions:                                                    */
/* - Update aes_key_label with your desired key label name          */
/*                                                                  */
/*      Note: An example key label naming scheme is                 */
/*              DATASET.<dataset_resource>.ENCRKEY.<seqno>          */
/*                                                                  */
/* - EXECUTE THIS CLIST FROM TSO                                    */
/*   (E.G. EX 'HLQ.MLD.LLQ(GENKEYC)')                               */
/*------------------------------------------------------------------*/
SIGNAL ON NOVALUE;

aes_key_label = ,
```

```
   LEFT('keylabel.to.be.created',64);

/*------------------------------------------------------------------*/
/* Check if the key exists in the CKDS (to prevent overwriting)      */
/*------------------------------------------------------------------*/
KRR2_rc    = 'FFFFFFFF'X;
KRR2_label = aes_key_label;
KRR2_token = COPIES('00'X,725);
CALL CSNBKRR2;                   /* If key is found, print and exit */
IF (KRR2_rc = '00000000'X) THEN
  DO;
    SAY 'Secure key label: '||STRIP(aes_key_label);
    SAY '  already exists. Stopping.';
    EXIT;
  END;

/*------------------------------------------------------------------*/
/* Build a skeleton with the correct key usage and key management    */
/*------------------------------------------------------------------*/
KTB2_rule_array_count        = '0000000C'X;
KTB2_rule_array              =,
   'INTERNALAES     CIPHER  XPRTCPACANY-MODE'||,
   'NOEX-SYMNOEX-RAWNOEXUASYNOEXAASYNOEX-DESNOEX-AESNOEX-RSA';
KTB2_target_key_token_length = '000002D5'X;
KTB2_target_key_token        = COPIES('DD'X,725);
CALL CSNBKTB2;
SAY 'Skeleton token created.';

/*------------------------------------------------------------------*/
/* Generate a 256-bit AES CIPHER key                                 */
/*------------------------------------------------------------------*/
KGN2_rule_array_count = '00000002'x;
KGN2_rule_array       = 'AES     '||,
                        'OP      ';
KGN2_clear_bit_length = '00000100'x; /* 256-bit */
KGN2_key_type_1       = 'TOKEN   ';
KGN2_key_type_2       = '';
KGN2_generated_key_id_1_length = '000002D5'x;
KGN2_generated_key_id_1        = LEFT(KTB2_target_key_token,725);
KGN2_generated_key_id_2_length = '00000000'x;
KGN2_generated_key_id_2        = '';
Call CSNBKGN2;
SAY 'Random secure key token generated.';

/*------------------------------------------------------------------*/
/* Store the key in the CKDS                                         */
/*------------------------------------------------------------------*/
KRC2_label        = aes_key_label;
KRC2_token_length = KGN2_generated_key_id_1_length;
KRC2_token        = KGN2_generated_key_id_1;
CALL CSNBKRC2;
SAY 'Successfully written to the CKDS.';

/*------------------------------------------------------------------*/
/* Read the key from the CKDS                                        */
```

```
/*-----------------------------------------------------------------*/
KRR2_label = aes_key_label;
KRR2_token = COPIES('00'X,725);
CALL CSNBKRR2;
IF (KRR2_rc \= '00000000'X) THEN
  DO;
    SAY 'KRR2 Failed   (rc=' C2X(KRR2_rc)' rs='C2X(KRR2_rs)')' ;
    SAY 'Secure key label: '||STRIP(aes_key_label);
    SAY '  was not successfully created';
    EXIT;
  END;
KRR2_token = LEFT(KRR2_token,C2D(KRR2_token_len));
IF (KRR2_token \= KGN2_generated_key_id_1) THEN
  DO;
    SAY 'Secure key label: '||STRIP(aes_key_label);
    SAY '  returned from KRR does not match!';
    EXIT;
  END;
SAY 'Secure key label: '||STRIP(aes_key_label);
SAY '  was successfully created';

SAY "-----------------------------------------------------------------"
SAY "End of Sample"
SAY "-----------------------------------------------------------------"

EXIT;

/* --------------------------------------------------------------- */
/* CSNBKRR2 - Key Record Read (CKDS)                               */
/*                                                                 */
/* Reads a key token from the CKDS.                                */
/*                                                                 */
/* See the ICSF Application Programmer's Guide for more details.   */
/* --------------------------------------------------------------- */
CSNBKRR2:
KRR2_rc              = 'FFFFFFFF'X;
KRR2_rs              = 'FFFFFFFF'X;
KRR2_exit_data_length = '00000000'X;
KRR2_exit_data       = '';
KRR2_rule_count      = '00000000'X;
KRR2_rule_array      = '';
KRR2_token_len       = D2C(725,4);
KRR2_token           = COPIES('00'X,725);
ADDRESS LINKPGM "CSNBKRR2",
                "KRR2_rc",
                "KRR2_rs",
                "KRR2_exit_data_length",
                "KRR2_exit_data",
                "KRR2_rule_count",
                "KRR2_rule_array",
                "KRR2_label",
                "KRR2_token_len",
                "KRR2_token";
RETURN;
```

```
/* ------------------------------------------------------------ */
/* CSNBKTB2 - Key Token Build2                                   */
/*                                                               */
/* Builds a variable-length AES skeleton token.                  */
/*                                                               */
/* See the ICSF Application Programmer's Guide for more details.   */
/* ------------------------------------------------------------ */
CSNBKTB2:
KTB2_return_code                 = 'FFFFFFFF'X;
KTB2_reason_code                 = 'FFFFFFFF'X;
KTB2_exit_data_length            = '00000000'X;
KTB2_exit_data                   = '';
KTB2_clear_key_bit_length        = '00000000'X;
KTB2_clear_key_value             = '00000000'X;
KTB2_key_name_length             = '00000000'X;
KTB2_key_name                    = '';
KTB2_user_associated_data_length = '00000000'X;
KTB2_user_associated_data        = '';
KTB2_token_data_length           = '00000000'X;
KTB2_token_data                  = '';
KTB2_service_data_length         = '00000000'X;
KTB2_service_data                = '';
KTB2_target_key_token_length     = '000002D5'X;
KTB2_target_key_token            = COPIES('DD'X,725);
ADDRESS LINKPGM "CSNBKTB2" ,
                "KTB2_return_code"                 ,
                "KTB2_reason_code"                 ,
                "KTB2_exit_data_length"            ,
                "KTB2_exit_data"                   ,
                "KTB2_rule_array_count"            ,
                "KTB2_rule_array"                  ,
                "KTB2_clear_key_bit_length"        ,
                "KTB2_clear_key_value"             ,
                "KTB2_key_name_length"             ,
                "KTB2_key_name"                    ,
                "KTB2_user_associated_data_length" ,
                "KTB2_user_associated_data"        ,
                "KTB2_token_data_length"           ,
                "KTB2_token_data"                  ,
                "KTB2_service_data_length"         ,
                "KTB2_service_data"                ,
                "KTB2_target_key_token_length"     ,
                "KTB2_target_key_token"            ;
IF (KTB2_return_code /= '00000000'x) THEN
  DO;
    SAY 'KTB2: RC='||C2X(KTB2_return_code)||,
             ' RS='||C2X(KTB2_reason_code);
    EXIT;
  END;
RETURN;

/* ------------------------------------------------------------ */
/* CSNBKGN - Key Generate                                        */
/*                                                               */
/* Generates either one or two DES or AES keys encrypted under a   */
```

```
/* master key (internal form) or KEK (external form).              */
/*                                                                 */
/* See the ICSF Application Programmer's Guide for more details.   */
/* --------------------------------------------------------------- */
CSNBKGN2:
KGN2_rc = 'FFFFFFFF'x;
KGN2_rs = 'FFFFFFFF'x;
KGN2_exit_data_length   = '00000000'x;
KGN2_exit_data          = '';
KGN2_key_name_1_length  = '00000000'x;
KGN2_key_name_1         = '';
KGN2_key_name_2_length  = '00000000'x;
KGN2_key_name_2         = '';
KGN2_user_data_1_length = '00000000'x;
KGN2_user_data_1        = '';
KGN2_user_data_2_length = '00000000'x;
KGN2_user_data_2        = '';
KGN2_kek_id_1_length    = '00000000'x;
KGN2_kek_id_1           = '';
KGN2_kek_id_2_length    = '00000000'x;
KGN2_kek_id_2           = '';
ADDRESS linkpgm 'CSNBKGN2' ,
   'KGN2_rc'                          'KGN2_rs'                 ,
   'KGN2_exit_data_length'            'KGN2_exit_data'          ,
   'KGN2_rule_array_count'            'KGN2_rule_array'         ,
   'KGN2_clear_bit_length'                                      ,
   'KGN2_key_type_1'                  'KGN2_key_type_2'         ,
   'KGN2_key_name_1_length'           'KGN2_key_name_1'         ,
   'KGN2_key_name_2_length'           'KGN2_key_name_2'         ,
   'KGN2_user_data_1_length'          'KGN2_user_data_1'        ,
   'KGN2_user_data_2_length'          'KGN2_user_data_2'        ,
   'KGN2_kek_id_1_length'             'KGN2_kek_id_1'           ,
   'KGN2_kek_id_2_length'             'KGN2_kek_id_2'           ,
   'KGN2_generated_key_id_1_length'  'KGN2_generated_key_id_1' ,
   'KGN2_generated_key_id_2_length'  'KGN2_generated_key_id_2' ;

IF KGN2_rc /= '00000000'x THEN
  DO;
    SAY 'KGN2 Failed   (rc=' c2x(KGN2_rc)' rs='c2x(KGN2_rs)')' ;
    EXIT;
  END;
ELSE
  KGN2_generated_key_id_1 = ,
     LEFT(KGN2_generated_key_id_1,C2D(KGN2_generated_key_id_1_length));
RETURN;

/* --------------------------------------------------------------- */
/* CSNBKRC2 - Key Record Create2                                   */
/*                                                                 */
/* Adds a key token to the CKDS.                                   */
/*                                                                 */
/* See the ICSF Application Programmer's Guide for more details.   */
/* --------------------------------------------------------------- */
CSNBKRC2:
KRC2_rc = 'FFFFFFFF'X;
```

```
KRC2_rs = 'FFFFFFFF'X;
KRC2_exit_data_length = '00000000'X;
KRC2_exit_data = '';
KRC2_rule_count = '00000000'X;
KRC2_rule_array = '';
ADDRESS LINKPGM "CSNBKRC2",
                "KRC2_rc",
                "KRC2_rs",
                "KRC2_exit_data_length",
                "KRC2_exit_data",
                "KRC2_rule_count",
                "KRC2_rule_array",
                "KRC2_label",
                "KRC2_token_length",
                "KRC2_token";
IF (KRC2_rc /= '00000000'X) THEN
  DO;
    SAY 'KRC2 Failed   (rc=' C2X(KRC2_rc)' rs='C2X(KRC2_rs)')' ;
    EXIT;
  END;
RETURN;

/* --------------------------------------------------------------- */
/* Debug                                                           */
/* --------------------------------------------------------------- */
NOVALUE:
SAY 'Condition NOVALUE was raised.';
SAY CONDITION('D')||' variable was not initialized.';
SAY SOURCELINE(sigl)
EXIT;
```

DATA key creation sample REXX script is shown in Example B-2.

Example B-2 DATA key creation sample

```
/* REXX */

/* Pervasive (Data Set) Encryption: Step 6 of 10                   */
/*-----------------------------------------------------------------*/
/* Generate a secure AES DATA key, store the key in the CKDS and    */
/* display the key label and key. If the key label already exists,  */
/* return the existing key label.                                   */
/*-----------------------------------------------------------------*/
/* Instructions:                                                    */
/* - Update aes_key_label with your desired key label name          */
/*                                                                  */
/*      Note: An example key label naming scheme is                 */
/*              DATASET.<dataset_resource>.ENCRKEY.<seqno>          */
/*                                                                  */
/* - EXECUTE THIS CLIST FROM TSO                                    */
/*   (E.G. EX 'HLQ.MLD.LLQ(GENKEY)')                                */
/*-----------------------------------------------------------------*/
SIGNAL ON NOVALUE;

aes_key_label = ,
   LEFT('keylabel.to.be.created',64);
```

```
/*-------------------------------------------------------------------*/
/* Check if the key exists in the CKDS (to prevent overwriting)      */
/*-------------------------------------------------------------------*/
KRR_rc    = 'FFFFFFFF'X;
KRR_label = aes_key_label;
KRR_token = COPIES('00'X,64);
CALL CSNBKRR;                    /* If key is found, print and exit */
IF (KRR_rc = '00000000'X) THEN
  DO;
    SAY 'Secure key label: '||STRIP(aes_key_label);
    SAY '  already exists. Stopping.';
    EXIT;
  END;

/*-------------------------------------------------------------------*/
/* Generate a 256-bit AES DATA key                                   */
/*-------------------------------------------------------------------*/
KGN_key_form                   = 'OP  ';
KGN_key_length                 = 'KEYLN32 ';
KGN_key_type_1                 = 'AESDATA ';
KGN_key_type_2                 = '';
KGN_kek_identifier_1           = COPIES('00'X,64);
KGN_kek_identifier_2           = '';
KGN_generated_key_identifier_1 = COPIES('00'X,64);
KGN_generated_key_identifier_2 = '';
CALL CSNBKGN;

/*-------------------------------------------------------------------*/
/* Store the key in the CKDS                                         */
/*-------------------------------------------------------------------*/
KRC2_label        = aes_key_label;
KRC2_token_length = '00000040'X;
KRC2_token        = KGN_generated_key_identifier_1;
CALL CSNBKRC2;

/*-------------------------------------------------------------------*/
/* Read the key from the CKDS                                        */
/*-------------------------------------------------------------------*/
KRR_label = aes_key_label;
KRR_token = COPIES('00'X,64);
CALL CSNBKRR;
IF (KRR_rc \= '00000000'X) THEN
  DO;
    SAY 'KRR Failed   (rc=' C2X(KRR_rc)' rs='C2X(KRR_rs)')' ;
    SAY 'Secure key label: '||STRIP(aes_key_label);
    SAY '  was not successfully created';
    EXIT;
  END;
IF (KRR_token \= KGN_generated_key_identifier_1) THEN
  DO;
    SAY 'Secure key label: '||STRIP(aes_key_label);
    SAY '  returned from KRR does not match!';
    EXIT;
  END;
```

```
SAY 'Secure key label: '||STRIP(aes_key_label);
SAY '  was successfully created';

SAY "----------------------------------------------------------------"
SAY "End of Sample"
SAY "----------------------------------------------------------------"

EXIT;

/* ------------------------------------------------------------- */
/* CSNBKGN - Key Generate                                         */
/*                                                                */
/* Generates either one or two DES or AES keys encrypted under a  */
/* master key (internal form) or KEK (external form).             */
/*                                                                */
/* See the ICSF Application Programmer's Guide for more details.  */
/* ------------------------------------------------------------- */
CSNBKGN:
KGN_rc = 'FFFFFFFF'X;
KGN_rs = 'FFFFFFFF'X;
KGN_exit_data_length         = '00000000'X;
KGN_exit_data                = '';
ADDRESS linkpgm "CSNBKGN",
   'KGN_rc'                          'KGN_rs'               ,
   'KGN_exit_data_length'            'KGN_exit_data'        ,
   'KGN_key_form'                    'KGN_key_length'       ,
   'KGN_key_type_1'                  'KGN_key_type_2'       ,
   'KGN_kek_identifier_1'            'KGN_kek_identifier_2' ,
   'KGN_generated_key_identifier_1'  'KGN_generated_key_identifier_2';
IF (KGN_rc /= '00000000'X) THEN
  DO;
    SAY 'KGN Failed   (rc=' C2X(KGN_rc)' rs='C2X(KGN_rs)')' ;
    EXIT;
  END;
RETURN;

/* ------------------------------------------------------------- */
/* CSNBKRC2 - Key Record Create2                                  */
/*                                                                */
/* Adds a key token to the CKDS.                                  */
/*                                                                */
/* See the ICSF Application Programmer's Guide for more details.  */
/* ------------------------------------------------------------- */
CSNBKRC2:
KRC2_rc = 'FFFFFFFF'X;
KRC2_rs = 'FFFFFFFF'X;
KRC2_exit_data_length = '00000000'X;
KRC2_exit_data = '';
KRC2_rule_count = '00000000'X;
KRC2_rule_array = '';
ADDRESS LINKPGM "CSNBKRC2",
                "KRC2_rc",
                "KRC2_rs",
                "KRC2_exit_data_length",
                "KRC2_exit_data",
```

```
                "KRC2_rule_count",
                "KRC2_rule_array",
                "KRC2_label",
                "KRC2_token_length",
                "KRC2_token";
IF (KRC2_rc /= '00000000'X) THEN
  DO;
    SAY 'KRC2 Failed   (rc=' C2X(KRC2_rc)' rs='C2X(KRC2_rs)')' ;
    EXIT;
  END;
RETURN;

/* --------------------------------------------------------------- */
/* CSNBKRR - Key Record Read (CKDS)                                */
/*                                                                 */
/* Reads a key token from the CKDS.                                */
/*                                                                 */
/* See the ICSF Application Programmer's Guide for more details.   */
/* --------------------------------------------------------------- */
CSNBKRR:
KRR_rc = 'FFFFFFFF'X;
KRR_rs = 'FFFFFFFF'X;
KRR_exit_data_length = '00000000'X;
KRR_exit_data    = '';
KRR_token        = COPIES('00'X, 64);
ADDRESS LINKPGM "CSNBKRR",
                "KRR_rc",
                "KRR_rs",
                "KRR_exit_data_length",
                "KRR_exit_data",
                "KRR_label",
                "KRR_token";
RETURN;

/* --------------------------------------------------------------- */
/* Debug                                                           */
/* --------------------------------------------------------------- */
NOVALUE:
SAY 'Condition NOVALUE was raised.'
SAY CONDITION('D')||' variable was not initialized.'
SAY SOURCELINE(sigl)
EXIT;
```

Abbreviations and acronyms

ACS	Automatic class selection
ACSP	Advanced Crypto Service Provider
AES	Advanced Encryption Standard
API	Application Programming Interface
AWS	Amazon Web Services
BPAM	Basic partitioned access method
BSAM	Basic sequential access method
CC	Crypto Connect
CCA	Common Cryptographic Architecture
CCPA	California Consumer Privacy Act
CKDS	Cryptographic key data set
CMK	Current master key
CPACF	Central Processor Assist for Cryptographic Function
CSR	Certificate signing requests
CVSS	Common Vulnerability Scoring System
DES	Data Encryption Standard
DFP	Data Facility Product
DFSMS	Data Facility Storage Management Subsystem
DHS	Department of Homeland Security
DK	Data key
DR	Disaster recovery
DRK	Disaster Recovery Key
EJB	Enterprise JavaBeans
EP11	Enterprise PKCS #11
ESDS	Entry-sequenced data set
ESM	External Security Manager
EU	European Union
EXCP	Execute Channel Program
GBIC	German Banking Industry Committee
GCP	Google Cloud Platform
GDPR	General Data Protection Regulation
GKLM	Guardium Key Lifecycle Manager
HIPAA	Health Insurance Portability and Accountability Act
HIPER	High Impact PERvasive
HLQ	High-level qualifier
HSA	Hardware system area
HSM	Hardware Security Module
IBM	International Business Machines Corporation
ICSF	Integrated Cryptographic Services Facility
IMS	IBM Information Management System
ISMF	Interactive Storage Management Facility
ISPF	Interactive System Productivity Facility
ISV	Independent Software Vendor
KDS	Key data set
KDSR	Key record format
KEK	Key-encrypting key
KGUP	Key Generator Utility Program
KSDS	Key-sequenced data set
LDS	Linear data set
LPARs	Logical partitions
MFA	Multi-Factor Authentication
MKVP	master key verification pattern
MM	Media Manager
MSDKE	Microsoft Azure Double Key Encryption
Mkab	Master Key
NMK	New master key
OMB	Office of Management and Budget's
OMK	Old master key
OUTPUT	OUTFILE
PAN	Primary Account Number
PERA	Pervasive Encryption Readiness Assessment
PIN	Personal identification number
PKDS	Public key data set
PPINIT	PassPhrase Initialization
PPRC	Peer-to-Peer Remote Copy
QSAM	Queued sequential access method
RACF	Resource Access Control Facility
RLS	Record-level sharing
RMF	Resource Measurement Facility
RNG	Random Number Generator

RNGs	Random number generators
RRDS	Relative record data set
SAF	System Authorization Facility
SDSF	System Display and Search Facility
SECINT	Security Vulnerability
SHCDS	Sharing control data set
SMF	System Management Facility
SMS	Storage Management Subsystem
SOX	Sarbanes-Oxley Act of 2002
STSM	Senior Technical Staff Member
SVL	Silicon Valley Laboratory
TK	Transport key
TKDS	Token data set
TKE	Trusted Key Entry
TSO	Time-Sharing Option
UKO	Unified Key Orchestrator
VRRDS	Variable relative record data set
VSAM	Virtual Storage Access Method
WLMGL	Workload Activity Report - Goal Mode
eIDAS	electronic IDentification, Authentication and trust Services
zBNA	Z Batch Network Analyzer
zEDC	Z Enterprise Data Compression
zERT	z/OS Encryption Readiness Technology

Related publications

The publications listed in this section are considered particularly suitable for a more detailed discussion of the topics covered in this book.

IBM Redbooks

The following IBM Redbooks publications provide additional information about the topic in this document. Note that some publications referenced in this list might be available in softcopy only.

- *IBM z/OS Cryptographic Services Integrated Cryptographic Service Facility System Programmer's Guide,* SC14-7507
- *Transitioning to Quantum-Safe Cryptography on IBM Z,* SG24-8525
- *IBM z/OS DFSMS Using Data Sets*, SC23-6855

You can search for, view, download or order these documents and other Redbooks, Redpapers, Web Docs, draft and additional materials, at the following website:

ibm.com/redbooks

Help from IBM

IBM Support and downloads

ibm.com/support

Services from IBM Consulting

ibm.com/services

IBM Training

ibm.com/training

Redbooks

Getting Started with z/OS Data Set Encryption

SG24-8410-02

ISBN 0738460222

(0.5” spine)
0.475”<->0.873”
250 <-> 459 pages

SG24-8410-02

ISBN 0738462527

Printed in U.S.A.

Get connected

ibm.com/redbooks